

EVA SQUARE

Health Interpretation Platform

Technical Manual

Full-Structure Codebase Reference

Frontend · Backend · Database — `Prototype1_Version2_FullStructure`

What this manual covers

- The complete three-tier architecture (frontend / backend / database)
- Every source file, module by module, function by function
- Schematics, entity-relationship, dependency and sequence diagrams
- Cross-cutting request & data flows end to end

Contents

1	Introduction	1
1.1	Purpose of this manual	1
1.2	What EVA SQUARE does	1
1.3	Relationship to the original prototype	1
1.4	Document conventions	1
1.5	New interactive features in this build	2
2	System Architecture	3
2.1	Three-tier overview	3
2.2	Why three independent directories	3
2.3	Request lifecycle	3
2.4	Technology stack summary	4
2.5	Design philosophy: “basis version” upgrade paths	4
3	Directory and File Structure	5
3.1	Top-level layout	5
3.2	Frontend files	7
3.3	Backend files	7
3.4	Database files	8
3.5	Root-level files	9
4	Database Tier	10
4.1	Overview	10
4.2	Entity–relationship diagram	10
4.3	Table reference	12
4.4	schema.sql: the generated snapshot	14
4.5	Upgrading to a deployed web version	14
5	Backend Tier	15
5.1	Core	16
5.1.1	app/main.py — application endpoint	16
5.1.2	app/database.py — engine and session factory	17
5.1.3	app/deps.py — shared dependencies	18
5.1.4	app/models.py — ORM models	18
5.1.5	app/schemas.py — Pydantic request/response models	21
5.1.6	app/security.py — password hashing and sessions	22
5.1.7	app/seed.py — optional CLI seed script	23
5.1.8	requirements.txt	24
5.2	Static reference content (app/data/)	25
5.2.1	app/data/lab_dictionary.py	25
5.2.2	app/data/legal_content.py	26
5.2.3	app/data/questionnaire.py	29
5.2.4	app/data/recommendations.py	31
5.2.5	app/data/reference_scales.py	33
5.2.6	app/data/sample_reports.py	37

5.3	Clinical rule engines (app/engines/)	41
5.3.1	app/engines/orchestrator.py — ties every engine together	41
5.3.2	app/engines/cardiovascular.py — the real-spec engine	43
5.3.3	app/engines/other_domains.py — the five illustrative domains	53
5.3.4	app/engines/rule_engine.py — generic range classifier	54
5.3.5	app/engines/safety_engine.py — overall pathway	55
5.3.6	app/engines/recommendation_engine.py	56
5.3.7	app/engines/report_generator.py	56
5.3.8	app/engines/trend_engine.py	59
5.3.9	app/engines/question_engine.py	59
5.4	Routers (app/routers/)	61
5.4.1	app/routers/auth.py	61
5.4.2	app/routers/consent.py	64
5.4.3	app/routers/settings.py	65
5.4.4	app/routers/reports.py — the core pipeline	66
5.4.5	app/routers/meta.py	70
5.4.6	app/routers/action_plan.py	71
6	Frontend Tier	72
6.1	Document shell and styling	72
6.1.1	index.html — document shell and script load order	72
6.1.2	css/style.css — design tokens and component styles	73
6.2	Data layer	86
6.2.1	js/api.js — REST fetch wrapper	86
6.2.2	js/store.js — client-side cache and need()	87
6.3	Application bootstrap	93
6.3.1	js/app.js — SQ.act.* and boot	93
6.4	UI library and chrome	97
6.4.1	js/ui/components.js — reusable component library	97
6.4.2	js/ui/shell.js — page chrome	100
6.4.3	js/ui/router.js — hash router	102
6.5	New interactive features	105
6.5.1	js/ui/anatomy.js — interactive anatomy board	105
6.5.2	js/ui/scaleRow.js — expandable reference-scale rows	111
6.5.3	js/ui/trendChart.js — enlarged interactive trend chart	113
6.6	Screens (js/ui/screens/)	119
6.6.1	js/ui/screens/auth.js — Welcome, Register, Login, Terms, Consent	119
6.6.2	js/ui/screens/dashboard.js — Dashboard	122
6.6.3	js/ui/screens/upload.js — Upload a Report	124
6.6.4	js/ui/screens/verify.js — Verify Extracted Results	125
6.6.5	js/ui/screens/questions.js — Triggered Health Questionnaire	126
6.6.6	js/ui/screens/analysis.js — Analysis Loading and Summary	128
6.6.7	js/ui/screens/results.js — single-marker Result Detail	130
6.6.8	js/ui/screens/domain.js — Health Domain detail (six tabs)	131
6.6.9	js/ui/screens/actionPlan.js — Action Plan	135
6.6.10	js/ui/screens/referral.js — Referral & Next Steps	137
6.6.11	js/ui/screens/report.js — Full Report	138
6.6.12	js/ui/screens/trends.js — Trends	139
6.6.13	js/ui/screens/profile.js — Health Profile	140
6.6.14	js/ui/screens/reportsLibrary.js — Blood Reports library	142
6.6.15	js/ui/screens/settings.js — Settings	142

7	Cross-Tier Data Flow	144
7.1	The core pipeline, end to end	144
7.2	Why the frontend navigates before some calls resolve	146
7.3	The need () cache lifecycle	146
7.4	Safety pathway aggregation	146
8	Build, Run and Deployment	148
8.1	Running the local basis version	148
8.2	run.bat — the Windows one-click launcher	148
8.3	README.md	151
8.4	Deployment upgrade path, consolidated	154
A	REST API Reference	155
A.1	Auth — /api/auth	155
A.2	Consent — /api/consent	155
A.3	Settings — /api/settings	155
A.4	Reports — /api/reports	156
A.5	Action plan — /api/action-plan	156
A.6	Meta (static reference content) — /api/meta	157
A.7	Health check	157
B	Static Reference Content: Quick Tables	158
B.1	Health domains	158
B.2	Lab dictionary	158
B.3	Anthropometric rules	159
B.4	Recommendation library (action plan)	159
B.5	Referral ladder	159
B.6	Demo accounts	160

List of Figures

2.1	The three tiers and the request/response paths between them. A single uvicorn process hosts the middle tier and also mounts the frontend's static files, so no CORS configuration is needed for local use.	3
3.1	Complete directory tree. Colour matches the tier legend used throughout this manual: blue = frontend, teal = backend, amber = database.	6
4.1	Entity-relationship diagram of all seven tables in <code>square.db</code> , generated from <code>backend/app/models.py</code> . Blue = account/identity; teal = normalized per-user state; amber = document-style (JSON-column) report state.	11
5.1	Backend module dependency graph. Solid arrows are direct imports; dashed arrows are read-only references to static reference content.	15
5.2	Data flow through <code>orchestrator.run_analysis()</code> : six parallel domain analyses feed one assembled findings bundle, which the safety, recommendation and report-generator engines each consume independently before their outputs are combined into the final result.	41
5.3	Registration sequence: a new account is created, a session token is issued and cached client-side, and the freshly-created (empty) consent/settings rows are fetched before the router hands off to the Terms screen. This is one instance of the general auth pattern — login and demo-login differ only in whether a new <code>users</code> row is inserted.	63
6.1	Frontend module dependency graph. <code>index.html</code> fixes the script load order; every screen file registers itself onto <code>SQ.screens.*</code> and is dispatched to by the router.	72
6.2	Schematic redrawing of the anatomy board's layout: six leader-lined domain cards, coloured by that domain's worst risk tone (§6.5.1), each pointing to a stylised organ anchor. The real board is one SVG with the cards embedded via <code><foreignObject></code> ; this figure reproduces its geometry, not its exact organ artwork.	105
7.1	The four-stage core pipeline traced across all three tiers. Every step after upload reads from and writes to the same <code>reports</code> row (and its two child rows); <code>GET /api/reports/{id}/analysis</code> (not shown) can recompute the same result lazily if a client asks for it before <code>/analyze</code> has run, as long as the report is already verified.	145
7.2	The cache lifecycle every <code>SQ.store.get*(id)</code> getter follows. A screen calling a getter while a value is loading always renders a loading state on that same tick — the getter never blocks or throws, so a screen never needs its own loading flag.	146
7.3	Six independent domain urgency signals collapse into one <code>safety.overall</code> value, which then selects the highlighted rung of the four-level referral ladder (§6.6.10).	147

Chapter 1

Introduction

1.1 Purpose of this manual

This document is the technical reference for **EVA SQUARE — Prototype 1, Version 2 (Full Structure)**, a three-tier rebuild of the original single-file EVA SQUARE prototype. Where the earlier prototype computed everything client-side in the browser and kept its state in `localStorage`, this build separates the application into three independent, deployable concerns:

- a static **frontend** single-page application (HTML/CSS/JavaScript, no build step),
- a Python **backend** REST API (FastAPI + SQLAlchemy), and
- a local **database** (SQLite, upgradeable to Postgres with a one-line configuration change).

This manual walks through every file in all three tiers, explains what it does and why it is structured the way it is, and illustrates the architecture, schema and cross-cutting request flows with diagrams. It is written for a developer who needs to maintain, extend, or hand off this codebase, and assumes familiarity with Python, JavaScript and basic web application concepts but no prior exposure to this specific project.

1.2 What EVA SQUARE does

EVA SQUARE is a health-interpretation and wellness-navigation platform. A user registers, uploads a blood report, verifies the values the platform extracted from it, answers a short adaptive questionnaire, and receives an analysed health snapshot: a cardiovascular risk assessment built against a real clinical specification, five further “illustrative” health domains (metabolic, liver, renal, haematology, anthropometric), a personalised action plan, a four-level safety/referral pathway, and longitudinal trend tracking across repeat reports. EVA SQUARE explicitly does not provide a medical diagnosis — this is stated throughout both the application’s own copy and its Terms & Consent content (see §5.2.2).

1.3 Relationship to the original prototype

The application’s *content* — every screen, all clinical logic, the two demo scenarios — is unchanged from the original `Prototype1_Version2` single-file build. This Full-Structure version only changes *how* the application is built: the same 20 screens and clinical rule engines have been ported line-for-line from a single HTML file’s inline JavaScript into a proper three-tier system, and four new interactive features were added on top (see §1.5).

1.4 Document conventions

Throughout this manual, file paths are given relative to the project root `Prototype1_Version2_FullStructure/` unless otherwise stated. Code excerpts are reproduced verbatim from the source with line numbers; ellipses (...) indicate omitted, unchanged lines. Endpoint paths are always prefixed with `/api` exactly as the backend registers them. Tone values — **good**, **warn**, and **bad** — are the application’s own three-level risk vocabulary, used consistently from the database-level rule engines through to every pixel of colour in the frontend.

1.5 New interactive features in this build

Four features were added specifically for this Full-Structure build, on top of the ported v2 content:

1. **Interactive anatomy board** (Dashboard) — a schematic 2D body illustration with a leader-lined summary card per health domain, coloured by that domain's worst risk tone, with hover cross-highlighting and click-through navigation. See §6.5.1.
2. **Expandable reference-scale rows** — every quantity in a domain's Results tab can be expanded to reveal its whole reference scale as a coloured band chart, with the patient's value marked on it. See §6.5.2.
3. **Enlarged interactive trend charts** — clicking a quantity on the Trends screen opens a zoomable, pannable canvas chart with Times New Roman axis labels, a Cambria Math trend-formula readout, and a line colour that follows the quantity's own risk level as a smooth gradient. See §6.5.3.
4. **Hover feedback everywhere** — buttons, cards, navigation links, tabs, table rows, toggles, and the three features above all carry a hover transition, consistent with the rest of the application's interaction design.

Chapter 2

System Architecture

2.1 Three-tier overview

EVA SQUARE Full-Structure is a classic three-tier web application, with one deliberate simplification for local/offline use: a single Python process serves both the REST API *and* the static frontend files, so the whole stack starts with one command and needs no separate web server, reverse proxy, or CORS configuration. Figure 2.1 shows the three tiers and how a browser request travels through them.

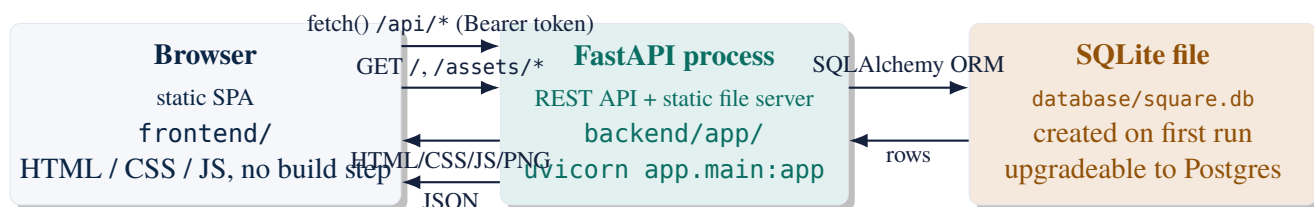


Figure 2.1: The three tiers and the request/response paths between them. A single uvicorn process hosts the middle tier and also mounts the frontend’s static files, so no CORS configuration is needed for local use.

2.2 Why three independent directories

frontend/, backend/ and database/ sit as three sibling directories at the project root, deliberately kept independent so that each concern can be redeployed separately later without restructuring the codebase:

- **frontend/** is a plain static site. It can be lifted onto any CDN or static host unchanged, provided `js/api.js`’s single `API_BASE` constant is pointed at wherever the backend ends up living.
- **backend/** is a self-contained FastAPI application with its own `requirements.txt`. It has no compile-time dependency on the frontend beyond optionally serving it as static files (see `backend/app/main.py`, §5.1.1).
- **database/** holds only the SQLite file and a generated schema snapshot — nothing in the backend code assumes SQLite specifically (see §4).

2.3 Request lifecycle

Two request shapes flow through the system:

1. **Static asset requests** — the browser’s very first request for `/` returns `index.html`; every subsequent `<script>`/`<link>` tag in that file requests a file under `/assets`, `/css` or `/js`, each mounted by FastAPI’s `StaticFiles`. The application itself is a *hash-routed* single-page app (§6.4.3), so navigating within the app (e.g. to `#/dashboard`) never triggers a new server request — only the JavaScript router re-renders `#root`.
2. **API requests** — every data-bearing operation (auth, consent, reports, questionnaire, analysis, trends, action plan, static reference content) is a `fetch()` call from `js/api.js` to a JSON endpoint under `/api/*`, carrying a bearer session token once the user is logged in. See Chapter 7 for full se-

quence diagrams of the two most important multi-step flows (registration/login and the upload → verify → questionnaire → analyse pipeline).

2.4 Technology stack summary

Tier	Technology	Notes
Frontend	Vanilla HTML/CSS/JavaScript	No framework, no bundler, no build step; every screen renders by returning an HTML string; <code>#root.innerHTML</code> is replaced wholesale on every state change.
Frontend	Hash routing	Client-side only; §6.4.3.
Backend	FastAPI (≥ 0.115)	ASGI web framework; also serves the OpenAPI docs at <code>/docs</code> .
Backend	Uvicorn	ASGI server; single process serves both API and static frontend.
Backend	SQLAlchemy (≥ 2.0)	ORM; declarative models in <code>models.py</code> .
Backend	Pydantic (≥ 2.7)	Request/response validation via FastAPI's <code>schemas.py</code> integration.
Database	SQLite	Local file, zero configuration; swappable for Postgres via one environment variable.
Auth	PBKDF2-HMAC-SHA256 (stdlib)	Dependency-free password hashing; opaque bearer session tokens. Documented upgrade path to <code>passlib/argon2</code> + JWT.

Table 2.1: Technology choices by tier, and the deliberate “basis version” simplifications each one documents an upgrade path for.

2.5 Design philosophy: “basis version” upgrade paths

A recurring pattern across the backend’s source comments (and repeated throughout this manual) is documenting, inline, exactly how a simplification made for this local prototype would be swapped out for a production deployment — without restructuring the surrounding code. Three examples recur throughout this manual: the SQLite-to-Postgres swap (§4), the PBKDF2-to-JWT/argon2 auth swap (§5.1.6), and the sample-matched-upload-to-real-OCR swap (§5.4.4). None of these require touching more than the one module each is documented in.

Chapter 3

Directory and File Structure

3.1 Top-level layout

Figure 3.1 shows the complete directory tree of `Prototype1_Version2_FullStructure/`. The three tiers are top-level siblings, exactly as described in §2; each subsection below tabulates every file in one tier with a one-line statement of its purpose, cross-referenced to the chapter where it is documented in full.

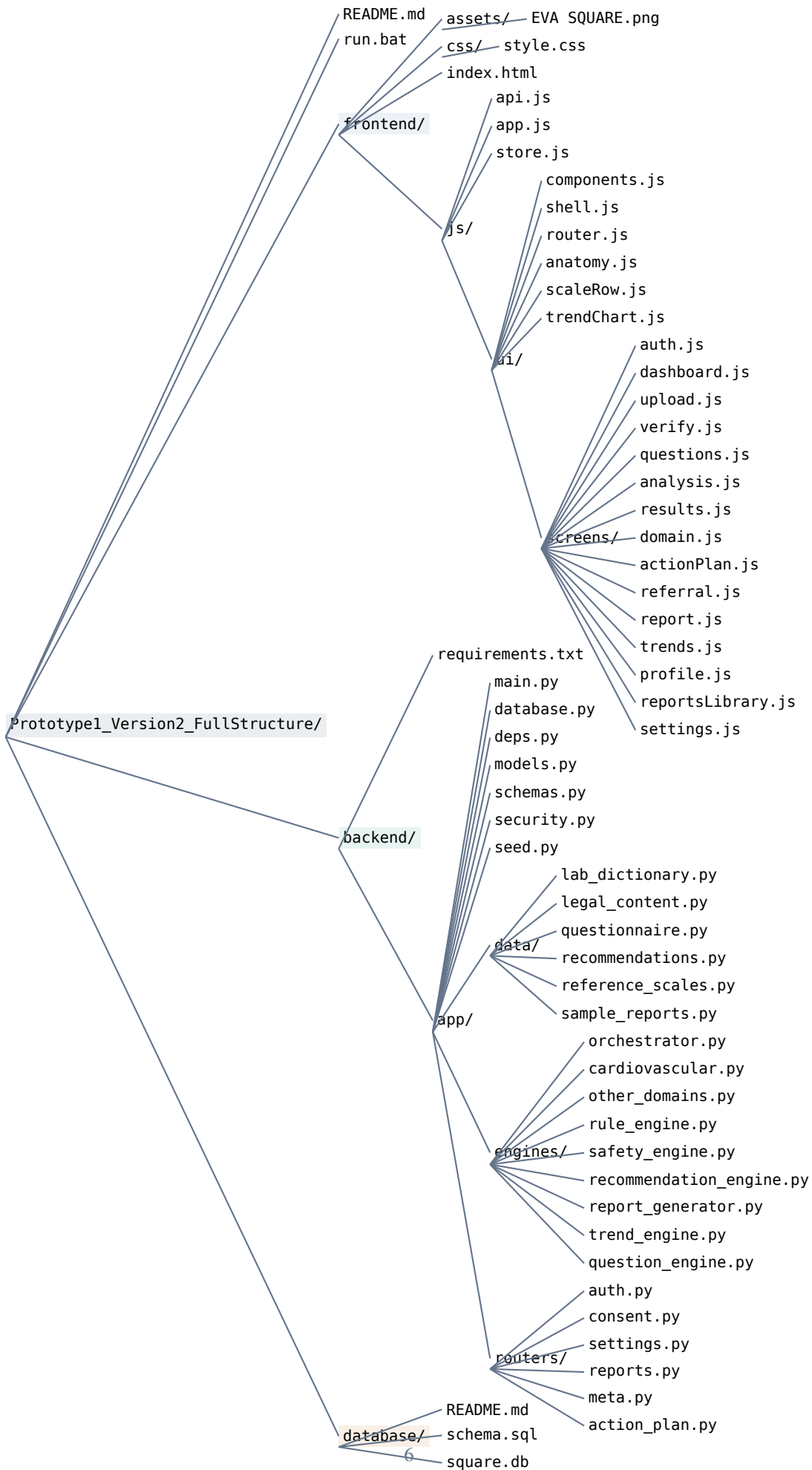


Figure 3.1: Complete directory tree. Colour matches the tier legend used throughout this manual: blue = frontend, teal = backend, amber = database.

3.2 Frontend files

File	Purpose	§
index.html	Document shell; fixed script load order (data layer → UI layer → screens → bootstrap last).	6.1.1
css/style.css	All design tokens and component styles; single stylesheet, no preprocessor.	6.1.2
assets/EVA SQUARE.png	The real product logo, served via /assets static mount.	6.1.1
js/api.js	Thin same-origin fetch() wrapper; every back-end call in the app funnels through SQ.api.*.	6.2.1
js/store.js	Client-side async cache over the REST API; the need() cache-or-fetch pattern.	6.2.2
js/app.js	Bootstraps the router/store subscription and defines every SQ.act.* cross-screen action.	6.3.1
js/ui/components.js	Reusable HTML-string component library (buttons, cards, pills, tables, toasts, ...).	6.4.1
js/ui/shell.js	Page chrome: auth shell, onboarding-stepper shell, app shell (sidebar + topline).	6.4.2
js/ui/router.js	Hash router; route table → screen dispatch, login/auth gating, boot-loading gate.	6.4.3
js/ui/anatomy.js	New feature. Interactive 2D anatomy board (Requirement #1).	6.5.1
js/ui/scaleRow.js	New feature. Expandable reference-scale rows (Requirement #2).	6.5.2
js/ui/trendChart.js	New feature. Enlarged interactive trend chart modal (Requirement #4).	6.5.3
js/ui/screens/*.js	15 files, one per screen group (20 screens total); each registers itself on SQ.screens.*.	6.6

3.3 Backend files

File	Purpose	§
requirements.txt	Pinned dependency ranges: FastAPI, Uvicorn, SQLAlchemy, Pydantic, python-multipart, email-validator.	5.1.8
app/main.py	FastAPI app endpoint; registers every router; mounts static frontend; SPA fallback route.	5.1.1
app/database.py	SQLAlchemy engine/session factory; SQLite path resolution; get_db() dependency.	5.1.2
app/deps.py	Shared FastAPI dependencies: get_current_user, get_owned_report.	5.1.3
app/models.py	Every SQLAlchemy ORM table (7 tables).	5.1.4
app/schemas.py	Pydantic request/response models for typed endpoints.	5.1.5
app/security.py	Password hashing (PBKDF2) and bearer session tokens.	5.1.6

File	Purpose	§
app/seed.py	Optional CLI script to pre-populate both demo accounts.	5.1.7
app/data/lab_dictionary.py	Domain registry, lab code dictionary, simple-range thresholds, anthropometric rules.	5.2.1
app/data/legal_content.py	Organisation info, Terms & Conditions sections, required/optional consent items.	5.2.2
app/data/questionnaire.py	Adaptive Health Questionnaire section/question definitions.	5.2.3
app/data/recommendations.py	Recommendation library, education snippets, four-level referral ladder.	5.2.4
app/data/reference_scales.py	Full-scale band definitions powering the expandable scale-row feature.	5.2.5
app/data/sample_reports.py	The two seed scenarios (“routine” and “urgent”), fully self-consistent demo datasets.	5.2.6
app/engines/orchestrator.py	Ties every engine together into <code>run_analysis()</code> .	5.3.1
app/engines/cardiovascular.py	The real-spec Cardiovascular Domain engine (CV-01 through CV-14).	5.3.2
app/engines/other_domains.py	Illustrative logic for Metabolic, Liver, Renal, Haematology, Anthropometric.	5.3.3
app/engines/rule_engine.py	Generic classify-by-range helper used by the illustrative domains.	5.3.4
app/engines/safety_engine.py	Aggregates every domain’s urgency signal into one overall pathway.	5.3.5
app/engines/recommendation_engine.py	Engines the recommendation library against assembled findings.	5.3.6
app/engines/report_generator.py	Assembles the printable structured report, block by block.	5.3.7
app/engines/trend_engine.py	Longitudinal direction/delta helpers for the Trends feature.	5.3.8
app/engines/question_engine.py	Adaptive-questionnaire section-activation logic.	5.3.9
app/routers/auth.py	Register / login / demo-login / logout / /me endpoints.	5.4.1
app/routers/consent.py	Consent-state read/patch endpoints.	5.4.2
app/routers/settings.py	User preference read/patch endpoints.	5.4.3
app/routers/reports.py	The core pipeline: sample seed, upload, verify, questionnaire, analyse, trends.	5.4.4
app/routers/meta.py	Static reference-content endpoints (domains, lab dictionary, legal text, ...).	5.4.5
app/routers/action_plan.py	Action-plan completion-state endpoints.	5.4.6

3.4 Database files

File	Purpose
square.db	The live SQLite database file; created automatically by <code>Base.metadata.create_all()</code> on first backend start; safe to delete to reset all app data.

File	Purpose
schema.sql	A generated, human-readable snapshot of the schema (via SQLAlchemy's CreateTable); the source of truth remains backend/app/models.py.
README.md	Explains the normalized-vs-JSON table design split and the Postgres upgrade path.

3.5 Root-level files

File	Purpose
README.md	Project overview, what's new versus Prototype1_Version2, run instructions, deployment upgrade summary.
run.bat	One-click Windows launcher: sets up a virtual environment, installs dependencies, seeds the database, opens the browser, and starts uvicorn.

Chapter 4

Database Tier

4.1 Overview

The database tier is a single SQLite file, `database/square.db`, created automatically the first time the backend starts (`Base.metadata.create_all()` in `backend/app/database.py`). It lives in its own top-level directory, outside `backend/`, so that frontend, backend and database read as three independent concerns — see §2. Deleting `square.db` resets all application data; it is silently recreated empty on the next backend start, or can be recreated *with* both demo accounts pre-populated by running `python -m app.seed` (§5.1.7).

Design note: two kinds of tables, deliberately

Account/session/consent/settings/action-plan state is fully *normalized* (one column per fact) because that state is small, queried individually, and benefits from real columns. Report, questionnaire and analysis payloads are deliberately stored as *JSON text columns* (`*_json`) because they are large, nested, always read/written as a whole document, and their shape mirrors the Python engine output one-to-one. Normalizing every lab value and calculator result into its own column or table is exactly the kind of migration a deployed version would make once the schema has proven itself against real usage — doing it now would be guessing.

4.2 Entity–relationship diagram

Figure 4.1 shows all seven tables and their relationships. Every foreign key ultimately traces back to users; a user has at most one `consent_records` row and one `settings` row (one-to-one), any number of `sessions` and `action_plan_items` rows (one-to-many), and any number of `reports`, each of which owns exactly one `questionnaire_answers` row and, once analysed, exactly one `analysis_results` row.

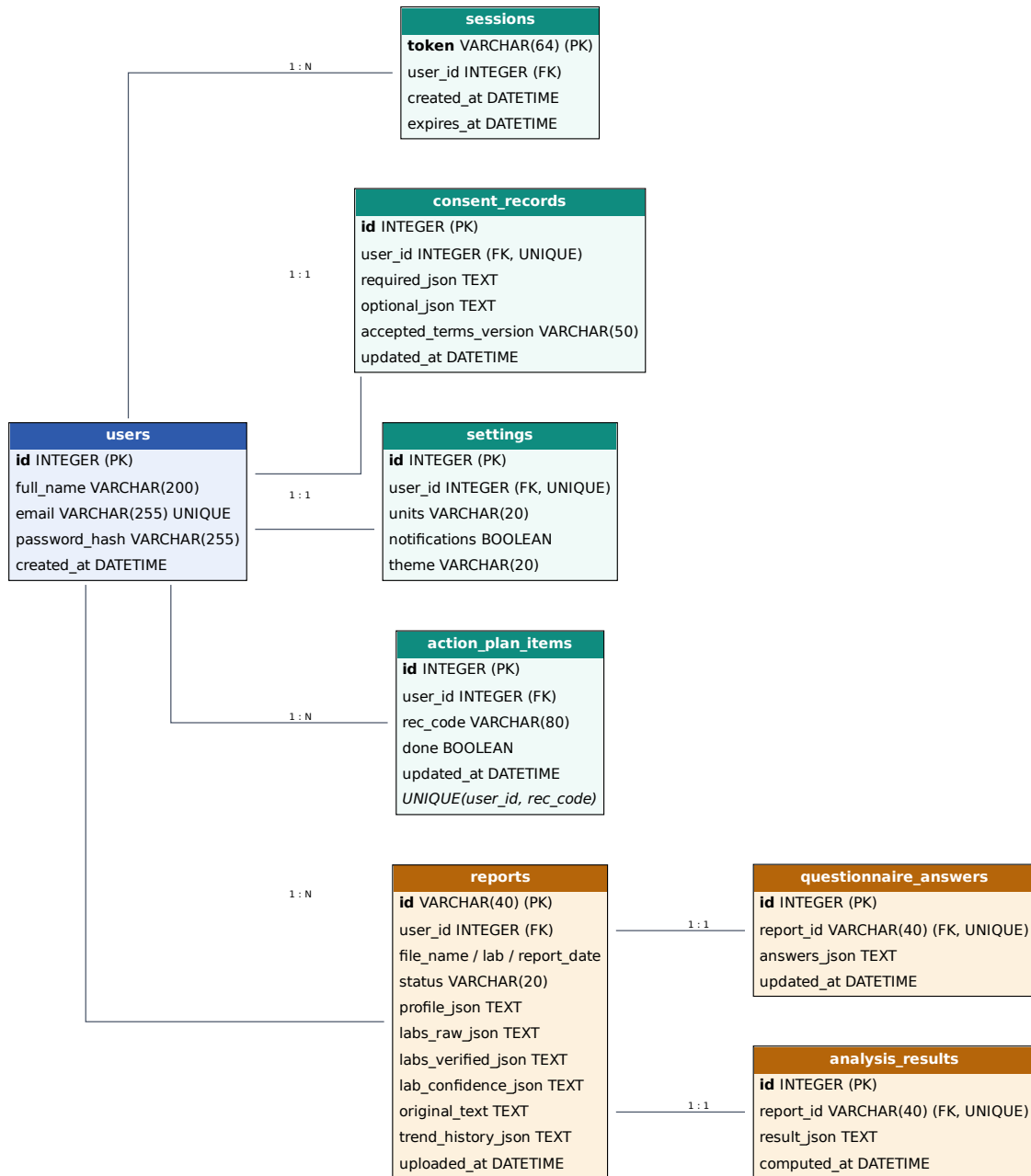


Figure 4.1: Entity-relationship diagram of all seven tables in `square.db`, generated from `backend/app/models.py`. Blue = account/identity; teal = normalized per-user state; amber = document-style (JSON-column) report state.

4.3 Table reference

users

The account table. `password_hash` stores the full PBKDF2 encoding produced by `security.hash_password()` (§5.1.6), not a raw hash — it is self-describing (`scheme$iterations$salt$digest`), so the hashing parameters can change in the future without a migration.

sessions

Bearer session tokens. Deliberately simple — no JWT, no refresh token — for this basis version; token is itself the 64-character primary key (a 32-byte `secrets.token_hex()` value), so looking up a session is a single indexed primary-key read in `deps.get_current_user()` (§5.1.3). `expires_at` is checked on every authenticated request; an expired session is treated identically to a missing one.

consent_records

One row per user (`UNIQUE(user_id)`), holding both required and optional consent state as two independent JSON objects (`{consentId: bool}`) plus the accepted Terms version string. Kept as JSON rather than one column per consent item because the set of consent items is itself content defined in `data/legal_content.py` (§5.2.2), not a fixed schema.

settings

One row per user (`UNIQUE(user_id)`): units (metric/imperial), notification preference, theme. The smallest table in the schema; a plain three-column normalized row since none of these values are ever queried in bulk or extended dynamically.

action_plan_items

One row per (user, recommendation code) pair the user has ever toggled, with a `UNIQUE(user_id, rec_code)` constraint enforcing that toggling the same recommendation twice updates the same row rather than inserting a duplicate. `rec_code` references a code defined in `data/recommendations.py` (§5.2.4), but is stored as a plain string rather than a foreign key, since the recommendation library is application content, not database-managed data.

reports

The largest and most important table — one row per uploaded/sample report. `status` is either "pending-verify" or "verified"; almost every other screen in the application first checks this field (via `SQ.ui.notReady()` on the frontend, and via a guard in the relevant router function on the backend) before showing analysis-dependent content. Five JSON text columns carry the report's substantive payload:

- `profile_json` — demographic/anthropometric profile (age, sex, height, weight, waist, home BP, ...).
- `labs_raw_json` — the lab panel as originally “extracted” (in this basis version, matched from a sample scenario — see §5.4.4).
- `labs_verified_json` — NULL until the user confirms/edits values on the Verify screen; once set, this supersedes `labs_raw_json` for every downstream calculation.
- `lab_confidence_json` — a per-code confidence percentage, shown as a pill next to each row on the Verify screen.
- `trend_history_json` — a per-quantity time series (LDL, weight, home blood pressure), used by both the Trends screen and the enlarged trend-chart modal.

questionnaire_answers

One row per report (`UNIQUE(report_id)`), storing the Adaptive Health Questionnaire's answers as one JSON object keyed by question id. Seeded with scenario defaults when a report is created (`see_create_from_scenario()` in §5.4.4) and patched incrementally as the user answers each question on the frontend (one `PATCH` call per answer, autosaving).

analysis_results

One row per report (`UNIQUE(report_id)`), storing the *entire* output of `orchestrator.run_analysis()` (§5.3.1) as a single JSON blob — cardiovascular results, all five illustrative domains, safety pathway, recommendations, and the printable structured report, all in one document. This row does not exist until a report has been verified and analysed at least once; `GET /api/reports/{id}/analysis` computes and caches it lazily on first request if it is missing (§5.4.4).

4.4 `schema.sql`: the generated snapshot

`database/schema.sql` is a *generated* artifact, produced from `models.py` via SQLAlchemy's `CreateTable` compiler, kept purely for human readability (e.g. browsing the schema without starting Python). The source of truth is always `backend/app/models.py`; the project's own `database/README.md` documents the exact one-line command to regenerate it after a model change:

```
1 cd backend
2 python -c "
3 from app.database import Base, engine
4 from sqlalchemy.schema import CreateTable
5 for t in Base.metadata.sorted_tables:
6     print(CreateTable(t).compile(engine))
7 " > ../database/schema.sql
```

Listing 4.1: Regenerating `schema.sql` after a model change (from `database/README.md`).

4.5 Upgrading to a deployed web version

Swapping SQLite for a managed Postgres or MySQL instance requires setting one environment variable before starting the backend — no code changes, because every query in `app/routers/` and `app/models.py` is plain SQLAlchemy Core/ORM with no SQLite-specific features:

```
1 export SQUARE_DATABASE_URL="postgresql+psycopg://user:pass@host:5432/eva_square"
```

Listing 4.2: The entire Postgres migration, from `database/README.md`.

The `*_json` text columns work identically on Postgres, and can later be upgraded to native JSONB columns for indexable queries — a small, isolated follow-up change confined to `models.py`.

Chapter 5

Backend Tier

The backend is a FastAPI application living under `backend/app/`. It is organised into four layers, shown in Figure 5.1: a thin **core** (database wiring, ORM models, request/response schemas, security primitives), a set of **routers** (one file per resource group, each a FastAPI APIRouter), a set of **engines** (the clinical rule logic, entirely framework-agnostic pure Python), and a **data** layer of static reference content that both the routers and engines read from.

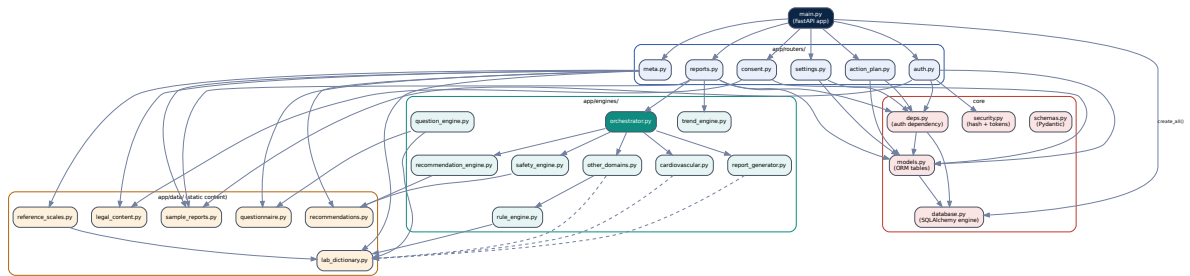


Figure 5.1: Backend module dependency graph. Solid arrows are direct imports; dashed arrows are read-only references to static reference content.

5.1 Core

5.1.1 app/main.py — application entrypoint

The single entrypoint that ties the whole backend together. Four things happen here, in order: (1) the database tables are created if they don't already exist; (2) every router is registered onto the FastAPI app; (3) a plain health-check endpoint is exposed at `/api/health`; (4) if a sibling `frontend/` directory exists, it is mounted as static files and a catch-all route serves `index.html` for any unmatched path — this is what lets the hash-routed single-page app survive a hard browser refresh on a deep link like `/#/dashboard`, since the server has no route matching `/#/dashboard` to 404 on in the first place (the fragment never even reaches the server).

```

1  """EVA SQUARE – FastAPI application entrypoint.
2
3  Local "basis version" wiring: this one process (1) creates/owns the SQLite
4  database in ../database/square.db, (2) exposes the REST API under /api/*,
5  and (3) serves the built frontend as static files at "/" – so the whole
6  stack starts with a single command (`uvicorn app.main:app`) and needs no
7  separate web server or CORS configuration. For a deployed web version, step
8  (3) goes away (the frontend is instead built/hosted separately, e.g. behind
9  a CDN) and step (1)'s DATABASE_URL points at a managed Postgres instance –
10 everything else here is unchanged. See the top-level README for that
11 upgrade path.
12 """
13 import os
14 from fastapi import FastAPI
15 from fastapi.staticfiles import StaticFiles
16 from fastapi.responses import FileResponse
17
18 from .database import Base, engine
19 from .routers import auth, consent, settings as settings_router, reports, meta, action_plan
20
21 Base.metadata.create_all(bind=engine)
22
23 app = FastAPI(title="EVA SQUARE API", version="1.0.0", description="Health interpretation platform
24 – FullStructure basis version.")
25
26 app.include_router(auth.router)
27 app.include_router(consent.router)
28 app.include_router(settings_router.router)
29 app.include_router(reports.router)
30 app.include_router(meta.router)
31 app.include_router(action_plan.router)
32
33 BACKEND_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
34 PROJECT_ROOT = os.path.dirname(BACKEND_DIR)
35 FRONTEND_DIR = os.path.join(PROJECT_ROOT, "frontend")
36
37 @app.get("/api/health")
38 def health():
39     return {"status": "ok", "service": "EVA SQUARE API"}
40
41
42 if os.path.isdir(FRONTEND_DIR):
43     app.mount("/assets", StaticFiles(directory=os.path.join(FRONTEND_DIR, "assets")), name="assets")
44     app.mount("/css", StaticFiles(directory=os.path.join(FRONTEND_DIR, "css")), name="css")
45     app.mount("/js", StaticFiles(directory=os.path.join(FRONTEND_DIR, "js")), name="js")
46
47 @app.get("/")
48 def index():

```

```

49     return FileResponse(os.path.join(FRONTEND_DIR, "index.html"))
50
51     @app.get("/{full_path:path}")
52     def spa_fallback(full_path: str):
53         """Hash-routed SPA: any non-API, non-asset path still serves
54         index.html so a hard refresh on e.g. /#/dashboard keeps working."""
55         candidate = os.path.join(FRONTEND_DIR, full_path)
56         if os.path.isfile(candidate):
57             return FileResponse(candidate)
58     return FileResponse(os.path.join(FRONTEND_DIR, "index.html"))

```

Listing 5.1: backend/app/main.py (full file).

Local vs. deployed

Step (4) — serving the frontend — only makes sense for this local basis version. In a deployed web version, the frontend would instead be built and hosted separately (e.g. behind a CDN), and this whole block would simply not run; step (1)’s DATABASE_URL would point at a managed Postgres instance instead. Nothing else in main.py changes.

5.1.2 app/database.py — engine and session factory

Resolves the SQLite file path (always database/square.db, a sibling of backend/, unless overridden), creates the SQLAlchemy engine and SessionLocal session factory, and exposes get_db(), the FastAPI dependency every router endpoint uses to obtain a request-scoped database session. SQUARE_DATABASE_URL is read from the environment first, defaulting to the local SQLite path — this one conditional is the entire Postgres upgrade path described in §4.

```

1  """EVA SQUARE – database engine/session setup.
2
3  The local "Database Environment" for this basis version is a single SQLite
4  file living in the sibling `database/` directory (kept OUTSIDE `backend/` on
5  purpose, so frontend/backend/database read as three independent concerns –
6  see the top-level README). Swapping to Postgres/MySQL for a deployed web
7  version is a one-line change: replace DATABASE_URL with a
8  `postgresql+psycopg://...` URL: every model and query below is plain
9  SQLAlchemy Core/ORM with no SQLite-specific features.
10 """
11 import os
12 from sqlalchemy import create_engine
13 from sqlalchemy.orm import sessionmaker, declarative_base
14
15 BACKEND_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
16 PROJECT_ROOT = os.path.dirname(BACKEND_DIR)
17 DATABASE_DIR = os.path.join(PROJECT_ROOT, "database")
18 os.makedirs(DATABASE_DIR, exist_ok=True)
19 DATABASE_PATH = os.path.join(DATABASE_DIR, "square.db")
20
21 DATABASE_URL = os.environ.get("SQUARE_DATABASE_URL", f"sqlite:/// {DATABASE_PATH}")
22
23 connect_args = {"check_same_thread": False} if DATABASE_URL.startswith("sqlite") else {}
24 engine = create_engine(DATABASE_URL, connect_args=connect_args)
25 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
26 Base = declarative_base()
27
28
29 def get_db():
30     db = SessionLocal()
31     try:
32         yield db
33     finally:

```

```
34 db.close()
```

Listing 5.2: backend/app/database.py (full file).

5.1.3 app/deps.py — shared dependencies

Two dependency functions used across almost every router: `get_current_user()` reads the `Authorization: Bearer <token>` header, looks the token up in the sessions table, checks its expiry, and resolves the owning User row (raising 401 at any failure point); `get_owned_report()` is a small helper every report-scoped endpoint calls to fetch a report *and* verify the current user actually owns it in one step, raising 404 rather than 403 if not, so a request for another user's report id is indistinguishable from a request for a report id that doesn't exist.

```
1 """EVA SQUARE – shared FastAPI dependencies: DB session + current-user auth."""
2 import datetime as dt
3 from fastapi import Depends, HTTPException, Header
4 from sqlalchemy.orm import Session as DBSession
5
6 from .database import get_db
7 from . import models
8
9
10 def get_current_user(authorization: str = Header(default=""), db: DBSession = Depends(get_db)) ->
11     models.User:
12     token = ""
13     if authorization.lower().startswith("bearer "):
14         token = authorization[7:].strip()
15     if not token:
16         raise HTTPException(status_code=401, detail="Missing bearer token – log in first.")
17     session = db.get(models.Session, token)
18     if not session or session.expires_at < dt.datetime.utcnow():
19         raise HTTPException(status_code=401, detail="Session expired or invalid – log in again.")
20     user = db.get(models.User, session.user_id)
21     if not user:
22         raise HTTPException(status_code=401, detail="User no longer exists.")
23     return user
24
25 def get_owned_report(report_id: str, db: DBSession, user: models.User) -> models.Report:
26     report = db.get(models.Report, report_id)
27     if not report or report.user_id != user.id:
28         raise HTTPException(status_code=404, detail="Report not found.")
29     return report
```

Listing 5.3: backend/app/deps.py (full file).

5.1.4 app/models.py — ORM models

All seven SQLAlchemy declarative models. Fully documented table-by-table in §4; reproduced here for completeness alongside the rest of the core layer.

```
1 """EVA SQUARE – SQLAlchemy ORM models.
2
3 Design note: account/session/consent/settings/action-plan state is fully
4 normalized (one column per fact) because that state is small, queried
5 individually, and benefits from real columns. Report/questionnaire/analysis
6 payloads are deliberately stored as JSON text columns (`*_json`) – they are
7 large, nested, always read/written as a whole document, and their shape
8 mirrors the JS engine output one-to-one. This is a pragmatic "basis version"
9 choice: normalizing every lab value and calculator result into its own
```

```

10 table is exactly the kind of migration a deployed v2 would do once the
11 schema has proven itself against real usage – see database/README.md.
12 """
13 import uuid
14 import datetime as dt
15 from sqlalchemy import (
16     Column, Integer, String, Boolean, Text, ForeignKey, DateTime, UniqueConstraint
17 )
18 from sqlalchemy.orm import relationship
19 from .database import Base
20
21
22 def _uuid():
23     return uuid.uuid4().hex
24
25
26 def _now():
27     return dt.datetime.utcnow()
28
29
30 class User(Base):
31     __tablename__ = "users"
32     id = Column(Integer, primary_key=True)
33     full_name = Column(String(200), nullable=False)
34     email = Column(String(255), nullable=False, unique=True, index=True)
35     password_hash = Column(String(255), nullable=False)
36     created_at = Column(DateTime, default=_now)
37
38     sessions = relationship("Session", back_populates="user", cascade="all, delete-orphan")
39     reports = relationship("Report", back_populates="user", cascade="all, delete-orphan")
40     consent = relationship("ConsentRecord", back_populates="user", uselist=False, cascade="all, delete-orphan")
41     settings = relationship("Settings", back_populates="user", uselist=False, cascade="all, delete-orphan")
42     action_items = relationship("ActionPlanItem", back_populates="user", cascade="all, delete-orphan")
43
44
45 class Session(Base):
46     """Bearer session tokens. Deliberately simple (no JWT/refresh) for this
47     basis version – see backend/app/security.py for the upgrade note."""
48     __tablename__ = "sessions"
49     token = Column(String(64), primary_key=True, default=_uuid)
50     user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
51     created_at = Column(DateTime, default=_now)
52     expires_at = Column(DateTime, nullable=False)
53
54     user = relationship("User", back_populates="sessions")
55
56
57 class ConsentRecord(Base):
58     __tablename__ = "consent_records"
59     id = Column(Integer, primary_key=True)
60     user_id = Column(Integer, ForeignKey("users.id"), nullable=False, unique=True)
61     required_json = Column(Text, default="{}") # {"processHealthData": true, ...}
62     optional_json = Column(Text, default="{}") # {"research": false, "updates": true}
63     accepted_terms_version = Column(String(50), nullable=True)
64     updated_at = Column(DateTime, default=_now, onupdate=_now)
65
66     user = relationship("User", back_populates="consent")
67
68
69 class Settings(Base):
70     __tablename__ = "settings"

```

```

71     id = Column(Integer, primary_key=True)
72     user_id = Column(Integer, ForeignKey("users.id"), nullable=False, unique=True)
73     units = Column(String(20), default="metric")
74     notifications = Column(Boolean, default=True)
75     theme = Column(String(20), default="light")
76
77     user = relationship("User", back_populates="settings")
78
79
80 class Report(Base):
81     __tablename__ = "reports"
82     id = Column(String(40), primary_key=True, default=_uuid)
83     user_id = Column(Integer, ForeignKey("users.id"), nullable=False, index=True)
84     file_name = Column(String(255))
85     lab = Column(String(255))
86     report_date = Column(String(20))
87     status = Column(String(20), default="pending-verify") # pending-verify | verified
88     profile_json = Column(Text, default="{}")
89     labs_raw_json = Column(Text, default="{}")
90     labs_verified_json = Column(Text, nullable=True)
91     lab_confidence_json = Column(Text, default="{}")
92     original_text = Column(Text, default="")
93     trend_history_json = Column(Text, default="{}")
94     uploaded_at = Column(DateTime, default=_now)
95
96     user = relationship("User", back_populates="reports")
97     questionnaire = relationship("QuestionnaireAnswer", back_populates="report", uselist=False,
98                                cascade="all, delete-orphan")
99     analysis = relationship("AnalysisResult", back_populates="report", uselist=False, cascade="all
100                            , delete-orphan")
101
102
103 class QuestionnaireAnswer(Base):
104     __tablename__ = "questionnaire_answers"
105     id = Column(Integer, primary_key=True)
106     report_id = Column(String(40), ForeignKey("reports.id"), nullable=False, unique=True)
107     answers_json = Column(Text, default="{}")
108     updated_at = Column(DateTime, default=_now, onupdate=_now)
109
110     report = relationship("Report", back_populates="questionnaire")
111
112
113 class AnalysisResult(Base):
114     __tablename__ = "analysis_results"
115     id = Column(Integer, primary_key=True)
116     report_id = Column(String(40), ForeignKey("reports.id"), nullable=False, unique=True)
117     result_json = Column(Text, default="{}")
118     computed_at = Column(DateTime, default=_now, onupdate=_now)
119
120     report = relationship("Report", back_populates="analysis")
121
122
123 class ActionPlanItem(Base):
124     __tablename__ = "action_plan_items"
125     id = Column(Integer, primary_key=True)
126     user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
127     rec_code = Column(String(80), nullable=False)
128     done = Column(Boolean, default=False)
129     updated_at = Column(DateTime, default=_now, onupdate=_now)
130     __table_args__ = (UniqueConstraint("user_id", "rec_code", name="uq_user_reccode"),)
131
132     user = relationship("User", back_populates="action_items")

```

Listing 5.4: backend/app/models.py (full file).

5.1.5 app/schemas.py — Pydantic request/response models

Request bodies that a human actually fills in (register, login, verify edits, questionnaire patches, settings, consent) are fully typed here so FastAPI validates them automatically and generates accurate interactive docs at /docs. Complex nested payloads — a full lab panel, a full analysis result — are typed simply as dict, on purpose: their real shape is defined by the engine modules (§5.3.1 onward) and would otherwise have to be duplicated field-by-field here for no benefit.

```

1  """EVA SQUARE – Pydantic request/response schemas.
2
3  Complex nested payloads (profile, labs, analysis results) are typed as
4  `dict` on purpose: their shape is defined by the engine modules (see
5  app/engines/) and documented in the top-level README rather than
6  duplicated here field-by-field. Request bodies that a human actually fills
7  in (register, login, verify edits, questionnaire answers) are fully typed
8  so FastAPI validates them and generates accurate OpenAPI docs at /docs.
9  """
10 from typing import Optional, Any, Dict
11 from pydantic import BaseModel, EmailStr, Field
12
13
14 class RegisterIn(BaseModel):
15     fullName: str
16     email: EmailStr
17     password: str = Field(min_length=4)
18
19
20 class LoginIn(BaseModel):
21     email: EmailStr
22     password: str
23
24
25 class AuthOut(BaseModel):
26     token: str
27     user: dict
28
29
30 class ConsentPatch(BaseModel):
31     kind: str # "required" | "optional"
32     id: str
33     value: bool
34
35
36 class AcceptTermsIn(BaseModel):
37     pass
38
39
40 class SampleReportIn(BaseModel):
41     scenario: str # "routine" | "urgent"
42     fileName: Optional[str] = None
43     fullPipeline: bool = False # true = also verify + seed questionnaire + run analysis
44
45
46 class VerifyIn(BaseModel):
47     values: Dict[str, float]
48
49
50 class AnswersPatch(BaseModel):
51     patch: Dict[str, Any]
52
53
54 class SettingsPatch(BaseModel):
55     units: Optional[str] = None

```

```

56     notifications: Optional[bool] = None
57     theme: Optional[str] = None
58
59
60 class ActionToggleIn(BaseModel):
61     code: str

```

Listing 5.5: backend/app/schemas.py (full file).

5.1.6 app/security.py — password hashing and sessions

Deliberately dependency-free: password hashing uses only the Python standard library's `hashlib.pbkdf2_hmac` (260,000 iterations, SHA-256, a fresh random salt per password), and session identity is a 32-byte random hex token with no external JWT library. `verify_password()` uses `hmac.compare_digest()` for a constant-time comparison, guarding against timing attacks.

```

1  """EVA SQUARE – password hashing + bearer session tokens.
2
3  Deliberately dependency-free (stdlib `hashlib`/`secrets` only) so the basis
4  version has zero native-build dependencies. Before a real web deployment,
5  swap `hash_password`/`verify_password` for a vetted library (passlib/argon2)
6  and consider short-lived JWTs with refresh instead of the long-lived opaque
7  bearer tokens used here – both are drop-in replacements for these two
8  functions and the Session model, nothing else in the app needs to change.
9  """
10 import hashlib
11 import hmac
12 import secrets
13 import datetime as dt
14
15 PBKDF2_ITERATIONS = 260_000
16 SESSION_TTL_DAYS = 30
17
18
19 def hash_password(password: str) -> str:
20     salt = secrets.token_hex(16)
21     digest = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), bytes.fromhex(salt),
22                                 PBKDF2_ITERATIONS)
23     return f"pbkdf2_sha256${PBKDF2_ITERATIONS}${salt}${digest.hex()}"
24
25 def verify_password(password: str, stored_hash: str) -> bool:
26     try:
27         scheme, iterations, salt, hex_digest = stored_hash.split("$")
28         if scheme != "pbkdf2_sha256":
29             return False
30         digest = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), bytes.fromhex(salt), int(
31             iterations))
32         return hmac.compare_digest(digest.hex(), hex_digest)
33     except Exception:
34         return False
35
36 def new_session_token() -> str:
37     return secrets.token_hex(32)
38
39
40 def session_expiry() -> dt.datetime:
41     return dt.datetime.utcnow() + dt.timedelta(days=SESSION_TTL_DAYS)

```

Listing 5.6: backend/app/security.py (full file).

Documented upgrade path

Before a real deployment, `hash_password/verify_password` should be swapped for a vetted library (`passlib/argon2`), and the long-lived opaque bearer token should become a short-lived JWT with refresh. Both are drop-in replacements for these two functions and the Session model — nothing else in the application needs to change, since every other module only ever calls through `security.*` and `deps.get_current_user()`.

5.1.7 app/seed.py — optional CLI seed script

Not required to run the app (both demo accounts are also seeded lazily by `routers/auth.py`'s `demo_login` endpoint, called by the frontend's "Load a sample report" buttons) — this script exists purely so a fresh `square.db` can be pre-populated with both demo accounts and a fully-analysed report in one command, useful for inspecting the database directly (e.g. with the `sqlite3` CLI) without opening the app first. Invoked as `python -m app.seed` from `backend/`.

```

1  """EVA SQUARE – optional CLI seed script.
2
3  The app already seeds demo accounts lazily (see routers/auth.py#demo_login,
4  called by the frontend's "Load a sample report" buttons), so this script is
5  not required to run the app. It exists for convenience: run it once after a
6  fresh `database/square.db` is created to pre-populate both demo accounts
7  with a fully-analysed report, so the database has example rows to inspect
8  immediately (e.g. with the sqlite3 CLI or a DB browser) without opening the
9  app first.
10
11  Usage:  cd backend && python -m app.seed
12  """
13  from .database import Base, engine, SessionLocal
14  from . import models, security
15  from .data.sample_reports import SAMPLE_SCENARIOS
16  from .routers.auth import DEMO_PASSWORD
17  from .routers.reports import _create_from_scenario
18
19
20  def run():
21      Base.metadata.create_all(bind=engine)
22      db = SessionLocal()
23      try:
24          for key, sc in SAMPLE_SCENARIOS.items():
25              email = sc["profile"]["email"]
26              user = db.query(models.User).filter_by(email=email).first()
27              if not user:
28                  user = models.User(full_name=sc["profile"]["fullName"], email=email,
29                                   password_hash=security.hash_password(DEMO_PASSWORD))
30                  db.add(user)
31                  db.commit()
32                  db.refresh(user)
33                  print(f"Created demo user: {email}")
34              if not db.query(models.Settings).filter_by(user_id=user.id).first():
35                  db.add(models.Settings(user_id=user.id))
36              if not db.query(models.ConsentRecord).filter_by(user_id=user.id).first():
37                  db.add(models.ConsentRecord(user_id=user.id, required_json={}, optional_json={}))
38              db.commit()
39              if not db.query(models.Report).filter_by(user_id=user.id).first():
40                  report = _create_from_scenario(db, user, key, None, full_pipeline=True)
41                  print(f"Seeded report {report.id} ({key}) for {email}")
42              print("Seed complete.")
43      finally:
44          db.close()

```

```
45
46
47 if __name__ == "__main__":
48     run()
```

Listing 5.7: backend/app/seed.py (full file).

5.1.8 requirements.txt

```
fastapi>=0.115,<1.0
uvicorn[standard]>=0.30,<1.0
SQLAlchemy>=2.0,<3.0
pydantic>=2.7,<3.0
python-multipart>=0.0.9
email-validator>=2.0
```

Listing 5.8: backend/requirements.txt (full file).

5.2 Static reference content (app/data/)

Every file in this layer is pure data — no request handling, no business logic — read by the routers (to serve as-is over `/api/meta/*`) and by the engines (to classify values against). Keeping this content in dedicated modules rather than scattered as literals through the engines means the routers, the engines, and the frontend (which fetches this content once at boot and binds it onto plain `SQ.*` globals — see §6.2.2) all share exactly one source of truth for each piece of reference content.

5.2.1 app/data/lab_dictionary.py

The registry every other module keys off: the six health domains (`DOMAINS`, `DOMAIN_ORDER`), the sixteen lab codes with their display name, alternate names, unit, decimal precision and real vs. illustrative source flag (`LAB_DICTIONARY`), the simple threshold ranges the five illustrative domains classify against (`SIMPLE_RANGES`), and the BMI/waist anthropometric rules (`ANTHRO_RULES`).

```

1  """EVA SQUARE – lab dictionary, reference ranges, domains.
2
3  Direct Python port of frontend-v2's js/data/labDictionary.js. Cardiovascular
4  entries are marked source="real" (interpreted by app/engines/cardiovascular.py
5  against the Cardiovascular Domain Specification); every other domain is
6  source="illustrative" pending its own domain specification.
7  """
8
9  DOMAINS = {
10     "cardiovascular": {"key": "cardiovascular", "label": "Cardiovascular", "icon": "♥", "color": "#0392b"},
11     "metabolic": {"key": "metabolic", "label": "Metabolic", "icon": "\U0001F4A7", "color": "#2e5aac"},
12     "liver": {"key": "liver", "label": "Liver", "icon": "\U0001FAC0", "color": "#2f9e44"},
13     "renal": {"key": "renal", "label": "Renal", "icon": "\U0001FAD8", "color": "#0f8c7f"},
14     "haematology": {"key": "haematology", "label": "Haematology", "icon": "\U0001FA78", "color": "#8e44ad"},
15     "anthropometric": {"key": "anthropometric", "label": "Anthropometric", "icon": "\U0001F4CF", "color": "#b5650a"},
16 }
17
18 DOMAIN_ORDER = ["cardiovascular", "metabolic", "liver", "renal", "haematology", "anthropometric"]
19
20 LAB_DICTIONARY = {
21     # ---- Cardiovascular (REAL spec) ----
22     "LAB_TC": {"code": "LAB_TC", "name": "Total Cholesterol", "altNames": ["TC", "Cholesterol, Total"], "domain": "cardiovascular", "unit": "mmol/L", "decimals": 1, "source": "real"},
23     "LAB_LDL": {"code": "LAB_LDL", "name": "LDL Cholesterol", "altNames": ["LDL-C", "LDL"], "domain": "cardiovascular", "unit": "mmol/L", "decimals": 1, "source": "real"},
24     "LAB_HDL": {"code": "LAB_HDL", "name": "HDL Cholesterol", "altNames": ["HDL-C", "HDL"], "domain": "cardiovascular", "unit": "mmol/L", "decimals": 1, "source": "real"},
25     "LAB_TG": {"code": "LAB_TG", "name": "Triglycerides", "altNames": ["TG"], "domain": "cardiovascular", "unit": "mmol/L", "decimals": 1, "source": "real"},
26     "LAB_APOB": {"code": "LAB_APOB", "name": "Apolipoprotein B", "altNames": ["ApoB"], "domain": "cardiovascular", "unit": "mg/dL", "decimals": 0, "source": "real", "advanced": True},
27     "LAB_APOA1": {"code": "LAB_APOA1", "name": "Apolipoprotein A-I", "altNames": ["ApoA-I", "ApoA1"], "domain": "cardiovascular", "unit": "mg/dL", "decimals": 0, "source": "real", "advanced": True},
28     "LAB_LPA": {"code": "LAB_LPA", "name": "Lipoprotein(a)", "altNames": ["Lp(a)"], "domain": "cardiovascular", "unit": "nmol/L", "decimals": 0, "source":

```

```

    "real", "advanced": True},
# ---- Metabolic (illustrative) ----
"LAB_GLU_FAST": {"code": "LAB_GLU_FAST", "name": "Fasting Glucose", "altNames": ["FBS", "FPG"],
    "domain": "metabolic", "unit": "mmol/L", "decimals": 1, "source": "illustrative"},
"LAB_HBA1C": {"code": "LAB_HBA1C", "name": "HbA1c", "altNames": ["Glycated Hb"],
    "domain": "metabolic", "unit": "%", "decimals": 1, "source": "illustrative"},
# ---- Liver (illustrative) ----
"LAB_ALT": {"code": "LAB_ALT", "name": "ALT", "altNames": ["SGPT"], "domain": "liver", "unit":
    "U/L", "decimals": 0, "source": "illustrative"},
"LAB_AST": {"code": "LAB_AST", "name": "AST", "altNames": ["SGOT"], "domain": "liver", "unit":
    "U/L", "decimals": 0, "source": "illustrative"},
# ---- Renal (illustrative) ----
"LAB_CREAT": {"code": "LAB_CREAT", "name": "Creatinine", "altNames": ["Serum Creatinine"],
    "domain": "renal", "unit": "µmol/L", "decimals": 0, "source": "illustrative"},
"LAB_EGFR": {"code": "LAB_EGFR", "name": "eGFR", "altNames": [],
    "domain": "renal", "unit": "mL/min/1.73m²", "decimals": 0, "source": "illustrative"},
# ---- Haematology (illustrative) ----
"LAB_HB": {"code": "LAB_HB", "name": "Haemoglobin", "altNames": ["Hb"], "domain": "
    haematology", "unit": "g/dL", "decimals": 1, "source": "illustrative"},
"LAB_WBC": {"code": "LAB_WBC", "name": "White Blood Cells", "altNames": ["TLC"], "domain": "
    haematology", "unit": "cells/µL", "decimals": 0, "source": "illustrative"},
"LAB_PLT": {"code": "LAB_PLT", "name": "Platelets", "altNames": [], "domain": "
    haematology", "unit": "Lakh/µL", "decimals": 2, "source": "illustrative"},
}

SIMPLE_RANGES = {
    "LAB_GLU_FAST": {"normalLow": 3.9, "normalHigh": 5.5, "mildHigh": 6.9, "severeHigh": 10, "
        source": "Demo threshold – illustrative, pending metabolic domain specification"},
    "LAB_HBA1C": {"normalHigh": 5.6, "mildHigh": 6.4, "severeHigh": 9, "source": "Demo
        threshold – illustrative, pending metabolic domain specification"},
    "LAB_ALT": {"normalHigh": 41, "mildHigh": 82, "severeHigh": 200, "source": "Demo
        threshold – illustrative, pending liver domain specification"},
    "LAB_AST": {"normalHigh": 40, "mildHigh": 80, "severeHigh": 200, "source": "Demo
        threshold – illustrative, pending liver domain specification"},
    "LAB_CREAT": {"normalLow": 60, "normalHigh": 110, "severeHigh": 180, "source": "Demo
        threshold – illustrative, pending renal domain specification"},
    "LAB_EGFR": {"normalLow": 90, "mildLow": 60, "severeLow": 30, "source": "Demo threshold –
        illustrative, pending renal domain specification"},
    "LAB_HB": {"normalLow": 13.0, "normalHigh": 17.0, "mildLow": 11, "severeLow": 8, "source
        ": "Demo threshold – illustrative, pending haematology domain specification"},
    "LAB_WBC": {"normalLow": 4000, "normalHigh": 11000, "source": "Demo threshold –
        illustrative, pending haematology domain specification"},
    "LAB_PLT": {"normalLow": 1.5, "normalHigh": 4.5, "source": "Demo threshold – illustrative
        , pending haematology domain specification"},
}

ANTHRO_RULES = {
    "bmi": [
        {"max": 18.5, "label": "Underweight", "tone": "warn"},
        {"max": 23, "label": "Normal", "tone": "good"},
        {"max": 27.5, "label": "Overweight", "tone": "warn"},
        {"max": 999, "label": "Obese", "tone": "bad"},
    ],
    "waist": {"men": 90, "women": 80},
}

```

Listing 5.9: backend/app/data/lab_dictionary.py (full file).

5.2.2 app/data/legal_content.py

The organisation identity block (ORG), all ten sections of the Terms & Conditions (TERMS_SECTIONS), and the three required plus two optional consent items (CONSENT_REQUIRED, CONSENT_OPTIONAL). This is sample

legal content for prototype purposes, explicitly flagged as such in the Terms screen itself (“This is a sample legal draft ...has not been reviewed by qualified Malaysian legal counsel”).

```

1  """EVA SQUARE – Terms & Conditions / Consent content. Python port of
2  js/data/legalContent.js, sample details filled in per agreed prototype
3  scope (see README)."""
4
5  ORG = {
6      "legalName": "EVA SQUARE Health Sdn Bhd (sample entity for this prototype)",
7      "supportEmail": "support@evasquare.example",
8      "legalEmail": "legal@evasquare.example",
9      "address": "Level 12, Menara EVA SQUARE, Jalan Ampang, 50450 Kuala Lumpur, Malaysia (sample
10         address)",
11      "effectiveDate": "29 August 2026",
12      "version": "Draft v1.0",
13  }
14
15  def _org_name(): return ORG["legalName"]
16  def _org_email(): return ORG["legalEmail"]
17  def _org_addr(): return ORG["address"]
18
19
20  TERMS_SECTIONS = [
21      {"id": "about", "title": "1. About EVA SQUARE", "body": [
22          "EVA SQUARE is a personal health interpretation and wellness-navigation platform. We help
23          you understand your screening information, organise important insights, and plan next
24          steps for your health and wellbeing.",
25          f'These Terms form a binding agreement between you and {_org_name()} ("EVA SQUARE", "we",
26          "us"). By creating an account or continuing to use EVA SQUARE, you confirm that you
27          have read and accepted these Terms.',
28      ]},
29      {"id": "scope", "title": "2. Scope of Service", "body": ["EVA SQUARE provides the following
30          services to support your health journey:"],
31      "list": ["Health information organisation", "Personalised wellness education", "Risk-
32          awareness guidance", "Follow-up and progress tracking"],
33      "note": {"tone": "info", "title": "Important: EVA SQUARE does not provide a medical diagnosis
34          .",
35          "text": "Platform guidance does not replace assessment, diagnosis or treatment by a
36          qualified healthcare professional."}},
37      {"id": "not-diagnosis", "title": "3. Not Medical Diagnosis or Emergency Care", "body": [
38          "Unless EVA SQUARE expressly identifies a separately booked consultation delivered by a
39          named, registered healthcare professional, EVA SQUARE's output is general
40          informational and educational content – not a diagnosis, medical opinion, prescription
41          or treatment plan.",
42          "Use of the automated platform alone does not create a doctor-patient relationship. EVA
43          SQUARE does not continuously monitor your data and may not detect deterioration or
44          emergencies.",
45      ], "note": {"tone": "bad", "title": "Emergency warning",
46          "text": "EVA SQUARE must not be used for urgent or emergency care. If you may have
47          a medical emergency, contact Malaysia's emergency services at 999 or go to
48          the nearest emergency department immediately."}},
49      {"id": "responsibilities", "title": "4. User Responsibilities", "body": [
50          "You agree to provide information that is accurate, current and complete; to answer
51          questionnaires honestly; and to avoid uploading altered, illegible or mismatched
52          reports.",
53          "You remain responsible for your decisions and for obtaining appropriate professional care
54          . You must not change medication, supplements or treatment solely because of EVA
55          SQUARE output.",
56      ]},
57      {"id": "privacy", "title": "5. Data and Privacy", "body": [
58          "EVA SQUARE may process identity, demographic, lifestyle, laboratory, medical-history and
59          other health-related information. Health information is sensitive personal data, and
60          its processing requires your separate, specific consent – recorded on the Consent

```

```

        screen.",
        "Optional consent for research or communications is never bundled with access to the core
        service, and may be withdrawn at any time.",
    ]},
    {"id": "referral", "title": "6. Professional Referral", "body": [
        "EVA SQUARE may display or facilitate referrals to independent doctors, clinics,
        laboratories or allied health professionals. Unless expressly identified as our
        employee or agent, each provider is independent and responsible for their own advice,
        treatment and fees.",
        "A referral, listing or link is not a guarantee or endorsement of outcome – you may choose
        another qualified provider.",
    ]},
    {"id": "limitations", "title": "7. Platform Limitations", "body": [
        "We aim to provide a reliable service but do not promise uninterrupted or error-free
        access. Automated outputs – including optical character recognition, unit conversion,
        calculators and rule-based classification – may occasionally misread, omit or
        misclassify information.",
        "Before relying on an output, compare the patient identity, test name, value, unit and
        reference interval against your original report. Nothing in these Terms excludes a
        right that cannot lawfully be excluded, including for fraud, wilful misconduct, or
        death or personal injury caused by negligence.",
    ]},
    {"id": "account", "title": "8. Account and Access", "body": [
        "You must be at least 18 years old and have legal capacity to use EVA SQUARE, unless a
        service is expressly made available to minors through a parent or guardian.",
        "You are responsible for safeguarding your credentials and for activity through your
        account. We may suspend or restrict access to protect users, data or the Services, and
        will provide notice where reasonably possible.",
    ]},
    {"id": "changes", "title": "9. Changes to These Terms", "body": [
        "We may update these Terms to reflect changes in law, safety, technology or business
        operations. We will post the updated version with its effective date, and request
        renewed acceptance for material changes where required.",
    ]},
    {"id": "contact", "title": "10. Contact Us", "body": [
        f"Questions, complaints, account requests and legal notices may be sent to {_org_email()}
        or {_org_addr()}.",
        "These Terms are governed by the laws of Malaysia. We will first attempt to resolve any
        complaint in good faith through our support channel.",
    ]},
]

CONSENT_REQUIRED = [
    {"id": "processHealthData", "icon": "\U0001F512", "title": "Process my health information",
     "body": "I consent to EVA SQUARE securely processing the information and health reports I
     provide to generate personalised interpretation, education and action guidance.",
     "detail": "Includes account and contact details; demographics; anthropometrics and lifestyle
     responses; uploaded laboratory reports and extracted values; and platform activity needed
     to provide, secure and audit the service."},
    {"id": "nonDiagnosticAck", "icon": "\U0001F9D1", "title": "Non-diagnostic platform
     acknowledgement",
     "body": "I understand that EVA SQUARE provides health interpretation, education and wellness
     guidance – not a diagnosis, medical treatment, prescription or emergency service.",
     "detail": "EVA SQUARE does not establish a doctor-patient relationship merely because I use
     the platform. Laboratory reference ranges can differ between laboratories, methods and
     populations."},
    {"id": "professionalCareAck", "icon": "\U0001FA7A", "title": "Professional-care
     acknowledgement",
     "body": "I understand that I should obtain qualified healthcare assessment when advised, when
     symptoms concern me, when results are unexpected, or whenever I am unsure.",
     "detail": "EVA SQUARE must not be used for urgent or emergency care. If I may have a medical
     emergency, I will contact Malaysia's emergency services at 999 or go to the nearest
     emergency department immediately."},
]

```

```

74
75 CONSENT_OPTIONAL = [
76     {"id": "research", "title": "Contribute to health research",
77      "body": "Allow de-identified platform data to be considered for ethically approved research
78        that may improve health understanding and services.",
79      "points": ["Your name and direct identifiers are removed", "Choosing no will not affect
80        platform access", "You can withdraw this permission later", "Additional approval may be
81        required for specific studies"]},
82     {"id": "updates", "title": "Receive helpful updates",
83      "body": "Get occasional health education and platform updates. No promotional messages
84        without permission.", "points": []},
85 ]

```

Listing 5.10: backend/app/data/legal_content.py (full file).

5.2.3 app/data/questionnaire.py

The Adaptive Health Questionnaire’s four sections (cardiovascular, metabolic, liver & lifestyle, medication & history), each carrying a triggerDomains list — a section only appears if the report’s lab panel actually contains a value from one of its trigger domains (the history section, with an empty trigger list, always appears). See §5.3.9 for the activation logic and §6.6.5 for where it’s rendered.

```

1  """EVA SQUARE – Triggered Health Questionnaire (Screen 9 spec). Python port
2  of js/data/questionnaire.js – see that file's header comment for the
3  adaptive-section rationale."""
4
5  QUESTIONNAIRE_SECTIONS = [
6      {
7          "id": "cardiovascular", "label": "Cardiovascular", "icon": "♥",
8          "triggerDomains": ["cardiovascular"],
9          "questions": [
10             {"id": "everDiagnosedHighBP", "prompt": "Have you ever been diagnosed with high blood
11               pressure?",
12              "type": "choice", "options": ["Yes", "No", "Not sure"],
13              "why": "A prior diagnosis changes how a single elevated reading should be interpreted
14                ."},
15             {"id": "recentBP", "prompt": "What is your most recent blood pressure reading?", "type":
16               "bp-pair",
17              "why": "A home or clinic reading lets SQUARE confirm or contextualise the reading on
18                your lab report."},
19             {"id": "bpMedication", "prompt": "Do you currently take medication for blood pressure?",
20              "type": "choice", "options": ["Yes", "No", "Prefer not to answer"],
21              "why": "Treated blood pressure is interpreted differently to untreated readings in
22                the risk calculators."},
23             {"id": "familyHistoryPrematureCVD", "prompt": "Do any parents or siblings have a
24               history of heart attack or stroke before age 60?",
25              "type": "choice", "options": ["Yes", "No", "Not sure"],
26              "why": "Premature family history is a risk-enhancing factor alongside your lipid
27                results."},
28         ],
29     },
30     {
31         "id": "metabolic", "label": "Metabolic Health", "icon": "\U0001F4A7",
32         "triggerDomains": ["metabolic"],
33         "questions": [
34             {"id": "diagnosedDiabetes", "prompt": "Have you been diagnosed with diabetes or
35               prediabetes?",
36              "type": "choice", "options": ["Yes", "No", "Not sure"],
37              "why": "A confirmed diagnosis is treated differently to one unconfirmed elevated
38                glucose reading."},

```

```

30     {"id": "diabetesMedication", "prompt": "Are you currently taking medication for blood
31         glucose?",
32         "type": "choice", "options": ["Yes", "No", "Prefer not to answer"],
33         "why": "Treatment status affects how a glucose or HbA1c result should be read."},
34     {"id": "familyHistoryDiabetes", "prompt": "Do any close family members have diabetes?"
35         ,
36         "type": "choice", "options": ["Yes", "No", "Not sure"],
37         "why": "Family history is one of several factors used to contextualise metabolic
38             findings."},
39     {"id": "activityLevel", "prompt": "How would you describe your typical physical
40         activity level?",
41         "type": "choice", "options": ["Sedentary", "Light", "Moderate", "Active"],
42         "why": "Activity level is a modifiable factor considered in your action plan."},
43 ],
44 {
45     "id": "liver", "label": "Liver & Lifestyle", "icon": "\U0001FAC0",
46     "triggerDomains": ["liver"],
47     "questions": [
48         {"id": "alcoholIntake", "prompt": "How would you describe your typical alcohol intake?"
49             ,
50             "type": "choice", "options": ["None", "Occasional", "Regular", "Heavy"],
51             "why": "Alcohol intake is one of the most common contributors to a raised liver
52                 enzyme result."},
53         {"id": "fattyLiverHistory", "prompt": "Have you been told you have fatty liver or
54             another liver condition?",
55             "type": "choice", "options": ["Yes", "No", "Not sure"],
56             "why": "A known liver condition changes how a liver enzyme result should be
57                 interpreted."},
58         {"id": "smokingStatus", "prompt": "What is your current smoking status?",
59             "type": "choice", "options": ["Never smoked", "Former smoker", "Current smoker"],
60             "why": "Smoking status is a required input to your cardiovascular risk calculation."
61         },
62     ],
63 },
64 {
65     "id": "history", "label": "Medication & History", "icon": "\U0001F48A",
66     "triggerDomains": [],
67     "questions": [
68         {"id": "regularMedication", "prompt": "Are you currently taking any regular
69             prescription medication?",
70             "type": "choice-with-text", "options": ["Yes", "No"], "textLabel": "If yes, please
71                 list what you're taking (optional)",
72             "why": "Some medications change how a result should be read or which advice is
73                 appropriate."},
74         {"id": "kidneyDisease", "prompt": "Have you been diagnosed with kidney disease?",
75             "type": "choice", "options": ["Yes", "No", "Not sure"],
76             "why": "Kidney function changes how several other findings should be weighed."},
77         {"id": "pregnancyStatus", "prompt": "Are you currently pregnant, or within 6 months
78             postpartum?",
79             "type": "choice", "options": ["Not applicable", "Pregnant", "Postpartum", "Prefer not
80                 to answer"],
81             "why": "Pregnancy changes reference ranges and routes some findings to a separate
82                 safety pathway."},
83     ],
84 },
85 ]

```

Listing 5.11: backend/app/data/questionnaire.py (full file).

5.2.4 app/data/recommendations.py

Three registries: RECOMMENDATIONS, eight action-plan entries each carrying a when predicate (a plain Python lambda over the assembled findings dict), an issue/action/why/benefit/precaution/followUp narrative, a category and a sort weight; EDUCATION, short explanatory snippets keyed by an education code; and REFERRAL_LEVELS, the four-rung safety ladder (emergency, prompt, routine, self) that both §5.3.5 and the Referral screen (§6.6.10) render from.

```

1  """EVA SQUARE – recommendation library, education snippets, referral ladder.
2  Python port of js/data/recommendations.js. Each recommendation's "when" is a
3  plain callable over the `fh` (findings) dict assembled in engines/__init__
4  so the recommendation engine can `filter(lambda r: r["when"](fh), ...)`
5  """
6
7  RECOMMENDATIONS = [
8      {
9          "code": "REC_CV_BP_TRACK", "domain": "cardiovascular",
10         "when": lambda fh: fh["cv"].get("bpCategoryRank") is not None and fh["cv"]["bpCategoryRank"]
11             >= 2,
12         "issue": "Blood pressure above the optimal range",
13         "action": "Track your blood pressure at home twice daily for 7 days using a validated
14             monitor, seated and rested.",
15         "why": "A single reading is a screening result, not confirmed hypertension – home readings
16             establish whether the pattern is persistent.",
17         "benefit": "Confirms whether follow-up with a clinician is warranted and gives them a full
18             week of data.",
19         "precaution": "Use a validated upper-arm monitor; readings taken immediately after
20             exercise, caffeine or stress can be misleadingly high.",
21         "followUp": "Bring your 7-day log to your next review.",
22         "category": "monitoring", "weight": 95,
23     },
24     {
25         "code": "REC_CV_LIPID_DIET", "domain": "cardiovascular",
26         "when": lambda fh: (fh["cv"].get("ldl") or 0) >= 3.4 or fh["cv"].get("apobFlag"),
27         "issue": "LDL cholesterol / ApoB above the goal linked to your risk category",
28         "action": "Increase soluble fibre and improve dietary fat quality – oats, legumes,
29             vegetables, and unsaturated oils in place of saturated fat.",
30         "why": "Excess saturated fat and low fibre intake are common contributors to elevated LDL-
31             cholesterol and particle burden.",
32         "benefit": "May improve LDL-C, non-HDL-C and ApoB over –812 weeks alongside other
33             lifestyle changes.",
34         "precaution": "Dietary advice may need modification for other medical conditions – discuss
35             significant changes with a healthcare professional.",
36         "followUp": "Repeat a fasting lipid panel in –812 weeks.",
37         "category": "nutrition", "weight": 88,
38     },
39     {
40         "code": "REC_CV_ACTIVITY", "domain": "cardiovascular", "when": lambda fh: True,
41         "issue": "General cardiovascular and metabolic health",
42         "action": "Build towards at least 150 minutes of moderate activity per week, adjusted to
43             your current ability.",
44         "why": "Regular activity supports blood pressure, lipid profile, insulin sensitivity and
45             weight management together.",
46         "benefit": "Supports multiple findings at once rather than addressing each in isolation.",
47         "precaution": "Increase intensity gradually; stop and seek care if you experience chest
48             pain, breathlessness or dizziness during activity.",
49         "followUp": "Reassess activity tolerance at your next review.",
50         "category": "activity", "weight": 60,
51     },
52     {
53         "code": "REC_CV_SMOKING", "domain": "cardiovascular",
54         "when": lambda fh: fh["profile"].get("smokingStatus") == "Current smoker",
55         "issue": "Current smoking status",
56     }
57 ]

```

```

44     "action": "Consider a structured smoking-cessation programme or speak with a pharmacist
45           about cessation aids.",
46     "why": "Smoking is one of the largest modifiable contributors to cardiovascular risk in
47           the risk calculators used by SQUARE.",
48     "benefit": "Cessation measurably reduces cardiovascular risk over time, independent of
49           other findings.",
50     "precaution": "Cessation aids may not be suitable for everyone – a pharmacist or doctor
51           can advise on the right approach.",
52     "followUp": "Revisit your risk score after –36 months smoke-free.",
53     "category": "lifestyle", "weight": 97,
54   },
55   {
56     "code": "REC_MET_SUGAR", "domain": "metabolic",
57     "when": lambda fh: fh["metabolic"].get("glucoseFlag"),
58     "issue": "Fasting glucose / HbA1c above the illustrative reference range",
59     "action": "Reduce sugary drinks and refined carbohydrates, and space meals to avoid large
60           glucose swings.",
61     "why": "Excess refined-carbohydrate intake is a common contributor to elevated glucose and
62           HbA1c.",
63     "benefit": "May improve fasting glucose and HbA1c at the next check.",
64     "precaution": "If you take glucose-lowering medication, discuss dietary changes with your
65           prescriber first.",
66     "followUp": "Repeat fasting glucose or HbA1c in 3 months.",
67     "category": "nutrition", "weight": 80,
68   },
69   {
70     "code": "REC_LIVER_ALCOHOL", "domain": "liver",
71     "when": lambda fh: fh["liver"].get("altFlag"),
72     "issue": "ALT above the illustrative reference range",
73     "action": "Reduce alcohol intake and review any supplements or over-the-counter medication
74           you take regularly.",
75     "why": "Alcohol, certain supplements and excess body weight are the most common
76           contributors to a raised ALT.",
77     "benefit": "Liver enzymes often normalise within weeks once the contributing factor is
78           addressed.",
79     "precaution": "Do not stop a prescribed medication without medical advice, even if you
80           suspect it is contributing.",
81     "followUp": "Repeat liver function tests in –46 weeks.",
82     "category": "lifestyle", "weight": 75,
83   },
84   {
85     "code": "REC_ANTHRO_WEIGHT", "domain": "anthropometric",
86     "when": lambda fh: fh["anthro"].get("bmiCategory") in ("Overweight", "Obese") or fh["
87           anthro"].get("waistFlag"),
88     "issue": "Body mass index and waist circumference above the healthy range",
89     "action": "Work towards a gradual, sustainable reduction in weight and waist circumference
90           through diet and activity together.",
91     "why": "Excess weight, particularly central adiposity, is a common upstream contributor to
92           several of your other findings.",
93     "benefit": "Weight and waist reduction may improve blood pressure, lipids and glucose
94           findings at the same time.",
95     "precaution": "Rapid or extreme weight loss is not advised without professional guidance,
96           particularly with other medical conditions.",
97     "followUp": "Re-check weight and waist circumference monthly.",
98     "category": "lifestyle", "weight": 85,
99   },
100  {
101    "code": "REC_GEN_SLEEP", "domain": "general", "when": lambda fh: True,
102    "issue": "General preventive health",
103    "action": "Aim for –79 hours of consistent, good-quality sleep.",
104    "why": "Poor sleep is linked with impaired glucose regulation, elevated blood pressure and
105          reduced motivation for other healthy behaviours.",
106    "benefit": "Supports metabolic, cardiovascular and general wellbeing findings together.",
107    "precaution": "If you suspect a sleep disorder (e.g. loud snoring with witnessed pauses in

```

```

    breathing), mention this to a clinician.",
    "followUp": "Track sleep for 2 weeks if unsure of your current pattern.",
    "category": "sleep", "weight": 40,
  },
]

EDUCATION = {
  "EDU_LDL_001": "LDL carries cholesterol through the bloodstream. Higher levels may contribute
    to long-term cardiovascular risk when considered alongside your overall profile.",
  "EDU_APOB_001": "Apolipoprotein B approximates the number of cholesterol-carrying particles in
    your blood, rather than the amount of cholesterol inside them – useful when it disagrees
    with your LDL-C result.",
  "EDU_BP_001": "Blood pressure is the force of circulating blood against your artery walls. A
    single reading is a screening result; a pattern over several readings is more informative.",
  "EDU_FRS_001": "The Framingham General Cardiovascular Risk Score estimates the chance of a
    major cardiovascular event over the next 10 years using age, sex, cholesterol, blood
    pressure, smoking and diabetes status. It is a population-based estimate, not a certainty
    for any one person.",
  "EDU_LPA_001": "Lipoprotein(a), or Lp(a), is largely inherited and usually only needs to be
    measured once in adulthood. An elevated result modifies your overall risk but does not by
    itself diagnose cardiovascular disease.",
  "EDU_GLUCOSE_001": "Fasting glucose and HbA1c both reflect how your body is managing blood
    sugar, over different timeframes – a single fasting reading versus an average over roughly
    three months.",
  "EDU_ALT_001": "ALT is an enzyme mainly found in the liver. A raised level can reflect several
    different causes, most commonly alcohol, certain medications, or excess body weight.",
}

REFERRAL_LEVELS = [
  {"id": "emergency", "label": "Emergency care", "tone": "bad", "rank": 1,
    "description": "New severe chest pain, severe breathing difficulty, fainting, sudden weakness
    , facial droop or speech difficulty.",
    "action": "Call local emergency services", "urgent": True},
  {"id": "prompt", "label": "Prompt medical review", "tone": "warn", "rank": 2,
    "description": "Concerning symptoms, markedly abnormal repeat readings, or a result
    specifically flagged for early review.",
    "action": "Seek medical advice soon"},
  {"id": "routine", "label": "Routine appointment", "tone": "teal", "rank": 3,
    "description": "Discuss non-urgent findings, prevention targets and ongoing monitoring.",
    "action": "Recommended for you"},
  {"id": "self", "label": "Wellness self-management", "tone": "good", "rank": 4,
    "description": "Continue appropriate lifestyle actions and routine monitoring when no
    clinical review is indicated.",
    "action": "View Action Plan"},
]

```

Listing 5.12: backend/app/data/recommendations.py (full file).

5.2.5 app/data/reference_scales.py

Powers Requirement #2, the expandable full-reference-scale rows (§6.5.2). Three registries are merged by `get_all_scales()` into the single object served at `GET /api/meta/reference-scales`: `CARDIO_SCALES` (thirteen hand-authored scales reusing the exact thresholds already implemented in §5.3.2, each carrying its own clinical source citation), `ILLUSTRATIVE_SCALES` (nine scales auto-derived from `SIMPLE_RANGES` via `_bands_from_simple_range()`, so the illustrative domains have exactly one source of threshold truth), and `ANTHRO_SCALES` (BMI and sex-specific waist circumference).

An honesty note worth preserving verbatim

The module's own docstring is explicit that the general LDL/TC/HDL/Non-HDL/Castelli population bands shown on the scale widget are *not* part of the Cardiovascular Domain Specification (which ties LDL to risk-linked ApoB goals rather than a fixed absolute scale) — they are standard NCEP ATP III-style reference bands, included only so the scale-row UI has something illustrative to show, and labelled as such (via the NCEP_NOTE constant) everywhere they appear.

```

1  """EVA SQUARE – reference-scale definitions for the "expand row to see the
2  full scale" UI feature (FullStructure spec item #2). Each scale is a
3  {unit, min, max, bands:[{from,to,label,tone}], source} object; `from`/`to`
4  are inclusive-lower/exclusive-upper except the last band, which extends to
5  `max`. Cardiovascular scales reuse the exact thresholds already implemented
6  in engines/cardiovascular.py; illustrative-domain scales are generated from
7  data/lab_dictionary.py#SIMPLE_RANGES so there is one source of truth for
8  those thresholds. LDL/TC/HDL/Non-HDL/Castelli general bands are NOT part of
9  the Cardiovascular Domain Specification (which ties LDL to risk-linked ApoB
10 goals, not a fixed absolute scale) – they are standard NCEP ATP III-style
11 population reference bands, added ONLY so the UI has something illustrative
12 to show on the scale widget, and are labelled as such everywhere they
13 appear.
14 """
15 from .lab_dictionary import SIMPLE_RANGES, ANTHRO_RULES, LAB_DICTIONARY
16
17 NCEP_NOTE = "General population reference bands (NCEP ATP III-style), shown for illustration only
18 – EVA SQUARE's actual guidance uses the risk-linked ApoB goals and discordance rules in the
19 Cardiovascular Domain Specification, not a fixed absolute scale."
20 SPEC_SOURCE = "Cardiovascular Domain Specification"
21
22 CARDIO_SCALES = {
23     "BP_SYSTOLIC": {"name": "Systolic blood pressure", "unit": "mmHg", "min": 80, "max": 200, "
24 source": "MOH Malaysia Hypertension CPG, 5th ed. 2018",
25     "bands": [
26         {"from": 80, "to": 120, "label": "Optimal", "tone": "good"},
27         {"from": 120, "to": 130, "label": "Normal", "tone": "good"},
28         {"from": 130, "to": 140, "label": "At-risk / high-normal", "tone": "warn"},
29         {"from": 140, "to": 160, "label": "Grade 1 Hypertension", "tone": "warn"},
30         {"from": 160, "to": 180, "label": "Grade 2 Hypertension", "tone": "bad"},
31         {"from": 180, "to": 200, "label": "Grade 3 Hypertension", "tone": "bad"},
32     ]},
33     "BP_DIASTOLIC": {"name": "Diastolic blood pressure", "unit": "mmHg", "min": 50, "max": 130, "
34 source": "MOH Malaysia Hypertension CPG, 5th ed. 2018",
35     "bands": [
36         {"from": 50, "to": 80, "label": "Optimal", "tone": "good"},
37         {"from": 80, "to": 85, "label": "Normal", "tone": "good"},
38         {"from": 85, "to": 90, "label": "At-risk / high-normal", "tone": "warn"},
39         {"from": 90, "to": 100, "label": "Grade 1 Hypertension", "tone": "warn"},
40         {"from": 100, "to": 110, "label": "Grade 2 Hypertension", "tone": "bad"},
41         {"from": 110, "to": 130, "label": "Grade 3 Hypertension", "tone": "bad"},
42     ]},
43     "LAB_LDL": {"name": "LDL Cholesterol", "unit": "mmol/L", "min": 0, "max": 8, "source":
44 NCEP_NOTE,
45     "bands": [
46         {"from": 0, "to": 2.6, "label": "Optimal", "tone": "good"},
47         {"from": 2.6, "to": 3.4, "label": "Near optimal", "tone": "good"},
48         {"from": 3.4, "to": 4.1, "label": "Borderline high", "tone": "warn"},
49         {"from": 4.1, "to": 4.9, "label": "High", "tone": "bad"},
50         {"from": 4.9, "to": 8, "label": "Very high", "tone": "bad"},
51     ]},
52     "LAB_HDL": {"name": "HDL Cholesterol", "unit": "mmol/L", "min": 0, "max": 3, "source":
53 NCEP_NOTE + " Higher is more favourable.",
54     "bands": [
55         {"from": 0, "to": 1.0, "label": "Low", "tone": "bad"},

```

```

50     {"from": 1.0, "to": 1.3, "label": "Below average", "tone": "warn"},
51     {"from": 1.3, "to": 1.6, "label": "Average", "tone": "good"},
52     {"from": 1.6, "to": 3, "label": "High / protective", "tone": "good"},
53   ]},
54   "LAB_TC": {"name": "Total Cholesterol", "unit": "mmol/L", "min": 0, "max": 9, "source":
55     NCEP_NOTE,
56     "bands": [
57       {"from": 0, "to": 5.2, "label": "Desirable", "tone": "good"},
58       {"from": 5.2, "to": 6.2, "label": "Borderline high", "tone": "warn"},
59       {"from": 6.2, "to": 9, "label": "High", "tone": "bad"},
60     ]},
61   "LAB_TG": {"name": "Triglycerides", "unit": "mmol/L", "min": 0, "max": 12, "source":
62     SPEC_SOURCE,
63     "bands": [
64       {"from": 0, "to": 1.7, "label": "Desirable", "tone": "good"},
65       {"from": 1.7, "to": 4.5, "label": "Raised", "tone": "warn"},
66       {"from": 4.5, "to": 10, "label": "High (Friedewald LDL-C invalid)", "tone": "bad"},
67       {"from": 10, "to": 12, "label": "Severe (pancreatitis risk)", "tone": "bad"},
68     ]},
69   "NON_HDL": {"name": "Non-HDL Cholesterol", "unit": "mmol/L", "min": 0, "max": 8, "source":
70     NCEP_NOTE,
71     "bands": [
72       {"from": 0, "to": 3.4, "label": "Desirable", "tone": "good"},
73       {"from": 3.4, "to": 4.9, "label": "Borderline high", "tone": "warn"},
74       {"from": 4.9, "to": 8, "label": "High", "tone": "bad"},
75     ]},
76   "LAB_APOB": {"name": "Apolipoprotein B", "unit": "mg/dL", "min": 0, "max": 200, "source":
77     SPEC_SOURCE + " (risk-linked goal table; shown here against the highest-risk goal of <65,
78     moderate goal <100)",
79     "bands": [
80       {"from": 0, "to": 65, "label": "Meets very-high-risk goal", "tone": "good"},
81       {"from": 65, "to": 80, "label": "Meets high-risk goal", "tone": "good"},
82       {"from": 80, "to": 100, "label": "Meets moderate-risk goal only", "tone": "warn"},
83       {"from": 100, "to": 200, "label": "Above goal for all risk categories", "tone": "bad"},
84     ]},
85   "LAB_LPA": {"name": "Lipoprotein(a)", "unit": "nmol/L", "min": 0, "max": 300, "source":
86     SPEC_SOURCE,
87     "bands": [
88       {"from": 0, "to": 75, "label": "Lower", "tone": "good"},
89       {"from": 75, "to": 125, "label": "Intermediate", "tone": "warn"},
90       {"from": 125, "to": 300, "label": "Elevated / risk-enhancing", "tone": "bad"},
91     ]},
92   "AIP": {"name": "Atherogenic Index of Plasma", "unit": "log10(TG/HDL-C)", "min": -0.5, "max":
93     1.0, "source": SPEC_SOURCE + " (supplementary index)",
94     "bands": [
95       {"from": -0.5, "to": 0.11, "label": "Lower", "tone": "good"},
96       {"from": 0.11, "to": 0.21, "label": "Intermediate", "tone": "warn"},
97       {"from": 0.21, "to": 1.0, "label": "Higher", "tone": "bad"},
98     ]},
99   "CASTELLI_I": {"name": "Castelli Risk Index I (TC/HDL-C)", "unit": "ratio", "min": 2, "max":
100     8, "source": SPEC_SOURCE + " (supplementary; shown using the men's cut-off – women's cut-
101     offs are slightly lower)",
102     "bands": [
103       {"from": 2, "to": 4.0, "label": "Desirable", "tone": "good"},
104       {"from": 4.0, "to": 5.0, "label": "Borderline", "tone": "warn"},
105       {"from": 5.0, "to": 8, "label": "Higher", "tone": "bad"},
106     ]},
107   "CASTELLI_II": {"name": "Castelli Risk Index II (LDL/HDL-C)", "unit": "ratio", "min": 1, "max":
108     6, "source": SPEC_SOURCE + " (supplementary; shown using the men's cut-off)",
109     "bands": [
110       {"from": 1, "to": 3.0, "label": "Desirable", "tone": "good"},
111       {"from": 3.0, "to": 6, "label": "Higher", "tone": "warn"},
112     ]},

```

```

103     "FRS_RISK": {"name": "Framingham 10-year CVD risk", "unit": "%", "min": 0, "max": 40, "source"
104                 : SPEC_SOURCE,
105                 "bands": [
106                     {"from": 0, "to": 10, "label": "Low", "tone": "good"},
107                     {"from": 10, "to": 20, "label": "Intermediate", "tone": "warn"},
108                     {"from": 20, "to": 40, "label": "High", "tone": "bad"},
109                 ]},
110 }
111
112 def _bands_from_simple_range(code, margin_low=0.15, margin_high=0.3):
113     """Derive a 3-4 zone band list from a SIMPLE_RANGES threshold dict, with
114     tone semantics matching engines/rule_engine.py#classify_by_range exactly:
115     only a severeLow/severeHigh breach is "bad"; every other deviation
116     (mild-* or below/above the normal bounds) is "warn", since that engine
117     maps both to the same tone. mildLow/mildHigh therefore only affect the
118     *severity threshold*, not a separately-colored zone – a mild deviation
119     and a below/above-range deviation render as one continuous warn region,
120     which is the correct, non-misleading picture for a color scale."""
121     r = SIMPLE_RANGES[code]
122     vals = [v for k, v in r.items() if isinstance(v, (int, float))]
123     lo = min(vals) * (1 - margin_low) if min(vals) > 0 else min(vals) - 1
124     hi = max(vals) * (1 + margin_high)
125     normal_low = r.get("normalLow")
126     normal_high = r.get("normalHigh")
127     severe_low = r.get("severeLow")
128     severe_high = r.get("severeHigh")
129     bands = []
130     cur = lo
131     if severe_low is not None:
132         bands.append({"from": cur, "to": severe_low, "label": "Markedly below range", "tone": "bad"
133                     }); cur = severe_low
134     if normal_low is not None:
135         bands.append({"from": cur, "to": normal_low, "label": "Below range", "tone": "warn"}); cur
136         = normal_low
137     band_hi = normal_high if normal_high is not None else hi
138     bands.append({"from": cur, "to": band_hi, "label": "Within range", "tone": "good"}); cur =
139     band_hi
140     if normal_high is not None:
141         upper_end = severe_high if severe_high is not None else hi
142         if upper_end > cur:
143             bands.append({"from": cur, "to": upper_end, "label": "Above range", "tone": "warn"});
144             cur = upper_end
145     if severe_high is not None and cur < hi:
146         bands.append({"from": cur, "to": hi, "label": "Markedly above range", "tone": "bad"})
147     meta = LAB_DICTIONARY.get(code, {})
148     return {"name": meta.get("name", code), "unit": meta.get("unit", ""), "min": round(lo, 2), "
149             max": round(hi, 2), "bands": bands, "source": r["source"]}
150
151 ILLUSTRATIVE_CODES = ["LAB_GLU_FAST", "LAB_HBA1C", "LAB_ALT", "LAB_AST", "LAB_CREAT", "LAB_EGFR",
152                       "LAB_HB", "LAB_WBC", "LAB_PLT"]
153 ILLUSTRATIVE_SCALES = {code: _bands_from_simple_range(code) for code in ILLUSTRATIVE_CODES}
154
155 ANTHRO_SCALES = {
156     "BMI": {"name": "Body Mass Index", "unit": "kg/m²", "min": 12, "max": 45,
157            "source": "General illustrative BMI bands (WHO Asian-population cut-offs)",
158            "bands": [
159                {"from": 12, "to": 18.5, "label": "Underweight", "tone": "warn"},
160                {"from": 18.5, "to": 23, "label": "Normal", "tone": "good"},
161                {"from": 23, "to": 27.5, "label": "Overweight", "tone": "warn"},
162                {"from": 27.5, "to": 45, "label": "Obese", "tone": "bad"},
163            ]},
164     "WAIST_M": {"name": "Waist circumference (male)", "unit": "cm", "min": 60, "max": 130,

```

```

160         "source": "Asian population cut-off referenced in the Cardiovascular Domain
161             Specification's metabolic-syndrome criterion",
162         "bands": [{"from": 60, "to": 90, "label": "Within cut-off", "tone": "good"}, {"
163             from": 90, "to": 130, "label": "Above cut-off (central obesity)", "tone": "bad
164             "}]},
165     "WAIST_F": {"name": "Waist circumference (female)", "unit": "cm", "min": 50, "max": 120,
166         "source": "Asian population cut-off referenced in the Cardiovascular Domain
167             Specification's metabolic-syndrome criterion",
168         "bands": [{"from": 50, "to": 80, "label": "Within cut-off", "tone": "good"}, {"
169             from": 80, "to": 120, "label": "Above cut-off (central obesity)", "tone": "bad
170             "}]},
171 }
172
173 def get_all_scales():
174     """Full registry keyed by quantity code, for the /api/reference-scales endpoint."""
175     out = {}
176     out.update(CARDIO_SCALES)
177     out.update(ILLUSTRATIVE_SCALES)
178     out.update(ANTHRO_SCALES)
179     return out

```

Listing 5.13: backend/app/data/reference_scales.py (full file).

5.2.6 app/data/sample_reports.py

The two demo scenarios — "routine" (Dexter Tan, 35M, unremarkable lipids) and "urgent" (Priya Kumar, 58F, triggers the emergency safety pathway via a blood pressure of 190/122 with symptoms) — each a fully self-consistent dataset: profile, full lab panel, per-lab confidence scores, questionnaire defaults, three-point trend history for LDL/weight/home-BP, and a realistic plain-text originalText block formatted like an actual lab report printout (shown on the Verify screen next to the extracted-values table). The urgent scenario's lipid values deliberately reproduce the Cardiovascular Domain Specification's own LDL/ApoB discordance worked example, so that pattern is exercised end-to-end by simply loading the sample.

```

1  """EVA SQUARE – two seed scenarios (Python port of js/data/sampleReports.js).
2  See that file's header for the rationale: an internally-consistent original
3  dataset chosen to exercise both the routine path and the safety-escalation
4  matrix, deliberately reproducing the spec document's LDL/ApoB discordance
5  worked example."""
6
7  SAMPLE_SCENARIOS = {
8      "routine": {
9          "key": "routine", "label": "Sample Blood Report (routine)",
10         "lab": "HealthPlus Labs", "reportDate": "2026-08-20", "fileName": "Sample Blood Report.pdf",
11         "profile": {
12             "fullName": "Dexter Tan", "email": "dexter@demo.square", "age": 35, "sex": "M", "
13             country": "Malaysia",
14             "height": 173, "weight": 78.2, "waist": 94, "restingHR": 72,
15             "homeBP": {"sbp": 131, "dbp": 84, "readings": 14, "treated": False},
16         },
17         "labs": {
18             "LAB_TC": 5.4, "LAB_LDL": 3.2, "LAB_HDL": 1.3, "LAB_TG": 1.4,
19             "LAB_APOB": 105, "LAB_APOA1": 145, "LAB_LPA": 45,
20             "LAB_GLU_FAST": 5.8, "LAB_HBA1C": 5.9, "LAB_ALT": 34, "LAB_AST": 26,
21             "LAB_CREAT": 82, "LAB_EGFR": 96, "LAB_HB": 14.2, "LAB_WBC": 6800, "LAB_PLT": 2.6,
22         },
23         "labConfidence": {"LAB_TC": 99, "LAB_LDL": 98, "LAB_HDL": 99, "LAB_TG": 97, "LAB_GLU_FAST":
24             : 82, "LAB_ALT": 95, "LAB_HB": 99},
25         "questionnaireDefaults": {
26             "everDiagnosedHighBP": "No", "recentBP": {"sbp": 131, "dbp": 84}, "bpMedication": "No",
27             "familyHistoryPrematureCVD": "No",

```

```

25     "diagnosedDiabetes": "No", "diabetesMedication": "No", "familyHistoryDiabetes": "Not
26       sure", "activityLevel": "Light",
27     "alcoholIntake": "Occasional", "fattyLiverHistory": "No", "smokingStatus": "Never
28       smoked",
29     "regularMedication": "No", "regularMedicationText": "", "kidneyDisease": "No", "
30       pregnancyStatus": "Not applicable",
31   },
32   "trendHistory": {
33     "LAB_LDL": [{"date": "2023-11-05", "value": 3.3}, {"date": "2024-05-02", "value":
34       3.6}, {"date": "2026-08-20", "value": 3.2}],
35     "weight": [{"date": "2026-03-01", "value": 79.8}, {"date": "2026-05-01", "value":
36       79.1}, {"date": "2026-07-01", "value": 78.6}, {"date": "2026-08-20", "value":
37       78.2}],
38     "homeBP": [
39       {"date": "2026-08-14", "sbp": 134, "dbp": 86}, {"date": "2026-08-15", "sbp": 129,
40         "dbp": 82}, {"date": "2026-08-16", "sbp": 132, "dbp": 85},
41       {"date": "2026-08-17", "sbp": 128, "dbp": 81}, {"date": "2026-08-18", "sbp": 133,
42         "dbp": 86}, {"date": "2026-08-19", "sbp": 130, "dbp": 83}, {"date": "
43         2026-08-20", "sbp": 131, "dbp": 84},
44     ],
45   },
46   "originalText": ""SAMPLE LABORATORY REPORT
47   Sample ID: 260820-00417      Age/Gender: 35 / Male      Report Date: 20 Aug 2026
48   Collected: 19 Aug 2026 07:40  Ref By: Dr. A. Physician
49   Reported: 20 Aug 2026 11:05    Lab ID: HL-224981
50
51   LIPID PROFILE
52   Total Cholesterol      5.4    mmol/L    <5.2
53   LDL Cholesterol       3.2    mmol/L    <3.4
54   HDL Cholesterol       1.3    mmol/L    >1.0
55   Triglycerides         1.4    mmol/L    <1.7
56   Apolipoprotein B      105    mg/dL     <100 (risk-linked)
57   Apolipoprotein A-I    145    mg/dL     -
58   Lipoprotein(a)        45     nmol/L    <75
59
60   GLYCAEMIC
61   Fasting Glucose       5.8    mmol/L    3.9-5.5
62   HbA1c                 5.9    %         <5.7
63
64   LIVER FUNCTION
65   ALT (SGPT)            34     U/L       7-41
66   AST (SGOT)            26     U/L       8-40
67
68   RENAL FUNCTION
69   Creatinine            82     umol/L    60-110
70   eGFR                  96     mL/min/1.73m2  >90
71
72   FULL BLOOD COUNT
73   Haemoglobin           14.2  g/dL     13.0-17.0
74   White Blood Cells     6800  cells/uL  4000-11000
75   Platelets             2.6   Lakh/uL  1.5-4.5
76   *** This is a simulated report generated for prototype demonstration. ****",
77   },
78   "urgent": {
79     "key": "urgent", "label": "Screening Report (urgent pathway demo)",
80     "lab": "Wellness Diagnostics", "reportDate": "2026-08-21", "fileName": "Screening Report.
81       pdf",
82     "profile": {
83       "fullName": "Priya Kumar", "email": "priya@demo.square", "age": 58, "sex": "F", "
84       country": "Malaysia",
85       "height": 160, "weight": 74, "waist": 96, "restingHR": 88,
86       "homeBP": {"sbp": 190, "dbp": 122, "readings": 3, "treated": False},
87     },
88   },
89   "labs": {

```

```

78     "LAB_TC": 7.8, "LAB_LDL": 6.1, "LAB_HDL": 1.0, "LAB_TG": 11.2,
79     "LAB_APOB": 168, "LAB_APOA1": 118, "LAB_LPA": 140,
80     "LAB_GLU_FAST": 9.1, "LAB_HbA1c": 8.2, "LAB_ALT": 30, "LAB_AST": 28,
81     "LAB_CREAT": 95, "LAB_EGFR": 72, "LAB_Hb": 12.8, "LAB_WBC": 7600, "LAB_PLT": 2.4,
82 },
83 "labConfidence": {"LAB_TC": 97, "LAB_LDL": 95, "LAB_HDL": 98, "LAB_TG": 93, "LAB_GLU_FAST":
84   : 96, "LAB_ALT": 98, "LAB_Hb": 97},
85 "questionnaireDefaults": {
86   "everDiagnosedHighBP": "Yes", "recentBP": {"sbp": 190, "dbp": 122}, "bpMedication": "
87     Yes", "familyHistoryPrematureCVD": "Yes",
88   "diagnosedDiabetes": "Yes", "diabetesMedication": "Yes", "familyHistoryDiabetes": "Yes
89     ", "activityLevel": "Sedentary",
90   "alcoholIntake": "None", "fattyLiverHistory": "Not sure", "smokingStatus": "Former
91     smoker",
92   "regularMedication": "Yes", "regularMedicationText": "Metformin, Amlodipine", "
93     kidneyDisease": "Not sure", "pregnancyStatus": "Not applicable",
94   "symptomsNow": ["Severe headache", "Blurred vision"],
95 },
96 "trendHistory": {
97   "LAB_LDL": [{"date": "2023-11-10", "value": 5.2}, {"date": "2024-05-08", "value":
98     5.7}, {"date": "2026-08-21", "value": 6.1}],
99   "weight": [{"date": "2026-03-01", "value": 75.6}, {"date": "2026-05-01", "value":
100     75.0}, {"date": "2026-07-01", "value": 74.4}, {"date": "2026-08-21", "value":
101     74.0}],
102   "homeBP": [
103     {"date": "2026-08-15", "sbp": 176, "dbp": 114}, {"date": "2026-08-16", "sbp": 182,
104       "dbp": 118}, {"date": "2026-08-17", "sbp": 179, "dbp": 116},
105     {"date": "2026-08-18", "sbp": 185, "dbp": 119}, {"date": "2026-08-19", "sbp": 188,
106       "dbp": 121}, {"date": "2026-08-20", "sbp": 184, "dbp": 120}, {"date": "
107       2026-08-21", "sbp": 190, "dbp": 122},
108   ],
109 },
110 "originalText": """"SCREENING REPORT
111
112 Sample ID: 260821-00933      Age/Gender: 58 / Female
113 Collected: 21 Aug 2026 08:10 Ref By: Self-referred screening
114 Reported: 21 Aug 2026 13:20 Lab ID: WD-551203
115
116 LIPID PROFILE
117 Total Cholesterol      7.8 mmol/L <5.2
118 LDL Cholesterol       6.1 mmol/L <3.4
119 HDL Cholesterol       1.0 mmol/L >1.0
120 Triglycerides         11.2 mmol/L <1.7
121 Apolipoprotein B     168 mg/dL <100 (risk-linked)
122 Apolipoprotein A-I   118 mg/dL -
123 Lipoprotein(a)       140 nmol/L <75
124
125 GLYCAEMIC
126 Fasting Glucose      9.1 mmol/L 3.9-5.5
127 HbA1c                8.2 % <5.7
128
129 LIVER FUNCTION
130 ALT (SGPT)           30 U/L 7-41
131 AST (SGOT)           28 U/L 8-40
132
133 RENAL FUNCTION
134 Creatinine           95 umol/L 60-110
135 eGFR                 72 mL/min/1.73m2 >90
136
137 FULL BLOOD COUNT
138 Haemoglobin          12.8 g/dL 13.0-17.0 (flag: below range)
139 White Blood Cells    7600 cells/uL 4000-11000
140 Platelets            2.4 Lakh/uL 1.5-4.5
141 *** This is a simulated report generated for prototype demonstration. ****",
142 },

```

```
131 }
```

Listing 5.14: backend/app/data/sample_reports.py (full file).

5.3 Clinical rule engines (app/engines/)

Every module in this layer is plain, framework-agnostic Python — no FastAPI, no SQLAlchemy imports — taking and returning plain dicts. This is what makes each engine independently unit-testable and, per the project’s own convention, a direct function-for-function port of an original JavaScript engine module (from the single-file `Prototype1_Version2` prototype), kept behaviourally identical to it by design.

5.3.1 app/engines/orchestrator.py — ties every engine together

The single function the `/api/reports/{id}/analyze` endpoint calls. `assemble_findings()` runs all six domain analyses (one cardiovascular, five illustrative) against the same `profile/labs/questionnaire` input and folds their summaries into one `fh` (“findings”) dict — the shape every recommendation’s when predicate (§5.2.4) is written against. `run_analysis()` then feeds that bundle through the safety engine, the recommendation engine, and the report generator, and returns the single combined result object that becomes the entire `analysis_results.result_json` row. Figure 5.2 traces this data flow visually.

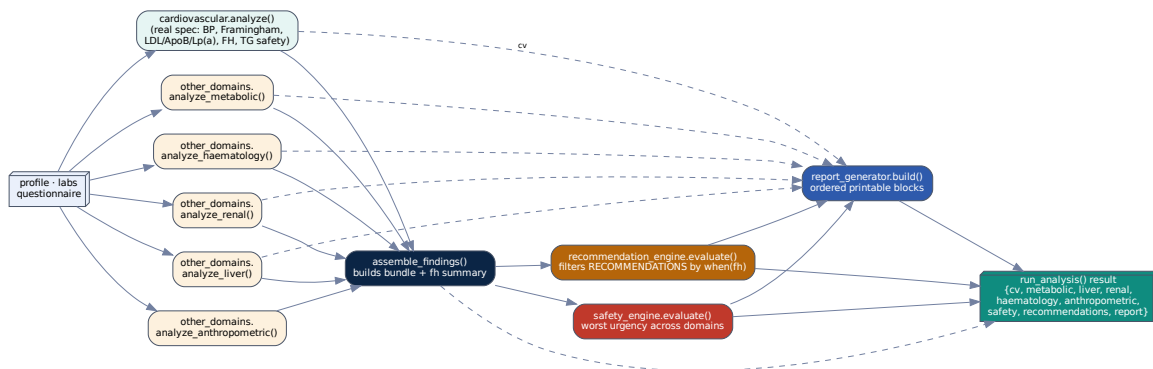


Figure 5.2: Data flow through `orchestrator.run_analysis()`: six parallel domain analyses feed one assembled findings bundle, which the safety, recommendation and report-generator engines each consume independently before their outputs are combined into the final result.

```

1  """EVA SQUARE – ties every engine together into one `run_analysis()` call,
2  mirroring frontend-v2's store.js#assembleFindings + #runAnalysis. This is
3  the single function the /api/reports/{id}/analyze endpoint calls."""
4  from . import cardiovascular, other_domains, safety_engine, recommendation_engine,
    report_generator
5
6
7  def assemble_findings(profile, labs, questionnaire):
8      q = questionnaire or {}
9      cv = cardiovascular.analyze(profile, labs, q)
10     metabolic = other_domains.analyze_metabolic(labs, q)
11     liver = other_domains.analyze_liver(labs, q)
12     renal = other_domains.analyze_renal(labs, q)
13     haematology = other_domains.analyze_haematology(labs, q)
14     anthropometric = other_domains.analyze_anthropometric(profile)
15
16     fh = {
17         "cv": cv["summary"],
18         "profile": {"smokingStatus": q.get("smokingStatus"), "age": profile.get("age"), "sex":
19             profile.get("sex")},
19         "metabolic": metabolic["summary"],
20         "liver": liver["summary"],
21         "renal": renal["summary"],
22         "haematology": haematology["summary"],

```

```

23     "anthro": {"bmiCategory": anthropometric["bmiCategory"], "waistFlag": anthropometric["
24         waistFlag"]},
25     }
26     return {"profile": profile, "labs": labs, "q": q, "cv": cv, "metabolic": metabolic, "liver":
27         liver,
28         "renal": renal, "haematology": haematology, "anthropometric": anthropometric, "fh": fh
29         }
30
31 def run_analysis(report_label, profile, labs, questionnaire):
32     bundle = assemble_findings(profile, labs, questionnaire)
33     safety = safety_engine.evaluate({"cv": bundle["cv"], "metabolic": bundle["metabolic"], "liver"
34         : bundle["liver"],
35         "renal": bundle["renal"], "haematology": bundle["haematology
36         "]])
37     recs_raw = recommendation_engine.evaluate(bundle["fh"])
38     recs = [recommendation_engine.serializable(r) for r in recs_raw]
39     report = report_generator.build(report_label, bundle["cv"],
40         {"metabolic": bundle["metabolic"], "liver": bundle["liver"],
41         "renal": bundle["renal"], "haematology": bundle["haematology
42         ""]},
43         safety, recs)
44
45     return {
46         "cv": bundle["cv"], "metabolic": bundle["metabolic"], "liver": bundle["liver"], "renal":
47         bundle["renal"],
48         "haematology": bundle["haematology"], "anthropometric": bundle["anthropometric"], "fh":
49         bundle["fh"],
50         "safety": safety, "recommendations": recs, "report": report,
51     }

```

Listing 5.15: backend/app/engines/orchestrator.py (full file).

5.3.2 app/engines/cardiovascular.py — the real-spec engine

The one domain implemented against an actual source document (the *Cardiovascular Domain Specification*), and by far the largest engine module. It implements calculator register entries CV-01 through CV-14, summarised in Table 5.1, and its `analyze()` function at the bottom of the file is the orchestration point that wires every one of them together against a single patient’s profile, labs and questionnaire answers.

ID	Calculator	Function
CV-01	Blood pressure classification (MOH Malaysia Hypertension CPG, 5th ed. 2018)	<code>classify_bp()</code>
—	Emergency/urgent escalation on BP $\geq 180/120$ or emergency symptoms	<code>bp_escalation()</code>
CV-02	Framingham General CVD 10-year risk score (D’Agostino 2008, sex-specific)	<code>framingham_risk()</code>
CV-03/04	Non-HDL-C and Friedewald-calculated LDL-C	<code>non_hdl()</code> , <code>calculated_ldl()</code>
CV-05	Atherogenic Index of Plasma (supplementary)	<code>aip()</code>
CV-06/07/08	Castelli Risk Index I & II, Atherogenic Coefficient	<code>castelli_i()</code> , <code>castelli_ii()</code> , <code>atherogenic_coefficient()</code>
CV-09	ApoB risk-linked goal assessment	<code>apo_b_goal()</code> , <code>apo_b_assessment()</code>
CV-10	ApoB/ApoA-I ratio (supplementary)	<code>apo_ratio()</code>
CV-11	Lp(a) dual-unit banding (nmol/L or mg/dL, never cross-converted)	<code>lpa_band()</code>
CV-12	Metabolic syndrome, any 3 of 5 criteria	<code>metabolic_syndrome()</code>
CV-13	“Possible FH” referral trigger (never an auto-diagnosis)	<code>fh_trigger()</code>
CV-14	Triglyceride safety tiers (pancreatitis-risk escalation)	<code>tg_safety()</code>
—	Lipid discordance interpretation (spec’s worked-example table)	<code>discordance()</code>

Table 5.1: The cardiovascular calculator register implemented in `cardiovascular.py`, keyed to the Cardiovascular Domain Specification’s own CV-*nn* numbering.

Framingham risk score, worked. The core of CV-02 is a sex-specific log-linear model. For coefficients c (Table lookup by sex), total cholesterol TC and HDL-C HDL in mg/dL, systolic blood pressure SBP , and boolean smoker/diabetes/treated flags:

$$L = c_{age} \ln(\text{age}) + c_{tc} \ln(TC) + c_{hdl} \ln(HDL) + c_{sbp} \ln(SBP) + c_{smoker} \cdot \mathbb{1}_{smoker} + c_{diabetes} \cdot \mathbb{1}_{diabetes}$$

$$\text{risk} = \max\left(0.1, \min\left(99, 100 \times \left(1 - s_0^{\exp(L - \bar{L})}\right)\right)\right) \%$$

where c_{sbp} itself switches between a treated and untreated coefficient depending on `treated`, and s_0 , $\bar{L}$ are the sex-specific baseline survival and mean-L constants in `_FRS_COEF`. Total cholesterol and HDL-C are converted from mmol/L to mg/dL first ($\times 38.67$) since the original Framingham coefficients were

derived in mg/dL. Eligibility is checked *before* the formula runs (`_framingham_eligibility()`): established CVD, pregnancy/postpartum, age outside 30–74, missing sex/lipids/BP/smoking/diabetes status, or an acute-symptom escalation in progress all disqualify a fresh 10-year estimate, each with its own explanatory reason string surfaced back to the frontend rather than a bare “N/A”.

```

1  """EVA SQUARE – Cardiovascular Domain engine. Direct, function-for-function
2  Python port of the frontend-v2 js/engine/cardiovascular.js – see that
3  file's header comment for the full rationale (this is the one domain
4  implemented against a real source document, the Cardiovascular Domain
5  Specification; CV-01 through CV-14 calculator register; prohibited-claims
6  list; unit conventions). Nothing here should ever diverge in behavior from
7  the JS version – if you change a threshold, change it in both places (or,
8  in a deployed version, delete the JS engine entirely and have the frontend
9  call this API exclusively, which is exactly what this FullStructure build
10 already does).
11 """
12 import math
13 import datetime as dt
14
15 SOURCE = ("Cardiovascular Domain Specification – MOH Malaysia Hypertension 5th ed. 2018; "
16          "D'Agostino RB Sr et al., Framingham General CVD, Circulation 2008;117:743-753; "
17          "ESC/EAS Dyslipidaemia guidelines.")
18
19 EMERGENCY_SYMPTOMS = [
20     "Chest pain", "Severe breathlessness", "Sudden weakness", "Facial droop",
21     "Speech difficulty", "Confusion", "Severe headache", "Blurred vision", "Collapse",
22 ]
23
24 # -----
25 # CV-01 Blood pressure classification (MOH Malaysia Hypertension CPG, 5th ed. 2018)
26 # -----
27 _BP_BANDS = [
28     ("optimal", "Optimal", 1, "good", lambda s, d: s < 120 and d < 80, "Within the optimal range."
29     ),
30     ("normal", "Normal", 2, "good", lambda s, d: 120 <= s <= 129 and 80 <= d <= 84, "Within the
31     normal range."),
32     ("atrisk", "At-risk / high-normal", 3, "warn", lambda s, d: (130 <= s <= 139) or (85 <= d <=
33     89), "Above optimal – worth monitoring."),
34     ("grade1", "Grade 1 Hypertension", 4, "warn", lambda s, d: (140 <= s <= 159) or (90 <= d <=
35     99), "Raised – confirmation with repeat readings is recommended."),
36     ("grade2", "Grade 2 Hypertension", 5, "bad", lambda s, d: (160 <= s <= 179) or (100 <= d <=
37     109), "High – prompt clinical review is recommended."),
38     ("grade3", "Grade 3 Hypertension", 6, "bad", lambda s, d: s >= 180 or d >= 110, "Very high –
39     urgent assessment is recommended."),
40 ]
41
42 def classify_bp(sbp, dbp):
43     if sbp is None or dbp is None:
44         return None
45     if sbp >= 140 and dbp < 90:
46         return {"id": "isolated-systolic", "label": "Isolated Systolic Hypertension", "rank": 4, "
47             tone": "warn",
48             "note": "Systolic pressure raised with a normal diastolic pressure.", "sbp": sbp,
49             "dbp": dbp, "source": SOURCE}
50     for bid, label, rank, tone, test, note in _BP_BANDS:
51         if test(sbp, dbp):
52             return {"id": bid, "label": label, "rank": rank, "tone": tone, "note": note, "sbp":
53                 sbp, "dbp": dbp, "source": SOURCE}
54     return {"id": "grade3", "label": "Grade 3 Hypertension", "rank": 6, "tone": "bad",
55         "note": "Very high – urgent assessment is recommended.", "sbp": sbp, "dbp": dbp, "
56         source": SOURCE}

```

```

49
50 def bp_escalation(sbp, dbp, symptoms_now):
51     symptoms = symptoms_now or []
52     has_emergency_symptom = any(s in EMERGENCY_SYMPTOMS for s in symptoms)
53     if sbp is not None and dbp is not None and (sbp >= 180 or dbp >= 120):
54         if has_emergency_symptom:
55             return {"urgency": "emergency", "reason": "Blood pressure ≥180/120 mmHg with symptoms
56                 that may indicate a medical emergency.",
57                     "action": "Advise immediate emergency services / emergency department. Stop
58                         routine risk scoring."}
59             return {"urgency": "urgent", "reason": "Blood pressure ≥180/120 mmHg without reported
60                 symptoms.",
61                     "action": "Repeat the reading correctly and arrange urgent, same-day clinical
62                         review."}
63         if has_emergency_symptom:
64             return {"urgency": "emergency", "reason": "Reported symptoms may indicate a medical
65                 emergency, independent of the blood pressure reading.",
66                     "action": "Advise immediate emergency services / emergency department."}
67         return None
68
69 # -----
70 # CV-02 Framingham General CVD Risk Score (D'Agostino 2008, sex-specific)
71 # -----
72 MMOL_TO_MGDL_CHOL = 38.67
73
74 _FRS_COEF = {
75     "M": {"age": 3.06117, "tc": 1.12370, "hdl": -0.93263, "sbpU": 1.93303, "sbpT": 1.99881, "
76         smoker": 0.65451, "diabetes": 0.57367, "mean": 23.9802, "s0": 0.88431},
77     "F": {"age": 2.32888, "tc": 1.20904, "hdl": -0.70833, "sbpU": 2.76157, "sbpT": 2.82263, "
78         smoker": 0.52873, "diabetes": 0.69154, "mean": 26.1931, "s0": 0.94833},
79 }
80
81 def _framingham_eligibility(ctx):
82     reasons = []
83     if ctx.get("hasEstablishedCVD"):
84         reasons.append("Established cardiovascular disease – routed to secondary-prevention
85             pathway instead of a fresh 10-year estimate.")
86     if ctx.get("pregnancyStatus") in ("Pregnant", "Postpartum"):
87         reasons.append("Pregnancy / postpartum – outside the model's validated population.")
88     age = ctx.get("age")
89     if age is None or age < 30 or age > 74:
90         reasons.append("Age outside the model's validated range –(3074 years).")
91     if ctx.get("sex") not in ("M", "F"):
92         reasons.append("Sex not specified.")
93     if ctx.get("tc") is None or ctx.get("hdl") is None:
94         reasons.append("Total cholesterol and/or HDL-cholesterol missing, or not from the same
95             sample.")
96     if ctx.get("sbp") is None:
97         reasons.append("Blood pressure missing.")
98     if ctx.get("smoker") is None:
99         reasons.append("Smoking status not confirmed.")
100     if ctx.get("diabetes") is None:
101         reasons.append("Diabetes status not confirmed.")
102     if ctx.get("acuteSymptoms"):
103         reasons.append("Acute symptoms reported – routine risk scoring is paused pending safety
104             review.")
105     return {"eligible": len(reasons) == 0, "reasons": reasons}
106
107 def framingham_risk(ctx):
108     elig = _framingham_eligibility(ctx)
109     if not elig["eligible"]:

```

```

103     return {"eligible": False, "reasons": elig["reasons"], "source": SOURCE}
104     c = _FRS_COEF[ctx["sex"]]
105     tc_mgdl = ctx["tc"] * MMOL_TO_MGDL_CHOL
106     hdl_mgdl = ctx["hdl"] * MMOL_TO_MGDL_CHOL
107     sbp_coef = c["sbpT"] if ctx.get("treated") else c["sbpU"]
108     L = (c["age"] * math.log(ctx["age"])) + c["tc"] * math.log(tc_mgdl) + c["hdl"] * math.log(
109         hdl_mgdl)
110         + sbp_coef * math.log(ctx["sbp"]) + c["smoker"] * (1 if ctx.get("smoker") else 0)
111         + c["diabetes"] * (1 if ctx.get("diabetes") else 0))
112     risk_fraction = 1 - (c["s0"] ** math.exp(L - c["mean"]))
113     risk_percent = max(0.1, min(99, risk_fraction * 100))
114
115     if risk_percent < 10:
116         band, tone, note = "Low", "good", "Below 10% – lower estimated 10-year risk."
117     elif risk_percent < 20:
118         band, tone, note = "Intermediate", "warn", "-1019.9% – intermediate estimated 10-year risk"
119         .
120     else:
121         band, tone, note = "High", "bad", "20% or higher – high estimated 10-year risk. Clinician
122         review is recommended."
123
124     return {
125         "eligible": True, "riskPercent": round(risk_percent, 1), "band": band, "tone": tone, "note
126         ": note,
127         "inputs": {"age": ctx["age"], "sex": ctx["sex"], "tcMgdl": round(tc_mgdl), "hdlMgdl":
128             round(hdl_mgdl),
129             "sbp": ctx["sbp"], "treated": bool(ctx.get("treated")), "smoker": bool(ctx.get(
130                 "smoker")), "diabetes": bool(ctx.get("diabetes"))},
131         "caution": "This is a population-based estimate, not a prediction for a specific
132             individual. CKD, very high LDL-C, familial hypercholesterolaemia and high Lp(a) can
133             push true risk above this percentage.",
134         "source": SOURCE,
135     }
136
137 # -----
138 # CV-03 / CV-04 Non-HDL-C and calculated (Friedewald) LDL-C
139 # -----
140 def non_hdl(tc, hdl):
141     if tc is None or hdl is None:
142         return None
143     return round((tc - hdl) * 10) / 10
144
145 def calculated_ldl(tc, hdl, tg):
146     if tc is None or hdl is None or tg is None:
147         return {"value": None, "valid": False, "reason": "One or more of TC, HDL-C, TG is missing."}
148     if tg >= 4.5:
149         return {"value": None, "valid": False, "reason": "Friedewald calculation is not valid when
150             triglycerides are ≥4.5 mmol/L. A direct LDL-C measurement is required."}
151     ldl = tc - hdl - tg / 2.2
152     return {"value": round(ldl * 10) / 10, "valid": True, "label": "Calculated LDL-C (Friedewald)"
153         , "source": SOURCE}
154
155 # -----
156 # CV-05 Atherogenic Index of Plasma (supplementary only)
157 # -----
158 def aip(tg, hdl):
159     if tg is None or hdl is None or tg <= 0 or hdl <= 0:
160         return None
161     value = math.log10(tg / hdl)
162     if value < 0.11:

```

```

156         band, tone = "Lower", "good"
157     elif value <= 0.21:
158         band, tone = "Intermediate", "warn"
159     else:
160         band, tone = "Higher", "bad"
161     return {"value": round(value, 2), "band": band, "tone": tone, "status": "Supplementary",
162           "note": "A supplementary index – it does not override LDL-C, non-HDL-C, ApoB, Lp(a) or
           your overall risk score.", "source": SOURCE}
163
164
165 # -----
166 # CV-06 / CV-07 / CV-08 Castelli I & II, Atherogenic coefficient
167 # -----
168 def castelli_i(tc, hdl, sex):
169     if tc is None or hdl is None or hdl <= 0:
170         return None
171     value = round((tc / hdl) * 100) / 100
172     desirable = 3.5 if sex == "F" else 4.0
173     higher = 4.5 if sex == "F" else 5.0
174     if value < desirable:
175         band, tone = "Desirable", "good"
176     elif value <= higher:
177         band, tone = "Borderline", "warn"
178     else:
179         band, tone = "Higher", "bad"
180     return {"value": value, "band": band, "tone": tone, "status": "Supplementary", "source":
           SOURCE}
181
182
183 def castelli_ii(ldl, hdl, sex, ldl_valid):
184     if ldl is None or hdl is None or hdl <= 0 or ldl_valid is False:
185         return None
186     value = round((ldl / hdl) * 100) / 100
187     desirable = 2.5 if sex == "F" else 3.0
188     if value < desirable:
189         band, tone = "Desirable", "good"
190     else:
191         band, tone = "Higher", "warn"
192     return {"value": value, "band": band, "tone": tone, "status": "Supplementary", "source":
           SOURCE}
193
194
195 def atherogenic_coefficient(tc, hdl):
196     if tc is None or hdl is None or hdl <= 0:
197         return None
198     return round(((tc - hdl) / hdl) * 100) / 100
199
200
201 # -----
202 # CV-09 ApoB risk-linked goal
203 # -----
204 def apo_b_goal(risk_band, secondary_prevention):
205     if secondary_prevention:
206         return {"target": 65, "label": "Very-high risk (established disease)"}
207     if risk_band == "High":
208         return {"target": 80, "label": "High risk"}
209     if risk_band == "Intermediate":
210         return {"target": 100, "label": "Moderate risk"}
211     return {"target": 100, "label": "Moderate risk (default goal pending full risk assessment)"}
212
213
214 def apo_b_assessment(apob, risk_band, secondary_prevention):
215     if apob is None:
216         return None

```

```

217     goal = apo_b_goal(risk_band, secondary_prevention)
218     above_goal = apob > goal["target"]
219     return {"value": apob, "unit": "mg/dL", "target": goal["target"], "goalLabel": goal["label"],
220            "aboveGoal": above_goal, "status": "Advanced core", "source": SOURCE}
221
222
223 def apo_ratio(apob, apoal):
224     if apob is None or apoal is None or apoal <= 0:
225         return None
226     return {"value": round((apob / apoal) * 100) / 100, "status": "Supplementary",
227            "note": "Lower is more favourable. No single universal cut-off – read alongside the
228                    lab's own interpretation.", "source": SOURCE}
229
230 # -----
231 # CV-11 Lp(a) – dual-unit bands, never cross-converted
232 # -----
233 def lpa_band(value, unit="nmol/L"):
234     if value is None:
235         return None
236     if unit == "nmol/L":
237         if value < 75:
238             band, tone = "Lower", "good"
239         elif value < 125:
240             band, tone = "Intermediate", "warn"
241         else:
242             band, tone = "Elevated / risk-enhancing", "bad"
243     else:
244         if value < 30:
245             band, tone = "Lower", "good"
246         elif value < 50:
247             band, tone = "Intermediate", "warn"
248         else:
249             band, tone = "Elevated / risk-enhancing", "bad"
250     return {"value": value, "unit": unit, "band": band, "tone": tone, "status": "Advanced core",
251            "note": "Reported in its own native unit – Lp(a) mass and molar units are not
252                    interconverted with a fixed factor.", "source": SOURCE}
253
254 # -----
255 # CV-12 Metabolic syndrome – any 3 of 5
256 # -----
257 def metabolic_syndrome(inp):
258     criteria = []
259     known = 0
260     met = 0
261     missing = False
262
263     def add(cid, label, value, ok):
264         nonlocal known, met, missing
265         if value is None:
266             criteria.append({"id": cid, "label": label, "status": "unknown"})
267             missing = True
268         return
269         known += 1
270         if ok:
271             met += 1
272         criteria.append({"id": cid, "label": label, "status": "met" if ok else "not-met"})
273
274     waist = inp.get("waist")
275     sex = inp.get("sex")
276     add("waist", "Central obesity (waist)", waist,
277         waist is not None and ((sex == "M" and waist >= 90) or (sex == "F" and waist >= 80)))
278     tg = inp.get("tg")

```

```

279     add("tg", "Triglycerides ≥1.7 mmol/L or treated", tg, tg is not None and (tg >= 1.7 or inp.get(
280         ("tgTreated")))
281     hdl = inp.get("hdl")
282     add("hdl", "Low HDL-C or treated", hdl,
283         hdl is not None and ((sex == "M" and hdl < 1.0) or (sex == "F" and hdl < 1.3) or inp.get("
284             hdlTreated")))
285     sbp, dbp = inp.get("sbp"), inp.get("dbp")
286     bp_met = (sbp is not None and dbp is not None and (sbp >= 130 or dbp >= 85)) or bool(inp.get("
287         bpTreated"))
288     add("bp", "Blood pressure ≥130/85 or treated", sbp, bp_met)
289     glucose = inp.get("glucose")
290     add("glucose", "Fasting glucose ≥5.6 mmol/L or known diabetes", glucose,
291         (glucose is not None and glucose >= 5.6) or bool(inp.get("diabetes")))
292
293     return {
294         "criteriaMet": met, "criteriaKnown": known, "criteria": criteria, "positive": met >= 3,
295         "indeterminate": (met < 3) and missing,
296         "note": "One or more components could not be fully determined – this result cannot be
297             fully confirmed either way." if (missing and met < 3) else None,
298         "source": SOURCE,
299     }
300
301 # -----
302 # CV-13 "Possible FH" referral trigger – never an auto-diagnosis
303 # -----
304 def fh_trigger(ldl, family_history_premature_cvd, xanthomata):
305     flags = []
306     if ldl is not None and ldl >= 4.9:
307         flags.append("Markedly elevated LDL-cholesterol ≥(4.9 mmol/L on this sample).")
308     if family_history_premature_cvd == "Yes":
309         flags.append("Family history of premature cardiovascular disease.")
310     if xanthomata:
311         flags.append("Reported tendon xanthomata.")
312     return {"triggered": len(flags) > 0, "flags": flags,
313         "note": "A single lipid value does not diagnose familial hypercholesterolaemia. This
314             flag routes to clinical assessment; it is not a diagnosis.", "source": SOURCE}
315
316 # -----
317 # CV-14 Triglyceride safety tiers
318 # -----
319 def tg_safety(tg):
320     if tg is None:
321         return None
322     if tg >= 10:
323         return {"tier": "severe", "tone": "bad", "urgency": "urgent", "note": "Severely elevated –
324             urgent clinical assessment is recommended (pancreatitis risk)."}
325     if tg >= 4.5:
326         return {"tier": "high", "tone": "bad", "urgency": "prompt", "note": "Elevated enough that
327             calculated LDL-C is not valid – prompt clinical review is recommended."}
328     if tg >= 1.7:
329         return {"tier": "raised", "tone": "warn", "urgency": "routine", "note": "Raised – consider
330             assessment for metabolic or secondary causes."}
331     return {"tier": "desirable", "tone": "good", "urgency": "none", "note": "Within the desirable
332         range."}
333
334 # -----
335 # Lipid discordance interpretation (spec's worked-example table)
336 # -----
337 def discordance(ldl, apob_assessment, tg_safety_result, hdl, aip_result, lpa_result):
338     notes = []
339     if ldl is not None and ldl < 3.4 and apob_assessment and apob_assessment["aboveGoal"]:

```

```

334     notes.append({"id": "ldl-apob", "title": "Balanced lipid explanation",
335                  "text": "Your LDL-cholesterol is within the stated laboratory range, but
                              ApoB is above the goal associated with your risk category. This means
                              the number of cholesterol-carrying particles may be higher than LDL-
                              cholesterol alone suggests. Any supplementary ratios support the same
                              overall pattern but are not treatment targets on their own."})
336     elif ldl is not None and ldl >= 3.4 and apob_assessment and not apob_assessment["aboveGoal"]:
337         notes.append({"id": "ldl-high-apob-lower", "title": "LDL-cholesterol remains meaningful",
338                      "text": "LDL-cholesterol is above range even though ApoB is closer to goal.
                              LDL-cholesterol should not be dismissed – both are shown together rather
                              than one overriding the other."})
339     if (tg_safety_result and tg_safety_result["tier"] in ("raised", "high") and hdl is not None
        and hdl < 1.0
340         and aip_result and aip_result["band"] != "Lower"):
341         notes.append({"id": "atherogenic-pattern", "title": "Atherogenic / metabolic pattern",
342                      "text": "Raised triglycerides together with lower HDL-cholesterol and a
                              higher supplementary index suggest an atherogenic metabolic pattern.
                              Checking glucose, waist circumference and blood pressure together with
                              this result is recommended."})
343     if ldl is not None and ldl < 3.4 and lpa_result and lpa_result["band"] == "Elevated / risk-
        enhancing":
344         notes.append({"id": "lpa-modifier", "title": "Inherited risk modifier",
345                      "text": "LDL-cholesterol is controlled, but Lp(a) is elevated. Lp(a) is
                              largely inherited and adds residual risk on top of a controlled LDL-
                              cholesterol – this profile should not be read as fully reassuring on its
                              own."})
346     return notes
347
348
349     # -----
350     # Orchestrator
351     # -----
352     def analyze(profile, labs, questionnaire):
353         q = questionnaire or {}
354         sex = profile.get("sex")
355         age = profile.get("age")
356
357         bp = q.get("recentBP") or profile.get("homeBP") or {}
358         home_bp = profile.get("homeBP") or {}
359         sbp = bp.get("sbp") if bp.get("sbp") is not None else home_bp.get("sbp")
360         dbp = bp.get("dbp") if bp.get("dbp") is not None else home_bp.get("dbp")
361         treated = q.get("bpMedication") == "Yes" or home_bp.get("treated") or False
362         smoker = q.get("smokingStatus") == "Current smoker"
363         diabetes = q.get("diagnosedDiabetes") == "Yes"
364         symptoms_now = q.get("symptomsNow") or []
365
366         bp_class = classify_bp(sbp, dbp)
367         escalation = bp_escalation(sbp, dbp, symptoms_now)
368
369         frs = framingham_risk({
370             "age": age, "sex": sex, "tc": labs.get("LAB_TC"), "hdl": labs.get("LAB_HDL"), "sbp": sbp,
371             "treated": treated,
372             "smoker": smoker, "diabetes": diabetes, "hasEstablishedCVD": bool(q.get("establishedCVD")),
373             "pregnancyStatus": q.get("pregnancyStatus"), "acuteSymptoms": bool(escalation),
374         })
375
376         non_hdl_v = non_hdl(labs.get("LAB_TC"), labs.get("LAB_HDL"))
377         ldl_calc = calculated_ldl(labs.get("LAB_TC"), labs.get("LAB_HDL"), labs.get("LAB_TG"))
378         ldl_value = labs.get("LAB_LDL") if labs.get("LAB_LDL") is not None else ldl_calc["value"]
379         ldl_is_measured = labs.get("LAB_LDL") is not None
380
381         aip_result = aip(labs.get("LAB_TG"), labs.get("LAB_HDL"))
382         cl = castelli_i(labs.get("LAB_TC"), labs.get("LAB_HDL"), sex)

```

```

382     c2 = castelli_ii(ldl_value, labs.get("LAB_HDL"), sex, True if ldl_is_measured else ldl_calc["
383         valid"])
384
385     ac = atherogenic_coefficient(labs.get("LAB_TC"), labs.get("LAB_HDL"))
386
387     risk_band = frs["band"] if frs["eligible"] else "Intermediate"
388     apob = apo_b_assessment(labs.get("LAB_APOB"), risk_band, bool(q.get("establishedCVD")))
389     apo_ratio_result = apo_ratio(labs.get("LAB_APOB"), labs.get("LAB_APOA1"))
390     lpa = lpa_band(labs.get("LAB_LPA"), "nmol/L")
391
392     met_syn = metabolic_syndrome({
393         "sex": sex, "waist": profile.get("waist"), "tg": labs.get("LAB_TG"), "hdl": labs.get("
394             LAB_HDL"),
395         "sbp": sbp, "dbp": dbp, "bpTreated": treated, "glucose": labs.get("LAB_GLU_FAST"), "
396             diabetes": diabetes,
397     })
398
399     fh = fh_trigger(ldl_value, q.get("familyHistoryPrematureCVD"), bool(q.get("tendonXanthomata"))
400     )
401     tg_safety_result = tg_safety(labs.get("LAB_TG"))
402     discordance_notes = discordance(ldl_value, apob, tg_safety_result, labs.get("LAB_HDL"),
403         aip_result, lpa)
404
405     urgency_order = ["emergency", "urgent", "prompt", "routine"]
406     urgency = "routine"
407     urgency_reasons = []
408
409     def raise_(level, reason):
410         nonlocal urgency
411         if not level:
412             return
413         urgency_reasons.append({"level": level, "reason": reason})
414         if urgency_order.index(level) < urgency_order.index(urgency):
415             urgency = level
416
417     if escalation:
418         raise_(escalation["urgency"], escalation["reason"])
419     if tg_safety_result and tg_safety_result.get("urgency") and tg_safety_result["urgency"] != "
420         none":
421         raise_(tg_safety_result["urgency"], f"Triglycerides {tg_safety_result['tier']} ({labs.get
422             ('LAB_TG')} mmol/L).")
423     if fh["triggered"]:
424         raise_("prompt", "Possible familial hypercholesterolaemia pathway triggered.")
425     if frs["eligible"] and frs["band"] == "High":
426         raise_("prompt", "High 10-year cardiovascular risk score.")
427     if lpa and lpa["band"] == "Elevated / risk-enhancing":
428         raise_("routine", "Elevated Lp(a) – risk modifier, clinician CV-risk review suggested.")
429     if apob and apob["aboveGoal"]:
430         raise_("routine", "ApoB above the goal linked to your risk category.")
431
432     bp_category_rank = bp_class["rank"] if bp_class else None
433
434     return {
435         "generatedAt": dt.datetime.utcnow().isoformat(),
436         "source": SOURCE, "status": "real",
437         "inputs": {"sbp": sbp, "dbp": dbp, "treated": treated, "smoker": smoker, "diabetes":
438             diabetes, "age": age, "sex": sex},
439         "safety": {"emergencySymptoms": [s for s in symptoms_now if s in EMERGENCY_SYMPTOMS], "
440             escalation": escalation},
441         "bp": bp_class,
442         "framingham": frs,
443         "lipids": {"tc": labs.get("LAB_TC"), "hdl": labs.get("LAB_HDL"), "tg": labs.get("LAB_TG"),
444             "ldl": {"value": ldl_value, "measured": ldl_is_measured, "calc": ldl_calc}, "
445             nonHdl": non_hdl_v},
446         "supplementary": {"aip": aip_result, "castelliI": c1, "castelliII": c2, "

```

```

436     "atherogenicCoefficient": ac},
437     "advanced": {"apoB": apob, "apoRatio": apo_ratio_result, "lpa": lpa},
438     "metabolicSyndrome": met_syn, "fh": fh, "tgSafety": tg_safety_result,
439     "discordanceNotes": discordance_notes, "urgency": urgency, "urgencyReasons":
440         urgency_reasons,
441     "summary": {
442         "bpCategoryRank": bp_category_rank, "bpCategory": bp_class["label"] if bp_class else
443             None,
444         "ldl": ldl_value, "nonHdl": non_hdl_v, "tg": labs.get("LAB_TG"),
445         "apobFlag": bool(apob and apob["aboveGoal"]), "lpaFlag": bool(lpa and lpa["band"] == "
            Elevated / risk-enhancing"),
446         "fhFlag": fh["triggered"], "frsBand": frs["band"] if frs["eligible"] else None, "
            urgency": urgency,
447     },
448 }

```

Listing 5.16: backend/app/engines/cardiovascular.py (full file — the core clinical engine).

5.3.3 app/engines/other_domains.py — the five illustrative domains

Metabolic, Liver, Renal and Haematology all follow the same shape: pull one or two lab codes, classify each via §5.3.4's `classify_by_range()`, derive a boolean flag and an urgency (always "routine" unless a severe band was hit, in which case "prompt"), and return a status: "illustrative" result carrying an explicit note that these thresholds are demo values pending a real domain specification — never rendered without that caveat anywhere in the frontend. Anthropometric is the exception: rather than lab-code classification, it computes BMI from height/weight, categorises it against `ANTHRO_RULES`, and flags waist circumference against a sex-specific Asian-population cut-off (the same cut-off CV-12's metabolic syndrome criterion uses).

```

1  """EVA SQUARE – simplified logic for Metabolic, Liver, Renal, Haematology
2  and Anthropometric domains. Python port of js/engine/otherDomains.js."""
3  from ..data.lab_dictionary import ANTHRO_RULES
4  from .rule_engine import classify_by_range
5
6
7  def analyze_metabolic(labs, q):
8      glucose = classify_by_range("LAB_GLU_FAST", labs.get("LAB_GLU_FAST"))
9      hba1c = classify_by_range("LAB_HBA1C", labs.get("LAB_HBA1C"))
10     glucose_flag = bool((glucose and glucose["tone"] != "good") or (hba1c and hba1c["tone"] != "
11     good") or q.get("diagnosedDiabetes") == "Yes")
12     urgency = "prompt" if ((glucose and glucose["band"] == "severe-high") or (hba1c and hba1c["
13     band"] == "severe-high")) else "routine"
14     return {"status": "illustrative", "domain": "metabolic", "results": [r for r in [glucose,
15     hba1c] if r],
16             "summary": {"glucoseFlag": glucose_flag, "urgency": urgency},
17             "note": "Illustrative demo thresholds – pending a full metabolic domain specification.
18             "}
19
20
21  def analyze_liver(labs, q):
22     alt = classify_by_range("LAB_ALT", labs.get("LAB_ALT"))
23     ast = classify_by_range("LAB_AST", labs.get("LAB_AST"))
24     alt_flag = bool(alt and alt["tone"] != "good")
25     urgency = "prompt" if ((alt and alt["band"] == "severe-high") or (ast and ast["band"] == "
26     severe-high")) else "routine"
27     return {"status": "illustrative", "domain": "liver", "results": [r for r in [alt, ast] if r],
28             "summary": {"altFlag": alt_flag, "urgency": urgency},
29             "note": "Illustrative demo thresholds – pending a full liver domain specification."}
30
31
32  def analyze_renal(labs, q):
33     creat = classify_by_range("LAB_CREAT", labs.get("LAB_CREAT"))
34     egfr = classify_by_range("LAB_EGFR", labs.get("LAB_EGFR"))
35     egfr_flag = bool(egfr and egfr["tone"] != "good")
36     urgency = "prompt" if (egfr and egfr["band"] == "severe-low") else "routine"
37     return {"status": "illustrative", "domain": "renal", "results": [r for r in [creat, egfr] if r
38     ],
39             "summary": {"egfrFlag": egfr_flag, "urgency": urgency},
40             "note": "Illustrative demo thresholds – pending a full renal domain specification."}
41
42
43  def analyze_haematology(labs, q):
44     hb = classify_by_range("LAB_HB", labs.get("LAB_HB"))
45     wbc = classify_by_range("LAB_WBC", labs.get("LAB_WBC"))
46     plt = classify_by_range("LAB_PLT", labs.get("LAB_PLT"))
47     any_flag = any(r and r["tone"] != "good" for r in (hb, wbc, plt))
48     urgency = "prompt" if (hb and hb["band"] == "severe-low") else "routine"
49     return {"status": "illustrative", "domain": "haematology", "results": [r for r in [hb, wbc,
50     plt] if r],
51             "summary": {"anyFlag": any_flag, "urgency": urgency},
52             "note": "Illustrative demo thresholds – pending a full haematology domain

```

```

specification."}
46
47
48 def bmi(height_cm, weight_kg):
49     if not height_cm or not weight_kg:
50         return None
51     m = height_cm / 100
52     return round((weight_kg / (m * m)) * 10) / 10
53
54
55 def bmi_category(value):
56     if value is None:
57         return None
58     for band in ANTHRO_RULES["bmi"]:
59         if value <= band["max"]:
60             return band["label"]
61     return "Obese"
62
63
64 def analyze_anthropometric(profile):
65     value = bmi(profile.get("height"), profile.get("weight"))
66     category = bmi_category(value)
67     cutoff = ANTHRO_RULES["waist"]["women"] if profile.get("sex") == "F" else ANTHRO_RULES["waist"]
68     ]["men"]
69     waist = profile.get("waist")
70     waist_flag = waist is not None and cutoff is not None and waist >= cutoff
71     return {"status": "illustrative", "domain": "anthropometric", "bmi": value, "bmiCategory":
72         category,
73         "waist": waist, "waistCutoff": cutoff, "waistFlag": waist_flag,
74         "summary": {"bmiCategory": category, "waistFlag": waist_flag},
75         "note": "BMI bands are general illustrative ranges; waist cut-off uses Asian
76             population thresholds referenced in the Cardiovascular Domain Specification's
77             metabolic-syndrome criterion."}

```

Listing 5.17: backend/app/engines/other_domains.py (full file).

5.3.4 app/engines/rule_engine.py — generic range classifier

Deliberately free of any clinical knowledge of its own: it only reads the threshold dicts in `data/lab_dictionary.py#SIMPLE_RANGES` and returns a six-band classification (severe-low, mild-low, below-range, within-range, above-range, mild-high, severe-high — collapsed to whichever bands that quantity's threshold dict actually defines), each carrying a human label and a good/warn/bad tone. This is the one function every illustrative domain in §5.3.3 calls through.

```

1  """EVA SQUARE – generic classify-by-range helper. Python port of
2  js/engine/ruleEngine.js. Kept free of clinical knowledge of its own: it only
3  reads the threshold dicts in data/lab_dictionary.py#SIMPLE_RANGES."""
4  from ..data.lab_dictionary import SIMPLE_RANGES, LAB_DICTIONARY
5
6
7  def _band_label(band):
8      return {
9          "severe-low": "Markedly below range", "severe-high": "Markedly above range",
10         "mild-low": "Below range", "mild-high": "Above range",
11         "below-range": "Below range", "above-range": "Above range",
12     }.get(band, "Within range")
13
14
15 def classify_by_range(code, value):
16     r = SIMPLE_RANGES.get(code)
17     meta = LAB_DICTIONARY.get(code)
18     if not r or value is None:

```

```

19     return None
20
21     band, tone = "normal", "good"
22     if r.get("severeLow") is not None and value <= r["severeLow"]:
23         band, tone = "severe-low", "bad"
24     elif r.get("severeHigh") is not None and value >= r["severeHigh"]:
25         band, tone = "severe-high", "bad"
26     elif r.get("mildLow") is not None and value <= r["mildLow"]:
27         band, tone = "mild-low", "warn"
28     elif r.get("mildHigh") is not None and value >= r["mildHigh"]:
29         band, tone = "mild-high", "warn"
30     elif r.get("normalLow") is not None and value < r["normalLow"]:
31         band, tone = "below-range", "warn"
32     elif r.get("normalHigh") is not None and value > r["normalHigh"]:
33         band, tone = "above-range", "warn"
34
35     return {
36         "code": code, "value": value, "band": band, "tone": tone, "label": _band_label(band),
37         "name": meta["name"] if meta else code, "unit": meta["unit"] if meta else "",
38         "source": r.get("source", "Demo threshold – illustrative."), "isIllustrative": True,
39     }

```

Listing 5.18: backend/app/engines/rule_engine.py (full file).

5.3.5 app/engines/safety_engine.py — overall pathway

Aggregates the cardiovascular engine’s own detailed urgency/escalation matrix with the four illustrative domains’ simplified urgency signals into one overall pathway — the worst level across all of them, using the ordering emergency > urgent > prompt > routine > self. This single value drives the safety banner on the Dashboard/Analysis-summary screens, the recommended rung on the Referral screen’s four-level ladder (§6.6.10), and the "View action plan" vs. urgent-guidance branching throughout the app.

```

1  """EVA SQUARE – aggregates the Cardiovascular safety/escalation matrix with
2  the other domains' simplified urgency signals into one overall pathway.
3  Python port of js/engine/safetyEngine.js."""
4  from ..data.recommendations import REFERRAL_LEVELS
5
6  ORDER = ["emergency", "urgent", "prompt", "routine", "self"]
7
8
9  def _worst(levels):
10     best = "self"
11     for level in levels:
12         if not level:
13             continue
14         if ORDER.index(level) < ORDER.index(best):
15             best = level
16     return best
17
18
19  def evaluate(analysis):
20     reasons = []
21     levels = []
22
23     cv = analysis.get("cv")
24     if cv:
25         levels.append(cv["urgency"])
26         for r in cv.get("urgencyReasons", []):
27             reasons.append({"domain": "cardiovascular", "level": r["level"], "reason": r["reason"]
28                             })
29
30     for key in ("metabolic", "liver", "renal", "haematology"):

```

```

30     d = analysis.get(key)
31     if d and d.get("summary", {}).get("urgency") and d["summary"]["urgency"] != "routine":
32         levels.append(d["summary"]["urgency"])
33         reasons.append({"domain": key, "level": d["summary"]["urgency"], "reason": f"
34             Illustrative {key} finding outside the demo reference range."})
35
36     overall = _worst(levels)
37     if overall == "routine" and not reasons:
38         overall = "self"
39
40     level = next((r for r in REFERRAL_LEVELS if r["id"] == overall), REFERRAL_LEVELS[-1])
41     return {"overall": overall, "level": level, "reasons": reasons}

```

Listing 5.19: backend/app/engines/safety_engine.py (full file).

5.3.6 app/engines/recommendation_engine.py

Filters data/recommendations.py#RECOMMENDATIONS by evaluating each entry's when(fh) predicate against the assembled findings bundle, then sorts matches by descending weight. serializable() strips the non-JSON-serialisable when callable before a recommendation is sent over the API — a reminder that this registry is Python-native data (with executable predicates), not JSON content, unlike everything else in app/data/.

```

1  """EVA SQUARE – evaluates data/recommendations.py#RECOMMENDATIONS against the
2  assembled findings dict ("fh"). Python port of js/engine/recommendationEngine.js."""
3  from ..data.recommendations import RECOMMENDATIONS, EDUCATION
4
5
6  def evaluate(fh):
7      if not fh:
8          return []
9      matched = []
10     for r in RECOMMENDATIONS:
11         try:
12             if r["when"](fh):
13                 matched.append(r)
14         except Exception:
15             continue
16     return sorted(matched, key=lambda r: -r["weight"])
17
18
19  def education_for(codes):
20     out = []
21     for c in (codes or []):
22         text = EDUCATION.get(c)
23         if text:
24             out.append({"code": c, "text": text})
25     return out
26
27
28  def serializable(rec):
29     """Strip the non-JSON-serializable `when` callable before sending a
30     recommendation over the API."""
31     return {k: v for k, v in rec.items() if k != "when"}

```

Listing 5.20: backend/app/engines/recommendation_engine.py (full file).

5.3.7 app/engines/report_generator.py

Assembles the printable structured report as an ordered list of {id, title, body} blocks, following the block order the Cardiovascular Domain Specification's own report-generation rules define: safety status,

headline, blood pressure, 10-year risk, core lipids, advanced lipids, supplementary indices, risk modifiers, “why this result” (the discordance notes), actions, follow-up, then one block per illustrative domain. `headline()` is a small standalone function used both here and (mirrored client-side as `headlineFallback()` in §6.6.6, since the backend doesn’t expose it as an isolated field) to produce the one-line summary shown at the top of the Analysis Summary screen.

```

1  """EVA SQUARE – assembles the printable report in the block order defined by
2  the Cardiovascular Domain Specification's report-generation rules. Python
3  port of js/engine/reportGenerator.js."""
4  import datetime as dt
5  from ..data.lab_dictionary import DOMAINS
6
7
8  def headline(cv):
9      if not cv:
10         return "Your report is ready to review."
11     bits = []
12     if cv.get("bp"):
13         bits.append(cv["bp"]["label"].lower() + " blood pressure")
14     ldl = cv.get("lipids", {}).get("ldl", {})
15     if ldl.get("value") is not None:
16         bits.append(("elevated" if ldl["value"] >= 3.4 else "acceptable") + " LDL-cholesterol")
17     if cv.get("framingham", {}).get("eligible"):
18         bits.append(cv["framingham"]["band"].lower() + " estimated 10-year cardiovascular risk")
19     if not bits:
20         return "Your cardiovascular results are summarised below."
21     return "This report shows " + ", ".join(bits) + "."
22
23
24 def build(scenario_label, cv, others, safety, recs):
25     blocks = []
26     blocks.append({"id": "safety", "title": "Safety status",
27                   "body": "No emergency or urgent flags were identified in this report." if
28                           safety["overall"] == "self"
29                           else (safety["level"]["description"] + " " + safety["level"]["action"])}
30     blocks.append({"id": "headline", "title": "Headline", "body": headline(cv)})
31
32     if cv and cv.get("bp"):
33         blocks.append({"id": "bp", "title": "Blood pressure",
34                       "body": f"{cv['bp']['sbp']}/{cv['bp']['dbp']} mmHg – {cv['bp']['label']}. {cv['bp']['note']}"})
35
36     if cv and cv.get("framingham"):
37         frs = cv["framingham"]
38         blocks.append({"id": "risk", "title": "10-year cardiovascular risk",
39                       "body": (f"Estimated at {frs['riskPercent']}% ({frs['band']}). {frs['caution']}" if frs["eligible"]
40                               else ("Not calculated: " + " ".join(frs["reasons"])))})
41
42     if cv and cv.get("lipids"):
43         li = cv["lipids"]
44         ldl_txt = (f"{li['ldl']['value']} mmol/L + (" if li['ldl']['measured'] else " (
45                   calculated)") if li['ldl']['value'] is not None else "not available"
46         blocks.append({"id": "core-lipids", "title": "Core lipids",
47                       "body": f"Total cholesterol {li['tc']} mmol/L, LDL-C {ldl_txt}, HDL-C {li['hdl']} mmol/L, "
48                               f"Triglycerides {li['tg']} mmol/L, Non-HDL-C {li['nonHdl']} mmol/L"
49                               ".")})
50
51     adv = cv.get("advanced", {}) if cv else {}
52     if adv.get("apoB") or adv.get("lpa"):
53         parts = []
54         if adv.get("apoB"):

```

```

52         parts.append(f"ApoB {adv['apoB']['value']} mg/dL (goal <{adv['apoB']['target']} mg/dL
53             for your risk category)")
54     if adv.get("apoRatio"):
55         parts.append(f"ApoB/ApoA-I ratio {adv['apoRatio']['value']}")
56     if adv.get("lpa"):
57         parts.append(f"Lp(a) {adv['lpa']['value']} {adv['lpa']['unit']} ({adv['lpa']['band']})")
58     blocks.append({"id": "advanced-lipids", "title": "Advanced lipids", "body": "; ".join(
59         parts) + "."})
60
61     supp = cv.get("supplementary", {}) if cv else {}
62     if supp:
63         parts = []
64         if supp.get("aip"):
65             parts.append(f"AIP {supp['aip']['value']} ({supp['aip']['band']})")
66         if supp.get("castelliI"):
67             parts.append(f"Castelli I {supp['castelliI']['value']}")
68         if supp.get("castelliII"):
69             parts.append(f"Castelli II {supp['castelliII']['value']}")
70         blocks.append({"id": "supplementary", "title": "Supplementary indices",
71             "body": (" ".join(parts) or "Not available") + ". These are supplementary
72             and do not override the core lipid or risk results above."})
73
74     if cv and (cv["fh"]["triggered"] or cv.get("metabolicSyndrome", {}).get("positive")
75         or (adv.get("lpa") and adv["lpa"]["band"] == "Elevated / risk-enhancing")):
76         parts = []
77         if cv["fh"]["triggered"]:
78             parts.append("Possible familial hypercholesterolaemia pathway flagged: " + " ".join(cv
79                 ["fh"]["flags"]))
80         if cv.get("metabolicSyndrome", {}).get("positive"):
81             parts.append(f"Metabolic syndrome criteria met ({cv['metabolicSyndrome']['criteriaMet
82                 ']} of 5).")
83         if adv.get("lpa") and adv["lpa"]["band"] == "Elevated / risk-enhancing":
84             parts.append("Elevated Lp(a) adds inherited residual risk.")
85         blocks.append({"id": "modifiers", "title": "Risk modifiers", "body": " ".join(parts)})
86
87     if cv and cv.get("discordanceNotes"):
88         blocks.append({"id": "why", "title": "Why this result",
89             "body": "\n\n".join(f"{n['title']} – {n['text']}" for n in cv["
90                 discordanceNotes"])})
91
92     if recs:
93         blocks.append({"id": "actions", "title": "Actions", "body": "\n\n".join("• " + r["action"]
94             for r in recs[:5])})
95
96     blocks.append({"id": "followup", "title": "Follow-up",
97         "body": "Follow the urgent guidance above before your next scheduled review."
98         if safety["overall"] in ("emergency", "urgent")
99         else "Repeat a fasting lipid panel and blood pressure check at your next
100             routine interval, or sooner if advised."})
101
102     if others:
103         for key, d in others.items():
104             if not d:
105                 continue
106             label = DOMAINS.get(key, {}).get("label", key)
107             blocks.append({"id": f"domain-{key}", "title": f"{label} (illustrative)", "body": d.
108                 get("note", "")})
109
110     return {"generatedAt": dt.datetime.utcnow().isoformat(), "scenario": scenario_label, "blocks":
111         blocks}

```

Listing 5.21: backend/app/engines/report_generator.py (full file).

5.3.8 app/engines/trend_engine.py

Two small pure functions — `direction()` (up/down/flat, with a 1% noise threshold so a near-flat series doesn't flicker between up and down) and `delta()` — computed once server-side per trend series so the frontend's Trends screen (§6.6.12) doesn't duplicate this logic. `_numeric()` normalises a series point that may carry either a plain value field or a blood-pressure sbp field into one comparable number.

```

1  """EVA SQUARE – longitudinal trend helpers. Python port of
2  js/engine/trendEngine.js, used by the /api/reports/{id}/trends endpoint so
3  direction/delta are computed once, server-side, rather than duplicated in
4  the frontend."""
5
6
7  def _numeric(point):
8      if point is None:
9          return None
10     if isinstance(point.get("value"), (int, float)):
11         return point["value"]
12     if isinstance(point.get("sbp"), (int, float)):
13         return point["sbp"]
14     return None
15
16
17  def direction(series):
18     if not series or len(series) < 2:
19         return "flat"
20     first = _numeric(series[0])
21     last = _numeric(series[-1])
22     if first is None or last is None:
23         return "flat"
24     delta = last - first
25     if abs(delta) < 0.01 * max(1, abs(first)):
26         return "flat"
27     return "up" if delta > 0 else "down"
28
29
30  def delta(series):
31     if not series or len(series) < 2:
32         return 0
33     a = _numeric(series[0])
34     b = _numeric(series[-1])
35     if a is None or b is None:
36         return 0
37     return round((b - a) * 100) / 100

```

Listing 5.22: backend/app/engines/trend_engine.py (full file).

5.3.9 app/engines/question_engine.py

`domains_present_in_labs()` inspects which health domains actually have a non-null lab value in the report, and `active_sections()` filters `QUESTIONNAIRE_SECTIONS` down to just the sections whose `triggerDomains` overlap that set (an empty trigger list — the Medication & History section — always stays active). `is_complete()` checks whether every question in a section has been answered, including the two-field special case of a bp-pair question. Note that the Questions screen on the frontend maintains its own small client-side copy of `domainsPresentInLabs/activeSections` (§6.6.5) rather than calling this endpoint, since it needs the filtering synchronously against already-cached report data.

```

1  """EVA SQUARE – Adaptive Health Questionnaire logic. Python port of
2  js/engine/questionEngine.js."""
3  from ..data.questionnaire import QUESTIONNAIRE_SECTIONS
4  from ..data.lab_dictionary import LAB_DICTIONARY

```

```
5
6
7 def active_sections(present_domains):
8     domains = present_domains or []
9     return [s for s in QUESTIONNAIRE_SECTIONS if not s["triggerDomains"] or any(d in domains for d
        in s["triggerDomains"])]
10
11
12 def domains_present_in_labs(lab_values):
13     present = set()
14     for code, value in (lab_values or {}).items():
15         meta = LAB_DICTIONARY.get(code)
16         if meta and value is not None:
17             present.add(meta["domain"])
18     return list(present)
19
20
21 def is_complete(section, answers):
22     for q in section["questions"]:
23         v = answers.get(q["id"])
24         if q["type"] == "bp-pair":
25             if not v or v.get("sbp") is None or v.get("dbp") is None:
26                 return False
27         else:
28             if v is None or v == "":
29                 return False
30     return True
```

Listing 5.23: backend/app/engines/question_engine.py (full file).

5.4 Routers (app/routers/)

Each router is a FastAPI `APIRouter` mounted onto `app` in `main.py` under a fixed `/api/*` prefix. Every authenticated endpoint depends on `deps.get_current_user` (§5.1.3); every report-scoped endpoint additionally depends on `deps.get_owned_report`.

5.4.1 app/routers/auth.py

Registration, login, demo-login, logout, and the combined `/me` endpoint (current user + consent + settings in one call, used once at boot by the frontend’s `store.init()`). `demo_login(scenario)` implements a get-or-create pattern for the two seeded demo accounts — the endpoint the frontend’s “Load a routine/urgent sample” shortcuts call, letting a first-time visitor explore the whole application without filling in a registration form. Figure 5.3 traces the full registration sequence across all three tiers.

```

1  """EVA SQUARE – registration / login / demo-login / session endpoints."""
2  from fastapi import APIRouter, Depends, HTTPException
3  from sqlalchemy.orm import Session as DBSession
4
5  from .. import models, schemas, security
6  from ..database import get_db
7  from ..deps import get_current_user
8  from ..data.sample_reports import SAMPLE_SCENARIOS
9
10 router = APIRouter(prefix="/api/auth", tags=["auth"])
11
12 DEMO_PASSWORD = "demo-password" # only ever used for the seeded demo accounts
13
14
15 def _user_out(user: models.User) -> dict:
16     return {"id": user.id, "fullName": user.full_name, "email": user.email}
17
18
19 def _issue_session(db: DBSession, user: models.User) -> str:
20     token = security.new_session_token()
21     db.add(models.Session(token=token, user_id=user.id, expires_at=security.session_expiry()))
22     db.commit()
23     return token
24
25
26 def _ensure_defaults(db: DBSession, user: models.User):
27     existing = db.query(models.ConsentRecord).filter_by(user_id=user.id).first()
28     if not existing:
29         db.add(models.ConsentRecord(user_id=user.id, required_json={}, optional_json={}))
30     if not db.query(models.Settings).filter_by(user_id=user.id).first():
31         db.add(models.Settings(user_id=user.id))
32     db.commit()
33
34
35 @router.post("/register", response_model=schemas.AuthOut)
36 def register(body: schemas.RegisterIn, db: DBSession = Depends(get_db)):
37     if db.query(models.User).filter_by(email=body.email).first():
38         raise HTTPException(status_code=409, detail="An account with this email already exists – try logging in instead.")
39     user = models.User(full_name=body.fullName, email=body.email, password_hash=security.hash_password(body.password))
40     db.add(user)
41     db.commit()
42     db.refresh(user)
43     _ensure_defaults(db, user)
44     token = _issue_session(db, user)
45     return {"token": token, "user": _user_out(user)}
46

```

```

47
48 @router.post("/login", response_model=schemas.AuthOut)
49 def login(body: schemas.LoginIn, db: DBSession = Depends(get_db)):
50     user = db.query(models.User).filter_by(email=body.email).first()
51     if not user or not security.verify_password(body.password, user.password_hash):
52         raise HTTPException(status_code=401, detail="Incorrect email or password.")
53     _ensure_defaults(db, user)
54     token = _issue_session(db, user)
55     return {"token": token, "user": _user_out(user)}
56
57
58 @router.post("/demo-login/{scenario}", response_model=schemas.AuthOut)
59 def demo_login(scenario: str, db: DBSession = Depends(get_db)):
60     """Get-or-create the seeded demo account for `scenario` and log in as
61     them – this is what the "Load a routine/urgent sample" shortcuts call so
62     a visitor can explore the whole app without filling in a registration
63     form first."""
64     sc = SAMPLE_SCENARIOS.get(scenario)
65     if not sc:
66         raise HTTPException(status_code=404, detail="Unknown demo scenario.")
67     email = sc["profile"]["email"]
68     user = db.query(models.User).filter_by(email=email).first()
69     if not user:
70         user = models.User(full_name=sc["profile"]["fullName"], email=email, password_hash=
71             security.hash_password(DEMO_PASSWORD))
72         db.add(user)
73         db.commit()
74         db.refresh(user)
75     _ensure_defaults(db, user)
76     token = _issue_session(db, user)
77     return {"token": token, "user": _user_out(user)}
78
79 @router.post("/logout")
80 def logout(authorization: str = "", db: DBSession = Depends(get_db), user: models.User = Depends(
81     get_current_user)):
82     db.query(models.Session).filter_by(user_id=user.id).delete()
83     db.commit()
84     return {"ok": True}
85
86 @router.get("/me")
87 def me(user: models.User = Depends(get_current_user), db: DBSession = Depends(get_db)):
88     consent = db.query(models.ConsentRecord).filter_by(user_id=user.id).first()
89     settings = db.query(models.Settings).filter_by(user_id=user.id).first()
90     import json
91     return {
92         "user": _user_out(user),
93         "consent": {
94             "required": json.loads(consent.required_json) if consent else {},
95             "optional": json.loads(consent.optional_json) if consent else {},
96             "acceptedTermsVersion": consent.accepted_terms_version if consent else None,
97         },
98         "settings": {"units": settings.units, "notifications": settings.notifications, "theme":
99             settings.theme} if settings else {},
100     }

```

Listing 5.24: backend/app/routers/auth.py (full file).

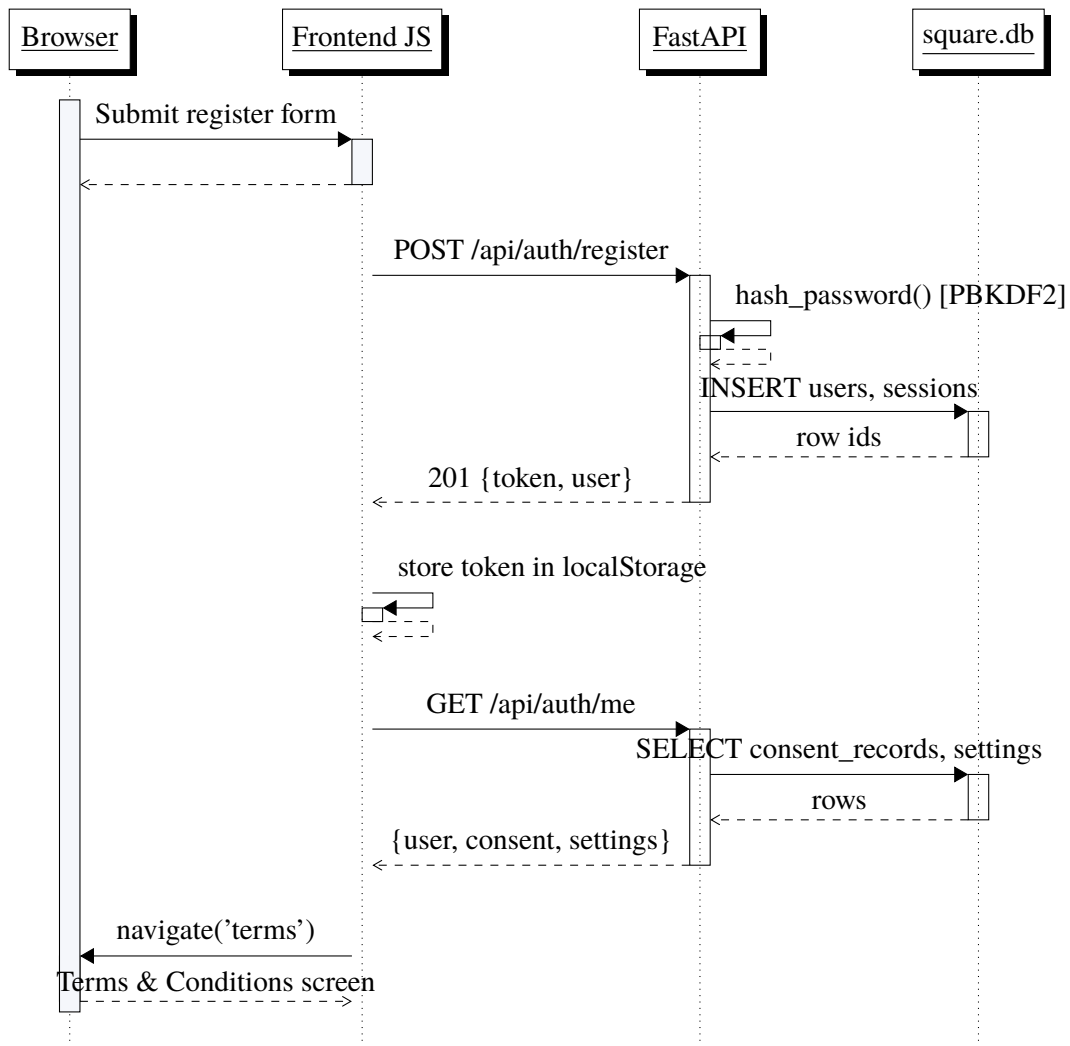


Figure 5.3: Registration sequence: a new account is created, a session token is issued and cached client-side, and the freshly-created (empty) consent/settings rows are fetched before the router hands off to the Terms screen. This is one instance of the general auth pattern — login and demo-login differ only in whether a new user's row is inserted.

5.4.2 app/routers/consent.py

Consent *state* (this file) is kept separate from consent *content* (§5.2.2, served by §5.4.5). `_get_or_create()` lazily creates an empty consent row the first time it's needed; `patch_consent()` updates a single consent item by id within either the required or optional JSON blob and returns whether all required items are now satisfied, which the frontend uses to enable/disable the Consent screen's continue button.

```

1  """EVA SQUARE – consent state endpoints (content lives in routers/meta.py)."""
2  import json
3  from fastapi import APIRouter, Depends
4  from sqlalchemy.orm import Session as DBSession
5
6  from .. import models, schemas
7  from ..database import get_db
8  from ..deps import get_current_user
9  from ..data.legal_content import CONSENT_REQUIRED, ORG
10
11 router = APIRouter(prefix="/api/consent", tags=["consent"])
12
13
14 def _get_or_create(db: DBSession, user: models.User) -> models.ConsentRecord:
15     rec = db.query(models.ConsentRecord).filter_by(user_id=user.id).first()
16     if not rec:
17         rec = models.ConsentRecord(user_id=user.id, required_json="{}", optional_json="{}")
18         db.add(rec)
19         db.commit()
20         db.refresh(rec)
21     return rec
22
23
24 @router.get("")
25 def get_consent(user: models.User = Depends(get_current_user), db: DBSession = Depends(get_db)):
26     rec = _get_or_create(db, user)
27     required = json.loads(rec.required_json)
28     complete = all(required.get(c["id"]) for c in CONSENT_REQUIRED)
29     return {"required": required, "optional": json.loads(rec.optional_json),
30           "acceptedTermsVersion": rec.accepted_terms_version, "requiredComplete": complete}
31
32
33 @router.post("/accept-terms")
34 def accept_terms(user: models.User = Depends(get_current_user), db: DBSession = Depends(get_db)):
35     rec = _get_or_create(db, user)
36     rec.accepted_terms_version = ORG["version"]
37     db.commit()
38     return {"acceptedTermsVersion": rec.accepted_terms_version}
39
40
41 @router.patch("")
42 def patch_consent(body: schemas.ConsentPatch, user: models.User = Depends(get_current_user), db:
43     DBSession = Depends(get_db)):
44     rec = _get_or_create(db, user)
45     field = "required_json" if body.kind == "required" else "optional_json"
46     data = json.loads(getattr(rec, field))
47     data[body.id] = body.value
48     setattr(rec, field, json.dumps(data))
49     db.commit()
50     required = json.loads(rec.required_json)
51     complete = all(required.get(c["id"]) for c in CONSENT_REQUIRED)
52     return {"required": json.loads(rec.required_json), "optional": json.loads(rec.optional_json),
53           "requiredComplete": complete}

```

Listing 5.25: backend/app/routers/consent.py (full file).

5.4.3 app/routers/settings.py

The simplest router in the codebase: get-or-create plus a partial patch over three fields (units, notifications, theme), mirroring the settings table exactly.

```
1  """EVA SQUARE – user settings endpoints."""
2  from fastapi import APIRouter, Depends
3  from sqlalchemy.orm import Session as DBSession
4
5  from .. import models, schemas
6  from ..database import get_db
7  from ..deps import get_current_user
8
9  router = APIRouter(prefix="/api/settings", tags=["settings"])
10
11
12 def _get_or_create(db: DBSession, user: models.User) -> models.Settings:
13     s = db.query(models.Settings).filter_by(user_id=user.id).first()
14     if not s:
15         s = models.Settings(user_id=user.id)
16         db.add(s)
17         db.commit()
18         db.refresh(s)
19     return s
20
21
22 def _out(s: models.Settings) -> dict:
23     return {"units": s.units, "notifications": s.notifications, "theme": s.theme}
24
25
26 @router.get("")
27 def get_settings(user: models.User = Depends(get_current_user), db: DBSession = Depends(get_db)):
28     return _out(_get_or_create(db, user))
29
30
31 @router.patch("")
32 def patch_settings(body: schemas.SettingsPatch, user: models.User = Depends(get_current_user), db:
33     DBSession = Depends(get_db)):
34     s = _get_or_create(db, user)
35     if body.units is not None:
36         s.units = body.units
37     if body.notifications is not None:
38         s.notifications = body.notifications
39     if body.theme is not None:
40         s.theme = body.theme
41     db.commit()
42     return _out(s)
```

Listing 5.26: backend/app/routers/settings.py (full file).

5.4.4 app/routers/reports.py — the core pipeline

The largest and most important router: sample-report seeding, upload, verify, questionnaire read/patch, analyse, and trends, all in one file because they operate on the same reports row through one continuous pipeline. `_create_from_scenario()` is the shared constructor both `/sample` and `/upload` funnel through — the difference between a “sample” and an “upload” in this basis version is purely cosmetic (the uploaded file’s own name is preserved), since there is no live OCR pipeline yet: any uploaded file is matched to the routine sample lab panel so the Verify screen still has something meaningful to check.

Where a real OCR pipeline would plug in

`upload_report()`’s docstring is explicit about this simplification. A production version would replace the call to `_create_from_scenario(db, user, "routine", ...)` with an actual extraction step (PDF text-layer parsing, falling back to OCR for scanned images), map the extracted text onto `LAB_DICTIONARY` codes (fuzzy-matching against each code’s `altNames`), and still route through the same Verify screen for human confirmation before anything downstream trusts the values. The rest of the pipeline — verify, questionnaire, analyse, trends — is completely unaffected by this change.

`_run_and_store_analysis()` is the bridge into §5.3.1: it reads back the verified labs and questionnaire answers, calls `orchestrator.run_analysis()`, and upserts the single `analysis_results` row. `get_analysis()` computes this *lazily* on first request if no cached row exists yet (as long as the report is verified) — so the frontend never has to explicitly call `POST /analyze` before reading analysis, only after a questionnaire is confirmed (where `SQ.act.confirmQuestions()` calls it explicitly to trigger the loading-screen transition; see §6.3.1).

```

1  """EVA SQUARE – reports: sample seeding / upload / verify / questionnaire /
2  analysis / trends. This is the core clinical-data pipeline endpoint group."""
3  import json
4  from typing import Optional
5  from fastapi import APIRouter, Depends, HTTPException, UploadFile, File
6  from sqlalchemy.orm import Session as DBSession
7
8  from .. import models, schemas
9  from ..database import get_db
10 from ..deps import get_current_user, get_owned_report
11 from ..data.sample_reports import SAMPLE_SCENARIOS
12 from ..engines import orchestrator, trend_engine
13
14 router = APIRouter(prefix="/api/reports", tags=["reports"])
15
16
17 def _report_out(r: models.Report, include_original_text=True) -> dict:
18     out = {
19         "id": r.id, "fileName": r.file_name, "lab": r.lab, "reportDate": r.report_date, "status":
20             r.status,
21         "profile": json.loads(r.profile_json), "labsRaw": json.loads(r.labs_raw_json),
22         "labsVerified": json.loads(r.labs_verified_json) if r.labs_verified_json else None,
23         "labConfidence": json.loads(r.lab_confidence_json), "trendHistory": json.loads(r.
24             trend_history_json),
25         "uploadedAt": r.uploaded_at.isoformat() if r.uploaded_at else None,
26     }
27     if include_original_text:
28         out["originalText"] = r.original_text
29     return out
30
31 @router.get("")
32 def list_reports(user: models.User = Depends(get_current_user), db: DBSession = Depends(get_db)):
33     reports = db.query(models.Report).filter_by(user_id=user.id).order_by(models.Report.
34         uploaded_at.desc()).all()

```

```

33     return [_report_out(r, include_original_text=False) for r in reports]
34
35
36 @router.get("/{report_id}")
37 def get_report(report_id: str, user: models.User = Depends(get_current_user), db: DBSession =
38     Depends(get_db)):
39     r = get_owned_report(report_id, db, user)
40     return _report_out(r)
41
42 def _create_from_scenario(db: DBSession, user: models.User, scenario_key: str, file_name: Optional
43     [str], full_pipeline: bool) -> models.Report:
44     sc = SAMPLE_SCENARIOS.get(scenario_key)
45     if not sc:
46         raise HTTPException(status_code=404, detail="Unknown demo scenario.")
47     report = models.Report(
48         user_id=user.id, file_name=file_name or sc["fileName"], lab=sc["lab"], report_date=sc["
49         reportDate"],
50         status="verified" if full_pipeline else "pending-verify",
51         profile_json=json.dumps(sc["profile"]), labs_raw_json=json.dumps(sc["labs"]),
52         labs_verified_json=json.dumps(sc["labs"]) if full_pipeline else None,
53         lab_confidence_json=json.dumps(sc["labConfidence"]), original_text=sc["originalText"],
54         trend_history_json=json.dumps(sc["trendHistory"]),
55     )
56     db.add(report)
57     db.commit()
58     db.refresh(report)
59     db.add(models.QuestionnaireAnswer(report_id=report.id, answers_json=json.dumps(sc["
60     questionnaireDefaults"])))
61     db.commit()
62     if full_pipeline:
63         _run_and_store_analysis(db, report)
64     return report
65
66 @router.post("/sample")
67 def create_sample_report(body: schemas.SampleReportIn, user: models.User = Depends(
68     get_current_user), db: DBSession = Depends(get_db)):
69     report = _create_from_scenario(db, user, body.scenario, body.fileName, body.fullPipeline)
70     return _report_out(report)
71
72 @router.post("/upload")
73 def upload_report(file: UploadFile = File(...), user: models.User = Depends(get_current_user), db:
74     DBSession = Depends(get_db)):
75     """This basis version has no live OCR pipeline – any uploaded file is
76     matched to the routine sample lab panel (see README) so Verify still has
77     something meaningful to check, and the file's own name is preserved."""
78     report = _create_from_scenario(db, user, "routine", file.filename, full_pipeline=False)
79     return _report_out(report)
80
81 @router.post("/{report_id}/verify")
82 def verify_report(report_id: str, body: schemas.VerifyIn, user: models.User = Depends(
83     get_current_user), db: DBSession = Depends(get_db)):
84     r = get_owned_report(report_id, db, user)
85     current = json.loads(r.labs_verified_json) if r.labs_verified_json else json.loads(r.
86     labs_raw_json)
87     current.update(body.values)
88     r.labs_verified_json = json.dumps(current)
89     r.status = "verified"
90     db.commit()
91     return _report_out(r)

```

```

89
90 @router.get("/{report_id}/questionnaire")
91 def get_questionnaire(report_id: str, user: models.User = Depends(get_current_user), db: DBSession
    = Depends(get_db)):
92     r = get_owned_report(report_id, db, user)
93     qa = db.query(models.QuestionnaireAnswer).filter_by(report_id=r.id).first()
94     return {"answers": json.loads(qa.answers_json) if qa else {}}
95
96
97 @router.patch("/{report_id}/questionnaire")
98 def patch_questionnaire(report_id: str, body: schemas.AnswersPatch, user: models.User = Depends(
    get_current_user), db: DBSession = Depends(get_db)):
99     r = get_owned_report(report_id, db, user)
100     qa = db.query(models.QuestionnaireAnswer).filter_by(report_id=r.id).first()
101     if not qa:
102         qa = models.QuestionnaireAnswer(report_id=r.id, answers_json="{}")
103         db.add(qa)
104     answers = json.loads(qa.answers_json) if qa.answers_json else {}
105     answers.update(body.patch)
106     qa.answers_json = json.dumps(answers)
107     db.commit()
108     return {"answers": answers}
109
110
111 def _run_and_store_analysis(db: DBSession, r: models.Report) -> dict:
112     profile = json.loads(r.profile_json)
113     labs = json.loads(r.labs_verified_json) if r.labs_verified_json else json.loads(r.
        labs_raw_json)
114     qa = db.query(models.QuestionnaireAnswer).filter_by(report_id=r.id).first()
115     answers = json.loads(qa.answers_json) if qa else {}
116     result = orchestrator.run_analysis(r.file_name, profile, labs, answers)
117
118     existing = db.query(models.AnalysisResult).filter_by(report_id=r.id).first()
119     if existing:
120         existing.result_json = json.dumps(result)
121     else:
122         db.add(models.AnalysisResult(report_id=r.id, result_json=json.dumps(result)))
123     db.commit()
124     return result
125
126
127 @router.post("/{report_id}/analyze")
128 def analyze_report(report_id: str, user: models.User = Depends(get_current_user), db: DBSession =
    Depends(get_db)):
129     r = get_owned_report(report_id, db, user)
130     if r.status != "verified":
131         raise HTTPException(status_code=400, detail="Verify the report's lab values before running
            analysis.")
132     return _run_and_store_analysis(db, r)
133
134
135 @router.get("/{report_id}/analysis")
136 def get_analysis(report_id: str, user: models.User = Depends(get_current_user), db: DBSession =
    Depends(get_db)):
137     r = get_owned_report(report_id, db, user)
138     existing = db.query(models.AnalysisResult).filter_by(report_id=r.id).first()
139     if existing:
140         return json.loads(existing.result_json)
141     if r.status != "verified":
142         raise HTTPException(status_code=400, detail="No analysis yet – verify the report first.")
143     return _run_and_store_analysis(db, r)
144
145
146 @router.get("/{report_id}/trends")

```

```
147 def get_trends(report_id: str, user: models.User = Depends(get_current_user), db: DBSession =  
    Depends(get_db)):  
148     r = get_owned_report(report_id, db, user)  
149     history = json.loads(r.trend_history_json)  
150     out = {}  
151     for key, series in history.items():  
152         out[key] = {"series": series, "direction": trend_engine.direction(series), "delta":  
            trend_engine.delta(series)}  
153     return out
```

Listing 5.27: backend/app/routers/reports.py (full file).

5.4.5 app/routers/meta.py

Eight endpoints, none of which depend on the logged-in user — purely static reference content re-exported from app/data/. Fetched exactly once, in parallel, by store.init() at application boot (§6.2.2), then bound onto plain SQ.* globals so every screen ported from the original single-file prototype can reference SQ.DOMAINS, SQ.labDictionary, etc. with no further changes.

```

1  """EVA SQUARE – static reference content endpoints: domains, lab dictionary,
2  questionnaire structure, legal text, reference scales, referral ladder,
3  education snippets. None of this depends on the logged-in user."""
4  from fastapi import APIRouter
5
6  from ..data.lab_dictionary import DOMAINS, DOMAIN_ORDER, LAB_DICTIONARY, SIMPLE_RANGES,
7  ANTHRO_RULES
8  from ..data.questionnaire import QUESTIONNAIRE_SECTIONS
9  from ..data.legal_content import ORG, TERMS_SECTIONS, CONSENT_REQUIRED, CONSENT_OPTIONAL
10 from ..data.recommendations import EDUCATION, REFERRAL_LEVELS
11 from ..data.reference_scales import get_all_scales
12 from ..data.sample_reports import SAMPLE_SCENARIOS
13
14 router = APIRouter(prefix="/api/meta", tags=["meta"])
15
16 @router.get("/domains")
17 def domains():
18     return {"domains": DOMAINS, "order": DOMAIN_ORDER}
19
20
21 @router.get("/lab-dictionary")
22 def lab_dictionary():
23     return {"labs": LAB_DICTIONARY, "simpleRanges": SIMPLE_RANGES, "anthroRules": ANTHRO_RULES}
24
25
26 @router.get("/questionnaire-sections")
27 def questionnaire_sections():
28     return {"sections": QUESTIONNAIRE_SECTIONS}
29
30
31 @router.get("/legal")
32 def legal():
33     return {"org": ORG, "terms": TERMS_SECTIONS, "consentRequired": CONSENT_REQUIRED, "
34           consentOptional": CONSENT_OPTIONAL}
35
36
37 @router.get("/reference-scales")
38 def reference_scales():
39     return get_all_scales()
40
41
42 @router.get("/referral-levels")
43 def referral_levels():
44     return {"levels": REFERRAL_LEVELS}
45
46
47 @router.get("/education")
48 def education():
49     return {"education": EDUCATION}
50
51
52 @router.get("/sample-scenarios")
53 def sample_scenarios():
54     """Lightweight listing (no lab values) for the "try a sample" buttons."""
55     return {k: {"key": v["key"], "label": v["label"], "profileName": v["profile"]["fullName"]} for
56             k, v in SAMPLE_SCENARIOS.items()}

```

Listing 5.28: backend/app/routers/meta.py (full file).

5.4.6 app/routers/action_plan.py

Global, per-user completion state for recommendation checklist items — not scoped to a report, matching the frontend store’s `actionPlanDone` shape (a flat `{code: bool}` map). `toggle()` flips a code’s done state, creating the row on first toggle courtesy of the `UNIQUE(user_id, rec_code)` constraint on `action_plan_items` (§4).

```

1  """EVA SQUARE – action plan completion-state endpoints (global per user,
2  matching frontend-v2's store.js#actionPlanDone shape: a flat {code: bool})."""
3  from fastapi import APIRouter, Depends
4  from sqlalchemy.orm import Session as DBSession
5
6  from .. import models, schemas
7  from ..database import get_db
8  from ..deps import get_current_user
9
10 router = APIRouter(prefix="/api/action-plan", tags=["action-plan"])
11
12 @router.get("")
13 def get_action_plan(user: models.User = Depends(get_current_user), db: DBSession = Depends(get_db)
14 ):
15     items = db.query(models.ActionPlanItem).filter_by(user_id=user.id).all()
16     return {"done": {i.rec_code: i.done for i in items}}
17
18 @router.post("/toggle")
19 def toggle(body: schemas.ActionToggleIn, user: models.User = Depends(get_current_user), db:
20     DBSession = Depends(get_db)):
21     item = db.query(models.ActionPlanItem).filter_by(user_id=user.id, rec_code=body.code).first()
22     if not item:
23         item = models.ActionPlanItem(user_id=user.id, rec_code=body.code, done=True)
24         db.add(item)
25     else:
26         item.done = not item.done
27     db.commit()
28     items = db.query(models.ActionPlanItem).filter_by(user_id=user.id).all()
29     return {"done": {i.rec_code: i.done for i in items}}

```

Listing 5.29: backend/app/routers/action_plan.py (full file).

Frontend Tier

6.1.1 index.html — document shell and script load order

```

1 <!doctype html>
2 <html lang="en">
3 <head>
4 <meta charset="utf-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1">
6 <title>EVA SQUARE – Health Interpretation Platform</title>
7 <link rel="icon" href="/assets/EVA%20SQUARE.png">
8 <link rel="stylesheet" href="/css/style.css">
9 </head>
10 <body>
11 <div id="root"><div class="loading-state"><div class="spinner"></div>Loading EVA ...SQUARE</div></div>
12
13 <!-- Data layer: fetch wrapper + async cache store -->
14 <script src="/js/api.js"></script>
15 <script src="/js/store.js"></script>
16
17 <!-- UI layer: components, chrome, and the three FullStructure feature modules -->
18 <script src="/js/ui/components.js"></script>
19 <script src="/js/ui/shell.js"></script>
20 <script src="/js/ui/anatomy.js"></script>
21 <script src="/js/ui/scaleRow.js"></script>
22 <script src="/js/ui/trendChart.js"></script>
23 <script src="/js/ui/router.js"></script>
24
25 <!-- Screens (order doesn't matter – each registers itself on SQ.screens.*) -->
26 <script src="/js/ui/screens/auth.js"></script>
27 <script src="/js/ui/screens/dashboard.js"></script>
28 <script src="/js/ui/screens/upload.js"></script>
29 <script src="/js/ui/screens/verify.js"></script>
30 <script src="/js/ui/screens/questions.js"></script>
31 <script src="/js/ui/screens/analysis.js"></script>
32 <script src="/js/ui/screens/results.js"></script>
33 <script src="/js/ui/screens/domain.js"></script>
34 <script src="/js/ui/screens/actionPlan.js"></script>
35 <script src="/js/ui/screens/referral.js"></script>
36 <script src="/js/ui/screens/report.js"></script>
37 <script src="/js/ui/screens/trends.js"></script>
38 <script src="/js/ui/screens/profile.js"></script>
39 <script src="/js/ui/screens/reportsLibrary.js"></script>
40 <script src="/js/ui/screens/settings.js"></script>
41
42 <!-- Bootstrap: must load last -->
43 <script src="/js/app.js"></script>
44 </body>
45 </html>

```

Listing 6.1: frontend/index.html (full file).

6.1.2 css/style.css — design tokens and component styles

One 569-line stylesheet, no preprocessor, organised top-to-bottom exactly as Table 6.1 lists it. The file opens with a `:root` block of CSS custom properties — the same palette reused as this manual’s own colour scheme (§1) — so every tone (**good**/**warn**/**bad**), every radius, and every shadow is defined exactly once and referenced everywhere via `var(--...)`.

```

1 /* =====
2 EVA SQUARE – Prototype 1, Version 2 (FullStructure)
3 Design tokens + component styles matching the Screen/ mockups
4 ===== */
5
6 :root {

```

Section	Styles
Custom properties	Colour palette (navy/teal/-good/warn/bad/info + tints), radii, card shadow.
Buttons, Forms	.btn variants, text inputs, .choice-row/.choice-btn, .toggle switch.
Cards / layout primitives	.card, .grid, .stack, .card-row.
Pills / badges, Banners	.pill tones, .callout, .disclaimer-banner.
Auth shell	Welcome/Register/Login layout, marketing strip, legal (Terms/Consent) stepper and two-column layout.
App shell	.app-sidebar, .app-topline, user menu, .section-title.
Stepper, Tabs	The four-step Upload/Verify/Answer/Analyse stepper (§1.5); domain-detail tab bar.
Domain grid / body summary, Safety ladder, Timeline, Progress ring	Domain overview cards; the four-level referral ladder (§6.6.10); the single-marker trend timeline; the loading-screen progress ring.
Upload dropzone, Tables, Misc, Toast	Drag/drop zone; .data-table; assorted small utility classes; toast notifications.
Anatomy board	Requirement #1 — §6.5.1.
Expandable reference-scale rows	Requirement #2 — §6.5.2.
Trends enlarge modal	Requirement #4 — §6.5.3.

Table 6.1: Top-to-bottom section map of style.css, in file order.

```

7  --navy: #0b2545;
8  --navy-2: #14213d;
9  --ink: #101828;
10 --muted: #64748b;
11 --muted-2: #8792a2;
12 --border: #e3e7ef;
13 --border-2: #eef1f6;
14 --bg: #f5f7fb;
15 --card: #ffffff;
16
17 --teal: #0f8c7f;
18 --teal-dark: #0b6e63;
19 --teal-light: #e6f5f3;
20 --teal-lighter: #f0faf8;
21
22 --good: #2f9e44;
23 --good-bg: #eaf7ec;
24 --good-border: #cdeed3;
25
26 --warn: #b5650a;
27 --warn-bg: #fdf1de;
28 --warn-border: #f6dfb4;
29
30 --bad: #c0392b;
31 --bad-bg: #fbe4e4;
32 --bad-border: #f3c6c6;
33
34 --info: #2e5aac;
35 --info-bg: #eaf0fb;
36 --info-border: #cddaf3;
37
38 --radius-lg: 14px;

```

```

39  --radius-md: 10px;
40  --radius-sm: 7px;
41  --shadow-card: 0 1px 2px rgba(16, 24, 40, 0.04), 0 1px 3px rgba(16, 24, 40, 0.06);
42  }
43
44  * { box-sizing: border-box; }
45  html, body { height: 100%; }
46  body {
47    margin: 0;
48    font-family: -apple-system, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
49    color: var(--ink);
50    background: var(--bg);
51    line-height: 1.5;
52    -webkit-font-smoothing: antialiased;
53  }
54  a { color: var(--teal); text-decoration: none; }
55  a:hover { text-decoration: underline; }
56  h1, h2, h3, h4 { color: var(--navy); margin: 0; font-weight: 800; }
57  p { margin: 0; }
58  button { font-family: inherit; cursor: pointer; }
59  input, select, textarea { font-family: inherit; }
60  ul { margin: 0; padding: 0; list-style: none; }
61  ::-webkit-scrollbar { width: 10px; height: 10px; }
62  ::-webkit-scrollbar-thumb { background: #d3d9e3; border-radius: 6px; }
63
64  .brand { display: flex; align-items: center; gap: 10px; }
65  .brand-logo { height: 38px; width: auto; display: block; flex-shrink: 0; }
66  .brand.sm .brand-logo { height: 28px; }
67  .brand-text { display: flex; flex-direction: column; line-height: 1.15; }
68  .brand-name { font-size: 1.35rem; font-weight: 800; letter-spacing: 0.06em; color: var(--navy); }
69  .brand-tag { font-size: 0.62rem; color: var(--muted); letter-spacing: 0.02em; }
70  .brand.sm .brand-name { font-size: 1.05rem; }
71
72  /* ----- Buttons ----- */
73  .btn {
74    display: inline-flex; align-items: center; justify-content: center; gap: 8px;
75    border-radius: var(--radius-sm); border: 1px solid transparent;
76    padding: 10px 18px; font-size: 0.92rem; font-weight: 700; white-space: nowrap;
77    transition: background .12s, border-color .12s, opacity .12s, transform .12s, box-shadow .12s;
78  }
79  .btn.primary { background: var(--teal); color: #fff; }
80  .btn.primary:hover { background: var(--teal-dark); transform: translateY(-1px); box-shadow: 0 4px 10px rgba(15,140,127,.25); }
81  .btn.primary.disabled { background: #a9c9c5; cursor: not-allowed; }
82  .btn.navy { background: var(--navy); color: #fff; }
83  .btn.navy:hover { background: #081a34; transform: translateY(-1px); box-shadow: 0 4px 10px rgba(11,37,69,.3); }
84  .btn.outline { background: #fff; color: var(--navy); border-color: var(--border); }
85  .btn.outline:hover { border-color: var(--navy); background: #f8fafc; }
86  .btn.ghost { background: transparent; color: var(--teal); border-color: transparent; }
87  .btn.ghost:hover { background: var(--teal-lighter); }
88  .btn.danger-ghost { background: transparent; color: var(--bad); }
89  .btn.danger-ghost:hover { background: var(--bad-bg); }
90  .btn.sm { padding: 6px 12px; font-size: 0.82rem; }
91  .btn.block { width: 100%; }
92  .btn.disabled { opacity: .6; cursor: not-allowed; transform: none !important; box-shadow: none !important; }
93  .icon { width: 1em; height: 1em; display: inline-block; }
94
95  /* ----- Forms ----- */
96  .field { display: flex; flex-direction: column; gap: 6px; margin-bottom: 16px; }
97  .field label { font-size: 0.85rem; font-weight: 700; color: var(--navy-2); }
98  .field .hint { font-size: 0.78rem; color: var(--muted); font-weight: 400; }
99  .field input[type=text], .field input[type=email], .field input[type=password],

```

```

100 .field input[type=number], .field input[type=date], .field select, .field textarea {
101   border: 1px solid var(--border); border-radius: var(--radius-sm);
102   padding: 11px 13px; font-size: 0.93rem; color: var(--ink); background: #fff;
103   width: 100%; transition: border-color .12s, box-shadow .12s;
104 }
105 .field input:focus, .field select:focus, .field textarea:focus {
106   outline: none; border-color: var(--teal); box-shadow: 0 0 3px var(--teal-light);
107 }
108 .field-pw { position: relative; }
109 .field-pw button { position: absolute; right: 10px; top: 50%; transform: translateY(-50%);
110   background: none; border: none; color: var(--muted); font-size: 0.8rem; }
111 .checkbox-row, .radio-row { display: flex; align-items: flex-start; gap: 10px; }
112 .checkbox-row input, .radio-row input { margin-top: 3px; width: 16px; height: 16px; accent-color:
113   var(--teal); flex-shrink: 0; }
114 .form-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 4px
115   20px; }
116 .segmented { display: flex; border: 1px solid var(--border); border-radius: var(--radius-sm);
117   overflow: hidden; }
118 .segmented button { flex: 1; padding: 10px; background: #fff; border: none; font-size: 0.88rem;
119   font-weight: 600; color: var(--muted); border-right: 1px solid var(--border); transition:
120   background .12s, color .12s; }
121 .segmented button:last-child { border-right: none; }
122 .segmented button:hover { background: #f4f6f9; }
123 .segmented button.active { background: var(--teal-lighter); color: var(--teal-dark); }
124 .choice-row { display: grid; grid-template-columns: repeat(auto-fit, minmax(140px, 1fr)); gap: 10px
125   ; }
126 .choice-btn {
127   border: 1.5px solid var(--border); border-radius: var(--radius-sm); background: #fff;
128   padding: 12px 14px; text-align: left; font-size: 0.9rem; font-weight: 600; color: var(--ink);
129   display: flex; align-items: center; gap: 10px; transition: border-color .12s, background .12s,
130   transform .12s;
131 }
132 .choice-btn:hover { border-color: #b9c2d0; transform: translateY(-1px); }
133 .choice-btn .dot { width: 15px; height: 15px; border-radius: 50%; border: 2px solid var(--border
134   -2); flex-shrink: 0; position: relative; }
135 .choice-btn.selected { border-color: var(--teal); background: var(--teal-lighter); color: var(--
136   teal-dark); }
137 .choice-btn.selected .dot { border-color: var(--teal); }
138 .choice-btn.selected .dot::after { content: ""; position: absolute; inset: 3px; border-radius:
139   50%; background: var(--teal); }
140 .toggle { position: relative; display: inline-block; width: 42px; height: 24px; flex-shrink: 0;
141   cursor: pointer; }
142 .toggle input { opacity: 0; width: 0; height: 0; }
143 .toggle .track { position: absolute; inset: 0; background: #ccd3de; border-radius: 999px;
144   transition: .15s; }
145 .toggle .track::after { content: ""; position: absolute; width: 18px; height: 18px; left: 3px; top
146   : 3px; background: #fff; border-radius: 50%; transition: .15s; box-shadow: 0 1px 2px rgba
147   (0,0,0,.25); }
148 .toggle input:checked + .track { background: var(--teal); }
149 .toggle input:checked + .track::after { transform: translateX(18px); }
150 .toggle:hover .track { box-shadow: 0 0 4px rgba(15,140,127,.12); }
151
152 /* ----- Cards / layout primitives ----- */
153 .card { background: var(--card); border: 1px solid var(--border); border-radius: var(--radius-lg);
154   padding: 22px 24px; box-shadow: var(--shadow-card); }
155 .card.pad-sm { padding: 16px 18px; }
156 .card-row { display: flex; align-items: flex-start; justify-content: space-between; gap: 14px; }
157 .grid { display: grid; gap: 18px; }
158 .grid.cols-2 { grid-template-columns: repeat(2, 1fr); }
159 .grid.cols-3 { grid-template-columns: repeat(3, 1fr); }
160 .grid.cols-4 { grid-template-columns: repeat(4, 1fr); }
161 .grid.cols-6 { grid-template-columns: repeat(6, 1fr); }
162 .grid.responsive { grid-template-columns: repeat(auto-fit, minmax(230px, 1fr)); }
163 @media (max-width: 1080px) {

```

```

148 .grid.cols-2, .grid.cols-3, .grid.cols-4, .grid.cols-6 { grid-template-columns: repeat(auto-fit,
    minmax(220px,1fr)); }
149 }
150 .stack { display: flex; flex-direction: column; gap: 16px; }
151 .hr { border: none; border-top: 1px solid var(--border-2); margin: 14px 0; }
152
153 /* ----- Pills / badges ----- */
154 .pill {
155   display: inline-flex; align-items: center; gap: 5px; padding: 4px 11px;
156   border-radius: 999px; font-size: 0.76rem; font-weight: 700; white-space: nowrap;
157   border: 1px solid transparent;
158 }
159 .pill.good { background: var(--good-bg); color: var(--good); border-color: var(--good-border); }
160 .pill.warn { background: var(--warn-bg); color: var(--warn); border-color: var(--warn-border); }
161 .pill.bad { background: var(--bad-bg); color: var(--bad); border-color: var(--bad-border); }
162 .pill.info { background: var(--info-bg); color: var(--info); border-color: var(--info-border); }
163 .pill.neutral { background: #f1f3f7; color: var(--muted); border-color: var(--border); }
164 .pill.teal { background: var(--teal-light); color: var(--teal-dark); border-color: #bfe6e1; }
165 .dot-ind { width: 8px; height: 8px; border-radius: 50%; display: inline-block; }
166 .dot-ind.good { background: var(--good); } .dot-ind.warn { background: var(--warn); } .dot-ind.bad
    { background: var(--bad); } .dot-ind.info { background: var(--info); }
167
168 /* ----- Banners / callouts ----- */
169 .callout { display: flex; gap: 12px; align-items: flex-start; padding: 14px 16px; border-radius:
    var(--radius-md); border: 1px solid; font-size: 0.86rem; }
170 .callout.info { background: var(--info-bg); border-color: var(--info-border); color: #1e3a70; }
171 .callout.warn { background: var(--warn-bg); border-color: var(--warn-border); color: #7a4406; }
172 .callout.bad { background: var(--bad-bg); border-color: var(--bad-border); color: #7d1f16; }
173 .callout.good { background: var(--good-bg); border-color: var(--good-border); color: #1c6b2c; }
174 .callout .ic { flex-shrink: 0; width: 20px; height: 20px; border-radius: 50%; display: flex; align
    -items: center; justify-content: center; font-size: 0.78rem; font-weight: 800; color: #fff; }
175 .callout.info .ic { background: var(--info); } .callout.warn .ic { background: var(--warn); } .
    callout.bad .ic { background: var(--bad); } .callout.good .ic { background: var(--good); }
176 .callout .callout-title { font-weight: 700; margin-bottom: 2px; }
177 .disclaimer-banner {
178   background: #fff7e0; border-top: 1px solid #eecf7a; color: #6b5300; font-size: 0.8rem;
179   padding: 10px 24px; display: flex; align-items: center; gap: 8px; justify-content: center; text-
    align: center;
180 }
181
182 /* ----- Auth shell (Welcome / Register / Login) ----- */
183 .auth-topbar {
184   display: flex; align-items: center; justify-content: space-between;
185   padding: 18px 40px; border-bottom: 1px solid var(--border-2);
186 }
187 .auth-topbar nav { display: flex; gap: 26px; align-items: center; }
188 .auth-topbar nav a { color: var(--navy-2); font-weight: 600; font-size: 0.9rem; }
189 .auth-hero {
190   display: grid; grid-template-columns: 1fr 1fr; gap: 60px; align-items: center;
191   max-width: 1320px; margin: 0 auto; padding: 56px 40px 70px;
192 }
193 .auth-hero h1 { font-size: 2.9rem; line-height: 1.12; letter-spacing: -0.01em; }
194 .auth-hero .lede { font-size: 1.05rem; color: var(--muted); margin-top: 18px; max-width: 480px; }
195 .auth-hero .cta-row { display: flex; gap: 12px; margin-top: 26px; flex-wrap: wrap; }
196 .auth-hero .trust-row { display: flex; gap: 22px; margin-top: 22px; flex-wrap: wrap; font-size:
    0.85rem; color: var(--muted); }
197 .auth-card { background: #fff; border: 1px solid var(--border); border-radius: var(--radius-lg);
    padding: 32px 36px; box-shadow: var(--shadow-card); max-width: 460px; margin-left: auto; }
198 .auth-card h2 { font-size: 1.55rem; }
199 .auth-card .sub { color: var(--muted); font-size: 0.9rem; margin-top: 6px; margin-bottom: 20px; }
200 .auth-progress { height: 4px; background: var(--border-2); border-radius: 999px; margin-bottom: 18
    px; overflow: hidden; }
201 .auth-progress > div { height: 100%; background: var(--teal); }
202 .auth-divider { display: flex; align-items: center; gap: 12px; color: var(--muted-2); font-size:

```

```

0.8rem; margin: 16px 0; }
203 .auth-divider::before, .auth-divider::after { content: ""; flex: 1; height: 1px; background: var
    (--border); }
204 .auth-foot { text-align: center; font-size: 0.85rem; color: var(--muted); margin-top: 14px; }
205 .auth-secure-note { display: flex; align-items: center; gap: 6px; justify-content: center; font-
    size: 0.78rem; color: var(--muted-2); margin-top: 14px; }
206 .marketing-strip { max-width: 1320px; margin: 0 auto; padding: 0 40px 70px; }
207 .journey-row { display: flex; align-items: center; justify-content: center; gap: 0; background: #
    fff; border: 1px solid var(--border); border-radius: var(--radius-lg); padding: 26px 20px;
    margin-bottom: 26px; flex-wrap: wrap; }
208 .journey-step { display: flex; align-items: center; gap: 12px; padding: 0 18px; }
209 .journey-step .num { width: 34px; height: 34px; border-radius: 50%; background: var(--teal); color
    : #fff; display: flex; align-items: center; justify-content: center; font-weight: 800; }
210 .journey-connector { width: 46px; height: 1px; border-top: 2px dotted var(--border); }
211 .feature-cards { display: grid; grid-template-columns: repeat(4, 1fr); gap: 18px; }
212 .feature-cards .card { transition: transform .15s, box-shadow .15s; }
213 .feature-cards .card:hover { transform: translateY(-3px); box-shadow: 0 10px 24px rgba
    (16,24,40,.09); }
214 @media (max-width: 980px) { .feature-cards, .auth-hero { grid-template-columns: 1fr; } .auth-card
    { margin-left: 0; } }
215
216 /* legal / consent stepper topbar (T&C, Consent screens) */
217 .setup-topbar { display: flex; align-items: center; justify-content: space-between; padding: 16px
    40px; border-bottom: 1px solid var(--border-2); }
218 .setup-steps { display: flex; align-items: center; gap: 10px; font-size: 0.92rem; font-weight:
    600; color: var(--muted-2); }
219 .setup-steps .active { color: var(--teal); }
220 .setup-steps .done { color: var(--navy-2); }
221 .setup-steps .chev { color: var(--muted-2); }
222 .legal-wrap { max-width: 1200px; margin: 0 auto; padding: 34px 40px 120px; }
223 .legal-wrap h1 { font-size: 2.1rem; }
224 .legal-wrap .sub { color: var(--muted); margin-top: 8px; }
225 .legal-body { display: grid; grid-template-columns: 260px 1fr; gap: 22px; margin-top: 22px; align-
    items: start; }
226 .toc-card { position: sticky; top: 20px; }
227 .toc-card h4 { font-size: 0.78rem; text-transform: uppercase; letter-spacing: .04em; color: var(--
    muted); margin-bottom: 10px; }
228 .toc-item { display: block; padding: 8px 10px; border-radius: var(--radius-sm); font-size: 0.87rem
    ; font-weight: 600; color: var(--navy-2); cursor: pointer; transition: background .12s; }
229 .toc-item:hover { background: #f4f6f9; text-decoration: none; }
230 .toc-item.active { background: var(--teal-lighter); color: var(--teal-dark); }
231 .legal-doc { max-height: 62vh; overflow-y: auto; padding-right: 6px; }
232 .legal-section { margin-bottom: 26px; }
233 .legal-section h3 { font-size: 1.15rem; margin-bottom: 8px; }
234 .legal-section p, .legal-section li { font-size: 0.92rem; color: #33405a; line-height: 1.65; }
235 .legal-section ul { padding-left: 20px; list-style: disc; margin-top: 6px; }
236 .legal-section ul li { margin-bottom: 4px; }
237 .sticky-footer-bar {
238     position: fixed; left: 0; right: 0; bottom: 0; background: #fff; border-top: 1px solid var(--
        border);
239     padding: 16px 40px; display: flex; align-items: center; justify-content: space-between; gap: 20
        px;
240     box-shadow: 0 -4px 14px rgba(16,24,40,.05);
241 }
242 .sticky-footer-bar .agree-text { font-size: 0.85rem; }
243 .sticky-footer-bar .agree-text .note { color: var(--muted); display: block; font-size: 0.78rem; }
244
245 .consent-grid { display: grid; grid-template-columns: 1.4fr 1fr; gap: 22px; margin-top: 24px; }
246 .consent-item { border: 1px solid var(--border); border-radius: var(--radius-md); padding: 16px 18
    px; display: flex; gap: 14px; align-items: flex-start; margin-bottom: 14px; transition: border
    -color .12s, box-shadow .12s; }
247 .consent-item:hover { border-color: #ccd4e0; box-shadow: var(--shadow-card); }
248 .consent-item .icon-badge { width: 42px; height: 42px; border-radius: 50%; background: var(--teal-
    lighter); color: var(--teal-dark); display: flex; align-items: center; justify-content: center

```

```

    ; font-size: 1.1rem; flex-shrink: 0; }
249 .consent-item h4 { font-size: 0.98rem; margin-bottom: 4px; }
250 .consent-item p { font-size: 0.85rem; color: var(--muted); }
251 .consent-optional-card { border: 1px solid var(--border); border-radius: var(--radius-md); padding
    : 16px 18px; margin-bottom: 14px; transition: border-color .12s, box-shadow .12s; }
252 .consent-optional-card:hover { border-color: #ccd4e0; box-shadow: var(--shadow-card); }
253 .consent-optional-card .top-row { display: flex; justify-content: space-between; align-items: flex
    -start; gap: 10px; }
254 .consent-optional-card h4 { font-size: 0.95rem; }
255 .consent-optional-card p { font-size: 0.83rem; color: var(--muted); margin-top: 6px; }
256 .consent-optional-card ul { margin-top: 10px; font-size: 0.8rem; color: var(--muted); }
257 .consent-optional-card ul li { display: flex; gap: 6px; margin-bottom: 4px; }
258
259 /* ----- App shell ----- */
260 .app-shell { display: flex; min-height: 100vh; align-items: stretch; }
261 .app-sidebar {
262     width: 232px; flex-shrink: 0; background: #fff; border-right: 1px solid var(--border-2);
263     padding: 22px 14px; display: flex; flex-direction: column; position: sticky; top: 0; height: 100
    vh; overflow-y: auto;
264 }
265 .app-sidebar .brand { padding: 0 8px 22px; }
266 .side-nav { display: flex; flex-direction: column; gap: 2px; margin-top: 6px; flex: 1; }
267 .side-link {
268     display: flex; align-items: center; gap: 11px; padding: 10px 12px; border-radius: var(--radius-
    sm);
269     color: #3a4658; font-size: 0.9rem; font-weight: 600; border-left: 3px solid transparent;
    transition: background .12s, color .12s;
270 }
271 .side-link:hover { background: #f4f6f9; text-decoration: none; }
272 .side-link.active { background: var(--teal-lighter); color: var(--teal-dark); border-left-color:
    var(--teal); }
273 .side-link .ico { width: 18px; text-align: center; flex-shrink: 0; }
274 .side-foot { border-top: 1px solid var(--border-2); padding-top: 10px; margin-top: 10px; }
275
276 .app-main { flex: 1; min-width: 0; padding: 26px 34px 70px; }
277 .app-topline { display: flex; align-items: flex-start; justify-content: space-between; gap: 20px;
    margin-bottom: 4px; }
278 .page-head h1 { font-size: 1.65rem; }
279 .page-head .sub { color: var(--muted); font-size: 0.92rem; margin-top: 4px; }
280 .topline-actions { display: flex; align-items: center; gap: 14px; flex-shrink: 0; }
281 .top-bell { position: relative; width: 38px; height: 38px; border-radius: 50%; border: 1px solid
    var(--border); background: #fff; display: flex; align-items: center; justify-content: center;
    color: var(--navy-2); transition: box-shadow .12s, border-color .12s; }
282 .top-bell:hover { border-color: var(--teal); box-shadow: 0 0 4px var(--teal-light); }
283 .top-bell .dot { position: absolute; top: 7px; right: 8px; width: 7px; height: 7px; border-radius:
    50%; background: var(--teal); border: 1.5px solid #fff; }
284 .user-chip { display: flex; align-items: center; gap: 8px; cursor: pointer; position: relative; }
285 .avatar { width: 34px; height: 34px; border-radius: 50%; background: linear-gradient(135deg,#0
    f8c7f,#0b2545); color: #fff; display: flex; align-items: center; justify-content: center; font
    -weight: 700; font-size: 0.85rem; }
286 .user-chip .name { font-weight: 700; font-size: 0.9rem; color: var(--navy-2); }
287 .user-menu { position: absolute; top: 44px; right: 0; background: #fff; border: 1px solid var(--
    border); border-radius: var(--radius-md); box-shadow: var(--shadow-card); min-width: 170px;
    padding: 6px; z-index: 40; }
288 .user-menu a, .user-menu button { display: block; width: 100%; text-align: left; padding: 9px 10px
    ; border-radius: 7px; font-size: 0.86rem; color: var(--ink); background: none; border: none; }
289 .user-menu a:hover, .user-menu button:hover { background: #f4f6f9; text-decoration: none; }
290 .user-menu button.danger { color: var(--bad); }
291 .section-title { font-size: 1.05rem; color: var(--navy); font-weight: 800; margin: 26px 0 12px; }
292 .section-title:first-child { margin-top: 0; }
293
294 /* ----- Stepper ----- */
295 .stepper { display: flex; align-items: center; gap: 0; margin: 6px 0 22px; flex-wrap: wrap; }
296 .step-node { display: flex; align-items: center; gap: 9px; }

```

```

297 .step-circle { width: 30px; height: 30px; border-radius: 50%; display: flex; align-items: center;
      justify-content: center; font-weight: 800; font-size: 0.85rem; border: 2px solid var(--border)
      ; color: var(--muted); background: #fff; }
298 .step-node.done .step-circle { background: var(--teal); border-color: var(--teal); color: #fff; }
299 .step-node.active .step-circle { background: var(--teal); border-color: var(--teal); color: #fff;
      }
300 .step-node .step-label { font-size: 0.88rem; font-weight: 700; color: var(--muted); }
301 .step-node.active .step-label, .step-node.done .step-label { color: var(--navy-2); }
302 .step-line { width: 60px; height: 2px; background: var(--border); margin: 0 8px; }
303 .step-line.done { background: var(--teal); }
304
305 /* ----- Tabs ----- */
306 .tabs { display: flex; gap: 6px; border-bottom: 1px solid var(--border); margin-bottom: 20px; flex
      -wrap: wrap; }
307 .tab-btn { padding: 11px 4px; margin-right: 22px; background: none; border: none; border-bottom:
      2.5px solid transparent; font-weight: 700; color: var(--muted); font-size: 0.94rem; transition
      : color .12s, border-color .12s; }
308 .tab-btn:hover { color: var(--teal-dark); }
309 .tab-btn.active { color: var(--teal-dark); border-bottom-color: var(--teal); }
310
311 /* ----- Domain grid / body summary ----- */
312 .domain-card { border: 1px solid var(--border); border-radius: var(--radius-md); padding: 15px 16
      px; display: flex; gap: 12px; align-items: flex-start; background: #fff; }
313 .domain-card .dic { width: 38px; height: 38px; border-radius: 10px; display: flex; align-items:
      center; justify-content: center; flex-shrink: 0; font-size: 1.1rem; }
314 .domain-card h4 { font-size: 0.95rem; margin-bottom: 3px; }
315 .domain-card p.d { font-size: 0.82rem; color: var(--muted); margin-top: 2px; }
316 .domain-card.clickable { cursor: pointer; transition: box-shadow .12s, border-color .12s,
      transform .12s; }
317 .domain-card.clickable:hover { box-shadow: var(--shadow-card); border-color: #ccd4e0; transform:
      translateY(-2px); }
318
319 .factor-table { width: 100%; border-collapse: collapse; font-size: 0.88rem; }
320 .factor-table th { text-align: left; color: var(--muted); font-size: 0.75rem; text-transform:
      uppercase; letter-spacing: .03em; padding: 8px 10px; border-bottom: 1px solid var(--border); }
321 .factor-table td { padding: 10px; border-bottom: 1px solid var(--border-2); }
322
323 .connect-flow { display: flex; align-items: center; justify-content: center; gap: 10px; flex-wrap:
      wrap; padding: 18px 0 6px; }
324 .connect-node { width: 92px; height: 92px; border-radius: 50%; display: flex; flex-direction:
      column; align-items: center; justify-content: center; text-align: center; font-size: 0.78rem;
      font-weight: 700; gap: 4px; }
325 .connect-arrow { color: var(--border-2); font-size: 1.3rem; }
326 .connect-result { margin-top: 14px; text-align: center; }
327 .connect-result .pill-lg { display: inline-flex; align-items: center; gap: 8px; padding: 10px 18px
      ; border: 1.5px solid var(--info-border); background: var(--info-bg); color: var(--info);
      border-radius: var(--radius-sm); font-weight: 700; }
328
329 /* ----- Safety ladder ----- */
330 .safety-ladder { display: flex; flex-direction: column; gap: 12px; }
331 .ladder-card { display: flex; align-items: center; gap: 16px; border: 1.5px solid var(--border);
      border-radius: var(--radius-md); padding: 14px 16px; position: relative; transition: box-
      shadow .12s; }
332 .ladder-card:hover { box-shadow: var(--shadow-card); }
333 .ladder-card.recommended { border-color: var(--teal); background: var(--teal-lighter); }
334 .ladder-card .lvl { width: 34px; height: 34px; border-radius: 50%; display: flex; align-items:
      center; justify-content: center; font-weight: 800; flex-shrink: 0; }
335 .ladder-card .lvl.l1 { background: var(--bad-bg); color: var(--bad); }
336 .ladder-card .lvl.l2 { background: var(--warn-bg); color: var(--warn); }
337 .ladder-card .lvl.l3 { background: var(--teal-light); color: var(--teal-dark); }
338 .ladder-card .lvl.l4 { background: var(--good-bg); color: var(--good); }
339 .ladder-card .body { flex: 1; }
340 .ladder-card h4 { font-size: 0.95rem; }
341 .ladder-card p { font-size: 0.82rem; color: var(--muted); margin-top: 2px; }

```

```

342 .ladder-card .check-badge { position: absolute; top: -9px; right: 14px; width: 22px; height: 22px;
    border-radius: 50%; background: var(--teal); color: #fff; display: flex; align-items: center;
    justify-content: center; font-size: 0.7rem; }
343
344 /* ----- Timeline ----- */
345 .timeline { display: flex; align-items: center; gap: 0; position: relative; padding: 14px 0; }
346 .timeline::before { content: ""; position: absolute; left: 20px; right: 20px; top: 50%; height: 2
    px; background: var(--border); }
347 .tl-point { flex: 1; text-align: center; position: relative; z-index: 1; }
348 .tl-point .dot2 { width: 12px; height: 12px; border-radius: 50%; background: var(--teal); border:
    3px solid #fff; box-shadow: 0 0 0 1px var(--teal); margin: 0 auto 8px; }
349 .tl-point .lbl { font-size: 0.82rem; font-weight: 700; color: var(--navy-2); }
350 .tl-point .sub { font-size: 0.78rem; color: var(--muted); }
351
352 /* ----- Progress ring ----- */
353 .progress-ring { width: 220px; height: 220px; border-radius: 50%; display: flex; align-items:
    center; justify-content: center; position: relative; margin: 0 auto; background: conic-
    gradient(var(--teal) calc(var(--pct,0)*1%), var(--border-2) 0); }
354 .progress-ring::before { content: ""; position: absolute; inset: 14px; border-radius: 50%;
    background: #fff; }
355 .progress-ring .val { position: relative; z-index: 1; font-size: 2.6rem; font-weight: 800; color:
    var(--navy); }
356 .progress-ring .val small { font-size: 1.3rem; }
357 .check-steps { display: flex; flex-direction: column; }
358 .check-step { display: flex; align-items: center; gap: 12px; padding: 9px 0; }
359 .check-step .num2 { width: 26px; height: 26px; border-radius: 50%; background: var(--border-2);
    color: var(--muted); display: flex; align-items: center; justify-content: center; font-size:
    0.78rem; font-weight: 800; flex-shrink: 0; }
360 .check-step.done .num2 { background: var(--good-bg); color: var(--good); }
361 .check-step.active .num2 { background: var(--teal); color: #fff; }
362 .check-step .lab { flex: 1; font-size: 0.9rem; font-weight: 600; }
363 .check-step.active .lab { color: var(--teal-dark); font-weight: 700; }
364 .check-step .st { font-size: 0.8rem; color: var(--muted); font-weight: 600; }
365 .check-step.done .st { color: var(--good); }
366 .check-connector { width: 2px; height: 12px; background: var(--border); margin-left: 13px; }
367
368 /* ----- Upload dropzone ----- */
369 .dropzone { border: 2px dashed #b7d9d5; border-radius: var(--radius-lg); background: var(--teal-
    lighter); padding: 44px 24px; text-align: center; transition: border-color .15s, background
    .15s; }
370 .dropzone.dragover { border-color: var(--teal); background: #e2f4f1; }
371 .dropzone .dz-icon { width: 62px; height: 62px; border-radius: 50%; background: #fff; display:
    flex; align-items: center; justify-content: center; margin: 0 auto 14px; font-size: 1.6rem;
    color: var(--teal-dark); box-shadow: var(--shadow-card); }
372 .dropzone h3 { font-size: 1.15rem; }
373 .dropzone .meta { color: var(--muted); font-size: 0.83rem; margin-top: 6px; }
374 .dropzone .dz-actions { display: flex; gap: 10px; justify-content: center; margin-top: 18px; flex-
    wrap: wrap; }
375
376 /* ----- Tables ----- */
377 table.data-table { width: 100%; border-collapse: collapse; font-size: 0.88rem; }
378 table.data-table th { text-align: left; color: var(--muted); font-size: 0.74rem; text-transform:
    uppercase; letter-spacing: .03em; padding: 10px; border-bottom: 1px solid var(--border); }
379 table.data-table td { padding: 12px 10px; border-bottom: 1px solid var(--border-2); vertical-align
    : middle; }
380 table.data-table tr:last-child td { border-bottom: none; }
381 table.data-table tbody tr:hover { background: #fafbfd; }
382 table.data-table input, table.data-table select { border: 1px solid var(--border); border-radius:
    6px; padding: 6px 8px; font-size: 0.85rem; width: 100%; min-width: 76px; }
383 .row-select { display: flex; align-items: center; gap: 10px; }
384
385 /* ----- Misc ----- */
386 .kv-grid { display: grid; grid-template-columns: repeat(3,1fr); gap: 12px 20px; }
387 .kv-grid .k { font-size: 0.72rem; text-transform: uppercase; color: var(--muted); letter-spacing:

```

```

    .03em; }
388 .kv-grid .v { font-size: 1.05rem; font-weight: 700; color: var(--navy); margin-top: 2px; }
389 .snap-tile { background: var(--teal-lighter); border-radius: var(--radius-md); padding: 14px 16px;
    transition: box-shadow .12s, transform .12s; }
390 .snap-tile:hover { box-shadow: var(--shadow-card); transform: translateY(-2px); }
391 .snap-tile .k { font-size: 0.72rem; text-transform: uppercase; color: var(--muted); letter-spacing
    : .03em; }
392 .snap-tile .v { font-size: 1.3rem; font-weight: 800; color: var(--navy); margin-top: 4px; }
393 .snap-tile .v.sm { font-size: 1rem; }
394 .actions-row { display: flex; gap: 10px; flex-wrap: wrap; margin-bottom: 20px; }
395 .chip-row { display: flex; gap: 8px; flex-wrap: wrap; }
396 .chip { background: #f1f3f7; border-radius: 999px; padding: 5px 12px; font-size: 0.8rem; font-
    weight: 600; color: var(--navy-2); }
397 .footer-note { text-align: center; font-size: 0.8rem; color: var(--muted); margin-top: 26px;
    padding-top: 18px; border-top: 1px solid var(--border-2); }
398 .link-row { display: flex; align-items: center; gap: 8px; font-size: 0.9rem; font-weight: 700;
    padding: 4px 0; transition: transform .12s; }
399 .link-row:hover { transform: translateX(2px); }
400 .list-plain li { padding: 8px 0; border-bottom: 1px solid var(--border-2); font-size: 0.88rem;
    display: flex; justify-content: space-between; gap: 10px; }
401 .list-plain li:last-child { border-bottom: none; }
402 .num-badge { width: 26px; height: 26px; border-radius: 50%; background: var(--teal-lighter); color
    : var(--teal-dark); display: flex; align-items: center; justify-content: center; font-weight:
    800; font-size: 0.82rem; flex-shrink: 0; }
403 .priority-card { border: 1px solid var(--border); border-radius: var(--radius-md); padding: 16px
    18px; transition: box-shadow .12s; }
404 .priority-card:hover { box-shadow: var(--shadow-card); }
405 .priority-card .top-row { display: flex; gap: 12px; align-items: flex-start; }
406 .priority-card h4 { font-size: 0.96rem; }
407 .priority-card p { font-size: 0.84rem; color: var(--muted); margin-top: 4px; }
408 .priority-card .actions { display: flex; align-items: center; justify-content: space-between;
    margin-top: 12px; font-size: 0.82rem; }
409 .mini-chart { display: block; }
410 .print-hidden {}
411 .empty-state { text-align: center; padding: 40px 20px; color: var(--muted); }
412 .loading-state { text-align: center; padding: 60px 20px; color: var(--muted); }
413 .spinner { width: 34px; height: 34px; border-radius: 50%; border: 3px solid var(--border); border-
    top-color: var(--teal); margin: 0 auto 14px; animation: sq-spin .8s linear infinite; }
414 @keyframes sq-spin { to { transform: rotate(360deg); } }
415
416 /* ----- Toast ----- */
417 #toast-root { position: fixed; top: 18px; right: 18px; z-index: 200; display: flex; flex-direction
    : column; gap: 8px; }
418 .toast { background: var(--navy); color: #fff; padding: 12px 16px; border-radius: var(--radius-sm)
    ; font-size: 0.86rem; box-shadow: 0 6px 18px rgba(0,0,0,.18); max-width: 320px; }
419 .toast.good { background: var(--good); } .toast.bad { background: var(--bad); }
420
421 @media print {
422 .app-sidebar, .topline-actions, .print-hidden, .disclaimer-banner { display: none !important; }
423 .app-main { padding: 0; }
424 body { background: #fff; }
425 .card { box-shadow: none; }
426 }
427
428 /* =====
429 FullStructure additions: anatomy board, expandable reference
430 scale rows, trends enlarge modal.
431 ===== */
432
433 /* ----- Anatomy board (Dashboard) -----
434 A body illustration with a leader-lined summary card per health
435 domain either side, matching the "medical infographic" reference the
436 user asked to match. See js/ui/anatomy.js for how the SVG + the
437 foreignObject cards are assembled. */

```

```

438 .anatomy-board { }
439 .anatomy-board-head { display: flex; align-items: center; justify-content: space-between; flex-
    wrap: wrap; gap: 10px; margin-bottom: 14px; }
440 .anatomy-board-head h3 { font-size: 1.05rem; color: var(--navy); margin: 0; }
441 .anatomy-legend-strip { display: flex; align-items: center; gap: 16px; flex-wrap: wrap; }
442 .anatomy-legend-chip { display: flex; align-items: center; gap: 6px; font-size: 0.78rem; font-
    weight: 700; color: var(--muted); }
443 .anatomy-legend-chip .dot { width: 9px; height: 9px; border-radius: 50%; display: inline-block;
    flex-shrink: 0; }
444 .anatomy-legend-chip .dot.tone-good { background: var(--good); }
445 .anatomy-legend-chip .dot.tone-warn { background: var(--warn); }
446 .anatomy-legend-chip .dot.tone-bad { background: var(--bad); }
447 .anatomy-figure-wide { display: block; width: 100%; height: auto; }
448 .anatomy-caption { text-align: center; color: var(--muted); font-size: 0.8rem; margin-top: 10px; }
449
450 /* Organs – SVG shapes (fill inherits from the tone-* class down onto
451    whichever child element(s) don't set their own fill, so one rule on
452    the wrapping <g> colours the whole shape). */
453 .anatomy-organ { cursor: pointer; transition: filter .18s, transform .18s, opacity .18s; transform
    -box: fill-box; transform-origin: center; }
454 .anatomy-organ:hover, .anatomy-organ.hovered { filter: brightness(0.85) saturate(1.3); transform:
    scale(1.08); }
455 .anatomy-organ:focus-visible { outline: 3px solid var(--teal); outline-offset: 2px; }
456 .anatomy-organ.tone-good { fill: var(--good); }
457 .anatomy-organ.tone-warn { fill: var(--warn); }
458 .anatomy-organ.tone-bad { fill: var(--bad); }
459 .anatomy-organ.tone-neutral { fill: #aab4c6; }
460 .anatomy-organ.tone-good:hover, .anatomy-organ.tone-good.hovered { filter: brightness(0.82)
    saturate(1.3) drop-shadow(0 0 8px var(--good)); }
461 .anatomy-organ.tone-warn:hover, .anatomy-organ.tone-warn.hovered { filter: brightness(0.85)
    saturate(1.3) drop-shadow(0 0 8px var(--warn)); }
462 .anatomy-organ.tone-bad:hover, .anatomy-organ.tone-bad.hovered { filter: brightness(0.85) saturate
    (1.3) drop-shadow(0 0 8px var(--bad)); }
463 .anatomy-organ.tone-neutral:hover, .anatomy-organ.tone-neutral.hovered { filter: brightness(0.85)
    saturate(1.1) drop-shadow(0 0 8px var(--muted)); }
464 .anatomy-silhouette { opacity: 0.2; stroke: #b7c1d1; stroke-width: 1.2; }
465 .anatomy-organ.tone-neutral .anatomy-silhouette { opacity: 0.14; }
466
467 /* Decorative-only anatomy (brain, lungs, a central vessel line) – always
468    present so the figure reads as a body, never clickable/tone-coloured. */
469 .anatomy-decor-organ { fill: #f4c9d1; opacity: 0.55; pointer-events: none; }
470 .anatomy-decor-line { fill: none; stroke: #d68a97; stroke-width: 1.2; opacity: 0.6; pointer-events
    : none; }
471 .anatomy-vessel { stroke: #c9455a; stroke-width: 1.4; stroke-dasharray: 1 4; stroke-linecap: round
    ; opacity: 0.35; }
472
473 /* Leader lines connecting each card to its organ */
474 .anatomy-leader line { stroke: #c7cfdc; stroke-width: 1.5; transition: stroke .15s, stroke-width
    .15s, opacity .15s; }
475 .anatomy-leader-dot { fill: #fff; stroke: #c7cfdc; stroke-width: 2; transition: stroke .15s; }
476 .anatomy-leader.tone-good line, .anatomy-leader.tone-good .anatomy-leader-dot { stroke: var(--good
    ); }
477 .anatomy-leader.tone-warn line, .anatomy-leader.tone-warn .anatomy-leader-dot { stroke: var(--warn
    ); }
478 .anatomy-leader.tone-bad line, .anatomy-leader.tone-bad .anatomy-leader-dot { stroke: var(--bad);
    }
479 .anatomy-leader line { opacity: 0.55; }
480 .anatomy-leader.hovered line { stroke-width: 2.5; opacity: 1; }
481
482 /* Domain cards – rendered as HTML inside <foreignObject> on desktop
483    (.anatomy-figure-wide) so they scale together with the illustration,
484    and as a plain stacked list below ~880px (.anatomy-compact-list). */
485 .anatomy-card {
486     display: flex; align-items: flex-start; gap: 12px; height: 100%; box-sizing: border-box;

```

```

487 background: #fff; border: 1px solid var(--border-2); border-radius: var(--radius-md);
488 padding: 14px 14px 14px 16px; box-shadow: var(--shadow-card); cursor: pointer;
489 font-family: -apple-system, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
490 transition: transform .15s, box-shadow .15s, border-color .15s;
491 }
492 .anatomy-card:hover, .anatomy-card.hovered { transform: translateY(-2px); box-shadow: 0 10px 22px
      rgba(16,24,40,.12); border-color: #ccd4e0; }
493 .anatomy-card-icon {
494   width: 40px; height: 40px; border-radius: 50%; border: 2px solid currentColor;
495   display: flex; align-items: center; justify-content: center; font-size: 1.15rem; flex-shrink: 0;
496 }
497 .anatomy-card.tone-good .anatomy-card-icon, .anatomy-card.tone-good .anatomy-card-status { color:
      var(--good); }
498 .anatomy-card.tone-warn .anatomy-card-icon, .anatomy-card.tone-warn .anatomy-card-status { color:
      var(--warn); }
499 .anatomy-card.tone-bad .anatomy-card-icon, .anatomy-card.tone-bad .anatomy-card-status { color:
      var(--bad); }
500 .anatomy-card.tone-neutral .anatomy-card-icon, .anatomy-card.tone-neutral .anatomy-card-status {
      color: var(--muted); }
501 .anatomy-card-body { flex: 1; min-width: 0; }
502 .anatomy-card-title { font-weight: 800; color: var(--navy); font-size: 0.92rem; }
503 .anatomy-card-status { font-weight: 700; font-size: 0.8rem; margin-top: 2px; }
504 .anatomy-card-desc { color: var(--muted); font-size: 0.78rem; margin-top: 4px; line-height: 1.35;
      display: -webkit-box; -webkit-line-clamp: 2; -webkit-box-orient: vertical; overflow: hidden; }
505 .anatomy-card-chevron { color: var(--muted-2); font-size: 1.3rem; align-self: center; flex-shrink:
      0; transition: transform .15s, color .15s; }
506 .anatomy-card:hover .anatomy-card-chevron { transform: translateX(3px); color: var(--teal-dark); }
507
508 .anatomy-compact-list { display: none; flex-direction: column; gap: 10px; margin-top: 4px; }
509 .anatomy-compact-list .anatomy-card { height: auto; }
510 @media (max-width: 880px) {
511   .anatomy-figure-wide { display: none; }
512   .anatomy-compact-list { display: flex; }
513 }
514
515 /* ----- Expandable reference-scale rows ----- */
516 .expand-toggle {
517   width: 26px; height: 26px; border-radius: 50%; border: 1px solid var(--border); background: #fff
      ;
518   color: var(--muted); display: inline-flex; align-items: center; justify-content: center;
519   font-size: 0.75rem; transition: background .12s, transform .18s, border-color .12s, color .12s;
520 }
521 .expand-toggle:hover { background: var(--teal-lighter); border-color: var(--teal); color: var(--
      teal-dark); }
522 .expand-toggle.open { transform: rotate(180deg); background: var(--teal-lighter); border-color:
      var(--teal); color: var(--teal-dark); }
523 .scale-row { display: none; }
524 .scale-row.open { display: table-row; }
525 .scale-row td { padding: 0 !important; border-bottom: 1px solid var(--border-2); }
526 .scale-panel { padding: 16px 18px 18px; background: #fafbfd; }
527 .scale-panel .scale-title { font-size: 0.82rem; font-weight: 700; color: var(--navy-2); margin-
      bottom: 10px; }
528 .scale-track { position: relative; height: 30px; border-radius: 999px; overflow: hidden; display:
      flex; box-shadow: inset 0 0 0 1px var(--border); }
529 .scale-band { height: 100%; position: relative; transition: filter .12s; }
530 .scale-band:hover { filter: brightness(0.94); }
531 .scale-band.good { background: var(--good); opacity: .82; }
532 .scale-band.warn { background: var(--warn); opacity: .82; }
533 .scale-band.bad { background: var(--bad); opacity: .82; }
534 .scale-marker {
535   position: absolute; top: -7px; width: 2px; height: 44px; background: var(--navy);
536   transform: translateX(-1px);
537 }
538 .scale-marker::after {

```

```

539   content: attr(data-value); position: absolute; top: -22px; left: 50%; transform: translateX
      (-50%);
540   background: var(--navy); color: #fff; font-size: 0.7rem; font-weight: 700; padding: 2px 6px;
      border-radius: 5px; white-space: nowrap;
541 }
542 .scale-labels { display: flex; justify-content: space-between; margin-top: 8px; font-size: 0.72rem
      ; color: var(--muted); }
543 .scale-legend { display: flex; flex-wrap: wrap; gap: 10px 16px; margin-top: 10px; }
544 .scale-legend-item { display: flex; align-items: center; gap: 6px; font-size: 0.76rem; color: var
      (--muted); }
545 .scale-legend-swatch { width: 10px; height: 10px; border-radius: 3px; }
546 .scale-source { margin-top: 10px; font-size: 0.72rem; color: var(--muted-2); }
547
548 /* ----- Trends enlarge modal ----- */
549 .trend-card { cursor: pointer; }
550 .modal-overlay {
551   position: fixed; inset: 0; background: rgba(11,37,69,.45); z-index: 300;
552   display: flex; align-items: center; justify-content: center; padding: 24px;
553 }
554 .trend-modal {
555   background: #fff; border-radius: var(--radius-lg); box-shadow: 0 20px 60px rgba(0,0,0,.35);
556   width: min(920px, 100%); max-height: 90vh; display: flex; flex-direction: column; overflow:
      hidden;
557 }
558 .trend-modal-head { display: flex; align-items: flex-start; justify-content: space-between;
      padding: 20px 24px; border-bottom: 1px solid var(--border-2); }
559 .trend-modal-head h3 { font-size: 1.2rem; }
560 .trend-modal-head .sub { color: var(--muted); font-size: 0.85rem; margin-top: 4px; }
561 .trend-modal-close { width: 32px; height: 32px; border-radius: 50%; border: 1px solid var(--border
      ); background: #fff; color: var(--muted); font-size: 1rem; transition: background .12s, color
      .12s; }
562 .trend-modal-close:hover { background: var(--bad-bg); color: var(--bad); border-color: var(--bad-
      border); }
563 .trend-modal-body { padding: 18px 24px 24px; overflow: auto; }
564 .trend-canvas-wrap { position: relative; border: 1px solid var(--border-2); border-radius: var(--
      radius-md); background: #fff; cursor: grab; touch-action: none; }
565 .trend-canvas-wrap:active { cursor: grabbing; }
566 .trend-canvas-wrap canvas { display: block; width: 100%; height: 380px; }
567 .trend-modal-controls { display: flex; align-items: center; justify-content: space-between; gap:
      12px; margin-top: 12px; flex-wrap: wrap; }
568 .trend-modal-hint { font-size: 0.78rem; color: var(--muted); }
569 .trend-formula { margin-top: 14px; padding: 12px 14px; border-radius: var(--radius-md); background
      : var(--teal-lighter); font-family: "Cambria Math", "Cambria", "STIX Two Math", serif; font-
      size: 1.02rem; color: var(--navy); }

```

Listing 6.2: frontend/css/style.css (full file).

6.2 Data layer

6.2.1 js/api.js — REST fetch wrapper

Every backend call in the application funnels through this one file. `request(method, path, body)` is the shared implementation behind `get/post/patch`: it attaches the `Authorization: Bearer <token>` header when a session token is cached (`getToken()`, reading `localStorage`), serialises the body as JSON, and throws an `Error` carrying the backend's own detail string plus an `.status` property on any non-2xx response, so every calling screen can show the backend's actual validation message rather than a generic failure. `upload()` is the one exception — a `multipart/form-data` `FormData` POST, used only by `uploadReport()`. `API_BASE` is the single constant a split deployment (frontend on a CDN, backend elsewhere) would need to change; every other line is unaffected.

```

1  /* EVA SQUARE – thin fetch wrapper around the backend REST API. Same-origin
2  * by default (the FastAPI backend serves this frontend as static files –
3  * see backend/app/main.py), so no base URL or CORS setup is needed for the
4  * local basis version. A deployed build only needs to change API_BASE.
5  */
6  window.SQ = window.SQ || {};
7
8  (function () {
9    "use strict";
10
11    const API_BASE = ""; // same-origin; set to e.g. "https://api.example.com" for a split
12    const TOKEN_KEY = "eva_square_token";
13
14    function getToken() { try { return localStorage.getItem(TOKEN_KEY); } catch (e) { return null; }
15    }
16
17    function setToken(t) { try { if (t) localStorage.setItem(TOKEN_KEY, t); else localStorage.
18    removeItem(TOKEN_KEY); } catch (e) { /* ignore */ } }
19
20    async function request(method, path, body) {
21      const headers = { "Content-Type": "application/json" };
22      const token = getToken();
23      if (token) headers["Authorization"] = "Bearer " + token;
24      const opts = { method, headers };
25      if (body !== undefined) opts.body = JSON.stringify(body);
26      const res = await fetch(API_BASE + path, opts);
27      if (!res.ok) {
28        let detail = res.statusText;
29        try { const j = await res.json(); detail = j.detail || detail; } catch (e) { /* non-JSON
30        error body */ }
31        const err = new Error(detail);
32        err.status = res.status;
33        throw err;
34      }
35      if (res.status === 204) return null;
36      return res.json();
37    }
38
39    async function upload(path, file) {
40      const headers = {};
41      const token = getToken();
42      if (token) headers["Authorization"] = "Bearer " + token;
43      const fd = new FormData();
44      fd.append("file", file);
45      const res = await fetch(API_BASE + path, { method: "POST", headers, body: fd });
46      if (!res.ok) { const err = new Error(res.statusText); err.status = res.status; throw err; }
47      return res.json();
48    }
49  })

```

```

46 const get = (path) => request("GET", path);
47 const post = (path, body) => request("POST", path, body === undefined ? {} : body);
48 const patch = (path, body) => request("PATCH", path, body);
49
50 SQ.api = {
51   getToken, setToken,
52   // auth
53   register: (fullName, email, password) => post("/api/auth/register", { fullName, email,
54     password }),
55   login: (email, password) => post("/api/auth/login", { email, password }),
56   demoLogin: (scenario) => post(`/api/auth/demo-login/${scenario}`),
57   logout: () => post("/api/auth/logout"),
58   me: () => get("/api/auth/me"),
59   // consent
60   getConsent: () => get("/api/consent"),
61   acceptTerms: () => post("/api/consent/accept-terms"),
62   patchConsent: (kind, id, value) => patch("/api/consent", { kind, id, value }),
63   // settings
64   getSettings: () => get("/api/settings"),
65   patchSettings: (body) => patch("/api/settings", body),
66   // reports
67   listReports: () => get("/api/reports"),
68   getReport: (id) => get(`/api/reports/${id}`),
69   createSampleReport: (scenario, fileName, fullPipeline) => post("/api/reports/sample", {
70     scenario, fileName: fileName || null, fullPipeline: !!fullPipeline }),
71   uploadReport: (file) => upload("/api/reports/upload", file),
72   verifyReport: (id, values) => post(`/api/reports/${id}/verify`, { values }),
73   getQuestionnaire: (id) => get(`/api/reports/${id}/questionnaire`),
74   patchQuestionnaire: (id, patchObj) => patch(`/api/reports/${id}/questionnaire`, { patch:
75     patchObj }),
76   analyzeReport: (id) => post(`/api/reports/${id}/analyze`),
77   getAnalysis: (id) => get(`/api/reports/${id}/analysis`),
78   getTrends: (id) => get(`/api/reports/${id}/trends`),
79   // action plan
80   getActionPlan: () => get("/api/action-plan"),
81   toggleActionItem: (code) => post("/api/action-plan/toggle", { code }),
82   // meta (static content)
83   metaDomains: () => get("/api/meta/domains"),
84   metaLabDictionary: () => get("/api/meta/lab-dictionary"),
85   metaQuestionnaireSections: () => get("/api/meta/questionnaire-sections"),
86   metaLegal: () => get("/api/meta/legal"),
87   metaReferenceScales: () => get("/api/meta/reference-scales"),
88   metaReferralLevels: () => get("/api/meta/referral-levels"),
89   metaEducation: () => get("/api/meta/education"),
90   metaSampleScenarios: () => get("/api/meta/sample-scenarios"),
91 };
92 }();

```

Listing 6.3: frontend/js/api.js (full file).

6.2.2 js/store.js — client-side cache and need()

The single most important file for understanding how this build differs from a v2-style local prototype. Unlike a synchronous local computation, every piece of state here may not be loaded yet, so `SQ.store` exposes two kinds of reads: a plain state object screens read directly (booted flag, user, consent, settings, meta, reports, per-report caches), and a family of `get*(id)` convenience getters (`getReport`, `getAnswers`, `getAnalysis`, `getTrends`) built on the shared `need(cacheValue, key, loaderFn)` helper: return the cached value if present, otherwise kick off `loaderFn()` exactly once per key (de-duplicated via an inflight `Set`) and return `null` so the calling screen can render a loading state; when the load resolves, every subscriber is notified and the router re-renders, at which point the same getter now returns the cached value. This is the pattern every screen in §6.6 that reads report/analysis/trend data follows.

`init()` is called once by `app.js` at boot: it fetches all eight pieces of static reference content in parallel (§5.4.5) and binds them onto plain `SQ.*` globals via `bindMetaGlobals()` — `SQ.DOMAINS`, `SQ.labDictionary`, `SQ.referenceScales`, and so on — specifically so that screen templates ported line-for-line from the original single-file prototype (which read these as local JavaScript constants) needed no further changes. If a session token is already cached, `init()` also restores the logged-in session (`SQ.api.me()`) before marking `state.booted = true`, which is what the router's boot-loading gate (§6.4.3) waits on.

```

1  /* EVA SQUARE – client-side cache over the backend REST API.
2  *
3  * Unlike frontend-v2 (which computed everything locally and persisted to
4  * localStorage), this FullStructure build treats the backend as the source
5  * of truth: every mutation calls the API, then updates this in-memory
6  * cache and notifies subscribers (the router) to re-render. `localStorage`
7  * is used ONLY for the session bearer token – a per-browser convenience so
8  * a refresh doesn't log you out – never for clinical data.
9  *
10 * `SQ.store.need(cacheValue, key, loaderFn)` is the pattern screens use to
11 * read data that might not be cached yet: it returns the cached value if
12 * present, otherwise kicks off `loaderFn()` (once – de-duplicated by
13 * `key`) and returns `null` to tell the screen to render a loading state.
14 * When the load resolves, listeners are notified and the router re-renders
15 * with the now-cached value.
16 */
17 window.SQ = window.SQ || {};
18
19 (function () {
20   "use strict";
21
22   function defaultState() {
23     return {
24       booted: false,
25       user: null,
26       consent: null,           // {required, optional, acceptedTermsVersion, requiredComplete}
27       settings: null,
28       meta: null,              // {domains, domainOrder, labDictionary, simpleRanges, anthroRules,
29                               // questionnaireSections, legal, referenceScales, referralLevels,
30                               // education, sampleScenarios}
31       reports: null,           // summary list
32       activeReportId: null,
33       reportDetail: {},        // { [id]: full report }
34       questionnaireAnswers: {}, // { [id]: answers }
35       analysisByReport: {},     // { [id]: analysis result }
36       trendsByReport: {},       // { [id]: trends }
37       actionPlanDone: {},
38     };
39   }
40
41   let state = defaultState();
42   const listeners = [];
43   const inflight = new Set();
44
45   function notify() { listeners.forEach((fn) => { try { fn(state); } catch (e) { console.error(e); } }); }
46
47   function subscribe(fn) { listeners.push(fn); return () => { const i = listeners.indexOf(fn); if (i >= 0) listeners.splice(i, 1); }; }
48
49   function need(cacheValue, key, loaderFn) {
50     if (cacheValue !== undefined && cacheValue !== null) return cacheValue;
51     if (!inflight.has(key)) {
52       inflight.add(key);
53       loaderFn().catch((e) => console.error("load failed:", key, e)).finally(() => { inflight.delete(key); notify(); });
54     }
55   }
56 }

```

```

53     return null;
54 }
55
56 // Bind rarely-changing reference content onto plain `SQ.*` globals once
57 // loaded, mirroring frontend-v2's data/*.js globals (SQ.DOMAINS,
58 // SQ.labDictionary, SQ.ORG, ...) so screen templates ported from v2 need
59 // almost no changes to how they reference static content.
60 function bindMetaGlobals() {
61     const m = state.meta;
62     if (!m) return;
63     SQ.DOMAINS = m.domains;
64     SQ.DOMAIN_ORDER = m.domainOrder;
65     SQ.labDictionary = m.labDictionary;
66     SQ.simpleRanges = m.simpleRanges;
67     SQ.anthroRules = m.anthroRules;
68     SQ.questionnaireSections = m.questionnaireSections;
69     SQ.ORG = m.legal.org;
70     SQ.termsSections = m.legal.terms;
71     SQ.consentRequired = m.legal.consentRequired;
72     SQ.consentOptional = m.legal.consentOptional;
73     SQ.referenceScales = m.referenceScales;
74     SQ.referralLevels = m.referralLevels;
75     SQ.education = m.education;
76     SQ.sampleScenarios = m.sampleScenarios;
77 }
78
79 // ---- boot ----
80 async function init() {
81     const metaCalls = await Promise.all([
82         SQ.api.metaDomains(), SQ.api.metaLabDictionary(), SQ.api.metaQuestionnaireSections(),
83         SQ.api.metaLegal(), SQ.api.metaReferenceScales(), SQ.api.metaReferralLevels(),
84         SQ.api.metaEducation(), SQ.api.metaSampleScenarios(),
85     ]);
86     const [domains, labDict, qSections, legal, refScales, referral, education, scenarios] =
87         metaCalls;
88     state.meta = {
89         domains: domains.domains, domainOrder: domains.order,
90         labDictionary: labDict.labs, simpleRanges: labDict.simpleRanges, anthroRules: labDict.
91             anthroRules,
92         questionnaireSections: qSections.sections, legal, referenceScales: refScales,
93         referralLevels: referral.levels, education: education.education, sampleScenarios: scenarios,
94     };
95     bindMetaGlobals();
96
97     if (SQ.api.getToken()) {
98         try {
99             const me = await SQ.api.me();
100             state.user = me.user; state.consent = me.consent; state.settings = me.settings;
101             await refreshReports();
102             await refreshActionPlan();
103         } catch (e) {
104             SQ.api.setToken(null);
105         }
106     }
107     state.booted = true;
108     notify();
109 }
110
111 async function refreshReports() {
112     state.reports = await SQ.api.listReports();
113     if (!state.activeReportId && state.reports.length) state.activeReportId = state.reports[0].id;
114 }
115
116 async function refreshActionPlan() {
117     const r = await SQ.api.getActionPlan();

```

```

115     state.actionPlanDone = r.done;
116 }
117 async function refreshConsent() {
118     state.consent = await SQ.api.getConsent();
119 }
120
121 // ---- auth ----
122 async function afterLogin(authOut) {
123     SQ.api.setToken(authOut.token);
124     state.user = authOut.user;
125     const me = await SQ.api.me();
126     state.consent = me.consent; state.settings = me.settings;
127     await refreshReports();
128     await refreshActionPlan();
129     notify();
130 }
131 async function register(fullName, email, password) { await afterLogin(await SQ.api.register(
132     fullName, email, password)); }
132 async function login(email, password) { await afterLogin(await SQ.api.login(email, password)); }
133 async function demoLogin(scenario) { await afterLogin(await SQ.api.demoLogin(scenario)); }
134 async function logout() {
135     try { await SQ.api.logout(); } catch (e) { /* token already invalid - fine */ }
136     SQ.api.setToken(null);
137     state = defaultState();
138     state.meta = state.meta; // meta refetched lazily on next init if needed
139     await init();
140 }
141
142 // ---- consent ----
143 async function acceptTerms() { await SQ.api.acceptTerms(); await refreshConsent(); notify(); }
144 async function setConsent(kind, id, value) { state.consent = await SQ.api.patchConsent(kind, id,
145     value); notify(); }
145 function requiredConsentComplete() { return !(state.consent && state.consent.requiredComplete);
146     }
147
148 // ---- reports ----
149 // Convenience getters: cache-or-fetch via `need()`. Screens call these
150 // directly (mirroring frontend-v2's synchronous getters); a `null`
151 // return means "fetch kicked off, re-render pending" - screens should
152 // treat that as a loading state, not "doesn't exist" (they can tell the
153 // two apart via state.activeReportId / report.status, since a fetch is
154 // only ever kicked off when there's something to fetch).
155 function getReport(id) {
156     if (!id) return null;
157     return need(state.reportDetail[id], "report:" + id, () => ensureReportDetail(id));
158 }
159 function getActiveReport() { return state.activeReportId ? getReport(state.activeReportId) :
160     null; }
161 function getReportSummary(id) { return (state.reports || []).find((r) => r.id === id) || null; }
162 async function ensureReportDetail(id) {
163     const detail = await SQ.api.getReport(id);
164     state.reportDetail[id] = detail;
165     return detail;
166 }
167 async function setActiveReport(id) { state.activeReportId = id; if (!state.reportDetail[id])
168     await ensureReportDetail(id); notify(); }
169 function getAnswers(id) {
170     if (!id) return null;
171     return need(state.questionnaireAnswers[id], "answers:" + id, () => ensureAnswers(id));
172 }
173 function getAnalysis(id) {
174     if (!id) return null;
175     return need(state.analysisByReport[id], "analysis:" + id, () => ensureAnalysis(id));
176 }

```

```

174 function getTrends(id) {
175   if (!id) return null;
176   return need(state.trendsByReport[id], "trends:" + id, () => ensureTrends(id));
177 }
178
179 async function addSampleReport(scenario, fileName) {
180   const report = await SQ.api.createSampleReport(scenario, fileName, false);
181   state.reportDetail[report.id] = report;
182   state.activeReportId = report.id;
183   await refreshReports();
184   notify();
185   return report.id;
186 }
187
188 async function uploadFile(file) {
189   const report = await SQ.api.uploadReport(file);
190   state.reportDetail[report.id] = report;
191   state.activeReportId = report.id;
192   await refreshReports();
193   notify();
194   return report.id;
195 }
196
197 async function seedDemo(scenario) {
198   await demoLogin(scenario);
199   const report = await SQ.api.createSampleReport(scenario, null, true);
200   state.reportDetail[report.id] = report;
201   state.activeReportId = report.id;
202   state.analysisByReport[report.id] = await SQ.api.getAnalysis(report.id);
203   await refreshReports();
204   notify();
205   return report.id;
206 }
207
208 async function verifyReport(id, values) {
209   const report = await SQ.api.verifyReport(id, values);
210   state.reportDetail[id] = report;
211   notify();
212 }
213
214 async function ensureAnswers(id) {
215   const r = await SQ.api.getQuestionnaire(id);
216   state.questionnaireAnswers[id] = r.answers;
217   return r.answers;
218 }
219
220 async function saveAnswers(id, patchObj) {
221   const r = await SQ.api.patchQuestionnaire(id, patchObj);
222   state.questionnaireAnswers[id] = r.answers;
223   notify();
224 }
225
226 async function runAnalysis(id) {
227   const result = await SQ.api.analyzeReport(id);
228   state.analysisByReport[id] = result;
229   notify();
230   return result;
231 }
232
233 async function ensureAnalysis(id) {
234   const result = await SQ.api.getAnalysis(id);
235   state.analysisByReport[id] = result;
236   return result;
237 }
238
239 async function ensureTrends(id) {
240   const result = await SQ.api.getTrends(id);
241   state.trendsByReport[id] = result;
242   return result;
243 }

```

```
238 }
239
240 async function toggleActionItem(code) {
241   const r = await SQ.api.toggleActionItem(code);
242   state.actionPlanDone = r.done;
243   notify();
244 }
245
246 async function updateSettings(patchObj) {
247   state.settings = await SQ.api.patchSettings(patchObj);
248   notify();
249 }
250
251 SQ.store = {
252   init, subscribe, notify, need,
253   get state() { return state; },
254   register, login, demoLogin, logout, reset: logout,
255   acceptTerms, setConsent, requiredConsentComplete,
256   getReport, getActiveReport, getReportSummary, setActiveReport, ensureReportDetail,
257   addSampleReport, uploadFile, seedDemo, verifyReport,
258   getAnswers, ensureAnswers, saveAnswers,
259   getAnalysis, runAnalysis, ensureAnalysis,
260   getTrends, ensureTrends,
261   toggleActionItem, updateSettings, refreshReports,
262 };
263 }());
```

Listing 6.4: frontend/js/store.js (full file).

6.3 Application bootstrap

6.3.1 js/app.js — SQ.act.* and boot

Loaded last. Two responsibilities: `boot()` subscribes the router to the store, wires `hashchange`, renders once immediately (showing the loading shell until `store.init()` resolves), and starts `store.init()`; and the `SQ.act` object defines every cross-screen action that a screen’s inline `onclick/onchange` handler references by name (screens themselves stay pure render functions with no event-handling logic of their own). Almost every action is `async` and `await`s its store call *before* navigating — called out explicitly in the file’s own header comment as the one behavioural difference from a v2-style synchronous action: a plain “mutate then navigate” would let the router’s login/auth gate (§6.4.3) see stale state and occasionally bounce a just-registered user back to Welcome, since the hash change fires a re-render before the `async register()` call has actually resolved. `confirmQuestions()` is a partial exception — it navigates to `analysis-loading` *before* awaiting `runAnalysis()`, deliberately, so the decorative loading screen (§6.6.6) has something to show while the real analysis runs server-side.

```

1  /* EVA SQUARE – Bootstrap: wires the store, router, and every cross-screen
2  * action (SQ.act.*) that screen templates reference via inline onclick /
3  * onchange handlers. Loaded last, after every data/engine/UI file.
4  *
5  * Ported from frontend-v2's app.js. The store is now backed by a real
6  * REST API (see js/store.js / js/api.js) rather than localStorage, so
7  * every action that used to be a synchronous local mutation is now
8  * `async` and `await`s the store call – critically, *before* navigating,
9  * so the router's login-gate (js/ui/router.js) sees the freshly-updated
10 * session/report state rather than racing it (a plain v2-style
11 * "mutate then navigate" would occasionally bounce a just-registered user
12 * back to Welcome because the router re-renders on the hashchange before
13 * the async register() call resolves).
14 */
15 window.SQ = window.SQ || {};
16
17 (function () {
18   "use strict";
19
20   function getReportId() { return SQ.store.state.activeReportId; }
21
22   SQ.act = {
23     // ---- Demo shortcuts ----
24     async tryDemo(scenarioKey) {
25       await SQ.store.seedDemo(scenarioKey);
26       SQ.ui.toast(scenarioKey === "urgent" ? "Loaded the urgent-pathway sample – see how EVA
27         SQUARE escalates." : "Loaded a routine sample report.", "good");
28       SQ.router.navigate("analysis-summary");
29     },
30     async startSampleUpload(scenarioKey) {
31       await SQ.store.addSampleReport(scenarioKey);
32       SQ.ui.toast("Sample report loaded – verify the extracted values below.", "good");
33       SQ.router.navigate("verify");
34     },
35     async handleFileUpload(event) {
36       const file = event.target.files && event.target.files[0];
37       if (!file) return;
38       await SQ.store.uploadFile(file);
39       SQ.ui.toast("This prototype has no live OCR yet, so "" + file.name + "" was matched to a
40         realistic sample lab panel for you to verify.", "info");
41       SQ.router.navigate("verify");
42     },
43     // ---- Auth / onboarding ----
44     async submitRegister(event) {

```

```

44     event.preventDefault();
45     const name = document.getElementById("reg-name").value.trim();
46     const email = document.getElementById("reg-email").value.trim();
47     const password = document.getElementById("reg-pw").value;
48     if (!name || !email || !password) { SQ.ui.toast("Please fill in your name, email and
49       password.", "bad"); return; }
50     try {
51       await SQ.store.register(name, email, password);
52       SQ.router.navigate("terms");
53     } catch (e) { SQ.ui.toast(e.message || "Could not create your account.", "bad"); }
54   },
55   async submitLogin(event) {
56     event.preventDefault();
57     const email = document.getElementById("login-email").value.trim();
58     const password = document.getElementById("login-pw").value;
59     if (!email || !password) { SQ.ui.toast("Please enter your email and password.", "bad");
60       return; }
61     try {
62       await SQ.store.login(email, password);
63       SQ.router.navigate("dashboard");
64     } catch (e) { SQ.ui.toast(e.message || "Incorrect email or password.", "bad"); }
65   },
66   async continueFromTerms() {
67     const agree = document.getElementById("tc-agree");
68     if (!agree || !agree.checked) { SQ.ui.toast("Please confirm you've read the Terms to
69       continue.", "bad"); return; }
70     await SQ.store.acceptTerms();
71     SQ.router.navigate("consent");
72   },
73   continueFromConsent() {
74     if (!SQ.store.requiredConsentComplete()) { SQ.ui.toast("All three required consents are
75       needed to continue.", "bad"); return; }
76     SQ.router.navigate("dashboard");
77   },
78   setConsent(kind, id, value) { SQ.store.setConsent(kind, id, value); },
79   async logout() { await SQ.store.logout(); SQ.router.navigate("welcome"); },
80   resetDemo() { SQ.act.logout(); },
81
82   // ---- User menu ----
83   toggleUserMenu(event) {
84     event.stopPropagation();
85     const menu = document.getElementById("user-menu");
86     if (menu) menu.hidden = !menu.hidden;
87   },
88
89   // ---- Verify screen ----
90   async confirmVerify() {
91     const report = SQ.store.getActiveReport();
92     if (!report) return;
93     const patch = {};
94     Object.keys(report.labsRaw || {}).forEach((code) => {
95       const el = document.getElementById("verify-" + code);
96       if (el && el.value !== "") patch[code] = parseFloat(el.value);
97     });
98     await SQ.store.verifyReport(report.id, patch);
99     SQ.ui.toast("Results confirmed.", "good");
100     SQ.router.navigate("questions");
101   },
102
103   // ---- Questionnaire ----
104   setAnswer(qid, value) { SQ.store.saveAnswers(getReportId(), { [qid]: value }); },
105   setAnswerText(field, value) { SQ.store.saveAnswers(getReportId(), { [field]: value }); },
106   setBP(qid, sbp, dbp) {
107     const current = (SQ.store.state.questionnaireAnswers[getReportId()] || {})[qid] || {};

```

```

104     const next = Object.assign({}, current);
105     if (sbp !== null) next.sbp = sbp === "" ? undefined : parseFloat(sbp);
106     if (dbp !== null) next.dbp = dbp === "" ? undefined : parseFloat(dbp);
107     SQ.store.saveAnswers(getReportId(), { [qid]: next });
108 },
109 async confirmQuestions() {
110     const reportId = getReportId();
111     if (!reportId) return;
112     SQ.router.navigate("analysis-loading");
113     await SQ.store.runAnalysis(reportId);
114 },
115
116 // ---- Action plan ----
117 toggleAction(code) { SQ.store.toggleActionItem(code); },
118
119 // ---- Reports library ----
120 async openReport(id) {
121     await SQ.store.setActiveReport(id);
122     const report = SQ.store.state.reportDetail[id];
123     if (!report) return;
124     SQ.router.navigate(report.status === "verified" ? "report" : "verify");
125 },
126
127 // ---- Settings ----
128 setSetting(key, value) { SQ.store.updateSettings({ [key]: value }); },
129
130 // ---- FullStructure additions ----
131 // Requirement #2: expand/collapse a quantity's full reference scale.
132 toggleScale(id) {
133     const btn = document.getElementById("scale-toggle-" + id);
134     const row = document.getElementById("scale-row-" + id);
135     if (!btn || !row) return;
136     const open = !row.classList.contains("open");
137     row.classList.toggle("open", open);
138     btn.classList.toggle("open", open);
139 },
140 // Requirement #4: open the enlarged interactive trend chart for one quantity.
141 async openTrendChart(key) {
142     const reportId = getReportId();
143     if (!reportId) return;
144     const trends = SQ.store.state.trendsByReport[reportId] || await SQ.store.ensureTrends(
145         reportId);
146     const entry = trends[key];
147     if (!entry) return;
148     SQ.ui.trendModal.open(key, entry, SQ.screens.trends.unitFor(key));
149 },
150
151 function closeMenusOnOutsideClick() {
152     document.addEventListener("click", () => {
153         const menu = document.getElementById("user-menu");
154         if (menu && !menu.hidden) menu.hidden = true;
155     });
156 }
157
158 function boot() {
159     SQ.store.subscribe(SQ.router.render);
160     window.addEventListener("hashchange", SQ.router.render);
161     closeMenusOnOutsideClick();
162     SQ.router.render(); // shows the loading shell until init() resolves
163     SQ.store.init();
164 }
165
166 if (document.readyState === "loading") document.addEventListener("DOMContentLoaded", boot);

```

```
167     else boot();  
168   })();
```

Listing 6.5: frontend/js/app.js (full file).

6.4 UI library and chrome

6.4.1 js/ui/components.js — reusable component library

A library of small functions that each return an HTML string fragment — buttons, pills, cards, call-outs, toggles, choice rows, a stepper, tabs, a progress ring, a check-step list, the safety ladder, a timeline, domain cards, key/value grids, snap tiles, toasts — everything visually repeated across more than one screen lives here exactly once. Two groups are specific to this FullStructure build rather than ported from v2: `loadingState()/notReady()` (async-backend loading and not-yet-analysed states v2 never needed, since it computed everything synchronously and locally) and `brand()`, which now renders the real EVA SQUARE.png logo served by the backend's `/assets` static mount (§5.1.1) instead of v2's CSS-drawn mark. `esc()` — a small HTML-escaping helper — is called by essentially every other function in this file and by every screen, since screens build markup by string concatenation rather than a templating engine with built-in escaping.

```

1  /* EVA SQUARE — Reusable HTML-string components. No framework/build step:
2   * every screen renders by returning an HTML string, and the router swaps
3   * #root.innerHTML wholesale on every state change (see ui/router.js).
4   *
5   * Ported from frontend-v2's ui/components.js. Two additions for this
6   * FullStructure build: `brand()` now renders the real EVA SQUARE.png logo
7   * (served by the backend's /assets StaticFiles mount) instead of the old
8   * CSS-drawn mark, and `loadingState()` / `notReady()` support the new
9   * async-backend loading states that v2 never needed (it computed
10  * everything locally, synchronously).
11  */
12  window.SQ = window.SQ || {};
13  SQ.ui = SQ.ui || {};
14
15  (function () {
16    "use strict";
17
18    function esc(s) {
19      if (s == null) return "";
20      return String(s).replace(/[\<>'"]/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", "'": "&quot;", "\"": "&#39;" }[c]));
21    }
22    function nl2br(s) { return esc(s).replace(/\n/g, "<br>"); }
23
24    function brand(opts) {
25      opts = opts || {};
26      const sm = opts.sm ? " sm" : "";
27      return `<div class="brand${sm}"><div class="brand-text">
29          <span class="brand-name">EVA SQUARE</span><span class="brand-tag">Health Interpretation
30            Platform</span>
31        </div></div>`;
32    }
33
34    function btn(label, opts) {
35      opts = opts || {};
36      const cls = ["btn", opts.variant || "primary", opts.sm ? "sm" : "", opts.block ? "block" :
37        ""].filter(Boolean).join(" ");
38      const attrs = [
39        opts.href ? `href="#${opts.href}"` : "",
40        opts.onClick ? `onclick="${opts.onClick}"` : "",
41        opts.disabled ? "disabled" : "",
42        opts.type ? `type="${opts.type}"` : "",
43        opts.id ? `id="${opts.id}"` : "",
44      ].filter(Boolean).join(" ");
45      const tag = opts.href ? "a" : "button";

```

```

43   return `${tag} class="${cls}" ${attrs}>${opts.icon ? `<span>${opts.icon}</span>` : ""}${esc(
      label)}</${tag}>`;
44 }
45
46 function pill(tone, label, icon) {
47   return `<span class="pill ${tone}">${icon ? icon + " " : ""}${esc(label)}</span>`;
48 }
49
50 function dotInd(tone) { return `<span class="dot-ind ${tone}"></span>`; }
51
52 function callout(tone, title, text, icon) {
53   icon = icon || (tone === "bad" ? "!" : tone === "warn" ? "!" : tone === "good" ? "✓" : "i");
54   return `<div class="callout ${tone}"><div class="ic">${icon}</div><div>
55     ${title ? `<div class="callout-title">${esc(title)}</div>` : ""}<div>${text}</div>
56   </div></div>`;
57 }
58
59 function card(inner, opts) {
60   opts = opts || {};
61   const cls = ["card", opts.pad === "sm" ? "pad-sm" : ""].filter(Boolean).join(" ");
62   return `<div class="${cls}"${opts.id ? ` id="${opts.id}"` : ""}>${inner}</div>`;
63 }
64
65 function toggle(id, checked, onchange) {
66   return `<label class="toggle"><input type="checkbox" id="${id}" ${checked ? "checked" : ""}
67     onchange="${onchange || ""}"><span class="track"></span></label>`;
68 }
69
70 function choiceBtn(label, selected, onclick) {
71   return `<button type="button" class="choice-btn ${selected ? "selected" : ""}" onclick="${
72     onclick}><span class="dot"></span>${esc(label)}</button>`;
73 }
74
75 function choiceRow(options, selectedValue, onclickFn) {
76   return `<div class="choice-row">${options.map((o) => choiceBtn(o, o === selectedValue, `
77     onclickFn('${esc(o).replace(/'/g, "\\'")}')`)).join("")}</div>`;
78 }
79
80 function stepper(steps, activeIndex) {
81   return `<div class="stepper">${steps.map((s, i) => {
82     const state = i < activeIndex ? "done" : i === activeIndex ? "active" : "";
83     const line = i < steps.length - 1 ? `<div class="step-line ${i < activeIndex ? "done" :
84       ""}></div>` : "";
85     return `<div class="step-node ${state}"><div class="step-circle">${i < activeIndex ? "✓" : i
86       + 1}</div><div class="step-label">${esc(s)}</div></div>${line}`;
87   }).join("")}</div>`;
88 }
89
90 function tabsNav(items, activeId) {
91   return `<div class="tabs">${items.map((t) =>
92     `<button type="button" class="tab-btn ${t.id === activeId ? "active" : ""}" onclick="${t.
93       onclick}">${esc(t.label)}</button>`
94   ).join("")}</div>`;
95 }
96
97 function progressRing(pct, valueLabel) {
98   return `<div class="progress-ring" style="--pct:${pct}"><div class="val">${valueLabel != null
99     ? valueLabel : pct + "%</small>"}</div></div>`;
100 }
101
102 function checkSteps(steps) {
103   return `<div class="check-steps">${steps.map((s, i) => {
104     const icon = s.status === "done" ? "✓" : i + 1;
105     return `<div class="check-step ${s.status}"><div class="num2">${icon}</div><div class="lab">

```

```

    ${esc(s.label)}</div><div class="st">${s.status === "done" ? "Done" : s.status === "
    active" ? "In progress" : "Waiting"}</div></div>`;
99   }).join('<div class="check-connector"></div>`)</div>`;
100  }
101
102  function safetyLadder(levels, recommendedId) {
103    return `<div class="safety-ladder">${levels.map((l) =>
104      `<div class="ladder-card ${l.id === recommendedId ? "recommended" : ""}>
105        ${l.id === recommendedId ? `<div class="check-badge"></div>` : ""}
106        <div class="lvl l${l.rank}">${l.rank}</div>
107        <div class="body"><h4>${esc(l.label)}</h4><p>${esc(l.description)}</p></div>
108        ${pill(l.tone, l.action)}
109      </div>`
110    ).join("")}</div>`;
111  }
112
113  function timeline(points) {
114    return `<div class="timeline">${points.map((p) =>
115      `<div class="tl-point"><div class="dot2"></div><div class="lbl">${esc(p.label)}</div><div
116        class="sub">${esc(p.sub || "")}</div></div>`
117    ).join("")}</div>`;
118  }
119
120  function domainCard(domain, resultSummary, opts) {
121    opts = opts || {};
122    const d = SQ.DOMAINS[domain];
123    return `<div class="domain-card ${opts.clickable ? "clickable" : ""}" ${opts.onclick ? `
124      onclick="${opts.onclick}"` : ""}>
125      <div class="dic" style="background:${d.color}a;color:${d.color}">${d.icon}</div>
126      <div style="flex:1"><h4>${esc(d.label)}</h4><p class="d">${resultSummary}</p></div>
127      ${opts.pill ? opts.pill : ""}
128    </div>`;
129  }
130
131  function kvGrid(items) {
132    return `<div class="kv-grid">${items.map((it) => `<div><div class="k">${esc(it.k)}</div><div
133      class="v">${it.v}</div></div>`).join("")}</div>`;
134  }
135
136  function snapTile(k, v, sm) {
137    return `<div class="snap-tile"><div class="k">${esc(k)}</div><div class="v ${sm ? "sm" : ""}">
138      ${v}</div></div>`;
139  }
140
141  function numBadge(n) { return `<div class="num-badge">${n}</div>`; }
142
143  function emptyState(text) { return `<div class="empty-state">${esc(text)}</div>`; }
144
145  // ---- FullStructure additions: async loading / not-ready states ----
146  function loadingState(message) {
147    return `<div class="loading-state"><div class="spinner"></div>${esc(message || "...Loading")}</
148      div>`;
149  }
150
151  // Shown instead of analysis-dependent content when a report hasn't been
152  // verified yet (the backend only computes analysis for verified
153  // reports – see backend/app/routers/reports.py#get_analysis). Frontend-v2
154  // never needed this because it computed analysis locally and unconditionally.
155  function notReady(report) {
156    if (!report) return emptyState("No report yet – upload one first.");
157    return card(`
158      <div class="empty-state">
159        <h3 style="color:var(--navy)">This report isn't analysed yet</h3>
160        <p style="margin-top:8px;color:var(--muted)">Verify its extracted lab values and answer a

```

```

156     few quick questions to generate your health snapshot.</p>
157     <div style="margin-top:16px">${btn("Continue verifying", { href: "verify" })}</div>
158   </div>`;
159 }
160 // ---- toast ----
161 function ensureToastRoot() {
162   let el = document.getElementById("toast-root");
163   if (!el) { el = document.createElement("div"); el.id = "toast-root"; document.body.appendChild
164     (el); }
165   return el;
166 }
167 function toast(msg, tone) {
168   const root = ensureToastRoot();
169   const el = document.createElement("div");
170   el.className = "toast" + (tone ? " " + tone : "");
171   el.textContent = msg;
172   root.appendChild(el);
173   setTimeout(() => { el.remove(); }, 3200);
174 }
175 SQ.ui.esc = esc; SQ.ui.nl2br = nl2br;
176 SQ.ui.brand = brand; SQ.ui.btn = btn; SQ.ui.pill = pill; SQ.ui.dotInd = dotInd;
177 SQ.ui.callout = callout; SQ.ui.card = card; SQ.ui.toggle = toggle;
178 SQ.ui.choiceBtn = choiceBtn; SQ.ui.choiceRow = choiceRow; SQ.ui.stepper = stepper;
179 SQ.ui.tabsNav = tabsNav; SQ.ui.progressRing = progressRing; SQ.ui.checkSteps = checkSteps;
180 SQ.ui.safetyLadder = safetyLadder; SQ.ui.timeline = timeline; SQ.ui.domainCard = domainCard;
181 SQ.ui.kvGrid = kvGrid; SQ.ui.snapTile = snapTile; SQ.ui.numBadge = numBadge;
182 SQ.ui.emptyState = emptyState; SQ.ui.loadingState = loadingState; SQ.ui.notReady = notReady;
183 SQ.ui.toast = toast;
184 })();

```

Listing 6.6: frontend/js/ui/components.js (full file).

6.4.2 js/ui/shell.js — page chrome

Three page shells the router wraps every screen's content in, selected by each route table entry's shell field (§6.4.3): `auth()` (a centred topbar for Welcome/Register/Login), `setup()` (a two-step Terms/Consent onboarding stepper topbar), and `app()` (the sidebar + topline chrome used from Dashboard onward, wrapping `sidebar()` and `topline()`). `NAV_ITEMS` is the single list that drives the sidebar; each entry's optional `matchPrefix` lets the Health Domains link stay highlighted across every domain/* sub-route. `topline()` renders the logged-in user's initials-avatar and dropdown menu, reading the user directly off `SQ.store.state.user` — the only change from v2's equivalent file, which read a local `session.user` object instead.

```

1  /* EVA SQUARE – Page chrome: the centered auth-shell (Welcome/Register/
2  * Login), the setup-shell (Terms/Consent onboarding stepper), and the
3  * app-shell (sidebar + topline used from Dashboard onward).
4  *
5  * Ported from frontend-v2's ui/shell.js – only change is reading the
6  * logged-in user off `SQ.store.state.user` instead of
7  * `SQ.store.state.session.user`, since this build's store talks to a real
8  * backend session (see js/store.js) rather than a local `session` object.
9  */
10 window.SQ = window.SQ || {};
11 SQ.ui = SQ.ui || {};
12
13 (function () {
14   "use strict";
15   const U = () => SQ.ui;
16

```

```

17 function authTopbar() {
18   return `<div class="auth-topbar">${U().brand()}
19     <nav>
20       <a href="#/welcome">Home</a>
21       <a href="#/login">Log in</a>
22       <a class="btn primary sm" href="#/register">Get started</a>
23     </nav>
24   </div>`;
25 }
26
27 function auth(content) {
28   return `${authTopbar()}${content}${footerNote()}`;
29 }
30
31 function footerNote() {
32   return `<div class="footer-note">EVA SQUARE is a health interpretation and education platform.
33     It does not provide a medical diagnosis. If you may have a medical emergency, call 999
34     immediately.</div>`;
35 }
36
37 function setup(content, stepId) {
38   const steps = [{ id: "terms", label: "Terms" }, { id: "consent", label: "Consent" }];
39   const nav = steps.map((s, i) => {
40     const cls = s.id === stepId ? "active" : (steps.findIndex((x) => x.id === stepId) > i ? "
41       done" : "");
42     const chev = i < steps.length - 1 ? `<span class="chev"></span>` : "";
43     return `<span class="${cls}">${i + 1}. ${s.label}</span>${chev}`;
44   }).join(" ");
45   return `<div class="setup-topbar">${U().brand({ sm: true })}<div class="setup-steps">${nav}</
46     div><div style="width:90px"></div></div>${content}`;
47 }
48
49 const NAV_ITEMS = [
50   { route: "dashboard", label: "Dashboard", icon: "" },
51   { route: "reports", label: "Blood Reports", icon: "" },
52   { route: "domain/cardiovascular", label: "Health Domains", icon: "", matchPrefix: "domain" },
53   { route: "action-plan", label: "Action Plan", icon: "" },
54   { route: "trends", label: "Trends", icon: "" },
55   { route: "referral", label: "Referrals", icon: "" },
56   { route: "profile", label: "Health Profile", icon: "" },
57   { route: "settings", label: "Settings", icon: "" },
58 ];
59
60 function initials(name) {
61   if (!name) return "?";
62   return name.trim().split(/\s+/).map((p) => p[0]).slice(0, 2).join("").toUpperCase();
63 }
64
65 function sidebar(currentRoute) {
66   const links = NAV_ITEMS.map((item) => {
67     const active = item.matchPrefix ? currentRoute.indexOf(item.matchPrefix) === 0 :
68       currentRoute === item.route;
69     return `<a class="side-link ${active ? "active" : ""}" href="#/${item.route}"><span class="
70       ico">${item.icon}</span>${item.label}</a>`;
71   }).join("");
72   return `<div class="app-sidebar">
73     ${U().brand()}
74     <div class="side-nav">${links}</div>
75     <div class="side-foot">
76       <button type="button" class="side-link" style="width:100%;border:none;background:none;"
77         onclick="SQ.act.logout()"><span class="ico"></span>Log out</button>
78     </div>
79   </div>`;
80 }

```

```

74
75 function topline(pageTitle, pageSub, extraActions) {
76   const user = SQ.store.state.user;
77   return `<div class="app-topline">
78     <div class="page-head"><h1>${U().esc(pageTitle)}</h1>${pageSub ? `<div class="sub">${pageSub
79       }</div>` : ""}</div>
80     <div class="topline-actions">
81       ${extraActions || ""}
82     <div class="top-bell"><span class="dot"></span></div>
83     <div class="user-chip" onclick="SQ.act.toggleUserMenu(event)">
84       <div class="avatar">${initials(user && user.fullName)}</div>
85       <span class="name">${U().esc(user ? user.fullName : "Guest")}</span>
86       <div id="user-menu" class="user-menu" hidden>
87         <a href="#/profile">Health Profile</a>
88         <a href="#/settings">Settings</a>
89         <button type="button" class="danger" onclick="SQ.act.logout()">Log out</button>
90       </div>
91     </div>
92   </div>`;
93 }
94
95 function app(route, content, opts) {
96   opts = opts || {};
97   return `<div class="app-shell">
98     ${sidebar(route)}
99     <div class="app-main">
100       ${topline(opts.title || "", opts.sub || "", opts.actions)}
101       ${content}
102     </div>
103   </div>
104   <div class="disclaimer-banner print-hidden"> EVA SQUARE provides health education and
105     interpretation, not a medical diagnosis. In an emergency, call 999.</div>`;
106 }
107
108 SQ.ui.shell = { auth, setup, app, NAV_ITEMS, initials };
109 }());

```

Listing 6.7: frontend/js/ui/shell.js (full file).

6.4.3 js/ui/router.js — hash router

A flat route table (routeTable()) mapping each hash path to a {shell, title, render, afterRender} descriptor (or, for the bare #/ route, a redirect function). render() is called on every hashchange and once at boot: it first checks state.booted — while the store's async init() is still in flight, every route renders the same loading shell rather than guessing a logged-in/logged-out state prematurely, the one addition this build makes over v2's synchronous-boot router. Once booted, it enforces the login/auth gate (AUTH_ROUTES — welcome/register/login — redirect away if already logged in; everything else redirects to welcome if not), calls the matched screen's render(params) inside a try/catch (an uncaught screen error becomes a visible bad callout instead of a blank white page), wraps the result in the route's shell, and finally calls the screen's optional afterRender(params) — used by Dashboard to wire the anatomy board's hover cross-highlighting (§6.5.1) and by Analysis Loading to schedule its timed auto-advance, both of which need real DOM nodes that only exist after the HTML string has actually been assigned to #root.innerHTML.

```

1  /* EVA SQUARE – Hash router. Routes: #/route/param1/param2. Screens register
2  * themselves on SQ.screens.<file>.<fn>; this file just wires hash → screen.
3  *
4  * Ported from frontend-v2's ui/router.js. Two changes: (1) `state.booted`
5  * gating – this build's store fetches meta + session state from the
6  * backend asynchronously on boot, so the very first render (before that
7  * resolves) shows a loading shell instead of assuming a logged-out state;

```

```

8  * (2) "logged in" is `!!SQ.store.state.user` rather than
9  * `state.session.loggedIn`.
10 */
11 window.SQ = window.SQ || {};
12
13 (function () {
14   "use strict";
15
16   function parseHash() {
17     const raw = (location.hash || "").replace(/^#\//, "");
18     const parts = raw.split("/").filter(Boolean);
19     return { route: parts[0] || "", params: parts.slice(1) };
20   }
21
22   function navigate(path) { location.hash = "#/" + path; }
23
24   const AUTH_ROUTES = new Set(["welcome", "register", "login"]);
25
26   function routeTable() {
27     const S = SQ.screens;
28     return {
29       "": () => navigate(SQ.store.state.user ? "dashboard" : "welcome"),
30       welcome: { shell: "auth", render: S.auth.welcome },
31       register: { shell: "auth", render: S.auth.register },
32       login: { shell: "auth", render: S.auth.login },
33       terms: { shell: "setup", step: "terms", render: S.auth.terms },
34       consent: { shell: "setup", step: "consent", render: S.auth.consent },
35       dashboard: { shell: "app", title: "Dashboard", render: S.dashboard.render, afterRender: S.
36         dashboard.afterRender },
37       upload: { shell: "app", title: "Upload a Report", render: S.upload.render },
38       verify: { shell: "app", title: "Verify Extracted Results", render: S.verify.render },
39       questions: { shell: "app", title: "A Few Quick Questions", render: S.questions.render },
40       "analysis-loading": { shell: "app", title: "Analysing Your Report", render: S.analysis.
41         loading, afterRender: S.analysis.afterLoading },
42       "analysis-summary": { shell: "app", title: "Your Health Snapshot", render: S.analysis.
43         summary },
44       results: { shell: "app", title: "Result Detail", render: S.results.render },
45       domain: { shell: "app", title: "Health Domains", render: S.domain.render, afterRender: S.
46         domain.afterRender },
47       "action-plan": { shell: "app", title: "Your Action Plan", render: S.actionPlan.render },
48       referral: { shell: "app", title: "Referral & Next Steps", render: S.referral.render },
49       report: { shell: "app", title: "Full Report", render: S.report.render },
50       trends: { shell: "app", title: "Trends", render: S.trends.render, afterRender: S.trends.
51         afterRender },
52       profile: { shell: "app", title: "Health Profile", render: S.profile.render },
53       reports: { shell: "app", title: "Blood Reports", render: S.reportsLibrary.render },
54       settings: { shell: "app", title: "Settings", render: S.settings.render },
55     };
56   }
57
58   function render() {
59     if (!SQ.store.state.booted) {
60       document.getElementById("root").innerHTML = SQ.ui.loadingState("Loading EVA ...SQUARE");
61       return;
62     }
63
64     const { route, params } = parseHash();
65     const table = routeTable();
66     let def = table[route];
67
68     if (typeof def === "function") { def(); return; }
69     if (!def) { navigate(SQ.store.state.user ? "dashboard" : "welcome"); return; }
70
71     const loggedIn = !!SQ.store.state.user;

```

```
67   if (!AUTH_ROUTES.has(route) && !loggedIn) { navigate("welcome"); return; }
68   if (AUTH_ROUTES.has(route) && loggedIn) { navigate("dashboard"); return; }
69
70   let contentHtml;
71   try {
72     contentHtml = def.render(params) || SQ.ui.emptyState("Nothing to show yet.");
73   } catch (e) {
74     console.error("Screen render error for route", route, e);
75     contentHtml = SQ.ui.callout("bad", "This screen hit a snag", SQ.ui.esc(e.message));
76   }
77
78   let wrapped;
79   if (def.shell === "auth") wrapped = SQ.ui.shell.auth(contentHtml);
80   else if (def.shell === "setup") wrapped = SQ.ui.shell.setup(contentHtml, def.step);
81   else wrapped = SQ.ui.shell.app(route, contentHtml, { title: def.title });
82
83   document.getElementById("root").innerHTML = wrapped;
84   window.scrollTo(0, 0);
85   if (typeof def.afterRender === "function") {
86     try { def.afterRender(params); } catch (e) { console.error("afterRender error for route",
87       route, e); }
88   }
89
90   SQ.router = { navigate, parseHash, render };
91 }());
```

Listing 6.8: frontend/js/ui/router.js (full file).

6.5 New interactive features

The three files in this section have no equivalent in the original single-file prototype — they were built specifically for this Full-Structure version (§1.5).

6.5.1 js/ui/anatomy.js — interactive anatomy board

Requirement #1. Renders one SVG (class="anatomy-figure-wide", viewBox 980 × 560) containing a small stylised body silhouette in the centre and, either side of it, three <foreignObject>-embedded HTML cards per side — one per health domain — each joined to its organ by a thin leader line with a dot at both ends. Figure 6.2 is a schematic redrawing of this layout. Below roughly 880px viewport width, CSS switches to a plain stacked list (.anatomy-compact-list) instead, built from the same cardHtml() function so the two layouts never drift out of sync with each other.

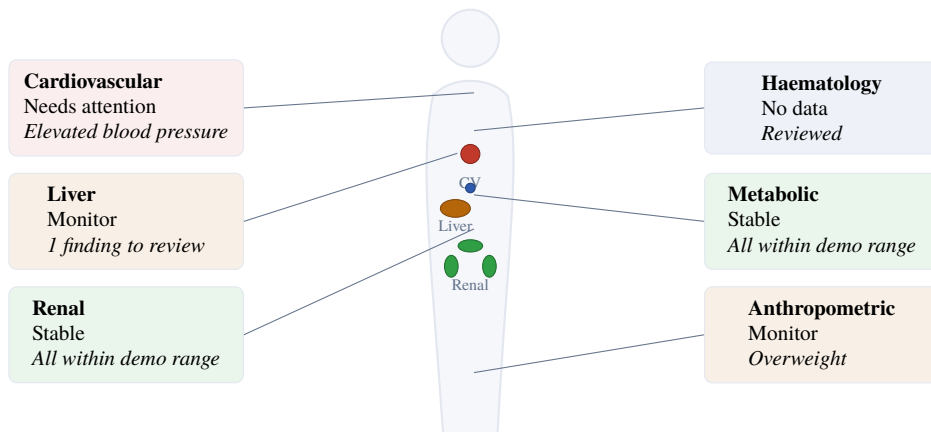


Figure 6.2: Schematic redrawing of the anatomy board's layout: six leader-lined domain cards, coloured by that domain's worst risk tone (§6.5.1), each pointing to a stylised organ anchor. The real board is one SVG with the cards embedded via <foreignObject>; this figure reproduces its geometry, not its exact organ artwork.

Every card and organ shares one tone- {good|warn|bad|neutral} CSS class, computed by domainTone(key, analysis): cardiovascular folds together seven separate sub-results (blood pressure, Framingham eligibility, LDL, AIP, Castelli I, ApoB-goal, Lp(a), triglyceride safety) via worstTone(); anthropometric derives its tone from BMI category plus a waist-circumference flag; the four remaining illustrative domains take the worst tone across their own results array. ANCHORS fixes each organ's click/leader-line target in a local 200 × 400 body coordinate space, mapped into the wider 980 × 560 board canvas by one shared BODY_TRANSFORM; afterRender() — called by the router immediately after the Dashboard's HTML is attached (§6.4.3) — wires the hover cross-highlighting between every element sharing a data-domain attribute, so hovering a card glows its organ and vice versa. Clicking either navigates straight to that domain's detail screen (§6.6.8). The module's own header comment is explicit that organ placement is stylised for readability, not anatomical precision — see the discussion of SVG-layer replacement options for a future, more anatomically faithful illustration.

```
1  /* EVA SQUARE – Interactive 2D anatomy board (Requirement #1).
2  *
3  * New for this FullStructure build (no v2 equivalent). Renders a body
4  * illustration in the centre with a "medical infographic" card for each
5  * health domain on either side (icon, status, one-line summary, chevron),
6  * each connected to its organ by a thin leader line – mirroring the
7  * reference layout the user asked to match, rather than the earlier
8  * version's plain floating shapes-on-a-body. Every card:
9  *   - is coloured by that domain's worst risk tone (good/warn/bad, or a
10 *     neutral grey when no data was available for it) – the same tone
11 *     vocabulary used everywhere else in the app;
```

```

12 * - darkens/lifts on hover, and cross-highlights its matching organ
13 * (and vice versa) via the shared `data-domain` attribute;
14 * - navigates to that domain's detail screen on click.
15 *
16 * The whole board (organs, leader lines and cards) is one SVG using
17 * <foreignObject> for the card HTML – that way the cards scale together
18 * with the illustration at any width instead of needing separate layout
19 * math. Below ~880px a plain stacked list (.anatomy-compact-list) is
20 * shown instead via CSS (see style.css) since the wide two-column layout
21 * doesn't have room to breathe on a narrow screen.
22 *
23 * This is a hand-authored illustrative diagram, not medical artwork –
24 * organ placement is stylised for readability, not anatomical precision.
25 */
26 window.SQ = window.SQ || {};
27 SQ.ui = SQ.ui || {};
28
29 (function () {
30   "use strict";
31   const U = () => SQ.ui;
32
33   const TONE_RANK = { bad: 3, warn: 2, good: 1, info: 0, neutral: -1 };
34   function worstTone(tones) {
35     let best = null;
36     tones.forEach((t) => {
37       if (!t) return;
38       if (best === null || (TONE_RANK[t] || 0) > (TONE_RANK[best] || 0)) best = t;
39     });
40     return best === null ? "neutral" : (best === "info" ? "good" : best);
41   }
42
43   function cardiovascularTone(cv) {
44     const tones = [];
45     if (cv.bp) tones.push(cv.bp.tone);
46     if (cv.framingham && cv.framingham.eligible) tones.push(cv.framingham.tone);
47     if (cv.lipids && cv.lipids.ldl && cv.lipids.ldl.value != null) tones.push(cv.lipids.ldl.value
48       >= 3.4 ? "warn" : "good");
49     if (cv.supplementary) {
50       if (cv.supplementary.aip) tones.push(cv.supplementary.aip.tone);
51       if (cv.supplementary.castelliI) tones.push(cv.supplementary.castelliI.tone);
52     }
53     if (cv.advanced && cv.advanced.apoB) tones.push(cv.advanced.apoB.aboveGoal ? "warn" : "good");
54     if (cv.advanced && cv.advanced.lpa) tones.push(cv.advanced.lpa.tone);
55     if (cv.tgSafety) tones.push(cv.tgSafety.tone);
56     return worstTone(tones);
57   }
58
59   function anthropometricTone(dom) {
60     if (!dom) return "neutral";
61     const cat = dom.bmiCategory;
62     let tone = cat === "Normal" ? "good" : cat === "Underweight" ? "warn" : cat === "Overweight" ?
63       "warn" : cat === "Obese" ? "bad" : "neutral";
64     if (dom.waistFlag && tone !== "bad") tone = "warn";
65     return tone;
66   }
67
68   function otherDomainTone(dom) {
69     if (!dom || !dom.results || !dom.results.length) return "neutral";
70     return worstTone(dom.results.map((r) => r.tone));
71   }
72
73   function domainTone(key, analysis) {
74     if (key === "cardiovascular") return cardiovascularTone(analysis.cv);
75     if (key === "anthropometric") return anthropometricTone(analysis.anthropometric);
76   }
77 }

```

```

74     return otherDomainTone(analysis[key]);
75 }
76
77 function domainSummary(key, analysis) {
78     if (key === "cardiovascular") {
79         const cv = analysis.cv;
80         return cv.bp ? cv.bp.label + " blood pressure" : "Reviewed";
81     }
82     if (key === "anthropometric") {
83         const dom = analysis.anthropometric;
84         return dom ? dom.bmiCategory + (dom.waistFlag ? " · waist above cut-off" : "") : "Reviewed";
85     }
86     const dom = analysis[key];
87     if (!dom || !dom.results) return "Reviewed";
88     const flagged = dom.results.filter((r) => r.tone !== "good").length;
89     return flagged ? `${flagged} finding(s) to review` : "All within demo range";
90 }
91
92 const STATUS_LABEL = { good: "Stable", warn: "Monitor", bad: "Needs attention", neutral: "No
    data" };
93 const STATUS_ICON = { cardiovascular: "♥", liver: "🔍", renal: "🔍", metabolic: "🔍", haematology:
    "🔍", anthropometric: "🔍" };
94
95 // ---- Body + organ illustration, in a fixed 200x400 local space; placed
96 // into the wider board canvas via a single shared transform (see
97 // BODY_TRANSFORM below), which is also used to map each organ's anchor
98 // point into board coordinates for the leader lines.
99 const BODY_SCALE = 1.15, BODY_TX = 375, BODY_TY = 40;
100 const BODY_TRANSFORM = `translate(${BODY_TX} ${BODY_TY}) scale(${BODY_SCALE})`;
101 function toBoard(x, y) { return { x: BODY_TX + x * BODY_SCALE, y: BODY_TY + y * BODY_SCALE }; }
102
103 // Anchor points or the organ's own click target, in local body space.
104 const ANCHORS = {
105     cardiovascular: { x: 97, y: 112 },
106     liver: { x: 76, y: 172 },
107     renal: { x: 100, y: 230 },
108     haematology: { x: 100, y: 144 },
109     metabolic: { x: 100, y: 200 },
110     anthropometric: { x: 150, y: 254 },
111 };
112
113 function bodyGroup(tones) {
114     // Decorative-only anatomy (no data / not clickable): the outline, a
115     // simple brain, and a pair of lungs, purely so the figure reads as a
116     // body rather than a bag of coloured blobs.
117     const outline = `

```

```

129 // twin-arc + point construction), centred on ANCHORS.cardiovascular -
130 // deliberately modest in size so it doesn't dominate the figure the
131 // way a hand-drawn curve did in the previous version.
132 const heart = `

```

```

179 function cardHtml(key, analysis, tone) {
180   const d = SQ.DOMAINS[key];
181   return `<div xmlns="http://www.w3.org/1999/xhtml" class="anatomy-card tone-${tone}" data-
      domain="${key}"
182     onclick="SQ.router.navigate('domain/${key}')" tabindex="0" role="button"
183     onkeydown="if(event.key==='Enter')SQ.router.navigate('domain/${key}')">
184     <div class="anatomy-card-icon">${STATUS_ICON[key]} || d.icon</div>
185     <div class="anatomy-card-body">
186       <div class="anatomy-card-title">${U().esc(d.label)}</div>
187       <div class="anatomy-card-status">${STATUS_LABEL[tone]}</div>
188       <div class="anatomy-card-desc">${U().esc(domainSummary(key, analysis))}</div>
189     </div>
190     <div class="anatomy-card-chevron">></div>
191   </div>`;
192 }
193
194 function foreignCard(key, x, y, analysis, tone) {
195   return `<foreignObject x="${x}" y="${y}" width="${COL_W}" height="${CARD_H}">${cardHtml(key,
      analysis, tone)}</foreignObject>`;
196 }
197
198 function leaderLine(key, cardX, cardY, fromRight, tone) {
199   const anchor = toBoard(ANCHORS[key].x, ANCHORS[key].y);
200   const startX = fromRight ? cardX : cardX + COL_W;
201   return `<g class="anatomy-leader tone-${tone}" data-domain="${key}">
202     <line x1="${startX}" y1="${cardY}" x2="${anchor.x}" y2="${anchor.y}" />
203     <circle class="anatomy-leader-dot" cx="${startX}" cy="${cardY}" r="4" />
204     <circle class="anatomy-leader-dot" cx="${anchor.x}" cy="${anchor.y}" r="4" />
205   </g>`;
206 }
207
208 function legendStrip() {
209   return `<div class="anatomy-legend-strip">
210     <span class="anatomy-legend-chip"><span class="dot tone-good"></span>Stable</span>
211     <span class="anatomy-legend-chip"><span class="dot tone-warn"></span>Monitor</span>
212     <span class="anatomy-legend-chip"><span class="dot tone-bad"></span>Needs attention</span>
213   </div>`;
214 }
215
216 function compactList(analysis) {
217   return `<div class="anatomy-compact-list">${SQ.DOMAIN_ORDER.map((key) => {
218     const tone = domainTone(key, analysis);
219     return cardHtml(key, analysis, tone);
220   }).join("")}</div>`;
221 }
222
223 function render(analysis) {
224   const tones = {};
225   SQ.DOMAIN_ORDER.forEach((key) => { tones[key] = domainTone(key, analysis); });
226
227   const leftCards = LEFT_KEYS.map((key, i) => foreignCard(key, LEFT_X, ROW_Y[i], analysis, tones
    [key])).join("");
228   const rightCards = RIGHT_KEYS.map((key, i) => foreignCard(key, RIGHT_X, ROW_Y[i], analysis,
    tones[key])).join("");
229   const leftLines = LEFT_KEYS.map((key, i) => leaderLine(key, LEFT_X, ROW_Y[i] + CARD_H / 2,
    false, tones[key])).join("");
230   const rightLines = RIGHT_KEYS.map((key, i) => leaderLine(key, RIGHT_X, ROW_Y[i] + CARD_H / 2,
    true, tones[key])).join("");
231
232   return `<div class="anatomy-board">
233     <div class="anatomy-board-head">
234       <h3>Body & Health Domains</h3>
235       ${legendStrip()}
236     </div>

```

```

237     <svg class="anatomy-figure-wide" viewBox="0 0 ${CANVAS_W} ${CANVAS_H}" xmlns="http://www.w3.
        org/2000/svg">
238         ${leftLines}${rightLines}
239         ${bodyGroup(tones)}
240         ${leftCards}${rightCards}
241     </svg>
242     ${compactList(analysis)}
243     <div class="anatomy-caption">Hover a card or organ to see how they connect. Click either to
        open that domain's full report.</div>
244 </div>`;
245 }
246
247 // Cross-highlight: hovering a card glows its matching organ/leader line
248 // (and vice versa). Wired after each render since #root is replaced
249 // wholesale by the router on every state change.
250 function afterRender() {
251     const root = document.getElementById("root");
252     if (!root) return;
253     const pairs = root.querySelectorAll("[data-domain]");
254     pairs.forEach((el) => {
255         const domain = el.getAttribute("data-domain");
256         el.addEventListener("mouseenter", () => {
257             root.querySelectorAll(`[data-domain="${domain}"]`).forEach((m) => m.classList.add("hovered
                "));
258         });
259         el.addEventListener("mouseleave", () => {
260             root.querySelectorAll(`[data-domain="${domain}"]`).forEach((m) => m.classList.remove("
                hovered"));
261         });
262     });
263 }
264
265 SQ.ui.anatomy = { render, afterRender, domainTone, domainSummary };
266 })();

```

Listing 6.9: frontend/js/ui/anatomy.js (full file).

6.5.2 js/ui/scaleRow.js — expandable reference-scale rows

Requirement #2. `toggleCell(id)` renders a small chevron button as the last cell of a results-table row; `panelRow(id, scaleKey, value, colspan)` renders the matching hidden `<tr>` placed immediately after it, toggled open/closed by `SQ.act.toggleScale()` (§6.3.1). `panelInner()` does the actual drawing: it looks `scaleKey` up in `SQ.referenceScales` (bound at boot from `GET /api/meta/reference-scales` — §5.2.5), renders one coloured `<div class="scale-band">` per band sized proportionally to its share of `[scale.min, scale.max]`, places a `.scale-marker` at the patient's own value's percentage position, and prints a legend and the scale's own clinical-source citation beneath it. `standalone()` is the non-table variant used by the single-marker Result Detail screen (§6.6.7), where the panel is always shown rather than toggled.

```

1  /* EVA SQUARE — Expandable reference-scale rows (Requirement #2).
2  *
3  * New for this FullStructure build. Every quantity row in a domain's
4  * Results tab (and the single-marker Result Detail screen) gets a small
5  * chevron button; clicking it expands a hidden row beneath showing the
6  * *whole* reference scale for that quantity as a coloured horizontal bar
7  * (green/amber/red bands = good/warn/bad, matching the tone vocabulary
8  * used everywhere else), with the patient's current value marked on it.
9  *
10 * Scale data comes from GET /api/meta/reference-scales (bound to
11 * SQ.referenceScales at boot — see backend/app/data/reference_scales.py
12 * for how each band was derived, and its honesty notes about which
13 * quantities have a real clinical source vs. an illustrative one).
14 */
15 window.SQ = window.SQ || {};
16 SQ.ui = SQ.ui || {};
17
18 (function () {
19   "use strict";
20   const U = () => SQ.ui;
21
22   function pctOf(value, min, max) {
23     if (value == null || !isFinite(value)) return null;
24     const clamped = Math.max(min, Math.min(max, value));
25     return ((clamped - min) / (max - min)) * 100;
26   }
27
28   // The clickable chevron cell — place this as the last <td> in a results row.
29   function toggleCell(id) {
30     return `<td style="width:36px;text-align:center"><button type="button" class="expand-toggle"
31       id="scale-toggle-${id}" onclick="SQ.act.toggleScale('${id}')" aria-label="Show full
32       reference scale" title="Show full reference scale"></button></td>`;
33   }
34
35   function panelInner(scaleKey, value) {
36     const scale = scaleKey && SQ.referenceScales ? SQ.referenceScales[scaleKey] : null;
37     if (!scale) {
38       return `<span style="color:var(--muted);font-size:.82rem">No reference scale is available
39         for this quantity yet.</span>`;
40     }
41     const range = scale.max - scale.min;
42     const bands = (scale.bands || []).map((b) => {
43       const from = Math.max(b.from, scale.min), to = Math.min(b.to, scale.max);
44       const width = Math.max(0, ((to - from) / range) * 100);
45       return `<div class="scale-band ${b.tone}" style="width:${width}%" title="${U().esc(b.label)}
46         : ${U().esc(from)} to ${U().esc(to)}${scale.unit ? " " + U().esc(scale.unit)} : """></div>`;
47     }).join("");
48     const markerPct = pctOf(value, scale.min, scale.max);
49     const marker = markerPct != null
50       ? `<div class="scale-marker" style="left:${markerPct}%" data-value="${U().esc(value)}${scale`

```

```

        .unit ? " " + U().esc(scale.unit) : ""}></div>` : "";
47 const seenTones = [];
48 const toneLabel = {};
49 (scale.bands || []).forEach((b) => { if (!toneLabel[b.tone]) { toneLabel[b.tone] = b.label;
    seenTones.push(b.tone); } });
50 const legend = seenTones.map((t) => `<span class="scale-legend-item"><span class="scale-legend
    -swatch" style="background:var(--${t})"></span>${U().esc(toneLabel[t])}</span>`).join("");
51
52 return `<div class="scale-title">${U().esc(scale.name)} – full reference scale</div>
53 <div class="scale-track">${bands}${marker}</div>
54 <div class="scale-labels"><span>${scale.min}${scale.unit ? " " + U().esc(scale.unit) : ""}</
    span><span>${scale.max}${scale.unit ? " " + U().esc(scale.unit) : ""}</span></div>
55 <div class="scale-legend">${legend}</div>
56 <div class="scale-source">Source: ${U().esc(scale.source)}</div>`;
57 }
58
59 // The hidden <tr> to place immediately after the main row. `colspan`
60 // should match the number of <td> columns (including the toggle cell)
61 // in the table this row belongs to.
62 function panelRow(id, scaleKey, value, colspan) {
63     return `<tr class="scale-row" id="scale-row-${id}"><td colspan="${colspan}"><div class="scale-
        panel">${panelInner(scaleKey, value)}</div></td></tr>`;
64 }
65
66 // Non-table variant: an always-visible scale card, used on the
67 // single-marker Result Detail screen (js/ui/screens/results.js).
68 function standalone(scaleKey, value) {
69     return `<div class="scale-panel" style="background:#fff;border:1px solid var(--border-2);
        border-radius:var(--radius-md)">${panelInner(scaleKey, value)}</div>`;
70 }
71
72 SQ.ui.scaleRow = { toggleCell, panelRow, standalone };
73 }());

```

Listing 6.10: frontend/js/ui/scaleRow.js (full file).

6.5.3 js/ui/trendChart.js — enlarged interactive trend chart

Requirement #4. `SQ.ui.trendModal.open(key, seriesEntry, unit)` — called by `SQ.act.openTrendChart()` (§6.3.1) when a card is clicked on the Trends screen (§6.6.12) — builds a full-screen overlay containing a `<canvas>` the user can zoom (mouse wheel, clamped to $0.05\times-3\times$ the data's natural span via `clampView()`) and pan (drag; single-finger touch drag on mobile). `draw()` does the actual rendering on every frame: grid-lines and axis labels are set with `ctx.font = "12px 'Times New Roman', ..."` directly on the canvas context, since a CSS font-family cannot reach text drawn by the Canvas API; each line's stroke is a `ctx.createLinearGradient()` with one colour stop per data point, where each stop's colour is that point's own risk tone (`toneForValue()`), reusing the same reference-scale band lookup as §6.5.2 — the canvas API interpolates smoothly between stops on its own, which is what produces the “line colour follows risk level as a smooth gradient” effect without any hand-rolled colour interpolation. `formulaHtml()` fits a simple least-squares line (`linearRegression()`) to the primary series and renders it in the `.trend-formula` CSS class (set to Cambria Math in `style.css`) alongside the first-to-last delta and a rising/falling/flat trend word.

```

1  /* EVA SQUARE – Trends "click to enlarge" interactive chart (Requirement #4).
2  *
3  * New for this FullStructure build. Clicking a quantity on the Trends
4  * screen opens this modal: a canvas line chart the user can zoom (mouse
5  * wheel / pinch) and pan (drag), with:
6  *   - axis labels set in Times New Roman directly via canvas ctx.font
7  *   (a CSS font-family can't reach canvas-drawn text, so this is done
8  *   in JS, not style.css);
9  *   - the plotted line coloured by the risk level of the quantity itself,
10 *   as a smooth gradient built from the same tone vocabulary as the
11 *   reference-scale bands (see js/ui/scaleRow.js / GET
12 *   /api/meta/reference-scales) – canvas linear gradients interpolate
13 *   between colour stops automatically, which gives the "smooth colour
14 *   change" the spec asks for without hand-rolled interpolation;
15 *   - a small formula readout rendered in Cambria Math (CSS class
16 *   .trend-formula, see style.css) showing the linear trend fitted to
17 *   the visible window.
18 * X axis = date, Y axis = the quantity's value, per spec.
19 */
20 window.SQ = window.SQ || {};
21 SQ.ui = SQ.ui || {};
22
23 (function () {
24   "use strict";
25   const U = () => SQ.ui;
26
27   function cssVar(name, fallback) {
28     try {
29       const v = getComputedStyle(document.documentElement).getPropertyValue(name).trim();
30       return v || fallback;
31     } catch (e) { return fallback; }
32   }
33
34   function toneColor(tone) {
35     if (tone === "good") return cssVar("--good", "#2f9e44");
36     if (tone === "warn") return cssVar("--warn", "#b5650a");
37     if (tone === "bad") return cssVar("--bad", "#c0392b");
38     return cssVar("--teal", "#0f8c7f");
39   }
40
41   function toneForValue(scaleKey, value) {
42     const scale = scaleKey && SQ.referenceScales ? SQ.referenceScales[scaleKey] : null;
43     if (!scale || value == null) return "neutral";
44     const band = (scale.bands || []).find((b) => value >= b.from && value <= b.to);
45     return band ? band.tone : "neutral";

```

```

46 }
47
48 // Builds { lines: [{name, scaleKey, points:[{x:index, date, value}]}] }
49 // from a /api/reports/:id/trends series for one key.
50 function buildLines(key, seriesEntry) {
51   const series = seriesEntry.series || [];
52   if (key === "homeBP") {
53     return [
54       { name: "Systolic", scaleKey: "BP_SYSTOLIC", points: series.map((p, i) => ({ x: i, date: p
55         .date, value: p.sbp })),
56       { name: "Diastolic", scaleKey: "BP_DIASTOLIC", points: series.map((p, i) => ({ x: i, date:
57         p.date, value: p.dbp })),
58     ];
59   }
60   const scaleKey = SQ.labDictionary && SQ.labDictionary[key] ? key : null;
61   const label = key === "weight" ? "Weight (kg)" : (SQ.labDictionary && SQ.labDictionary[key] ?
62     SQ.labDictionary[key].name : key);
63   return [{ name: label, scaleKey, points: series.map((p, i) => ({ x: i, date: p.date, value: p.
64     value }))] };
65 }
66
67 function linearRegression(points) {
68   const pts = points.filter((p) => p.value !== null);
69   const n = pts.length;
70   if (n < 2) return null;
71   const xs = pts.map((p) => p.x), ys = pts.map((p) => p.value);
72   const xBar = xs.reduce((a, b) => a + b, 0) / n, yBar = ys.reduce((a, b) => a + b, 0) / n;
73   let num = 0, den = 0;
74   for (let i = 0; i < n; i++) { num += (xs[i] - xBar) * (ys[i] - yBar); den += (xs[i] - xBar) *
75     (xs[i] - xBar); }
76   const slope = den === 0 ? 0 : num / den;
77   const intercept = yBar - slope * xBar;
78   return { slope, intercept };
79 }
80
81 // ---- modal state ----
82 let view = null; // { xMin, xMax, yMin, yMax }
83 let dragging = null; // { startX, startY, view0 }
84 let currentLines = null;
85 let currentUnit = "";
86
87 function dataBounds(lines) {
88   let xMin = Infinity, xMax = -Infinity, yMin = Infinity, yMax = -Infinity;
89   lines.forEach((line) => line.points.forEach((p) => {
90     if (p.value == null) return;
91     xMin = Math.min(xMin, p.x); xMax = Math.max(xMax, p.x);
92     yMin = Math.min(yMin, p.value); yMax = Math.max(yMax, p.value);
93   }));
94   if (!isFinite(xMin)) { xMin = 0; xMax = 1; }
95   if (!isFinite(yMin)) { yMin = 0; yMax = 1; }
96   if (xMin === xMax) { xMin -= 1; xMax += 1; }
97   const pad = (yMax - yMin) * 0.15 || 1;
98   return { xMin: xMin - 0.5, xMax: xMax + 0.5, yMin: yMin - pad, yMax: yMax + pad };
99 }
100
101 function draw() {
102   const canvas = document.getElementById("trend-canvas");
103   if (!canvas || !currentLines || !view) return;
104   const wrap = canvas.parentElement;
105   const cssW = wrap.clientWidth, cssH = 380;
106   const dpr = window.devicePixelRatio || 1;
107   canvas.width = cssW * dpr; canvas.height = cssH * dpr;
108   canvas.style.width = cssW + "px"; canvas.style.height = cssH + "px";
109   const ctx = canvas.getContext("2d");

```

```

105   ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
106   ctx.clearRect(0, 0, cssW, cssH);
107
108   const padL = 58, padR = 18, padT = 18, padB = 40;
109   const plotW = cssW - padL - padR, plotH = cssH - padT - padB;
110
111   function xPix(x) { return padL + ((x - view.xMin) / (view.xMax - view.xMin)) * plotW; }
112   function yPix(y) { return padT + plotH - ((y - view.yMin) / (view.yMax - view.yMin)) * plotH;
113     }
114
115   // gridlines + axis labels (Times New Roman, set directly on canvas –
116   // this is the one place a font-family has to be applied in JS rather
117   // than CSS, since canvas text ignores stylesheet fonts).
118   ctx.strokeStyle = "#e3e7ef"; ctx.lineWidth = 1;
119   ctx.fillStyle = "#64748b";
120   ctx.font = "12px 'Times New Roman', Times, serif";
121   ctx.textAlign = "right"; ctx.textBaseline = "middle";
122   const yTicks = 5;
123   for (let i = 0; i <= yTicks; i++) {
124     const y = view.yMin + (i / yTicks) * (view.yMax - view.yMin);
125     const py = yPix(y);
126     ctx.beginPath(); ctx.moveTo(padL, py); ctx.lineTo(cssW - padR, py); ctx.stroke();
127     ctx.fillText(Math.round(y * 100) / 100 + (currentUnit ? " " + currentUnit : ""), padL - 8,
128       py);
129   }
130
131   // x labels: pick evenly spaced visible points
132   const allDates = [];
133   currentLines[0].points.forEach((p) => { allDates[p.x] = p.date; });
134   ctx.textAlign = "center"; ctx.textBaseline = "top";
135   const visibleFrom = Math.max(0, Math.floor(view.xMin)), visibleTo = Math.min(allDates.length -
136     1, Math.ceil(view.xMax));
137   const span = Math.max(1, visibleTo - visibleFrom);
138   const step = Math.max(1, Math.round(span / 6));
139   for (let i = visibleFrom; i <= visibleTo; i += step) {
140     if (allDates[i] == null) continue;
141     ctx.fillText(allDates[i], xPix(i), cssH - padB + 8);
142   }
143
144   // axis lines
145   ctx.strokeStyle = "#b9c2d0"; ctx.beginPath();
146   ctx.moveTo(padL, padT); ctx.lineTo(padL, cssH - padB); ctx.lineTo(cssW - padR, cssH - padB);
147   ctx.stroke();
148
149   // clip to plot area so pan/zoom doesn't paint over axes
150   ctx.save();
151   ctx.beginPath(); ctx.rect(padL, padT, plotW, plotH); ctx.clip();
152
153   currentLines.forEach((line, li) => {
154     const pts = line.points.filter((p) => p.value != null);
155     if (pts.length < 1) return;
156     // gradient built from each point's risk tone – canvas interpolates
157     // smoothly between the colour stops we add here.
158     const grad = ctx.createLinearGradient(xPix(pts[0].x), 0, xPix(pts[pts.length - 1].x), 0);
159     const denom = Math.max(1, pts.length - 1);
160     pts.forEach((p, i) => {
161       const tone = toneForValue(line.scaleKey, p.value);
162       const stop = Math.min(1, Math.max(0, i / denom));
163       try { grad.addColorStop(stop, toneColor(tone)); } catch (e) { /* out-of-range stop guard */ }
164     });
165     ctx.strokeStyle = pts.length > 1 ? grad : toneColor(toneForValue(line.scaleKey, pts[0].value));
166     ctx.lineWidth = li === 0 ? 3 : 2;

```

```

163     ctx.setLineDash(li === 0 ? [] : [6, 4]);
164     ctx.beginPath();
165     pts.forEach((p, i) => { const x = xPix(p.x), y = yPix(p.value); if (i === 0) ctx.moveTo(x, y); else ctx.lineTo(x, y); });
166     ctx.stroke();
167     ctx.setLineDash([]);
168     pts.forEach((p) => {
169         const tone = toneForValue(line.scaleKey, p.value);
170         ctx.fillStyle = toneColor(tone);
171         ctx.beginPath(); ctx.arc(xPix(p.x), yPix(p.value), li === 0 ? 4 : 3, 0, Math.PI * 2); ctx.fill();
172         ctx.strokeStyle = "#fff"; ctx.lineWidth = 1.4; ctx.stroke();
173     });
174 });
175 ctx.restore();
176 }
177
178 function clampView(v, bounds) {
179     const minSpanX = (bounds.xMax - bounds.xMin) * 0.05;
180     const maxSpanX = (bounds.xMax - bounds.xMin) * 3;
181     let spanX = v.xMax - v.xMin;
182     spanX = Math.max(minSpanX, Math.min(maxSpanX, spanX));
183     let cx = (v.xMin + v.xMax) / 2;
184     v.xMin = cx - spanX / 2; v.xMax = cx + spanX / 2;
185     const minSpanY = (bounds.yMax - bounds.yMin) * 0.05;
186     const maxSpanY = (bounds.yMax - bounds.yMin) * 3;
187     let spanY = v.yMax - v.yMin;
188     spanY = Math.max(minSpanY, Math.min(maxSpanY, spanY));
189     let cy = (v.yMin + v.yMax) / 2;
190     v.yMin = cy - spanY / 2; v.yMax = cy + spanY / 2;
191     return v;
192 }
193
194 function wireInteractions(bounds) {
195     const wrap = document.getElementById("trend-canvas-wrap");
196     const canvas = document.getElementById("trend-canvas");
197     if (!wrap || !canvas) return;
198
199     wrap.addEventListener("wheel", (e) => {
200         e.preventDefault();
201         const rect = canvas.getBoundingClientRect();
202         const mx = e.clientX - rect.left, my = e.clientY - rect.top;
203         const padL = 58, padR = 18, padT = 18, padB = 40;
204         const plotW = rect.width - padL - padR, plotH = rect.height - padT - padB;
205         const dataX = view.xMin + ((mx - padL) / plotW) * (view.xMax - view.xMin);
206         const dataY = view.yMax - ((my - padT) / plotH) * (view.yMax - view.yMin);
207         const factor = e.deltaY > 0 ? 1.12 : 0.89;
208         view.xMin = dataX - (dataX - view.xMin) * factor;
209         view.xMax = dataX + (view.xMax - dataX) * factor;
210         view.yMin = dataY - (dataY - view.yMin) * factor;
211         view.yMax = dataY + (view.yMax - dataY) * factor;
212         clampView(view, bounds);
213         draw();
214     }, { passive: false });
215
216     wrap.addEventListener("mousedown", (e) => {
217         dragging = { startX: e.clientX, startY: e.clientY, view0: Object.assign({}, view) };
218     });
219     window.addEventListener("mousemove", (e) => {
220         if (!dragging) return;
221         const rect = canvas.getBoundingClientRect();
222         const padL = 58, padR = 18, padT = 18, padB = 40;
223         const plotW = rect.width - padL - padR, plotH = rect.height - padT - padB;
224         const dx = (e.clientX - dragging.startX) / plotW * (dragging.view0.xMax - dragging.view0.

```

```

xMin);
225   const dy = (e.clientY - dragging.startY) / plotH * (dragging.view0.yMax - dragging.view0.
      yMin);
226   view.xMin = dragging.view0.xMin - dx; view.xMax = dragging.view0.xMax - dx;
227   view.yMin = dragging.view0.yMin + dy; view.yMax = dragging.view0.yMax + dy;
228   draw();
229 });
230 window.addEventListener("mouseup", () => { dragging = null; });
231
232 // touch support (pinch not implemented; single-finger drag pans)
233 let touchStart = null;
234 wrap.addEventListener("touchstart", (e) => {
235   if (e.touches.length === 1) touchStart = { x: e.touches[0].clientX, y: e.touches[0].clientY,
      view0: Object.assign({}, view) };
236 }, { passive: true });
237 wrap.addEventListener("touchmove", (e) => {
238   if (!touchStart || e.touches.length !== 1) return;
239   const rect = canvas.getBoundingClientRect();
240   const padL = 58, padR = 18, padT = 18, padB = 40;
241   const plotW = rect.width - padL - padR, plotH = rect.height - padT - padB;
242   const dx = (e.touches[0].clientX - touchStart.x) / plotW * (touchStart.view0.xMax -
      touchStart.view0.xMin);
243   const dy = (e.touches[0].clientY - touchStart.y) / plotH * (touchStart.view0.yMax -
      touchStart.view0.yMin);
244   view.xMin = touchStart.view0.xMin - dx; view.xMax = touchStart.view0.xMax - dx;
245   view.yMin = touchStart.view0.yMin + dy; view.yMax = touchStart.view0.yMax + dy;
246   draw();
247 }, { passive: true });
248 wrap.addEventListener("touchend", () => { touchStart = null; });
249
250 window.addEventListener("resize", draw);
251 }
252
253 function formulaHtml(lines) {
254   const primary = lines[0];
255   const reg = linearRegression(primary.points);
256   if (!reg) return "";
257   const trendWord = reg.slope > 0.001 ? "rising" : reg.slope < -0.001 ? "falling" : "flat";
258   return `<div class="trend-formula">= ${reg.slope.toFixed(3)}x + ${reg.intercept.toFixed(2)}&
      nbps&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
      y<sub>final</sub> - y<sub>initial</sub> = ${(primary.points.
        filter(p=>p.value!=null).slice(-1)[0].value - primary.points.filter(p=>p.value!=null)[0].
        value).toFixed(2)}&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~trend: ${trendWord}</div>`;
259 }
260
261 function open(key, seriesEntry, unit) {
262   currentLines = buildLines(key, seriesEntry);
263   currentUnit = unit || "";
264   const bounds = dataBounds(currentLines);
265   view = Object.assign({}, bounds);
266
267   const label = key === "homeBP" ? "Blood pressure" : key === "weight" ? "Weight" : (SQ.
      labDictionary && SQ.labDictionary[key] ? SQ.labDictionary[key].name : key);
268   const overlay = document.createElement("div");
269   overlay.className = "modal-overlay";
270   overlay.id = "trend-modal-overlay";
271   overlay.innerHTML = `<div class="trend-modal">
272     <div class="trend-modal-head">
273       <div><h3>${U().esc(label)}</h3><div class="sub">Horizontal axis: date &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
          Vertical axis: ${U().esc(label)}${unit ? " (" + U().esc(unit) + ")" : ""}. Line colour
          follows risk level.</div></div>
274       <button type="button" class="trend-modal-close" onclick="SQ.ui.trendModal.close()">x</
          button>
275     </div>
276     <div class="trend-modal-body">

```

```

277     <div class="trend-canvas-wrap" id="trend-canvas-wrap"><canvas id="trend-canvas"></canvas
278         ></div>
279     <div class="trend-modal-controls">
280         <span class="trend-modal-hint">Scroll to zoom · drag to pan</span>
281         <button type="button" class="btn outline sm" onclick="SQ.ui.trendModal.reset()">Reset
282             view</button>
283     </div>
284     ${formulaHtml(currentLines)}
285 </div>`;
286 overlay.addEventListener("click", (e) => { if (e.target === overlay) close(); });
287 document.body.appendChild(overlay);
288
289 requestAnimationFrame(() => { draw(); wireInteractions(bounds); });
290 this._bounds = bounds;
291 SQ.ui.trendModal._bounds = bounds;
292 }
293
294 function reset() {
295     if (SQ.ui.trendModal._bounds) { view = Object.assign({}, SQ.ui.trendModal._bounds); draw(); }
296 }
297
298 function close() {
299     const el = document.getElementById("trend-modal-overlay");
300     if (el) el.remove();
301     currentLines = null; view = null; dragging = null;
302 }
303
304 SQ.ui.trendModal = { open, close, reset };
305 })();

```

Listing 6.11: frontend/js/ui/trendChart.js (full file).

6.6 Screens (`js/ui/screens/`)

Fifteen files, twenty screens (several files hold more than one closely related screen), each registering itself onto `SQ.screens.<file>` and consumed only by the router's route table (§6.4.3). Every screen follows the same shape established by `store.js`'s `need()` pattern: check `state.activeReportId/report` existence/`report.status/analysis` presence, in that order, returning an empty/loading/not-ready state as soon as one of those checks fails, and only building the real screen once every dependency is actually cached. Content in this layer is, with the exception of the three modules just covered, ported line-for-line from the original single-file prototype — each subsection below notes explicitly where behaviour changed and where it is verbatim.

6.6.1 `js/ui/screens/auth.js` — Welcome, Register, Login, Terms, Consent

Five screens. `welcome()` is the marketing landing page (hero, the two one-click demo-scenario buttons, a five-step journey strip, four feature cards). `register()/login()` are plain forms submitted to `SQ.act.submitRegister/submitLogin` (§6.3.1); the only functional change from v2 is that Register now collects and submits a real password, since the backend hashes and verifies it (§5.1.6) rather than v2's placeholder "any email logs you in". `terms()` renders all ten Terms sections (§5.2.2) with a sticky table of contents and a required agree-checkbox gating `SQ.act.continueFromTerms()`. `consent()` renders the three required and two optional consent items (§5.2.2) as toggles bound to `SQ.act.setConsent()`, with the continue button disabled until `SQ.store.requiredConsentComplete()` is true.

```

1  /* EVA SQUARE – Screens –15: Welcome, Register, Login, Terms & Conditions,
2   * Consent. Ported from frontend-v2's ui/screens/auth.js – content is
3   * unchanged (still the same five screens, same copy, same legal draft,
4   * "SQUARE" renamed "EVA SQUARE" per the real brand). The only functional
5   * change is that Register now collects and submits a real password field
6   * (the backend hashes it – see backend/app/security.py) since v2's fake
7   * "any email logs you in" auth is replaced by real accounts.
8   */
9  window.SQ = window.SQ || {};
10 SQ.screens = SQ.screens || {};
11
12 (function () {
13   "use strict";
14   const U = () => SQ.ui;
15
16   function welcome() {
17     return `
18     <div class="auth-hero">
19       <div>
20         <h1>Understand your health results – clearly, safely, one report at a time.</h1>
21         <p class="lede">Upload a blood report and EVA SQUARE turns it into plain-language
22           explanations, a real cardiovascular risk assessment, and a personalised action plan –
23           never a diagnosis, always a next step you can act on.</p>
24         <div class="cta-row">
25           ${U().btn("Get started free", { href: "register", variant: "primary" })}
26           ${U().btn("I already have an account", { href: "login", variant: "outline" })}
27         </div>
28         <div class="trust-row">
29           <span> Private & encrypted</span>
30           <span> Built with real clinical guidelines</span>
31           <span> Malaysia-ready</span>
32         </div>
33       </div>
34       <div class="auth-card">
35         <h2>Try it instantly</h2>
36         <p class="sub">Skip typing – load a sample report and see the full experience end to end
37           .</p>

```

```

35     ${U().btn("Load a routine sample report", { block: true, variant: "navy", onclick: "SQ.act
36       .tryDemo('routine')" })}
37     <div style="height:10px"></div>
38     ${U().btn("Load an urgent-pathway sample", { block: true, variant: "outline", onclick: "SQ
39       .act.tryDemo('urgent')" })}
40     <div class="auth-secure-note"> Demo data only – no real patient information</div>
41   </div>
42   <div class="marketing-strip">
43     <div class="journey-row">
44       ${["Upload", "Verify", "Answer", "Analyse", "Act"].map((s, i) =>
45         `<div class="journey-step"><div class="num">${i + 1}</div><div><strong>${s}</strong></
46         div></div>${i < 4 ? `<div class="journey-connector"></div>` : ""}`
47       )}.join("")}
48     </div>
49     <div class="feature-cards">
50       ${U().card(`<h4> Real cardiovascular science</h4><p class="d" style="margin-top:6px;color
51         :var(--muted);font-size:.85rem">Blood pressure, Framingham risk, LDL/ApoB, Lp(a) –
52         implemented from a clinical specification, not guesses.</p>`)}
53       ${U().card(`<h4> Plain-language explanations</h4><p class="d" style="margin-top:6px;color
54         :var(--muted);font-size:.85rem">Every number comes with why it matters and what
55         discordant results mean together.</p>`)}
56       ${U().card(`<h4> Safety-first escalation</h4><p class="d" style="margin-top:6px;color:var
57         (--muted);font-size:.85rem">Emergency, prompt, routine or self-managed – you always
58         know what to do next.</p>`)}
59       ${U().card(`<h4> Longitudinal tracking</h4><p class="d" style="margin-top:6px;color:var
60         (--muted);font-size:.85rem">See how your markers move over time, not just a single
61         snapshot.</p>`)}
62     </div>
63   </div>`;
64 }
65
66 function register() {
67   return `
68     <div class="auth-hero" style="padding-top:40px">
69       <div>
70         <h2 style="font-size:2rem">Create your EVA SQUARE account</h2>
71         <p class="lede">Takes under a minute. You'll review our Terms and Consent next, before any
72           health information is processed.</p>
73       </div>
74       <div class="auth-card">
75         <h2>Sign up</h2>
76         <p class="sub">Step 1 of 3 – Account details</p>
77         <div class="auth-progress"><div style="width:33%"></div></div>
78         <form onsubmit="SQ.act.submitRegister(event)">
79           <div class="field"><label>Full name</label><input type="text" id="reg-name" required
80             placeholder="e.g. Dexter Tan"></div>
81           <div class="field"><label>Email address</label><input type="email" id="reg-email"
82             required placeholder="you@example.com"></div>
83           <div class="field"><label>Password</label><input type="password" id="reg-pw" required
84             minlength="4" placeholder="At least 4 characters"></div>
85           <div class="checkbox-row" style="margin-bottom:16px"><input type="checkbox" id="reg-
86             updates"><label for="reg-updates" style="font-size:.85rem;color:var(--muted)">Send
87             me occasional health education updates (optional)</label></div>
88           ${U().btn("Continue", { block: true, type: "submit" })}
89         </form>
90         <div class="auth-foot">Already have an account? <a href="#/login">Log in</a></div>
91         <div class="auth-secure-note"> Your details are encrypted in transit and at rest</div>
92       </div>
93     </div>`;
94 }
95
96 function login() {
97   return `

```

```

82   <div class="auth-hero" style="padding-top:60px">
83     <div>
84       <h2 style="font-size:2rem">Welcome back</h2>
85       <p class="lede">Log in to continue reviewing your health reports, action plan and trends
86         .</p>
87     </div>
88     <div class="auth-card">
89       <h2>Log in</h2>
90       <p class="sub">Use the email and password you registered with.</p>
91       <form onsubmit="SQ.act.submitLogin(event)">
92         <div class="field"><label>Email address</label><input type="email" id="login-email"
93           required placeholder="you@example.com"></div>
94         <div class="field"><label>Password</label><input type="password" id="login-pw" required
95           placeholder="....."></div>
96         ${U().btn("Log in", { block: true, type: "submit" })}
97       </form>
98       <div class="auth-divider">or</div>
99       ${U().btn("Continue with routine sample account", { block: true, variant: "outline",
100         onclick: "SQ.act.tryDemo('routine')" })}
101       <div class="auth-foot">New to EVA SQUARE? <a href="#/register">Create an account</a></div>
102     </div>`;
103   }
104
105   function terms() {
106     const toc = SQ.termsSections.map((s) => `<a class="toc-item" href="#tc-${s.id}">${U().esc(s.
107       title)}</a>`).join("");
108     const body = SQ.termsSections.map((s) => `
109       <div class="legal-section" id="tc-${s.id}">
110         <h3>${U().esc(s.title)}</h3>
111         ${s.body.map((p) => `<p>${U().esc(p)}</p>`).join("")}
112         ${s.list ? `<ul>${s.list.map((li) => `<li>${U().esc(li)}</li>`).join("")}</ul>` : ""}
113         ${s.note ? U().callout(s.note.tone, s.note.title, U().esc(s.note.text)) : ""}
114       </div>`).join("");
115     return `
116     <div class="legal-wrap">
117       <h1>Terms & Conditions</h1>
118       <div class="sub">${U().esc(SQ.ORG.legalName)} · Effective ${U().esc(SQ.ORG.effectiveDate)} ·
119         ${U().esc(SQ.ORG.version)}</div>
120       ${U().callout("info", null, "This is a sample legal draft for prototype purposes and has not
121         been reviewed by qualified Malaysian legal counsel.")}
122       <div class="legal-body">
123         <div class="toc-card card pad-sm">${toc}</div>
124         <div class="legal-doc">${body}</div>
125       </div>
126     </div>
127     <div class="sticky-footer-bar">
128       <div class="agree-text">
129         <label class="checkbox-row"><input type="checkbox" id="tc-agree"><span>I have read and
130           agree to the Terms & Conditions<span class="note">Required to continue</span></span></label>
131       </div>
132       ${U().btn("Continue to Consent", { onclick: "SQ.act.continueFromTerms()" })}
133     </div>`;
134   }
135
136   function consentItemHtml(c) {
137     const checked = SQ.store.state.consent.required[c.id];
138     return `<div class="consent-item">
139       <div class="icon-badge">${c.icon}</div>
140       <div style="flex:1"><h4>${U().esc(c.title)}</h4><p>${U().esc(c.body)}</p>
141       <p style="font-size:.78rem;color:var(--muted-2);margin-top:6px">${U().esc(c.detail)}</p></div>
142       ${U().toggle("req-" + c.id, checked, `SQ.act.setConsent('required', '${c.id}', this.checked)`)}
143     </div>`;
144   }

```

```

136     }
137   </div>`;
138 }
139
140 function consentOptionalHtml(c) {
141   const checked = SQ.store.state.consent.optional[c.id];
142   return `

Listing 6.12: frontend/js/ui/screens/auth.js (full file).



## 6.6.2 js/ui/screens/dashboard.js — Dashboard



The application's home screen once a report is verified and analysed: a four-tile snapshot row (blood pressure, LDL, 10-year risk, action items done), the anatomy board (§6.5.1) rendered via U().anatomy.render(analysis), and two shortcut cards into the action plan and full report. afterRender() is exactly one line — U().anatomy.afterRender() — wiring the anatomy board's hover cross-highlighting once the HTML is actually in the DOM (§6.4.3). emptyDashboard() is shown instead for a first-time user with no reports at all, offering the same upload-/sample shortcuts as Welcome.



```

1 /* EVA SQUARE – Screen 6: Dashboard (User Control Centre).
2 *
3 * Ported from frontend-v2's ui/screens/dashboard.js. Two changes: (1)
4 * analysis is now fetched async from the backend (SQ.store.getAnalysis
5 * returns null while loading, or SQ.ui.notReady() if the report hasn't
6 * been verified yet – see js/ui/components.js); (2) Requirement #1 – after
7 * a report is analysed, an interactive 2D anatomy board is shown, one

```



122


```

```

8  * organ per health domain, coloured by that domain's risk level, that
9  * navigates to the domain's detail screen on click (js/ui/anatomy.js).
10 */
11 window.SQ = window.SQ || {};
12 SQ.screens = SQ.screens || {};
13
14 (function () {
15   "use strict";
16   const U = () => SQ.ui;
17
18   function render() {
19     if (SQ.store.state.reports === null) return U().loadingState("Loading your ...reports");
20     const report = SQ.store.getActiveReport();
21     if (!SQ.store.state.activeReportId) return emptyDashboard();
22     if (!report) return U().loadingState("Loading your latest ...report");
23     if (report.status !== "verified") return U().notReady(report);
24
25     const analysis = SQ.store.getAnalysis(report.id);
26     if (!analysis) return U().loadingState("Analysing your ...results");
27     const cv = analysis.cv;
28     const safety = analysis.safety;
29     const recs = analysis.recommendations;
30     const doneCount = recs.filter((r) => SQ.store.state.actionPlanDone[r.code]).length;
31
32     const safetyBanner = safety.overall === "emergency" || safety.overall === "urgent"
33       ? U().callout(safety.overall === "emergency" ? "bad" : "warn", safety.level.label, U().esc(
34         safety.level.description) + " " + U().esc(safety.level.action) + ` <a href="#/referral">
35           View guidance </a>`)
36       : "";
37
38     return `
39     ${safetyBanner}
40     <div class="section-title" style="margin-top:${safetyBanner ? "18px" : "0"}">Your latest
41       snapshot</div>
42     <div class="grid cols-4">
43       ${U().snapTile("Blood pressure", cv.bp ? cv.bp.sbp + "/" + cv.bp.dbp : "-", false)}
44       ${U().snapTile("LDL cholesterol", cv.lipids.ldl.value != null ? cv.lipids.ldl.value + " mmol
45         /L" : "-")}
46       ${U().snapTile("10-yr CV risk", cv.framingham.eligible ? cv.framingham.riskPercent + "%" : "
47         N/A", true)}
48       ${U().snapTile("Action items done", doneCount + " / " + recs.length)}
49     </div>
50
51     <div class="section-title">Your health, at a glance</div>
52     ${U().card(U().anatomy.render(analysis))}
53
54     <div class="grid cols-2" style="margin-top:22px">
55       ${U().card(`<h3 style="font-size:1.05rem">Continue your action plan</h3>
56         <p style="color:var(--muted);font-size:.88rem;margin-top:6px">${recs.length} personalised
57           recommendations from this report.</p>
58         <div style="margin-top:14px">${U().btn("Open action plan", { href: "action-plan" })}</div>
59       `)}
60       ${U().card(`<h3 style="font-size:1.05rem">Full report</h3>
61         <p style="color:var(--muted);font-size:.88rem;margin-top:6px">Download or print a
62           structured summary of this result.</p>
63         <div style="margin-top:14px">${U().btn("View full report", { href: "report", variant: "
64           outline" })}</div>`)}
65     </div>
66
67     <div class="section-title">Report used for this snapshot</div>
68     ${U().card(`<div class="card-row">
69       <div><strong>${U().esc(report.fileName)}</strong><div style="color:var(--muted);font-size
70         :.85rem;margin-top:4px">${U().esc(report.lab)} · ${U().esc(report.reportDate)}</div></div>

```

```

61     <div style="display:flex;gap:8px">${U().btn("View library", { href: "reports", variant: "
        outline", sm: true })}${U().btn("Upload new", { href: "upload", sm: true })}</div>
62   </div>`)}
63   `;
64 }
65
66 function afterRender() {
67   U().anatomy.afterRender();
68 }
69
70 function emptyDashboard() {
71   return `
72     ${U().card(`
73       <div class="empty-state">
74         <h3 style="color:var(--navy)">No reports yet</h3>
75         <p style="margin-top:8px;color:var(--muted)">Upload a blood report to get your first
          personalised health snapshot, or load a sample to explore EVA SQUARE.</p>
76         <div style="display:flex;gap:10px;justify-content:center;margin-top:18px;flex-wrap:wrap">
77           ${U().btn("Upload a report", { href: "upload" })}
78           ${U().btn("Load routine sample", { variant: "outline", onclick: "SQ.act.tryDemo('routine
              ')" })}
79           ${U().btn("Load urgent sample", { variant: "outline", onclick: "SQ.act.tryDemo('urgent')
              " })}
80         </div>
81       </div>`)}
82   `;
83 }
84
85 SQ.screens.dashboard = { render, afterRender };
86 })();

```

Listing 6.13: frontend/js/ui/screens/dashboard.js (full file).

6.6.3 js/ui/screens/upload.js — Upload a Report

A drag/drop dropzone plus two “use a sample instead” shortcuts. Content is unchanged from v2 — the underlying simplification (no live OCR; §5.4.4) exists on both the frontend copy here and the backend, and the screen’s own info callout says so explicitly to the user rather than pretending otherwise.

```

1  /* EVA SQUARE – Screen 7: Upload a Report (Data Acquisition & Processing,
2   * step 1). Ported unchanged from frontend-v2 – the upload flow itself
3   * (drag/drop → sample-matched extraction, since there's no live OCR in
4   * this basis version either) is identical; only SQ.act.handleFileUpload /
5   * startSampleUpload now call the backend (see js/app.js).
6   */
7  window.SQ = window.SQ || {};
8  SQ.screens = SQ.screens || {};
9
10 (function () {
11   "use strict";
12   const U = () => SQ.ui;
13
14   function render() {
15     return `
16       ${U().stepper(["Upload", "Verify", "Answer", "Analyse"], 0)}
17       ${U().card(`
18         <div class="dropzone">
19           <div class="dz-icon"></div>
20           <h3>Drag & drop your blood report</h3>
21           <div class="meta">PDF, JPG or PNG – up to 10MB. Your file is processed securely.</div>
22           <div class="dz-actions">
23             <label class="btn primary" for="file-input" style="cursor:pointer">Choose a file</label>

```

```

24     <input type="file" id="file-input" style="display:none" onchange="SQ.act.
        handleFileUpload(event)" accept=".pdf,.jpg,.jpeg,.png">
25     ${U().btn("Use a routine sample instead", { variant: "outline", onclick: "SQ.act.
        startSampleUpload('routine')" })}
26     ${U().btn("Use an urgent-pathway sample", { variant: "outline", onclick: "SQ.act.
        startSampleUpload('urgent')" })}
27     </div>
28   </div>
29   `)}
30   <div style="margin-top:18px">
31     ${U().callout("info", "This prototype simulates OCR", "Choosing any file loads a realistic
        sample lab report so you can see the full extraction → verification → questionnaire →
        analysis flow.")}
32   </div>
33   `;
34 }
35
36 SQ.screens.upload = { render };
37 }());

```

Listing 6.14: frontend/js/ui/screens/upload.js (full file).

6.6.4 js/ui/screens/verify.js — Verify Extracted Results

A two-column layout: an editable table of every extracted lab value (each row also showing a good/warn/bad confidence pill from `report.labConfidence`) beside a read-only `<pre>` of the report's original text, so the user can visually cross-check. Confirming calls `SQ.act.confirmVerify()`, which reads every `#verify-<code>` input back out of the DOM and PATCHes only the ones that were touched.

```

1  /* EVA SQUARE – Screen 8: Verify Extracted Results (Data Acquisition &
2  * Processing, step 2). Ported from frontend-v2 – content unchanged; only
3  * addition is a loading gate for the (usually instant) case where the
4  * active report isn't cached yet.
5  */
6  window.SQ = window.SQ || {};
7  SQ.screens = SQ.screens || {};
8
9  (function () {
10   "use strict";
11   const U = () => SQ.ui;
12
13   function confidencePill(pct) {
14     if (pct == null) return "";
15     const tone = pct >= 95 ? "good" : pct >= 80 ? "warn" : "bad";
16     return U().pill(tone, pct + "% confidence");
17   }
18
19   function render() {
20     if (!SQ.store.state.activeReportId) return U().emptyState("No report to verify yet – upload
        one first.");
21     const report = SQ.store.getActiveReport();
22     if (!report) return U().loadingState("Loading ...report");
23     const labs = report.labsRaw || {};
24     const conf = report.labConfidence || {};
25
26     const rows = Object.keys(labs).map((code) => {
27       const meta = SQ.labDictionary[code];
28       if (!meta) return "";
29       return `<tr>
30         <td>${U().esc(meta.name)}${meta.source === "illustrative" ? ` <span class="pill neutral"
            style="margin-left:6px">illustrative</span>` : ""}</td>
31         <td><input type="number" step="any" id="verify-${code}" value="${labs[code]}"></td>

```

```

32     <td>${U().esc(meta.unit)}</td>
33     <td>${confidencePill(conf[code])}</td>
34 </tr>`;
35 }).join("");
36
37 return `
38 ${U().stepper(["Upload", "Verify", "Answer", "Analyse"], 1)}
39 <div class="grid cols-2" style="align-items:start">
40   ${U().card(`
41     <h3 style="font-size:1.05rem">Extracted results</h3>
42     <p style="color:var(--muted);font-size:.86rem;margin:6px 0 14px">Compare each value
43       against your original report. Edit anything that looks wrong before continuing.</p>
44     <table class="data-table"><thead><tr><th>Test</th><th>Value</th><th>Unit</th><th>
45       Confidence</th></tr></thead><tbody>${rows}</tbody></table>
46     <div style="margin-top:18px;display:flex;gap:10px;flex-wrap:wrap">
47       ${U().btn("Confirm & continue", { onclick: "SQ.act.confirmVerify()" })}
48       ${U().btn("Re-upload instead", { href: "upload", variant: "outline" })}
49     </div>
50   `)}
51   ${U().card(`
52     <h3 style="font-size:1.05rem">Original report text</h3>
53     <p style="color:var(--muted);font-size:.86rem;margin:6px 0 14px">${U().esc(report.fileName)}
54       · ${U().esc(report.lab)} · ${U().esc(report.reportDate)}</p>
55     <pre style="white-space:pre-wrap;font-size:.78rem;background:#f8f9fb;border:1px solid var
56       (--border-2);border-radius:8px;padding:14px;max-height:420px;overflow:auto">${U().esc(
57       report.originalText || "")}</pre>
58   `)}
59 </div>
60 `;
61 }
62
63 SQ.screens.verify = { render };
64 })();

```

Listing 6.15: frontend/js/ui/screens/verify.js (full file).

6.6.5 js/ui/screens/questions.js — Triggered Health Questionnaire

Renders whichever questionnaire sections are currently active, using a small client-side re-implementation of `question_engine.py`'s `domains_present_in_labs()/active_sections()` (§5.3.9) — roughly ten lines, inlined here rather than factored into a shared “engine” layer, since this is the only screen that needs it and it must run synchronously against already-cached report data. Three question widget types are handled: plain choice rows, choice-with-a-conditional-text-field (shown only when “Yes” is selected), and the two-input bp-pair type for home blood pressure. Every change autosaves via `SQ.act.setAnswer/setAnswerText/setBP`, one PATCH per answer.

```

1  /* EVA SQUARE – Screen 9: Triggered Health Questionnaire (adaptive, per
2   * section). Ported from frontend-v2 – content and behaviour unchanged.
3   * The one addition is a small client-side re-implementation of
4   * question_engine.py's domainsPresentInLabs()/activeSections() (a ~10
5   * line pure function): v2 had this in js/engine/questionEngine.js as a
6   * local module; here it's inlined since it's the only screen that needs
7   * it and pulling in a whole "engine" layer for one filter function would
8   * be pure overhead – the *actual* clinical/questionnaire content
9   * (SQ.questionnaireSections) still comes straight from the backend
10  * (GET /api/meta/questionnaire-sections), this just decides which
11  * sections to show.
12  */
13 window.SQ = window.SQ || {};
14 SQ.screens = SQ.screens || {};
15

```

```

16 (function () {
17   "use strict";
18   const U = () => SQ.ui;
19
20   function domainsPresentInLabs(labValues) {
21     const present = new Set();
22     Object.keys(labValues || {}).forEach((code) => {
23       const meta = SQ.labDictionary[code];
24       if (meta && labValues[code] !== null) present.add(meta.domain);
25     });
26     return Array.from(present);
27   }
28   function activeSections(presentDomains) {
29     return (SQ.questionnaireSections || []).filter((s) => !s.triggerDomains || !s.triggerDomains.
       length || s.triggerDomains.some((d) => presentDomains.indexOf(d) >= 0));
30   }
31
32   function choiceRowFor(qid, options, selected) {
33     return `<div class="choice-row">${options.map((o) =>
34       U().choiceBtn(o, o === selected, `SQ.act.setAnswer('${qid}', '${o.replace(/'/g, '\\\'')}')`)
35     )}.join("")</div>`;
36   }
37
38   function questionHtml(q, answers) {
39     const val = answers[q.id];
40     if (q.type === "bp-pair") {
41       const v = val || {};
42       return `<div class="field"><label>${U().esc(q.prompt)}</label>
43         <div style="display:flex;gap:10px;align-items:center;max-width:260px">
44           <input type="number" placeholder="Systolic" value="${v.sbp !== null ? v.sbp : ""}"
45             onchange="SQ.act.setBP('${q.id}', this.value, null)">
46           <span style="color:var(--muted)"></span>
47           <input type="number" placeholder="Diastolic" value="${v.dbp !== null ? v.dbp : ""}"
48             onchange="SQ.act.setBP('${q.id}', null, this.value)">
49           <span style="color:var(--muted);font-size:.8rem">mmHg</span>
50         </div>
51         <span class="hint">${U().esc(q.why)}</span></div>`;
52     }
53     if (q.type === "choice-with-text") {
54       return `<div class="field"><label>${U().esc(q.prompt)}</label>
55         ${choiceRowFor(q.id, q.options, val)}
56         ${val === "Yes" ? `<input type="text" style="margin-top:10px" placeholder="${U().esc(q.
57           textLabel)}" value="${U().esc(answers[q.id + "Text"] || "")}" onchange="SQ.act.
58           setAnswerText('${q.id}Text', this.value)">` : ""}
59         <span class="hint">${U().esc(q.why)}</span></div>`;
60     }
61     return `<div class="field"><label>${U().esc(q.prompt)}</label>
62       ${choiceRowFor(q.id, q.options, val)}
63       <span class="hint">${U().esc(q.why)}</span></div>`;
64   }
65
66   function render() {
67     if (!SQ.store.state.activeReportId) return U().emptyState("No report yet – upload one first.");
68     ;
69     const report = SQ.store.getActiveReport();
70     if (!report) return U().loadingState("Loading ...report");
71     const answers = SQ.store.getAnswers(report.id);
72     if (!answers) return U().loadingState("Loading ...questionnaire");
73
74     const labs = report.labsVerified || report.labsRaw || {};
75     const presentDomains = domainsPresentInLabs(labs);
76     const sections = activeSections(presentDomains);
77
78     const body = sections.map((section) => `

```

```

74     ${U().card(`
75         <h3 style="font-size:1.02rem">${section.icon} ${U().esc(section.label)}</h3>
76         <div style="margin-top:14px">${section.questions.map((q) => questionHtml(q, answers)).join(
77             ""}</div>
78     `)}
79     `).join("<div style='height:16px'></div>");
80
81     return `
82     ${U().stepper(["Upload", "Verify", "Answer", "Analyse"], 2)}
83     ${U().callout("info", null, "These questions adapt to what was found in your report – sections
84         only appear when relevant.")}
85     <div style="height:14px"></div>
86     ${body}
87     <div style="margin-top:20px">${U().btn("Continue to analysis", { onclick: "SQ.act.
88         confirmQuestions()" })}</div>
89     `;
90 }
91
92 SQ.screens.questions = { render };
93 }());

```

Listing 6.16: frontend/js/ui/screens/questions.js (full file).

6.6.6 js/ui/screens/analysis.js — Analysis Loading and Summary

Two screens. `loading()` is a purely decorative fixed-progress animation — by the time it shows, the real analysis has already been triggered via `SQ.act.confirmQuestions()` (§6.3.1); its `afterRender()` just schedules a 1.3-second timed advance to `analysis-summary`. `summary()` is the first real look at the results: a one-line headline (`cv.headline`, or a client-side `headlineFallback()` mirroring `report_generator.headline()`'s logic, §5.3.7, for the one place that needs it standalone), a safety-level callout, a three-column grid of domain summary cards (each built by `U().anatomy.domainSummary()`, reused from the anatomy module rather than duplicated), and two “why this result” / “what’s next” cards.

```

1  /* EVA SQUARE — Screen 10 (Analysis Loading) & Screen 11 (Analysis
2  * Summary). Ported from frontend-v2 — the loading screen is still a
3  * purely decorative fixed animation (real analysis already ran via
4  * SQ.act.confirmQuestions() → POST /analyze before this screen shows);
5  * summary now reads analysis async with a loading/not-ready gate.
6  */
7  window.SQ = window.SQ || {};
8  SQ.screens = SQ.screens || {};
9
10 (function () {
11     "use strict";
12     const U = () => SQ.ui;
13
14     function loading() {
15         const steps = [
16             { label: "Reading verified results", status: "done" },
17             { label: "Applying cardiovascular calculators", status: "done" },
18             { label: "Cross-checking risk modifiers", status: "active" },
19             { label: "Preparing your report", status: "pending" },
20         ];
21         return `
22         ${U().stepper(["Upload", "Verify", "Answer", "Analyse"], 3)}
23         ${U().card(`
24             ${U().progressRing(72)}
25             <p style="text-align:center;color:var(--muted);margin-top:16px;font-size:.9rem">Analysing
26                 your results against the Cardiovascular Domain ...Specification</p>
27             <div style="max-width:360px;margin:20px auto 0">${U().checkSteps(steps)}</div>
28         `)}

```

```

28 `;
29 }
30
31 function afterLoading() {
32   setTimeout(() => { if (SQ.router.parseHash().route === "analysis-loading") SQ.router.navigate
33     ("analysis-summary"); }, 1300);
34 }
35
36 function summary() {
37   if (!SQ.store.state.activeReportId) return U().emptyState("No report yet – upload one first.")
38   ;
39   const report = SQ.store.getActiveReport();
40   if (!report) return U().loadingState("Loading ...report");
41   if (report.status !== "verified") return U().notReady(report);
42   const analysis = SQ.store.getAnalysis(report.id);
43   if (!analysis) return U().loadingState("Analysing your ...results");
44   const cv = analysis.cv;
45   const safety = analysis.safety;
46
47   const safetyBlock = safety.overall === "self"
48     ? U().callout("good", "No emergency or urgent flags", "Nothing in this report needs
49       immediate action. Continue to your action plan for routine, preventive steps.")
50     : U().callout(safety.overall === "emergency" ? "bad" : safety.overall === "urgent" ? "bad" :
51       "warn", safety.level.label, U().esc(safety.level.description) + " " + U().esc(safety.
52       level.action));
53
54   return `
55     <div class="section-title" style="margin-top:0">${U().esc(cv.headline || headlineFallback(cv))}
56       </div>
57     ${safetyBlock}
58     <div style="height:16px"></div>
59     <div class="grid cols-3">
60       ${SQ.DOMAIN_ORDER.map((key) => U().domainCard(key, U().anatomy.domainSummary(key, analysis),
61         {
62           clickable: true, onclick: `SQ.router.navigate('domain/${key}')`,
63           pill: key === "cardiovascular" ? U().pill("info", "Real spec") : U().pill("neutral", "
64             Illustrative"),
65         })}.join(""))}
66     </div>
67     <div class="grid cols-2" style="margin-top:20px">
68       ${U().card(`<h3 style="font-size:1.02rem">Why this result</h3>
69         <div class="stack" style="margin-top:12px">${(cv.discordanceNotes.length ? cv.
70           discordanceNotes : [{ title: "Consistent picture", text: "Your core results point in
71             the same direction – no discordant patterns were flagged this time." }]).map((n) => U
72             ().callout("info", n.title, U().esc(n.text))).join("")}</div>`)}
73       ${U().card(`<h3 style="font-size:1.02rem">What's next</h3>
74         <div class="stack" style="margin-top:12px">
75           <a class="link-row" href="#/action-plan"> Review your personalised action plan →</a>
76           <a class="link-row" href="#/referral"> See referral & escalation guidance →</a>
77           <a class="link-row" href="#/report"> View the full structured report →</a>
78           <a class="link-row" href="#/trends"> Track this over time →</a>
79         </div>`)}
80     </div>
81 `;
82 }
83
84 // report_generator.headline() is only exposed server-side via the
85 // "headline" report block, not as a standalone field – this mirrors its
86 // logic client-side for the one place (this summary heading) that needs
87 // it in isolation rather than the full report.
88 function headlineFallback(cv) {
89   if (!cv) return "Your report is ready to review.";
90   const bits = [];
91   if (cv.bp) bits.push(cv.bp.label.toLowerCase() + " blood pressure");

```

```

81   const ldl = (cv.lipids || {}).ldl || {};
82   if (ldl.value !== null) bits.push((ldl.value >= 3.4 ? "elevated" : "acceptable") + " LDL-
      cholesterol");
83   if (cv.framingham && cv.framingham.eligible) bits.push(cv.framingham.band.toLowerCase() + "
      estimated 10-year cardiovascular risk");
84   if (!bits.length) return "Your cardiovascular results are summarised below.";
85   return "This report shows " + bits.join(", ") + ".";
86 }
87
88 SQ.screens.analysis = { loading, afterLoading, summary };
89 })();

```

Listing 6.17: frontend/js/ui/screens/analysis.js (full file).

6.6.7 js/ui/screens/results.js — single-marker Result Detail

Route results/:code. The deepest detail view on any single quantity: latest value, classification, source (real spec vs. illustrative), a plain-language explanation pulled from SQ.education keyed by a small hard-coded code → education-code map, a mini value-history timeline, and — the one addition over v2 — an always-expanded reference-scale panel via U().scaleRow.standalone() (§6.5.2), since the whole screen is already scoped to this one marker.

```

1  /* EVA SQUARE – Screen 12: single-marker result detail. Route: results/:code
2  * Ported from frontend-v2. Cardiovascular classification text still says
3  * "see domain tab" (v2 didn't have a scale visual); this build additionally
4  * shows the quantity's full reference scale (Requirement #2) right on this
5  * page – always expanded here, since the whole screen is already about
6  * this one marker.
7  */
8  window.SQ = window.SQ || {};
9  SQ.screens = SQ.screens || {};
10
11 (function () {
12   "use strict";
13   const U = () => SQ.ui;
14
15   function render(params) {
16     const code = params[0];
17     if (!SQ.store.state.activeReportId) return U().emptyState("No report yet – upload one first.");
18     const report = SQ.store.getActiveReport();
19     if (!report) return U().loadingState("Loading ...report");
20     if (!code) return U().emptyState("Choose a marker from your report to see its detail here.");
21     const meta = SQ.labDictionary[code];
22     if (!meta) return U().emptyState("Unknown marker.");
23     if (report.status !== "verified") return U().notReady(report);
24     const analysis = SQ.store.getAnalysis(report.id);
25     if (!analysis) return U().loadingState("Analysing your ...results");
26     const value = (report.labsVerified || report.labsRaw || {})[code];
27     const history = (report.trendHistory && report.trendHistory[code]) || [];
28
29     let band, tone, source;
30     if (meta.domain === "cardiovascular") {
31       band = "See domain tab for full classification"; tone = "info"; source = "Cardiovascular
        Domain Specification";
32     } else {
33       const scale = SQ.referenceScales ? SQ.referenceScales[code] : null;
34       const b = scale && value !== null ? (scale.bands || []).find((x) => value >= x.from && value
        <= x.to) : null;
35       band = b ? b.label : "Not classified"; tone = b ? b.tone : "neutral"; source = scale ? scale
        .source : "";
36     }

```

```

37  const eduCode = { LAB_LDL: "EDU_LDL_001", LAB_APOB: "EDU_APOB_001", LAB_LPA: "EDU_LPA_001",
38    LAB_GLU_FAST: "EDU_GLUCOSE_001", LAB_ALT: "EDU_ALT_001" }[code];
39  const explanation = (eduCode && SQ.education[eduCode]) || "This value is compared against a
40    reference range to help you understand whether it needs attention.";
41  const scaleKey = SQ.referenceScales && SQ.referenceScales[code] ? code : null;
42
43  return `
44    <div class="card-row" style="margin-bottom:16px">
45      <div><h2 style="font-size:1.3rem;color:var(--navy)">${U().esc(meta.name)}</h2>
46      <p style="color:var(--muted);font-size:.88rem;margin-top:4px">${U().esc((meta.altNames ||
47        []).join(", "))}</p></div>
48      ${U().btn("Back to domain", { onclick: `SQ.router.navigate('domain/${meta.domain}')`,
49        variant: "outline", sm: true })}
50    </div>
51    <div class="grid cols-3">
52      ${U().snapTile("Latest value", value != null ? value + " " + meta.unit : —")}
53      ${U().snapTile("Classification", band, true)}
54      ${U().snapTile("Source", meta.source === "real" ? "Clinical spec" : "Illustrative", true)}
55    </div>
56    <div style="height:16px"></div>
57    <div class="grid cols-2">
58      ${U().card(`<h3 style="font-size:1rem">What this means</h3><p style="margin-top:10px;color
59        :#33405a;font-size:.9rem">${U().esc(explanation)}</p>${source ? `<p style="margin-top:10
60        px;font-size:.76rem;color:var(--muted-2)">Source: ${U().esc(source)}</p>` : ""}`)}
61      ${U().card(history.length ? `<h3 style="font-size:1rem">Trend</h3><div style="margin-top:14
62        px">${U().timeline(history.map((h) => ({ label: h.value, sub: h.date })))}</div>` : `<h3
63        style="font-size:1rem">Trend</h3><p style="margin-top:10px;color:var(--muted);font-size
64        :.88rem">Not enough history yet — this value will appear on Trends after your next
65        report.</p>`)}
66    </div>
67    ${scaleKey ? `<div style="height:16px"></div>${U().card(U().scaleRow.standalone(scaleKey,
68      value))}` : ""}
69  `;
70  }
71
72  SQ.screens.results = { render };
73  }());

```

Listing 6.18: frontend/js/ui/screens/results.js (full file).

6.6.8 js/ui/screens/domain.js — Health Domain detail (six tabs)

The single largest and most structurally important screen file, rendering all six health domains through one shared six-tab shell (Overview, Results, How It’s Calculated, Risk & Modifiers, Recommendations, Education) via route `domain/:key/:tab`. Cardiovascular (`cardiovascularTab()`) renders from the real engine’s full calculator register — an eleven-row results table (CV-01 through CV-14, skipping only the metabolic-syndrome/FH rows which surface instead under Risk & Modifiers), a “how it’s calculated” Friedewald-LDL flow diagram built from three `.connect-node` boxes, and the discordance notes. The five illustrative domains (`otherDomainTab()`) share one generic renderer instead, reading whatever `dom.results` array that domain’s engine produced (§5.3.3), with anthropometric as the one special case (BMI and waist rows built directly from `analysis.anthropometric` rather than a generic results array). `resultRow()` is the shared helper every results table row in this file goes through, building the visible `<tr>` plus its hidden scale-panel `<tr>` pair (§6.5.2) in one call.

```

1  /* EVA SQUARE – Screen 13: Domain detail (6-tab view), used for all six
2  * domains. Cardiovascular renders from the real engine result; the other
3  * five render from the illustrative otherDomains engine, always visibly
4  * labelled. Ported from frontend-v2's ui/screens/domain.js.
5  *
6  * Requirement #2 addition: every quantity row in the "Results" tab now
7  * carries an expand chevron ▼() that reveals the quantity's full

```

```

8  * reference scale beneath it (js/ui/scaleRow.js), coloured band-by-band,
9  * with the patient's value marked on it. SQ.act.toggleScale() (js/app.js)
10 * toggles the hidden row.
11 */
12 window.SQ = window.SQ || {};
13 SQ.screens = SQ.screens || {};
14
15 (function () {
16   "use strict";
17   const U = () => SQ.ui;
18   const TABS = [
19     { id: "overview", label: "Overview" },
20     { id: "results", label: "Results" },
21     { id: "how", label: "How It's Calculated" },
22     { id: "modifiers", label: "Risk & Modifiers" },
23     { id: "recs", label: "Recommendations" },
24     { id: "education", label: "Education" },
25   ];
26
27   function render(params) {
28     const key = params[0] || "cardiovascular";
29     const tab = params[1] || "overview";
30     if (!SQ.store.state.activeReportId) return U().emptyState("No report yet – upload one first.");
31
32     const report = SQ.store.getActiveReport();
33     if (!report) return U().loadingState("Loading ...report");
34     if (report.status !== "verified") return U().notReady(report);
35     const analysis = SQ.store.getAnalysis(report.id);
36     if (!analysis) return U().loadingState("Analysing your ...results");
37     const d = SQ.DOMAINS[key];
38     if (!d) return U().emptyState("Unknown domain.");
39
40     const tabsHtml = U().tabsNav(TABS.map((t) => ({ id: t.id, label: t.label, onclick: `SQ.router.
41       navigate('domain/${key}/${t.id}')` })), tab);
42
43     let body;
44     if (key === "cardiovascular") body = cardiovascularTab(tab, analysis);
45     else body = otherDomainTab(tab, key, analysis, report);
46
47     return `
48     <div class="card-row" style="margin-bottom:16px">
49       <div style="display:flex;gap:14px;align-items:center">
50         <div class="dic" style="width:46px;height:46px;border-radius:12px;background:${d.color}1a;
51           color:${d.color};display:flex;align-items:center;justify-content:center;font-size:1.3
52           rem">${d.icon}</div>
53         <div><h2 style="font-size:1.3rem;color:var(--navy)">${U().esc(d.label)}</h2>
54           ${key === "cardiovascular" ? U().pill("info", "Real clinical specification") : U().pill("
55             neutral", "Illustrative demo logic")}</div>
56       </div>
57       ${U().btn("Back to domains", { href: "dashboard", variant: "outline", sm: true })}
58     </div>
59     ${tabsHtml}
60     ${body}
61   `;
62 }
63
64 // Builds a [main <tr>, hidden scale <tr>] pair for one results row.
65 function resultRow(id, cells, tone, scaleKey, value) {
66   const main = `<tr><td>${cells[0]}</td><td>${cells[1]}</td><td>${cells[2]}</td><td>${U().dotInd
67     (tone)}</td>${U().scaleRow.toggleCell(id)}</tr>`;
68   const panel = U().scaleRow.panelRow(id, scaleKey, value, 5);
69   return main + panel;
70 }

```

```

66 function cardiovascularTab(tab, analysis) {
67   const cv = analysis.cv;
68   if (tab === "results") {
69     // scaleKey is null where no reference-scale entry exists (e.g. the
70     // ApoB/ApoA-I ratio, which has no single-number cut-off) – the
71     // expand row then explains that instead of showing a scale.
72     const rows = [
73       { id: "cv-bp", cells: ["CV-01 Blood pressure", cv.bp ? cv.bp.sbp + "/" + cv.bp.dbp + "
74         mmHg" : "-", cv.bp ? cv.bp.label : "-"], tone: cv.bp ? cv.bp.tone : "neutral",
75         scaleKey: "BP_SYSTOLIC", value: cv.bp ? cv.bp.sbp : null },
76       { id: "cv-frs", cells: ["CV-02 Framingham 10-yr risk", cv.framingham.eligible ? cv.
77         framingham.riskPercent + "%" : "N/A", cv.framingham.eligible ? cv.framingham.band : "
78         Not eligible"], tone: cv.framingham.eligible ? cv.framingham.tone : "neutral",
79         scaleKey: "FRS_RISK", value: cv.framingham.eligible ? cv.framingham.riskPercent : null
80       },
81       { id: "cv-nonhdl", cells: ["CV-03 Non-HDL-C", cv.lipids.nonHdl + " mmol/L", "-"], tone: "
82         neutral", scaleKey: "NON_HDL", value: cv.lipids.nonHdl },
83       { id: "cv-ldl", cells: ["CV-04 LDL-C", cv.lipids.ldl.value + " mmol/L" + (cv.lipids.ldl.
84         measured ? "" : " (calculated)"), "-"], tone: "neutral", scaleKey: "LAB_LDL", value:
85         cv.lipids.ldl.value },
86       { id: "cv-aip", cells: ["CV-05 AIP", cv.supplementary.aip ? cv.supplementary.aip.value : -
87         "", cv.supplementary.aip ? cv.supplementary.aip.band : "-"], tone: cv.supplementary.
88         aip ? cv.supplementary.aip.tone : "neutral", scaleKey: "AIP", value: cv.supplementary.
89         aip ? cv.supplementary.aip.value : null },
90       { id: "cv-castelli1", cells: ["CV-06 Castelli I", cv.supplementary.castelliI ? cv.
91         supplementary.castelliI.value : "-"], cv.supplementary.castelliI ? cv.supplementary.
92         castelliI.band : "-"], tone: cv.supplementary.castelliI ? cv.supplementary.castelliI.
93         tone : "neutral", scaleKey: "CASTELLI_I", value: cv.supplementary.castelliI ? cv.
94         supplementary.castelliI.value : null },
95       { id: "cv-castelli2", cells: ["CV-07 Castelli II", cv.supplementary.castelliII ? cv.
96         supplementary.castelliII.value : "-"], cv.supplementary.castelliII ? cv.supplementary.
97         castelliII.band : "-"], tone: cv.supplementary.castelliII ? cv.supplementary.
98         castelliII.tone : "neutral", scaleKey: "CASTELLI_II", value: cv.supplementary.
99         castelliII ? cv.supplementary.castelliII.value : null },
100      { id: "cv-apob", cells: ["CV-09 ApoB", cv.advanced.apoB ? cv.advanced.apoB.value + " mg/dL
101        " : "-", cv.advanced.apoB ? (cv.advanced.apoB.aboveGoal ? "Above goal" : "At/under
102        goal") : "-"], tone: cv.advanced.apoB ? (cv.advanced.apoB.aboveGoal ? "warn" : "good")
103        : "neutral", scaleKey: "LAB_APOB", value: cv.advanced.apoB ? cv.advanced.apoB.value :
104        null },
105      { id: "cv-apor", cells: ["CV-10 ApoB/ApoA-I", cv.advanced.apoRatio ? cv.advanced.apoRatio.
106        value : "-", "Lower is more favourable"], tone: "neutral", scaleKey: null, value: null
107      },
108      { id: "cv-lpa", cells: ["CV-11 Lp(a)", cv.advanced.lpa ? cv.advanced.lpa.value + " " + cv.
109        advanced.lpa.unit : "-"], cv.advanced.lpa ? cv.advanced.lpa.band : "-"], tone: cv.
110        advanced.lpa ? cv.advanced.lpa.tone : "neutral", scaleKey: "LAB_LPA", value: cv.
111        advanced.lpa ? cv.advanced.lpa.value : null },
112      { id: "cv-tg", cells: ["CV-14 Triglycerides", cv.lipids.tg + " mmol/L", cv.tgSafety ? cv.
113        tgSafety.tier : "-"], tone: cv.tgSafety ? cv.tgSafety.tone : "neutral", scaleKey: "
114        LAB_TG", value: cv.lipids.tg },
115    ];
116    return U().card(`<table class="data-table"><thead><tr><th>Calculator</th><th>Value</th><th>
117      Classification</th><th></th><th></th></tr></thead><tbody>
118      ${rows.map((r) => resultRow(r.id, r.cells, r.tone, r.scaleKey, r.value)).join("")}
119    </tbody></table>`);
120  }
121  if (tab === "how") {
122    return U().card(`
123      <div class="connect-flow">
124        <div class="connect-node" style="background:var(--info-bg);color:var(--info)">TC · HDL-C
125          · TG</div>
126        <div class="connect-arrow"></div>
127        <div class="connect-node" style="background:var(--teal-lighter);color:var(--teal-dark)">
128          Friedewald<br>LDL-C engine</div>
129        <div class="connect-arrow"></div>

```

```

96     <div class="connect-node" style="background:${cv.lipids.ldr.value >= 3.4 ? "var(--warn-
          bg);color:var(--warn)" : "var(--good-bg);color:var(--good)"}">LDL-C<br>${cv.lipids.
          ldr.value} mmol/L</div>
97   </div>
98   <div class="connect-result"><span class="pill-lg"> ${cv.lipids.ldr.measured ? "Measured
          directly by the lab" : (cv.lipids.ldr.calc.valid ? "Calculated via Friedewald formula
          (TG < 4.5 mmol/L)" : cv.lipids.ldr.calc.reason)}</span></div>
99   <div class="hr"></div>
100  <p style="font-size:.88rem;color:var(--muted)">Every cardiovascular figure on this screen
          traces back to the Cardiovascular Domain Specification's calculator register (CV--01CV
          -14) – nothing here is a hard-coded guess.</p>
101  `);
102  }
103  if (tab === "modifiers") {
104    return `<div class="stack">
105      ${U().callout(cv.fh.triggered ? "warn" : "good", "Familial hypercholesterolaemia trigger",
        U().esc(cv.fh.triggered ? cv.fh.flags.join(" ") + " " + cv.fh.note : "No FH trigger
        flags present. " + cv.fh.note))}
106      ${U().callout(cv.metabolicSyndrome.positive ? "warn" : "info", "Metabolic syndrome (3 of 5
        criteria)", `${cv.metabolicSyndrome.criteriaMet} of ${cv.metabolicSyndrome.
        criteriaKnown || 5} criteria met.` + (cv.metabolicSyndrome.note ? " " + U().esc(cv.
        metabolicSyndrome.note) : ""))}
107      ${cv.discordanceNotes.map((n) => U().callout("info", n.title, U().esc(n.text))).join("")}
108    </div>`;
109  }
110  if (tab === "recs") return recsList("cardiovascular", analysis);
111  if (tab === "education") return educationList(["EDU_LDL_001", "EDU_APOB_001", "EDU_BP_001", "
        EDU_FRS_001", "EDU_LPA_001"]);
112  return `<div class="grid cols-3">
113    ${U().snapTile("Blood pressure", cv.bp ? cv.bp.sbp + "/" + cv.bp.dbp : "-")}
114    ${U().snapTile("10-yr CV risk", cv.framingham.eligible ? cv.framingham.riskPercent + "%" : "
        N/A")}
115    ${U().snapTile("LDL-C", cv.lipids.ldr.value + " mmol/L")}
116  </div>
117  <div style="height:16px"></div>
118  ${U().card(`<h3 style="font-size:1rem">Headline</h3><p style="margin-top:8px;color:#33405a">${
        U().esc(headlineText(cv))</p>`)}
119  `;
120  }
121
122  function headlineText(cv) {
123    if (!cv) return "Your report is ready to review.";
124    const bits = [];
125    if (cv.bp) bits.push(cv.bp.label.toLowerCase() + " blood pressure");
126    const ldr = (cv.lipids || {}).ldr || {};
127    if (ldr.value !== null) bits.push((ldr.value >= 3.4 ? "elevated" : "acceptable") + " LDL-
        cholesterol");
128    if (cv.framingham && cv.framingham.eligible) bits.push(cv.framingham.band.toLowerCase() + "
        estimated 10-year cardiovascular risk");
129    if (!bits.length) return "Your cardiovascular results are summarised below.";
130    return "This report shows " + bits.join(", ") + ".";
131  }
132
133  function otherDomainTab(tab, key, analysis, report) {
134    const dom = analysis[key];
135    if (tab === "results") {
136      const results = key === "anthropometric"
137        ? [
138          { id: "anthro-bmi", name: "BMI", value: dom.bmi, band: dom.bmiCategory, tone: dom.
            bmiCategory === "Normal" ? "good" : "warn", scaleKey: "BMI" },
139          { id: "anthro-waist", name: "Waist circumference", value: dom.waist, band: dom.
            waistFlag ? "Above cut-off" : "Within cut-off", tone: dom.waistFlag ? "warn" : "
            good", scaleKey: (report.profile && report.profile.sex === "F") ? "WAIST_F" : "
            WAIST_M" }

```

```

140     ].map((r) => ({ id: r.id, cells: [r.name, r.scaleKey === "BMI" ? r.value : r.value + "
      cm", r.band], tone: r.tone, scaleKey: r.scaleKey, value: r.value }));
141     : (dom.results || []).map((r) => ({ id: "res-" + r.code, cells: [r.name, r.value + (r.unit
      ? " " + r.unit : ""), r.label || r.band], tone: r.tone, scaleKey: r.code, value: r.
      value }));
142     if (!results.length) return U().emptyState("No results available for this domain in the
      current report.");
143     return U().card(`<table class="data-table"><thead><tr><th>Test</th><th>Value</th><th>Band</
      th><th></th><th></th></tr></thead><tbody>
144       ${results.map((r) => resultRow(r.id, r.cells, r.tone, r.scaleKey, r.value)).join("")}
145     </tbody></table>`);
146   }
147   if (tab === "how") return U().card(`${U().callout("info", "Illustrative logic", U().esc((dom.
      note || "") + " Values are compared against simple fixed thresholds, not a validated
      clinical model.))}`);
148   if (tab === "modifiers") return U().card(`<p style="color:var(--muted);font-size:.9rem">No
      cross-marker modifier logic is implemented for this illustrative domain yet – it will be
      added alongside a real domain specification.</p>`);
149   if (tab === "recs") return recsList(key, analysis);
150   if (tab === "education") {
151     const map = { metabolic: ["EDU_GLUCOSE_001"], liver: ["EDU_ALT_001"], renal: [], haematology
      : [], anthropometric: [] };
152     return educationList(map[key] || []);
153   }
154   const flagged = (dom.results || []).filter((r) => r.tone !== "good");
155   return `${U().callout("info", null, U().esc(dom.note || ""))}
156   <div style="height:14px"></div>
157   ${U().card(flagged.length ? `<h3 style="font-size:1rem">${flagged.length} result(s) outside
      the demo range</h3><ul class="list-plain" style="margin-top:10px">${flagged.map((r) => `<
      li>${U().esc(r.name)}<span>${r.value} ${U().esc(r.unit || "")}</span></li>`).join("")}</ul>
      >` : `<p>All illustrative results for this domain are within the demo reference range.</p>
      `)});
158 }
159
160 function recsList(domain, analysis) {
161   const recs = analysis.recommendations.filter((r) => r.domain === domain || r.domain === "
      general");
162   if (!recs.length) return U().emptyState("No specific recommendations for this domain right now
      .");
163   return `<div class="stack">${recs.map((r) => `
164     ${U().card(`<div class="card-row"><div><h4 style="font-size:.95rem;color:var(--navy)">${U().
      esc(r.issue)}</h4><p style="margin-top:6px;color:#33405a;font-size:.88rem">${U().esc(r.
      action)}</p></div>${U().pill("teal", r.category)}</div>`)}
165   `).join("")}</div>`;
166 }
167
168 function educationList(codes) {
169   const items = codes.map((c) => SQ.education[c]).filter(Boolean).map((text) => ({ text }));
170   if (!items.length) return U().emptyState("No education notes linked to this domain yet.");
171   return `<div class="stack">${items.map((e) => U().card(`<p style="font-size:.9rem;color:#33405
      a">${U().esc(e.text)}</p>`)).join("")}</div>`;
172 }
173
174 SQ.screens.domain = { render };
175 }());

```

Listing 6.19: frontend/js/ui/screens/domain.js (full file).

6.6.9 js/ui/screens/actionPlan.js — Action Plan

A snapshot row (recommendation count, completed count, overall safety pathway) followed by one priority card per recommendation returned by analysis.recommendations (§5.3.6), each showing its issue/action/why/benefit/prec

up fields and a completion toggle wired to `SQ.act.toggleAction()` (§5.4.6).

```

1  /* EVA SQUARE – Screen 14: Action Plan (Intervention & Escalation).
2  * Ported from frontend-v2 – content unchanged; analysis + completion
3  * state now come from the backend (GET /api/reports/:id/analysis,
4  * GET/POST /api/action-plan – see js/store.js).
5  */
6  window.SQ = window.SQ || {};
7  SQ.screens = SQ.screens || {};
8
9  (function () {
10   "use strict";
11   const U = () => SQ.ui;
12
13   function render() {
14     if (!SQ.store.state.activeReportId) return U().emptyState("No report yet – upload one first.");
15     ;
16     const report = SQ.store.getActiveReport();
17     if (!report) return U().loadingState("Loading ...report");
18     if (report.status !== "verified") return U().notReady(report);
19     const analysis = SQ.store.getAnalysis(report.id);
20     if (!analysis) return U().loadingState("Analysing your ...results");
21     const recs = analysis.recommendations;
22     const done = SQ.store.state.actionPlanDone;
23     const doneCount = recs.filter((r) => done[r.code]).length;
24
25     const cards = recs.map((r, i) => `
26       ${U().card(`
27         <div class="priority-card">
28           <div class="top-row">
29             ${U().numBadge(i + 1)}
30             <div style="flex:1">
31               <h4>${U().esc(r.issue)}</h4>
32               <p>${U().esc(r.action)}</p>
33             </div>
34             ${U().toggle("done-" + r.code, !!done[r.code], `SQ.act.toggleAction('${r.code}'))`)}
35           </div>
36           <div class="hr"></div>
37           <p style="font-size:.84rem;color:#33405a"><strong>Why:</strong> ${U().esc(r.why)}</p>
38           <p style="font-size:.84rem;color:#33405a;margin-top:6px"><strong>Benefit:</strong> ${U().
39             .esc(r.benefit)}</p>
40           <div class="actions">
41             <span style="color:var(--warn)Δ"> ${U().esc(r.precaution)}</span>
42             <span style="color:var(--muted)">Follow-up: ${U().esc(r.followUp)}</span>
43           </div>
44         </div>
45       `)}
46     `).join("");
47
48     return `
49     <div class="grid cols-3" style="margin-bottom:20px">
50       ${U().snapTile("Recommendations", recs.length)}
51       ${U().snapTile("Completed", doneCount + " / " + recs.length)}
52       ${U().snapTile("Overall pathway", analysis.safety.level.label, true)}
53     </div>
54     ${recs.length ? `<div class="stack">${cards}</div>` : U().emptyState("No recommendations were
55       triggered for this report.")}
56   `;
57 }
58
59 SQ.screens.actionPlan = { render };
60 })();

```

Listing 6.20: frontend/js/ui/screens/actionPlan.js (full file).

6.6.10 js/ui/screens/referral.js — Referral & Next Steps

Renders the four-level safety ladder (U().safetyLadder(), §6.4.1) against SQ.referralLevels (§5.2.4), highlighting the rung matching safety.overall (§5.3.5), followed by a table of every individual trigger reason that fed into that overall level. An emergency-tier report additionally shows a prominent bad-tone callout with the 999 guidance repeated verbatim from the app's own safety copy.

```

1  /* EVA SQUARE – Screen 15: Referral & Next Steps (four-level safety
2   * ladder). Ported from frontend-v2 – content unchanged.
3   */
4  window.SQ = window.SQ || {};
5  SQ.screens = SQ.screens || {};
6
7  (function () {
8    "use strict";
9    const U = () => SQ.ui;
10
11   function render() {
12     if (!SQ.store.state.activeReportId) return U().emptyState("No report yet – upload one first.");
13     ;
14     const report = SQ.store.getActiveReport();
15     if (!report) return U().loadingState("Loading ...report");
16     if (report.status !== "verified") return U().notReady(report);
17     const analysis = SQ.store.getAnalysis(report.id);
18     if (!analysis) return U().loadingState("Analysing your ...results");
19     const safety = analysis.safety;
20
21     const reasonRows = safety.reasons.length
22       ? `<table class="data-table" style="margin-top:16px"><thead><tr><th>Domain</th><th>Trigger</th><th>Level</th></tr></thead><tbody>
23         ${safety.reasons.map((r) => `<tr><td>${U().esc(r.domain)}</td><td>${U().esc(r.reason)}</td><td>${U().pill(r.level === "emergency" || r.level === "urgent" ? "bad" : "warn",
24           r.level)}</td></tr>`).join("")}
25       </tbody></table>`
26       : "";
27
28     return `
29     ${safety.overall === "emergency" ? U().callout("bad", "This looks like it may need emergency
30       care", "If you have chest pain, severe breathlessness, sudden weakness, facial droop or
31       speech difficulty – call 999 or go to the nearest emergency department now.") : ""}
32     ${U().card(`
33       <h3 style="font-size:1.05rem">Recommended pathway</h3>
34       <p style="color:var(--muted);font-size:.87rem;margin:6px 0 16px">Based on every safety
35         trigger across your cardiovascular results and the other illustrative domains.</p>
36       ${U().safetyLadder(SQ.referralLevels, safety.overall)}
37       ${reasonRows}
38     `)}
39     <div style="margin-top:18px;display:flex;gap:10px;flex-wrap:wrap">
40       ${U().btn("View action plan", { href: "action-plan", variant: "outline" })}
41       ${U().btn("View full report", { href: "report" })}
42     </div>
43     <div style="margin-top:18px">${U().callout("info", "Not a substitute for professional care", "
44       EVA SQUARE does not diagnose or treat. This referral guidance reflects the information you
45       provided and should be confirmed with a qualified healthcare professional.")}</div>
46   `;
47 }
48
49 SQ.screens.referral = { render };
50 })();

```

Listing 6.21: frontend/js/ui/screens/referral.js (full file).

6.6.11 js/ui/screens/report.js — Full Report

The printable/downloadable view: iterates `analysis.report.blocks` (§5.3.7) and renders each as its own titled section, in the exact block order the report generator assembled them in. `window.print()` is wired directly to a toolbar button; the `.print-hidden` CSS class hides the toolbar itself and the app-shell chrome when the browser's print stylesheet is active.

```

1  /* EVA SQUARE – Screen 16: Full structured report (printable/downloadable).
2   * Ported from frontend-v2 – content unchanged; org name reads EVA SQUARE.
3   */
4  window.SQ = window.SQ || {};
5  SQ.screens = SQ.screens || {};
6
7  (function () {
8    "use strict";
9    const U = () => SQ.ui;
10
11   function render() {
12     if (!SQ.store.state.activeReportId) return U().emptyState("No report yet – upload one first.");
13     ;
14     const report = SQ.store.getActiveReport();
15     if (!report) return U().loadingState("Loading ...report");
16     if (report.status !== "verified") return U().notReady(report);
17     const analysis = SQ.store.getAnalysis(report.id);
18     if (!analysis) return U().loadingState("Analysing your ...results");
19     const r = analysis.report;
20
21     return `
22     <div class="card-row print-hidden" style="margin-bottom:16px">
23       <div style="color:var(--muted);font-size:.85rem">Generated ${new Date(r.generatedAt).
24         toLocaleString()}</div>
25       <div style="display:flex;gap:10px">${U().btn("Print / Save as PDF", { onclick: "window.print
26         ()", variant: "outline", sm: true })}</div>
27     </div>
28     ${U().card(`
29       <h2 style="font-size:1.4rem">EVA SQUARE Health Report</h2>
30       <p style="color:var(--muted);font-size:.88rem;margin-top:4px">${U().esc(r.scenario)} · ${U().
31         esc(report.reportDate)} · ${U().esc(SQ.ORG.legalName)}</p>
32       <div class="hr"></div>
33       ${r.blocks.map((b) => `
34         <div class="legal-section">
35           <h3 style="font-size:1.02rem">${U().esc(b.title)}</h3>
36           <p style="white-space:pre-line;color:#33405a;font-size:.9rem;margin-top:6px">${U().nl2br
37             (b.body)}</p>
38         </div>
39       `).join("")}
40       <div class="hr"></div>
41       <p style="font-size:.78rem;color:var(--muted-2)">This report is generated for informational
42         and educational purposes only and is not a medical diagnosis. If you may have a medical
43         emergency, call 999.</p>
44     `)}}
45   `;
46 }
47
48 SQ.screens.report = { render };
49 })();

```

Listing 6.22: frontend/js/ui/screens/report.js (full file).

6.6.12 js/ui/screens/trends.js — Trends

Reads GET /api/reports/:id/trends (direction/delta pre-computed server-side by trend_engine.py, §5.3.8) and renders one clickable card per tracked quantity, each with a small inline SVG sparkline (sparkline(), built from a hand-rolled polyline — no charting library) and a direction-arrow pill. Clicking a card is the Requirement #4 entry point: onclick="SQ.act.openTrendChart(key)" opens the enlarged interactive modal (§6.5.3). unitFor() is exported (SQ.screens.trends.unitFor) since SQ.act.openTrendChart() needs the same unit lookup to pass into the modal.

```

1  /* EVA SQUARE — Screen 17: Trends (Longitudinal Management). Ported from
2  * frontend-v2's ui/screens/trends.js, reading from the backend's
3  * GET /api/reports/:id/trends (direction/delta computed server-side by
4  * backend/app/engines/trend_engine.py — see js/store.js#ensureTrends).
5  *
6  * Requirement #4: clicking a quantity's card now opens an enlarged,
7  * interactive canvas chart (js/ui/trendChart.js) instead of just showing
8  * the small sparkline — scroll to zoom, drag to pan, axis labels in Times
9  * New Roman, line colour following the quantity's own risk level.
10 */
11 window.SQ = window.SQ || {};
12 SQ.screens = SQ.screens || {};
13
14 (function () {
15     "use strict";
16     const U = () => SQ.ui;
17
18     function numeric(p) { return typeof p.value === "number" ? p.value : (typeof p.sbp === "number"
19         ? p.sbp : null); }
20     function sparklinePoints(series, width, height) {
21         if (!series || !series.length) return "";
22         const values = series.map(numeric).filter((v) => v != null);
23         const min = Math.min(...values), max = Math.max(...values);
24         const span = max - min || 1;
25         return series.map((p, i) => {
26             const v = numeric(p);
27             const x = (i / Math.max(1, series.length - 1)) * width;
28             const y = height - ((v - min) / span) * height;
29             return x.toFixed(1) + "," + y.toFixed(1);
30         }).join(" ");
31     }
32     function sparkline(series, color) {
33         const pts = sparklinePoints(series, 220, 60);
34         return `<svg viewBox="0 0 220 60" width="220" height="60" class="mini-chart"><polyline points
35             ="${pts}" fill="none" stroke="${color}" stroke-width="2.5" stroke-linecap="round" stroke-
36             linejoin="round"/></svg>`;
37     }
38     function unitFor(key) {
39         if (key === "homeBP") return "mmHg";
40         if (key === "weight") return "kg";
41         return (SQ.labDictionary && SQ.labDictionary[key]) ? SQ.labDictionary[key].unit : "";
42     }
43
44     function render() {
45         if (!SQ.store.state.activeReportId) return U().emptyState("No trend history yet — trends build
46             up as you add more reports.");
47         const report = SQ.store.getActiveReport();
48         if (!report) return U().loadingState("Loading ...report");
49         const trends = SQ.store.getTrends(report.id);
50         if (!trends) return U().loadingState("Loading ...trends");
51         if (!Object.keys(trends).length) return U().emptyState("No trend history yet — trends build up
52             as you add more reports.");
53
54         const cards = Object.keys(trends).map((key) => {

```

```

50     const t = trends[key];
51     const series = t.series;
52     const dir = t.direction, delta = t.delta;
53     const label = key === "homeBP" ? "Blood pressure" : key === "weight" ? "Weight (kg)" : (SQ.
        labDictionary[key] ? SQ.labDictionary[key].name : key);
54     const arrow = dir === "up" ? ↑ : dir === "down" ? ↓ : →;
55     const tone = key === "homeBP" || key === "weight" ? (dir === "down" ? "good" : dir === "up"
        ? "warn" : "neutral") : "info";
56     const last = series[series.length - 1];
57     const lastVal = last.value !== null ? last.value : (last.sbp + "/" + last.dbp);
58     return `<div class="card trend-card" onclick="SQ.act.openTrendChart('${key}')" title="Click
        to explore as an interactive chart">
59         <div class="card-row"><div><h4 style="font-size:.95rem;color:var(--navy)">${U().esc(label)}
            </h4>
60         <p style="font-size:.82rem;color:var(--muted);margin-top:2px">Latest: ${lastVal} · ${
            series[series.length - 1].date}</p></div>
61         ${U().pill(tone, arrow + " " + (delta > 0 ? "+" : "")) + delta}</div>
62         <div style="margin-top:10px">${sparkline(series, "#0f8c7f")}</div>
63     </div>`;
64     }).join("");
65
66     return `<div class="grid cols-2">${cards}</div>
67     <div style="margin-top:18px">${U().callout("info", null, "Trends compare values across your
        uploaded reports over time. More reports make these lines more meaningful. Click any card
        to explore it as an interactive, zoomable chart.")}</div>`;
68 }
69
70 function afterRender() { /* click handling is wired inline via onclick – nothing to bind post-
    render */ }
71
72 SQ.screens.trends = { render, afterRender, unitFor };
73 }());

```

Listing 6.23: frontend/js/ui/screens/trends.js (full file).

6.6.13 js/ui/screens/profile.js — Health Profile

A read-only summary screen aggregating state from three independent sources — the session user, the active report's profile block, and its questionnaire answers — each gated by its own loading check rather than one screen-wide gate, since none of the three blocks the others from rendering. `humanLabel()` is a small hard-coded map from questionnaire answer keys to display labels, used only here.

```

1  /* EVA SQUARE – Screen 18: Health Profile. Ported from frontend-v2 –
2  * content unchanged; account/consent come from the backend session,
3  * profile/answers/BMI come from the active report + its (possibly still
4  * loading) analysis, each gated independently since none of them block
5  * the rest of the page the way a dedicated analysis screen would.
6  */
7  window.SQ = window.SQ || {};
8  SQ.screens = SQ.screens || {};
9
10 (function () {
11     "use strict";
12     const U = () => SQ.ui;
13
14     function render() {
15         const user = SQ.store.state.user || {};
16         const report = SQ.store.getActiveReport();
17         const profile = report ? (report.profile || {}) : {};
18         const answers = report ? SQ.store.getAnswers(report.id) : null;
19         const analysis = (report && report.status === "verified") ? SQ.store.getAnalysis(report.id) :
            null;

```

```

20  const anthro = analysis ? analysis.anthropometric : null;
21
22  return `
23  <div class="grid cols-2">
24    ${U().card(`
25      <h3 style="font-size:1.02rem">Account</h3>
26      ${U().kvGrid([
27        { k: "Name", v: U().esc(user.fullName || -"") },
28        { k: "Email", v: U().esc(user.email || -"") },
29        { k: "Country", v: U().esc(profile.country || -"") },
30      ])}
31    `)}
32    ${U().card(`
33      <h3 style="font-size:1.02rem">Demographics & anthropometrics</h3>
34      ${U().kvGrid([
35        { k: "Age", v: (profile.age !== null ? profile.age : -"") },
36        { k: "Sex", v: profile.sex === "M" ? "Male" : profile.sex === "F" ? "Female" : -" " },
37        { k: "Height", v: (profile.height !== null ? profile.height + " cm" : -"") },
38        { k: "Weight", v: (profile.weight !== null ? profile.weight + " kg" : -"") },
39        { k: "Waist", v: (profile.waist !== null ? profile.waist + " cm" : -"") },
40        { k: "BMI", v: anthro ? anthro.bmi + " (" + anthro.bmiCategory + ")" : -" " },
41      ])}
42    `)}
43  </div>
44  <div class="section-title">Lifestyle & history (from your questionnaire)</div>
45  ${report ? (answers ? U().card(`
46    <ul class="list-plain">
47      ${Object.keys(answers).filter((k) => typeof answers[k] === "string").map((k) => `<li>${
48        humanLabel(k)}<span>${U().esc(answers[k])}</span></li>`).join("")}
49    </ul>
50    `) : U().loadingState("Loading ...questionnaire")) : U().emptyState("Answer the questionnaire on
51    a report to populate this section.")}
52  <div class="section-title">Consent status</div>
53  ${SQ.store.state.consent ? U().card(`
54    ${SQ.consentRequired.map((c) => `<div class="card-row" style="margin-bottom:8px"><span>${U().
55      .esc(c.title)}</span>${U().pill(SQ.store.state.consent.required[c.id] ? "good" : "bad",
56      SQ.store.state.consent.required[c.id] ? "Given" : "Not given"))</div>`).join("")}
57    ${SQ.consentOptional.map((c) => `<div class="card-row" style="margin-bottom:8px"><span>${U().
58      .esc(c.title)}</span>${U().pill(SQ.store.state.consent.optional[c.id] ? "good" : "
59      neutral", SQ.store.state.consent.optional[c.id] ? "Opted in" : "Opted out"))</div>`).
60      join("")}
61    <div style="margin-top:10px">${U().btn("Manage consent in Settings", { href: "settings",
62      variant: "outline", sm: true })}</div>
63    `) : U().loadingState("Loading consent ...status")}
64  `;
65  }
66
67  function humanLabel(key) {
68    const map = {
69      everDiagnosedHighBP: "Diagnosed with high blood pressure", bpMedication: "On BP medication",
70      familyHistoryPrematureCVD: "Family history of premature CVD", diagnosedDiabetes: "Diagnosed
71      with diabetes",
72      diabetesMedication: "On diabetes medication", familyHistoryDiabetes: "Family history of
73      diabetes",
74      activityLevel: "Activity level", alcoholIntake: "Alcohol intake", fattyLiverHistory: "Fatty
75      liver history",
76      smokingStatus: "Smoking status", regularMedication: "Regular medication", kidneyDisease: "
77      Kidney disease",
78      pregnancyStatus: "Pregnancy status",
79    };
80    return U().esc(map[key] || key);
81  }
82
83  SQ.screens.profile = { render };

```

```
72 }());
```

Listing 6.24: frontend/js/ui/screens/profile.js (full file).

6.6.14 js/ui/screens/reportsLibrary.js — Blood Reports library

A simple table of every report the user has ever uploaded or sampled (GET /api/reports), each row showing its verify status and whether it is the currently active report; “View” calls `SQ.act.openReport(id)`, which sets it active and routes to either Verify or the Full Report screen depending on `report.status`.

```
1  /* EVA SQUARE – Screen 19: Blood Reports library. Ported from
2  * frontend-v2 – content unchanged; reads the list from
3  * SQ.store.state.reports (GET /api/reports) instead of a local array.
4  */
5  window.SQ = window.SQ || {};
6  SQ.screens = SQ.screens || {};
7
8  (function () {
9    "use strict";
10   const U = () => SQ.ui;
11
12   function render() {
13     const library = SQ.store.state.reports;
14     if (library === null) return U().loadingState("Loading your ...reports");
15     if (!library.length) return ` ${U().emptyState("No reports uploaded yet.")}<div style="text-align:center">${U().btn("Upload a report", { href: "upload" })}</div>`;
16
17     const rows = library.map((r) => `
18       <tr>
19         <td><strong>${U().esc(r.fileName)}</strong><div style="color:var(--muted);font-size:.78rem">
20           >${U().esc(r.lab)}</div></td>
21         <td>${U().esc(r.reportDate)}</td>
22         <td>${U().pill(r.status === "verified" ? "good" : "warn", r.status === "verified" ? "Verified" : "Pending verify")}</td>
23         <td>${r.id === SQ.store.state.activeReportId ? U().pill("teal", "Active") : ""}</td>
24         <td style="display:flex;gap:8px">
25           ${U().btn("View", { sm: true, variant: "outline", onclick: `SQ.act.openReport('${r.id}')` })}
26         </td>
27       </tr>` ).join("");
28
29   return `
30     <div class="actions-row">${U().btn("Upload new report", { href: "upload" })}</div>
31     ${U().card(`<table class="data-table"><thead><tr><th>Report</th><th>Date</th><th>Status</th><th></th></tr></thead><tbody>${rows}</tbody></table>`)}
32   `;
33 }
34
35 SQ.screens.reportsLibrary = { render };
36 }());
```

Listing 6.25: frontend/js/ui/screens/reportsLibrary.js (full file).

6.6.15 js/ui/screens/settings.js — Settings

Units (metric/imperial) and notification preference (§5.4.3), the two optional consent toggles (mirroring the Consent screen’s own toggles, kept in sync since both write through the same `SQ.act.setConsent()/PATCH /api/consent` path), a log-out button, and links back to the Terms and Consent screens for review.

```
1  /* EVA SQUARE – Screen 20: Settings. Ported from frontend-v2 – content
2  * unchanged; settings/consent now come from the backend session.
```

```

3  */
4  window.SQ = window.SQ || {};
5  SQ.screens = SQ.screens || {};
6
7  (function () {
8    "use strict";
9    const U = () => SQ.ui;
10
11    function render() {
12      const s = SQ.store.state.settings || {};
13      const consent = SQ.store.state.consent;
14      return `
15      <div class="grid cols-2">
16        ${U().card(`
17          <h3 style="font-size:1.02rem">Preferences</h3>
18          <div class="card-row" style="margin-top:14px"><span>Units</span>
19            <div class="segmented" style="width:180px"><button class="${s.units === "metric" ? "
20              active" : ""}" onclick="SQ.act.setSetting('units','metric')">Metric</button><button
21                class="${s.units === "imperial" ? "active" : ""}" onclick="SQ.act.setSetting('units
22                  ', 'imperial')">Imperial</button></div>
23            </div>
24            <div class="card-row" style="margin-top:16px"><span>Notifications</span>${U().toggle("set-
25              notif", s.notifications, "SQ.act.setSetting('notifications', this.checked)")}</div>
26          `)}
27          ${U().card(`
28            <h3 style="font-size:1.02rem">Optional consent</h3>
29            ${consent ? SQ.consentOptional.map((c) => `<div class="card-row" style="margin-top:14px"><
30              span>${U().esc(c.title)}</span>${U().toggle("set-opt-" + c.id, consent.optional[c.id],
31                `SQ.act.setConsent('optional', '${c.id}', this.checked)`)}</div>`)}).join("") : U().
32              loadingState("...Loading")}
33            `)}
34          </div>
35          <div class="section-title">Account</div>
36          ${U().card(`
37            <div class="card-row"><span>Log out</span>${U().btn("Log out", { variant: "outline", sm:
38              true, onclick: "SQ.act.logout()" })}</div>
39          `)}
40          <div class="section-title">Legal</div>
41          ${U().card(`
42            <div class="card-row"><span>Terms & Conditions</span>${U().btn("Review", { href: "terms
43              ", variant: "outline", sm: true })}</div>
44            <div class="hr"></div>
45            <div class="card-row"><span>Consent record</span>${U().btn("Review", { href: "consent",
46              variant: "outline", sm: true })}</div>
47          `)}
48          `;
49    }
50
51    SQ.screens.settings = { render };
52  })();

```

Listing 6.26: frontend/js/ui/screens/settings.js (full file).

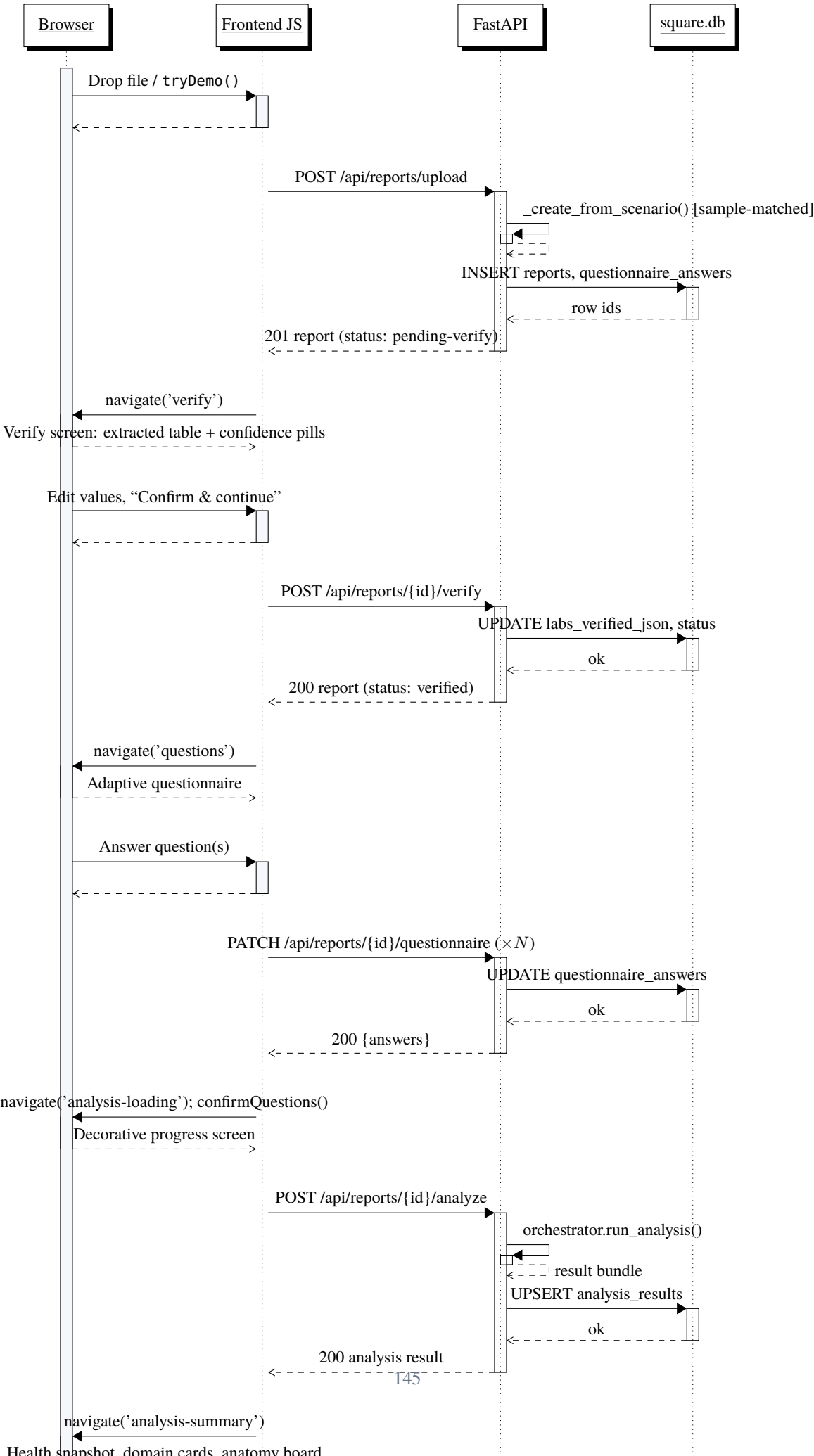
Chapter 7

Cross-Tier Data Flow

The previous two chapters documented the backend and frontend module by module. This chapter instead follows the data itself, end to end, across all three tiers, for the two flows that touch the most code in the application: establishing a session (already traced in Figure 5.3, §5.4.1), and the core upload → verify → questionnaire → analyse pipeline that turns an uploaded file into a full health snapshot.

7.1 The core pipeline, end to end

Figure 7.1 traces every request the frontend makes against a single report from the moment a file is chosen to the moment the Analysis Summary screen has real data to show. All four stages are handled by one router file, `app/routers/reports.py` (§5.4.4), operating on the same `reports` row throughout.



7.2 Why the frontend navigates before some calls resolve

§6.3.1 already calls out that `SQ.act.confirmQuestions()` is a deliberate exception to the “await, then navigate” rule every other action follows: it calls `SQ.router.navigate('analysis-loading')` *immediately*, and only then awaits `SQ.store.runAnalysis()`. This is safe specifically because the Analysis Loading screen (§6.6.6) has nothing real to show yet regardless — it is a fixed decorative animation with its own 1.3-second timer, not a screen that reads analysis out of the store. Every other action (register, login, verify, ...) navigates to a screen that *does* immediately try to read freshly-mutated state, so those must wait for the mutation to actually land in the store first, or the router would render against stale data for one frame.

7.3 The `need()` cache lifecycle

Every piece of report-scoped data (report detail, questionnaire answers, analysis, trends) passes through the same three-state cycle on the frontend, implemented once by `SQ.store.need()` (§6.2.2) and reused by four different getters. Figure 7.2 shows that cycle.

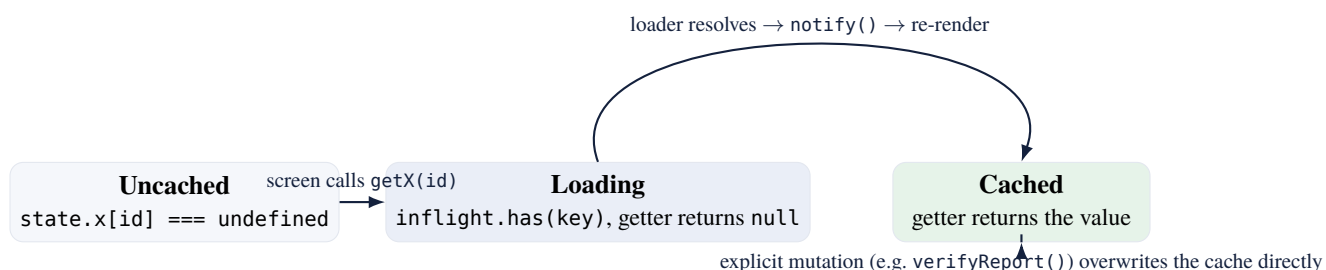


Figure 7.2: The cache lifecycle every `SQ.store.get*(id)` getter follows. A screen calling a getter while a value is loading always renders a loading state on that same tick — the getter never blocks or throws, so a screen never needs its own loading flag.

The de-duplication is what makes this safe to call from every screen’s render function on every re-render without triggering duplicate network requests: `need()` checks `inflight.has(key)` before starting a new load, so if Dashboard and the router’s own `afterRender` both end up asking for the same analysis in the same render pass, only one `fetch()` actually goes out.

7.4 Safety pathway aggregation

Figure 7.3 shows how the four-level referral ladder shown on the Referral screen (§6.6.10) is derived. Every domain analysis — the real cardiovascular engine and the five illustrative domains — produces its own local urgency signal first; `safety_engine.py` (§5.3.5) then takes the single worst signal across all six as `safety.overall`, using a fixed total ordering (`emergency > urgent > prompt > routine > self`). This one value is what the anatomy board’s tones (§6.5.1), the Dashboard/Analysis-summary safety banners, and the Referral screen’s highlighted rung all key off.

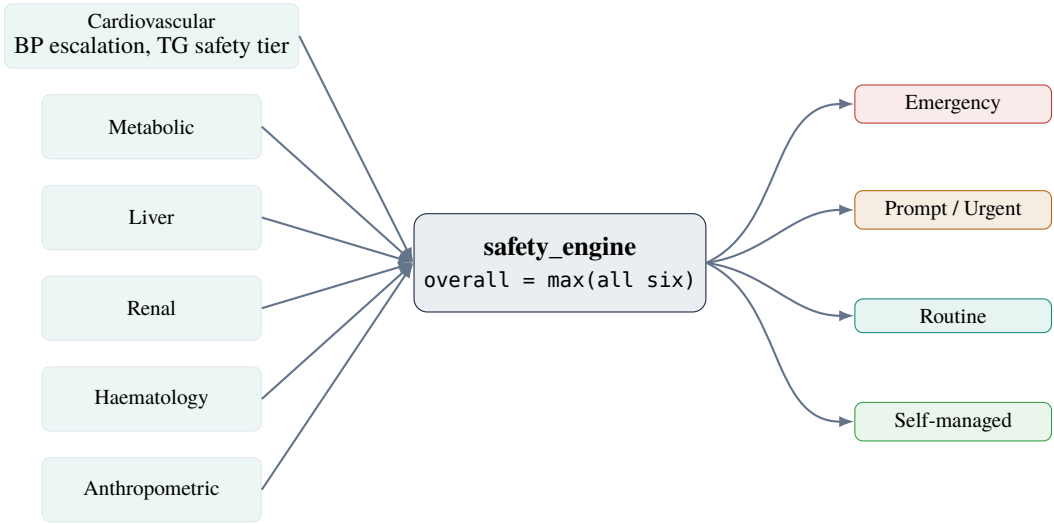


Figure 7.3: Six independent domain urgency signals collapse into one `safety.overall` value, which then selects the highlighted rung of the four-level referral ladder (§6.6.10).

Chapter 8

Build, Run and Deployment

8.1 Running the local basis version

There is no build step anywhere in this codebase — the frontend is static files served as-is, and the backend is plain Python. “Building” the application means only: create a virtual environment, install the backend’s pinned dependencies, and start uvicorn. Table 8.1 summarises the two supported paths; §8.2 walks through the Windows one-click launcher in full, since it is the path most users of this project will actually take.

Platform	Path
Windows	Double-click <code>run.bat</code> at the project root. Everything — virtual environment, dependency install, database seeding, browser launch, server start — is automated and idempotent (safe to run again).
macOS / Linux, or a manual Windows run	<code>cd backend && python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt && python -m app.seed && uvicorn app.main:app --reload</code>

Table 8.1: The two ways to start EVA SQUARE locally, from `README.md`.

Either path ends the same way: one uvicorn process listening on `http://localhost:8000`, serving both the REST API under `/api/*` and the static frontend (§5.1.1), with interactive OpenAPI documentation automatically available at `/docs`. Two demo accounts (`dexter@demo.square` and `priya@demo.square`, password `demo-password` for both) are either pre-seeded by `python -m app.seed` (§5.1.7) or created lazily the first time either `sample-report` shortcut is used (§5.4.1).

8.2 `run.bat` — the Windows one-click launcher

Five steps, each defensive about what might already exist so the script is safe to double-click on every subsequent run, not just the first:

1. **Locate Python.** Tries `python`, then `py`; if neither is found, prints an install link and exits cleanly rather than failing with a cryptic error.
2. **Create the virtual environment** at `backend/.venv`, but only if it doesn’t already exist (`if not exist ".venv\Scripts\activate.bat"`) — this is what makes every run after the first one fast.
3. **Install/upgrade dependencies** via `pip install -q -r requirements.txt` — a near-instant no-op once already satisfied.
4. **Seed the database**, but only if `database/square.db` doesn’t exist yet, so re-running `run.bat` never resets a user’s existing data.
5. **Launch the browser and start the server:** a backgrounded `cmd /c` opens `http://localhost:8000` after a three-second delay (giving uvicorn time to bind the port first), while the foreground process runs uvicorn itself — closing the console window or pressing `Ctrl+C` stops the server.

```
@echo off
```

```

setlocal enabledelayedexpansion
title EVA SQUARE - Local Server
cd /d "%~dp0backend"

rem ---- Find a Python interpreter ----
where python >nul 2>nul
if errorlevel 1 (
    where py >nul 2>nul
    if errorlevel 1 (
        echo.
        echo Python was not found on this computer.
        echo Please install Python 3.9 or later from https://www.python.org/downloads/
        echo and make sure to check "Add python.exe to PATH" during installation.
        echo Then double-click run.bat again.
        echo.
        pause
        exit /b 1
    )
    set "PYEXE=py"
) else (
    set "PYEXE=python"
)

rem ---- Create the virtual environment on first run ----
if not exist ".venv\Scripts\activate.bat" (
    echo Setting up EVA SQUARE for the first time - this can take a minute...
    !PYEXE! -m venv .venv
    if errorlevel 1 (
        echo.
        echo Could not create a Python virtual environment. See the error above.
        pause
        exit /b 1
    )
)

call .venv\Scripts\activate.bat

rem ---- Install/update dependencies (fast no-op once already installed) ----
echo Checking dependencies...
python -m pip install -q --disable-pip-version-check -r requirements.txt
if errorlevel 1 (
    echo.
    echo Failed to install dependencies. Check your internet connection and try again.
    pause
    exit /b 1
)

rem ---- Seed the database with demo accounts on first run ----
if not exist "..\database\square.db" (
    echo Creating the local database with sample accounts...
    python -m app.seed
)

echo.
echo =====
echo EVA SQUARE is starting at http://localhost:8000
echo Your browser will open automatically in a few seconds.
echo Keep this window open while you use the app.
echo Close this window (or press Ctrl+C) to stop the server.
echo =====
echo.

start "" cmd /c "timeout /t 3 >nul & start http://localhost:8000"
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000

```

```
echo.  
echo EVA SQUARE has stopped.  
pause
```

Listing 8.1: run.bat (full file).

8.3 README.md

The project root's own README.md is reproduced here in full — it is the first document a new developer opens, and duplicates several points made throughout this manual (what changed versus Prototype1_Version2, both run paths, the database reset behaviour, and the deployment upgrade summary below) in one condensed, skimmable page.

```
# EVA SQUARE – Prototype 1, Version 2 (Full Structure)

This is the **full frontend / backend / database** build of the EVA SQUARE
prototype: the same 20-screen application and clinical content as
`Prototype1_Version2`, rebuilt on a real three-tier architecture so it is
ready to grow into a deployable web product, rather than a single
self-contained HTML file.

...

Prototype1_Version2_FullStructure/|—
  frontend/      static HTML/CSS/JS single-page app (no build step)|—
  backend/       FastAPI + SQLAlchemy Python application (the REST API)|—
  database/      local SQLite database + schema snapshot
...

Each layer is deliberately independent – see `database/README.md` for the
schema, and every backend module under `backend/app/` for inline comments
explaining the deployment upgrade path.

## What's new versus Prototype1_Version2

Prototype1_Version2 computed everything client-side in the browser and
stored data in `localStorage`. This build moves that logic into a real
Python backend with its own database, and adds four interactive features
requested for this version:

1. **Interactive anatomy board** (Dashboard) – after a report is analysed,
   a schematic 2D body diagram shows one shape per health domain
   (heart = Cardiovascular, liver, kidneys = Renal, etc.), coloured by that
   domain's risk level. Hovering an organ (or its matching legend row)
   darkens it and adds a glow; clicking it opens that domain's full report.
   See `frontend/js/ui/anatomy.js`.

2. **Expandable reference-scale rows** – every quantity in a domain's
   Results tab (and the single-marker Result Detail page) has a ▼ button
   that reveals the quantity's whole reference scale as a coloured bar
   (green/amber/red), with the patient's value marked on it. Backed by a
   new `backend/app/data/reference_scales.py` and the
   `GET /api/meta/reference-scales` endpoint. See `frontend/js/ui/scaleRow.js`.

3. **Enlarged interactive trend charts** – clicking a quantity on the
   Trends page opens a zoomable, pannable canvas chart (scroll to zoom,
   drag to pan). Axis labels are set in Times New Roman; the fitted-trend
   formula is shown in Cambria Math; the plotted line's colour follows the
   quantity's own risk level as a smooth gradient. See
   `frontend/js/ui/trendChart.js`.

4. **Hover feedback everywhere** – buttons, cards, nav links, tabs, table
   rows, toggles and the two features above all have a hover transition
   (see the end of `frontend/css/style.css`).

The content itself – every screen, all clinical logic, both sample
scenarios – is unchanged from `Prototype1_Version2`: the real
Cardiovascular Domain Specification engine (blood pressure, Friedewald
LDL, Framingham risk, ApoB/Lp(a)/AIP/Castelli indices, FH and metabolic
syndrome triggers, four-level safety escalation) has been ported line-for-line
to Python in `backend/app/engines/`, and the other five domains keep their
clearly-labelled illustrative demo logic.
```

Running it locally

Windows – one click

Double-click `**`run.bat`**` in this folder. The first run sets up a Python virtual environment, installs dependencies, and creates the local database with both demo accounts (this can take a minute); every run after that starts in a few seconds. Your browser opens automatically at `**http://localhost:8000**`. Close the black console window (or press Ctrl+C in it) to stop the server.

Requires Python 3.9+ installed with "Add python.exe to PATH" checked during setup – ``run.bat`` will tell you if it can't find Python, with a link to install it. (A `true` .exe`` would need to bundle its own Python interpreter via a packaging tool like PyInstaller, which makes **for** a much larger, harder-to-update file **for** no real benefit at this stage – ``run.bat`` gives the same "double-click and it opens" experience without that.)

macOS / Linux, or the manual steps on Windows

Requires Python 3.9+.

```
``bash
cd backend
python3 -m venv .venv
source .venv/bin/activate           # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python -m app.seed                  # optional – pre-populates both demo accounts
uvicorn app.main:app --reload
``
```

Then open `**http://localhost:8000**` – the same FastAPI process serves both the API (under ``/api/*``) and the frontend, so nothing **else** needs to run. Interactive API docs are at `**http://localhost:8000/docs**`.

Two demo accounts exist (or are created on first use **if** you skip the seed step): sign **in** from the Welcome screen with "Load a routine sample report" / "Load an urgent-pathway sample", or log **in** directly as ``dexter@demo.square`` / ``priya@demo.square`` with password ``demo-password``.

The database

A single SQLite file at ``database/square.db``, created automatically on first run. Delete it any time to reset all app data – it's recreated empty on the next backend start. See ``database/README.md`` for the schema and the ``SQUARE_DATABASE_URL`` environment variable that swaps it for Postgres.

Upgrading to a deployed web version

This basis version was built so that becoming a real deployed product is a matter of configuration, not rewriting:

- **Database**: set ``SQUARE_DATABASE_URL`` to a managed Postgres instance (``database/README.md``). No code changes – every query is plain SQLAlchemy.
- **Auth**: ``backend/app/security.py`` currently hashes passwords with `stdlib` PBKDF2`` and issues opaque bearer tokens with no external dependencies, by design – the module's docstring notes exactly where to swap **in** ``passlib`/`argon2`` and JWTs **for** production.
- **Frontend hosting**: split ``frontend/`` out from FastAPI's `StaticFiles`` mount onto its own host/CDN, and point ``frontend/js/api.js``'s single ``API_BASE`` constant at the deployed API's URL – everything else in the frontend already talks to the backend exclusively through ``SQ.api.*``.
- **OCR**: ``POST` /api/reports/upload`` currently matches any uploaded file

```
to the routine sample lab panel (there is no live OCR in this basis
version, matching Prototype1_Version2's behaviour) – `backend/app/routers/reports.py`
documents exactly where a real extraction pipeline would plug in.
```

Content parity

Every screen's content matches the corresponding page in the project's
`Screen/` mockup directory and `Prototype1\_Version2` exactly – this build
only changes *how* the app is built (three tiers instead of one file), not
what it says.

Listing 8.2: README.md (full file, reproduced verbatim; Markdown source).

8.4 Deployment upgrade path, consolidated

Every simplification this basis version makes for local/offline use is documented, in isolation, at the exact module responsible for it — Table 8.2 collects all four in one place. None of them require touching more than the one module listed; this is a deliberate property of the codebase’s layering, not an accident.

Concern	Change required	Where
Database	Set <code>SQUARE_DATABASE_URL</code> to a managed Postgres/MySQL connection string. No SQLite-specific code exists anywhere in <code>app/routers/</code> or <code>app/models.py</code> .	§4, §5.1.2
Auth	Swap <code>hash_password/verify_password</code> for <code>passlib/argon2</code> ; swap the long-lived opaque bearer token for a short-lived JWT with refresh. Every other module only ever calls through <code>security.*/deps.get_current_user()</code> .	§5.1.6
OCR / extraction	Replace <code>_create_from_scenario()</code> ’s sample-matching in <code>upload_report()</code> with real PDF text-layer parsing (falling back to OCR for scans) and fuzzy-matching against <code>LAB_DICTIONARY.altNames</code> . Verify, questionnaire, analyse and trends are completely unaffected.	§5.4.4
Frontend hosting	Split frontend/ off FastAPI’s <code>StaticFiles</code> mount onto its own CDN/static host; point <code>js/api.js</code> ’s <code>API_BASE</code> constant at the deployed API’s URL. Every frontend call already goes through <code>SQ.api.*</code> , so nothing else changes.	§6.2.1, §5.1.1

Table 8.2: Every “basis version” simplification in this codebase, the change that replaces it, and the one module each is confined to.

A note on scope

None of these four changes are speculative — each is called out explicitly, in the source’s own comments or docstrings, at the exact line where the simplification lives (see the cross-references above). This manual’s role is to make that existing, distributed documentation discoverable in one place, not to invent a migration plan that doesn’t already exist in the code.

Appendix A

REST API Reference

Every endpoint below is registered by exactly one router file (§5) under the fixed prefix shown; “Auth” means the request must carry `Authorization: Bearer <token>` or the backend returns 401 (§5.1.3). Endpoints marked *owned* additionally 404 (never 403) if the report id belongs to a different user — `deps.get_owned_report()`’s deliberate choice (§5.1.3). Every row is exercised by exactly one function in `frontend/js/api.js` (§6.2.1), named in the last column.

A.1 Auth — /api/auth

Method	Path	Auth	Body	SQ.api.*
POST	/register	—	{fullName, email, password}	register
POST	/login	—	{email, password}	login
POST	/demo-login/{scenario}	—	—	demoLogin
POST	/logout	yes	—	logout
GET	/me	yes	—	me

register/login/demo-login all return `schemas.AuthOut ({token, user})` and issue a fresh session row (§5.1.6); demo-login get-or-creates the named demo account rather than requiring one to already exist. /me returns the combined {user, consent, settings} bundle `store.init()` needs in one round trip.

A.2 Consent — /api/consent

Method	Path	Auth	Body	SQ.api.*
GET	(root)	yes	—	getConsent
POST	/accept-terms	yes	—	acceptTerms
PATCH	(root)	yes	{kind, id, value}	patchConsent

kind is "required" or "optional"; id is a consent item id from `data/legal_content.py` (§5.2.2). The response always includes `requiredComplete`, the boolean the Consent screen’s continue button gates on.

A.3 Settings — /api/settings

Method	Path	Auth	Body	SQ.api.*
GET	(root)	yes	—	getSettings

Method	Path	Auth	Body	SQ.api.*
PATCH	(root)	yes	any of {units, notifications, theme}	

A.4 Reports — /api/reports

Method	Path	Auth	Body	SQ.api.*
GET	(root)	yes	—	listReports
GET	{id}	owned	—	getReport
POST	/sample	yes	{scenario, fileName?, fullPipeline?}	createSampleReport
POST	/upload	yes	multipart file	uploadReport
POST	{id}/verify	owned	{values}	verifyReport
GET	{id}/questionnaire	owned	—	getQuestionnaire
PATCH	{id}/questionnaire	owned	{patch}	patchQuestionnaire
POST	{id}/analyze	owned	—	analyzeReport
GET	{id}/analysis	owned	—	getAnalysis
GET	{id}/trends	owned	—	getTrends

The largest and most stateful group in the API (§5.4.4). Notes worth calling out explicitly:

- POST /sample and POST /upload both funnel through `_create_from_scenario()` — the only difference is that upload preserves the uploaded file’s own name and always starts pending-verify, while /sample accepts a `fullPipeline` flag used only by `demoLogin`’s “try it instantly” shortcuts to create an already-verified, already-analysed report in one call.
- POST {id}/analyze 400s if the report isn’t verified yet; GET {id}/analysis instead transparently computes and caches the same result lazily on first request (as long as the report is verified) — this is why the frontend can call `getAnalysis` unconditionally from five different screens (§6.6.2, §6.6.6, §6.6.8, §6.6.9, §6.6.10) without each of them needing to know whether /analyze already ran.
- GET {id}/trends is the one endpoint in this group that returns pre-computed derived fields (`direction`, `delta`) rather than raw stored data — computed fresh on every call by `trend_engine.py` (§5.3.8), not cached server-side.

A.5 Action plan — /api/action-plan

Method	Path	Auth	Body	SQ.api.*
GET	(root)	yes	—	getActionPlan
POST	/toggle	yes	{code}	toggleActionItem

Global per-user state, not report-scoped (§5.4.6); both endpoints return the full `{done: {code: bool}}` map.

A.6 Meta (static reference content) — `/api/meta`

Method	Path	Auth	SQ.api.*
GET	<code>/domains</code>	—	<code>metaDomains</code>
GET	<code>/lab-dictionary</code>	—	<code>metaLabDictionary</code>
GET	<code>/questionnaire-sections</code>	—	<code>metaQuestionnaireSections</code>
GET	<code>/legal</code>	—	<code>metaLegal</code>
GET	<code>/reference-scales</code>	—	<code>metaReferenceScales</code>
GET	<code>/referral-levels</code>	—	<code>metaReferralLevels</code>
GET	<code>/education</code>	—	<code>metaEducation</code>
GET	<code>/sample-scenarios</code>	—	<code>metaSampleScenarios</code>

The only unauthenticated group besides registration/login — pure static content re-exported from `app/data/` (§5.2.1–§5.2.6). Fetched exactly once, in parallel, by `store.init()` (§6.2.2) and bound onto plain `SQ.*` globals; no screen calls any of these eight functions directly after boot.

A.7 Health check

Method	Path	Auth	Notes
GET	<code>/api/health</code>	—	Registered directly in <code>main.py</code> (§5.1.1), not a router file; returns

Appendix B

Static Reference Content: Quick Tables

This appendix consolidates the registries defined across `app/data/` (§5.2.1–§5.2.6) into single lookup tables, for reference while reading Chapters 5 and 6 without paging back to the full source listings.

B.1 Health domains

Key	Label	Source
cardiovascular	Cardiovascular	Real — Cardiovascular Domain Specification (§5.3.2)
metabolic	Metabolic	Illustrative (§5.3.3)
liver	Liver	Illustrative
renal	Renal	Illustrative
haematology	Haematology	Illustrative
anthropometric	Anthropometric	Illustrative (BMI/waist rules)

DOMAIN_ORDER fixes this exact sequence everywhere a list of domains is rendered (the anatomy board’s card columns, the Analysis Summary grid, the sidebar’s Health Domains link).

B.2 Lab dictionary

Code	Name	Domain	Unit	Source
LAB_TC	Total Cholesterol	cardiovascular	mmol/L	Real
LAB_LDL	LDL Cholesterol	cardiovascular	mmol/L	Real
LAB_HDL	HDL Cholesterol	cardiovascular	mmol/L	Real
LAB_TG	Triglycerides	cardiovascular	mmol/L	Real
LAB_APOB	Apolipoprotein B	cardiovascular	mg/dL	Real (advanced)
LAB_APOA1	Apolipoprotein A-I	cardiovascular	mg/dL	Real (advanced)
LAB_LPA	Lipoprotein(a)	cardiovascular	nmol/L	Real (advanced)
LAB_GLU_FAST	Fasting Glucose	metabolic	mmol/L	Illustrative
LAB_HBA1C	HbA1c	metabolic	%	Illustrative
LAB_ALT	ALT	liver	U/L	Illustrative
LAB_AST	AST	liver	U/L	Illustrative
LAB_CREAT	Creatinine	renal	μmol/L	Illustrative
LAB_EGFR	eGFR	renal	mL/min/1.73m ²	Illustrative
LAB_HB	Haemoglobin	haematology	g/dL	Illustrative
LAB_WBC	White Blood Cells	haematology	cells/μL	Illustrative
LAB_PLT	Platelets	haematology	Lakh/μL	Illustrative

The seven cardiovascular codes are interpreted by the real engine (§5.3.2); the remaining nine are classified only against the fixed SIMPLE_RANGES thresholds in the same module, via the generic `rule_engine.classify_by_range()` (§5.3.4) — every one of them is rendered with an explicit “illustrative” badge in the frontend (§6.6.4).

B.3 Anthropometric rules

BMI upper bound	Category	Tone
18.5	Underweight	warn
23	Normal	good
27.5	Overweight	warn
(unbounded)	Obese	bad

Waist circumference cut-off: 90 cm (men), 80 cm (women) — the same cut-off both §5.3.3's anthropometric domain and CV-12's metabolic-syndrome criterion (§5.3.2) key off.

B.4 Recommendation library (action plan)

Code	Issue	Category	Weight
REC_CV_BP_TRACK	Blood pressure above the optimal range	monitoring	95
REC_CV_SMOKING	Current smoking status	lifestyle	97
REC_CV_LIPID_DIET	LDL cholesterol / ApoB above the goal linked to your risk category	nutrition	88
REC_ANTHRO_WEIGHT	Body mass index and waist circumference above the healthy range	lifestyle	85
REC_MET_SUGAR	Fasting glucose / HbA1c above the illustrative reference range	nutrition	80
REC_LIVER_ALCOHOL	ALT above the illustrative reference range	lifestyle	75
REC_CV_ACTIVITY	General cardiovascular and metabolic health	activity	60
REC_GEN_SLEEP	General preventive health	sleep	40

Sorted by descending weight at serve time (§5.3.6); REC_CV_ACTIVITY and REC_GEN_SLEEP both carry when: `lambda fh: True` and so always match, functioning as the two baseline preventive-care entries every analysed report receives regardless of findings.

B.5 Referral ladder

Rank	Level	Description	Action
1	Emergency care	New severe chest pain, severe breathing difficulty, fainting, sudden weakness, facial droop or speech difficulty.	Call local emergency services
2	Prompt medical review	Concerning symptoms, markedly abnormal repeat readings, or a result specifically flagged for early review.	Seek medical advice soon
3	Routine appointment	Discuss non-urgent findings, prevention targets and ongoing monitoring.	Recommended for you

Rank	Level	Description	Action
4	Wellness self-management	Continue appropriate lifestyle actions and routine monitoring when no clinical review is indicated.	View Action Plan

`safety_engine.py` (§5.3.5) selects the recommended rank via `safety.overall`; see Figure 7.3 for how the six domain-level signals feed into that single value.

B.6 Demo accounts

Scenario	Account	Password
Routine (Dexter Tan, 35M)	dexter@demo.square	demo-password
Urgent (Priya Kumar, 58F)	priya@demo.square	demo-password

Both are get-or-created lazily by `POST /api/auth/demo-login/{scenario}` (§5.4.1) the first time either is used, or pre-populated in one step by running `python -m app.seed` (§5.1.7).